Handwriting robot

Final Report

D.R. Braune
23539195

Submitted as partial fulfilment of the requirements of Project EPR400
in the Department of Electrical, Electronic and Computer Engineering
University of Pretoria

November 2026

Study leader: Prof. H.C. Myburgh

Part 1. Preamble

This report describes work I completed in my final year project, designing and implementing a handwriting robot that recognises handwritten text and its writer and reproduces text on paper in a chosen writer's handwriting style.

1. Project proposal and technical documentation
This main report contains an unaltered copy of the approved Project Proposal (as Part 2 of the report).

All the code that I developed appears as a separate submission on the AMS.

2. Project history
This project does not continue the work of a previous student project, and no part of a previous project's design or code was reused. The following parts of the work are not my own and are identified as such wherever they appear in the report. The physical components of the gantry and the capture rig (the two stepper motors, the two driver modules, the camera, the single-board computer, the switches, the pen, the power supply and the standard belts, pulleys, rods, bearings and fasteners) were bought, as the project proposal stated they would be. The layer arrangement of the text recogniser follows a published network described in the literature study, and is cited where it is used in Section 3.4; the implementation of that arrangement is my own. The public handwriting database used for training, and its transcriptions, are the work of its compilers and are cited. The software libraries listed under declaration 6 below are the work of their authors. Everything else reported on here, including all of the image processing, the training and inference code, the style extraction and synthesis, the G-code generation and the motion control, I designed and implemented myself. Section 6.3 states this in full.

MUST STILL BE COMPLETED BY ME

Confirm the first sentence above before submission. Appendix 4 requires this paragraph to state explicitly whether the project builds on an earlier project, and to cite that project by author and year if it does. I could not establish this from the project files, so I have written the statement that matches my own understanding — if any part of this project continues a previous student's work, name that work here by author and year and say exactly what was taken from it.

3. Language editing
This document has been language edited by a knowledgeable person. By submitting this document in its present form, I declare that this is the written material that I wish to be examined on.

My language editor was _____.

_____ _____

Language editor signature *Date*

MUST STILL BE COMPLETED BY ME

Fill in the language editor's name and obtain their signature and date, or attach their written confirmation at the end of the report and write "proof of language editing is attached" in place of the signature. The report is rejected without this.

4. Plagiarism declaration

I, D.R. Braune, understand what plagiarism is and have carefully studied the plagiarism policy of the University. I hereby declare that all the work described in this report is my own, except where explicitly indicated otherwise. Although I may have discussed the design and investigation with my study leader, fellow students or consulted various books, articles or the internet, the design/investigative work is my own. I have mastered the design and I have made all the required calculations in my lab book (and/or they are reflected in this report) to authenticate this. I am not presenting a complete solution of someone else or that I found somewhere.

Wherever I have used information from other sources, I have given credit by proper and complete referencing of the source material so that it can be clearly discerned what is my own work and what was quoted from other sources. I acknowledge that failure to comply with the instructions regarding referencing will be regarded as plagiarism. If there is any doubt about the authenticity of my work, I understand that I may have to attend an oral ancillary examination/evaluation about the work.

5. AI usage declaration

Artificial intelligence tools, including large language models and code-generation tools, were used during this project. They were used to draft and edit the text of this report, to assist with writing, refactoring and reviewing source code, and to help build scripts that audited the code and checked the reported measurements. They were not used to produce any of the measured results reported in Section 4: every number in this report comes from running the project's own code on the project's own data, and the raw output files are part of the code submission. The choice of methods, the design of the system, the decisions about what to build and what to abandon, and the interpretation of the results in Section 5 are my own.

MUST STILL BE COMPLETED BY ME

Read declaration 5 above and make it exactly true before submission, adding or removing items as needed. The template's default wording, which denies any AI use, was deleted because it is not true of this project. A declaration that does not match the facts is itself an integrity violation, independently of the quality of the work, so this paragraph must be checked word by word and signed off by me.

6. Software library usage declaration

Unless explicitly stated otherwise in the next paragraph, the author confirms that no software library functions, packages, or toolboxes were used in the design, implementation, or analysis presented in this report. All code and computations were developed entirely by the author.

The exceptions to the above are the following. NumPy (`numpy.org`) was used for array storage and elementary array arithmetic, and Pillow (`python-pillow.org`) for reading and writing image files and for drawing text labels and boundary lines onto diagnostic images that no network reads. No image-processing, machine-learning or scientific routine from either was used in the signal path. Every operation that changes a pixel value the networks are judged on — scaling a crop, halving and rotating a page, warping a training image, blurring a stroke edge and drawing the synthesised ink — is performed by one module written for this project, and each of those operations was verified against the library call it replaced. The convolution, normalisation, pooling, recurrent, loss and optimisation routines were likewise written from first principles, as Section 3.4 describes. SciPy (`scipy.org`) was used in two places that are not part of the deployed system: `scipy.ndimage.gaussian_filter` and `scipy.ndimage.map_coordinates` in the training-time image augmentation, and `scipy.signal.find_peaks` in the script that cuts the public database pages into lines. Flask and Flask-CORS (`flask.palletsprojects.com`) provide the web server. OpenCV (`opencv.org`) is used only to read frames from the USB camera, and libgpiod (`libgpiod` Python bindings) only to drive and read the general-purpose input and output pins of the single-board computer; both are hardware-interfacing libraries and are imported only inside the single function that needs each. The Hugging Face `datasets` package (`huggingface.co/docs/datasets`) was used once, in a standalone script, to download the public line-level handwriting database; nothing in the running system imports it. Finally, the weights of the writer-identification network were trained with PyTorch (`pytorch.org`) and then converted into this project's own weight format; the network that runs on the machine, and the from-scratch training program described in Section 3.4, contain no PyTorch code and do not import it.

MUST STILL BE COMPLETED BY ME

Two things to settle in declaration 6. First, confirm whether the deployed text recogniser's weights (`paper_cnn_bilstm_ctc_scratch_best.npz`) were trained from random initialisation by my own training program, or were warm-started from the PyTorch-trained checkpoint using the `--warm-start` option. I could not determine this from the saved file, and the answer changes what Section 3.4 and this declaration may claim. Second, check this list against the code submission and add anything I have missed.

7. Unaltered Project Proposal

I certify that the Project Proposal appearing as the Introduction section of the report is an unaltered copy of the approved Project Proposal.

Signature

With my signature below, I hereby certify that all statements, declarations, and information presented above are complete.

D.R. Braune Date

MUST STILL BE COMPLETED BY ME

Sign and date the report here. The report is rejected without this signature.

TABLE OF CONTENTS

Part 2. Project definition: approved Project Proposal

1. Project description
2. Technical challenges in this project
3. Functional analysis
4. System requirements and specifications
5. Field conditions
6. Student tasks

Part 3. Main Report

1. Literature study 1
    1.1. Existing concepts 1
        1.1.1. A need exists to be able to decode the writer's style and written text from an image 1
        1.1.2. A need exists to reproduce text on paper accurately and fast in any writer's style 5
    1.2. Intended contribution and discussion 6

2. Approach 8
    2.1. From functional analysis to concept 8
    2.2. Order in which the subsystems were built 8
    2.3. Design alternatives and trade-offs 9

3. Design and implementation 10
    3.1. Design summary 10
    3.2. System block diagram 10
        3.2.1. Hardware block diagram 13
        3.2.2. Software block diagram and systems-level description 13
    3.3. Hardware design 14
        3.3.1. Components and pin assignment 15
        3.3.2. Complete circuit and its operation 15
        3.3.3. Mechanical arrangement and the torque calculation 16
        3.3.4. Step generation, resolution, speed and calibration 18
        3.3.5. End-stops, emergency stop and pen mechanism 20
    3.4. Page conditioning and line segmentation 20
        3.4.1. Design goals and the measured scale unit 21
        3.4.2. The three array primitives 23
        3.4.3. Stage 2: lighting normalisation and the ink mask 23
        3.4.4. Stage 3: deskew by searching rotations 24
        3.4.5. Stage 4: removing what is not handwriting 25
        3.4.6. Stage 5: line pitch and line centres from the row-ink profile 26
        3.4.7. Stage 6: seams, pixel ownership and the line crops 27
        3.4.8. Testing the segmentation block 28

3.4.9. Design alternatives and limitations 30
3.5. Text recognition 31
3.5.1. Problem statement and design approach 31
3.5.2. Input pre-processing 32
3.5.3. Network architecture 32
3.5.4. Connectionist temporal classification 35
3.5.5. Training data and splits 36
3.5.6. Training procedure 36
3.5.7. From-scratch forward-and-backward implementation 38
3.5.8. Evaluation metrics and measured accuracy 39
3.5.9. Inference on the single-board computer, and alternatives 40
3.6. Writer identification 41
3.6.1. Why two classifiers were needed 41
3.6.2. Architecture 41
3.6.3. Stroke normalisation 43
3.6.4. Training data, protocol and the training fault 43
3.6.5. Page decisions, evidence and alternatives 44
3.6.6. Known faults and limitations 45
3.7. Style extraction and glyph library 46
3.7.1. Forced alignment: where is each character? 46
3.7.2. From ink to ordered pen paths 48
3.7.3. Variants, quality gates and the cross-writer letter prior 49
3.7.4. Style statistics and permanent storage 50
3.7.5. Design alternatives and justification 51
3.7.6. Known faults and limitations 51
3.8. Handwriting synthesis 52
3.8.1. Global quantities and glyph choice 52
3.8.2. Placement, joins and randomness 54
3.8.3. Best-of-*N* search with joint scoring 55
3.8.4. Design-stage measurements of the synthesiser 57
3.8.5. Design alternatives and justification 58
3.8.6. Known faults and limitations 58
3.9. G-code generation 59
3.9.1. Placement, ordering and program structure 59
3.9.2. Time model and verification 60
3.9.3. Known faults and limitations 61
3.10. Modifications made after the chain could be measured 62
3.10.1. A second writer classifier, judging shape instead of ink 62
3.10.2. End-to-end training in place of a frozen backbone 62
3.10.3. Selecting a candidate on both criteria instead of legibility alone 63
3.10.4. Line-level style calibration 63
3.11. Software design and system integration 63
3.11.1. Process flow and start-up 64
3.11.2. Timing chains of the two modes 65

4. Results 66
4.1. Summary of results achieved 66
4.2. Qualification tests 66
Qualification test 1: Reproduction fidelity of the synthesised handwriting (R1)
Objectives of the test or experiment 67
Equipment used 67
Test setup and experimental parameters 67
Steps followed in the test or experiment 68
Results or measurements 68
Observations 69
Statistical analysis 70
Qualification test 2: Classification of text and writer from real photographed pages (R2)
Objectives of the test or experiment 71
Equipment used 71
Test setup and experimental parameters 71
Steps followed in the test or experiment 71
Results or measurements 71
Observations 71
Statistical analysis 72
Qualification test 3: Reproduction time per character (R3)
Objectives of the test or experiment 72
Equipment used 72
Test setup and experimental parameters 72
Steps followed in the test or experiment 72
Results or measurements 73
Observations 73
Statistical analysis 73
Qualification test 4: Classification time (R4)
Objectives of the test or experiment 73
Equipment used 74
Test setup and experimental parameters 74
Steps followed in the test or experiment 74
Results or measurements 74
Observations 74
Statistical analysis 74
Qualification test 5: Absolute positional accuracy of the gantry (R5)
Objectives of the test or experiment 75
Equipment used 75
Test setup and experimental parameters 75
Steps followed in the test or experiment 75
Results or measurements 75
Observations 76
Statistical analysis 76
Qualification test 6: Multi-writer storage (R6)
Objectives of the test or experiment 76
Equipment used 76
Test setup and experimental parameters 76

Steps followed in the test or experiment 76
Results or measurements 76
Observations 76
Statistical analysis 77

5. Discussion 82
5.1. Critical evaluation of the design 82
5.1.1. Interpretation of results 82
5.1.2. Critical evaluation 84
5.1.3. Unsolved problems 85
5.1.4. Strong points of the design 87
5.1.5. Expected failure conditions 87
5.2. Considerations in the design 88
5.2.1. Ergonomics 88
5.2.2. Health and safety 88
5.2.3. Environmental impact 89
5.2.4. Social and legal impact 89
5.2.5. Ethics clearance 89

6. Conclusion 91
6.1. Summary of the work completed 91
6.2. Summary of observations and findings 91
6.3. Contribution 92
6.4. Suggestions for future work 93

7. References 94

Appendix

Metric definitions of the qualification tests 99
Confidence intervals and resampling 99
Character accuracy and its three averages 99
Word accuracy is a recall, not one minus the word error rate 100
Writer accuracy is reported per line, not per page 100
Decoding 100
Inter-rater agreement of the panel that was not convened 100
Style correlation on geometric features 100
Overlay distances in the pixel domain 101
Positional error and its uncertainty budget 102
Additional results and prepared measurement sheets 102
Provenance of the data sets and checkpoints 102
Per-feature and per-judge breakdown of qualification test 1 103
Per-writer breakdowns 103
Storage check of qualification test 6 104
Prepared sheets for the sub-tests not carried out 104
Features prescribed for the style correlation 105
Distance distributions behind the overlay test 106

Supporting detail of the page conditioning and segmentation block 106
Constants of the minor operations 107
The eight artefact-discard predicates 107
The six pixel-ownership correction rules 108
Pseudocode of the four routines that carry the arithmetic 110
Supporting detail of the text recogniser 111
Parameter budget of the network 112
The CTC forward–backward recursion 113
Pseudocode of the routines that carry the arithmetic 114
The backward passes and their verification 115
Data sets and their leakage assessment 117
Edit distance as the accuracy measure 118
Supporting detail of the writer classifiers 119
Parameter budget 119
The optimiser 119
Pseudocode of the forward pass and the stroke normalisation 120
Supporting detail of the style builder and the synthesiser 120
From a thinned image to ordered pen paths 121
The five quality gates 122
Schema of a stored style profile 123
Constants of the synthesiser 123
Cost of the candidate search 123
Supporting detail of the hardware design 125
Pin assignment 125
Stepper and belt dynamics behind the torque requirement 125
The three interface-circuit derivations 126
Interpolation and calibration arithmetic 127
Pseudocode of the motion program 128
Supporting detail of the G-code generator and the software architecture 128
Configuration of the G-code generator 128
Apportionment of the two timing budgets 128
Pseudocode of the emitting loop and the start-up warm-up 130

LIST OF ABBREVIATIONS

AdamW	Adam optimiser with decoupled weight decay
AMS	assignment management system (code submission)
API	application programming interface
ARM	processor architecture of the ODROID N2+ (Advanced RISC Machine)
AUC	area under the receiver operating characteristic curve
BiLSTM	bidirectional long short-term memory
BN	batch normalisation
CAD	computer-aided design
CER	character error rate
CNN	convolutional neural network
CRNN	convolutional recurrent neural network
CTC	connectionist temporal classification
EMNIST	Extended Modified National Institute of Standards and Technology (data set)
FC	field condition (FC1, FC2); also fully connected layer where stated
FU	functional unit
G-code	line-oriented machine-control instruction language
GPIO	general-purpose input/output
HSV	hue-saturation-value
HTR	handwritten text recognition
HTTP	hypertext transfer protocol
HWR	handwriting recognition
IAM	Institute of Computer Science and Applied Mathematics (handwriting database)
ID	identification (writer identification)
IoU	intersection over union
JPEG	Joint Photographic Experts Group (image format)
JSON	JavaScript Object Notation
LCS	longest common subsequence
LSTM	long short-term memory
MAC	multiply-accumulate operation
PC	personal computer
POPIA	Protection of Personal Information Act (South Africa)
PSU	power supply unit
R1–R6	requirements 1 to 6 of the project proposal
ReLU	rectified linear unit
RMSprop	root-mean-square propagation (optimiser)
RNN	recurrent neural network
SBC	single-board computer
STEP/DIR	step and direction control signals of a stepper-motor driver
USB	Universal Serial Bus
WER	word error rate

NOTATION

x_{h}	x-height of the handwriting (height of lower-case letters without ascenders or descenders)
x	number of characters of a sentence or page (in the time requirements R3 and R4)
N	number of candidate syntheses tried in the best-of-N search (“tries”)
$A_{\mathrm{c}}, A_{\mathrm{w}}$	character accuracy and writer accuracy

Part 2. Project definition: approved Project Proposal

This section contains the problem identification in the form of the complete approved Project Proposal, unaltered from the final approved version that appears on the AMS.

[MUST STILL BE COMPLETED BY ME: check these eight pages against the approved proposal on the AMS and confirm that every answer, the title block and the Feedback block now appear. The included file is a flattened copy; the original is kept unaltered as `project_proposal.pdf`.]

For use by the Project lecturer	Approved		Revision required	

Feedback

For use by the Project module lecturer only

To be completed by the student

PROJECT PROPOSAL 2026

Project no	HCM4	Revision no	1

Title	Surname	Initials	Student no	Study leader (title, initials, surname)
Mr	**Braune**	**D.R**	23539195	Prof. H.C. Myburgh

Project title (this is the title on the project concept note)

Handwriting robot

Language editor details	Language editor signature
MTBraune	
Student declaration I understand what plagiarism is and that I have to complete my project on my own.	**Study leader declaration** This is a clear and unambiguous description of what is required in this project. Approved for submission (Yes/No)
Student signature	Study leader signature and date 20 April 2026

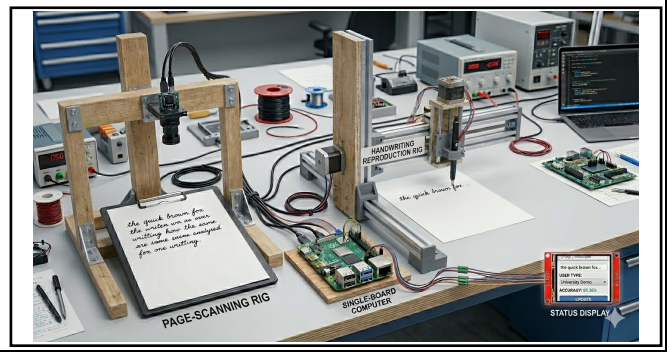
1. Project description.

What is the problem to be solved with your project? Describe in non-technical language what your project is about. What does your system have to do? Refer explictly to Figure 1 that you inserted on the right.

The need for handwriting still exists for authenticity, ease of communication and personal touches. The problem is that many individuals do not possess the time, skills or ability to write neatly and legibly. This leads to the solution presented in this proposal: a handwriting robot capable of recognizing a user's handwriting style and being able to replicate it.

The proposed solution is a handwriting robot capable of training on multiple different user's handwritten text (using a page-scanning rig to capture the handwritten page like the one seen in the image) and being able to produce any desired piece of text in any of these user's writing styles (using a text reproduction rig like the one seen in the image). The system will be able to recognize which user authored a piece of text and identify what text was written (shown to the user through a display as seen in the image). A gantry with 2-dimensional movement will perform actions with a pen to automatically reproduce the desired text.

Figure 1. Insert an image below that captures the idea with your project. Explain in the text on the left.



2. Technical challenges in this project

Describe the technical challenges that are *beyond* those encountered up to the end of third year and in other final year modules.

2.1 Primary *design* challenges

What is the intended core technical challenge in the project? Which aspect or aspects of the design of the system do you expect to be the most challenging?

1. Text image processing will be performed from first principles, including the isolation of letters from an image of various fonts and sizes of handwritten text.
2. A neural network will be designed and trained from first principles to be able to categorize text to different user's styles based on certain style elements, and identify the text written.
3. A gantry rig will be designed to be capable of performing the necessary precise 2-dimensional movements using a pen, whilst holding the pen and paper securely in place.
4. Pen path planning will be implemented, which involves the conversion of written characters to a 2D motion the gantry needs to perform, and then mapping the movement to stepper motor controls.

2.2 Primary *implementation* challenges

Which aspect or aspects of the implementation do you expect to be the most challenging?

1. Image processing and deep learning will be performed on a single board computer (instead of on a personal computer), which has limited memory, storage and power.
2. Smooth motor control will be needed to produce precise and consistent pen movement to create accurate text.
3. Near real-time classification of handwritten text and reproduction of digital text on paper will need to be achieved.

All processing will be done on an embedded platform, and not on a PC platform. Confirmed by having ticked this box. ✔

Library functions will be used for hardware interfacing only. Confirmed by having ticked this box. ✔

3. Functional analysis

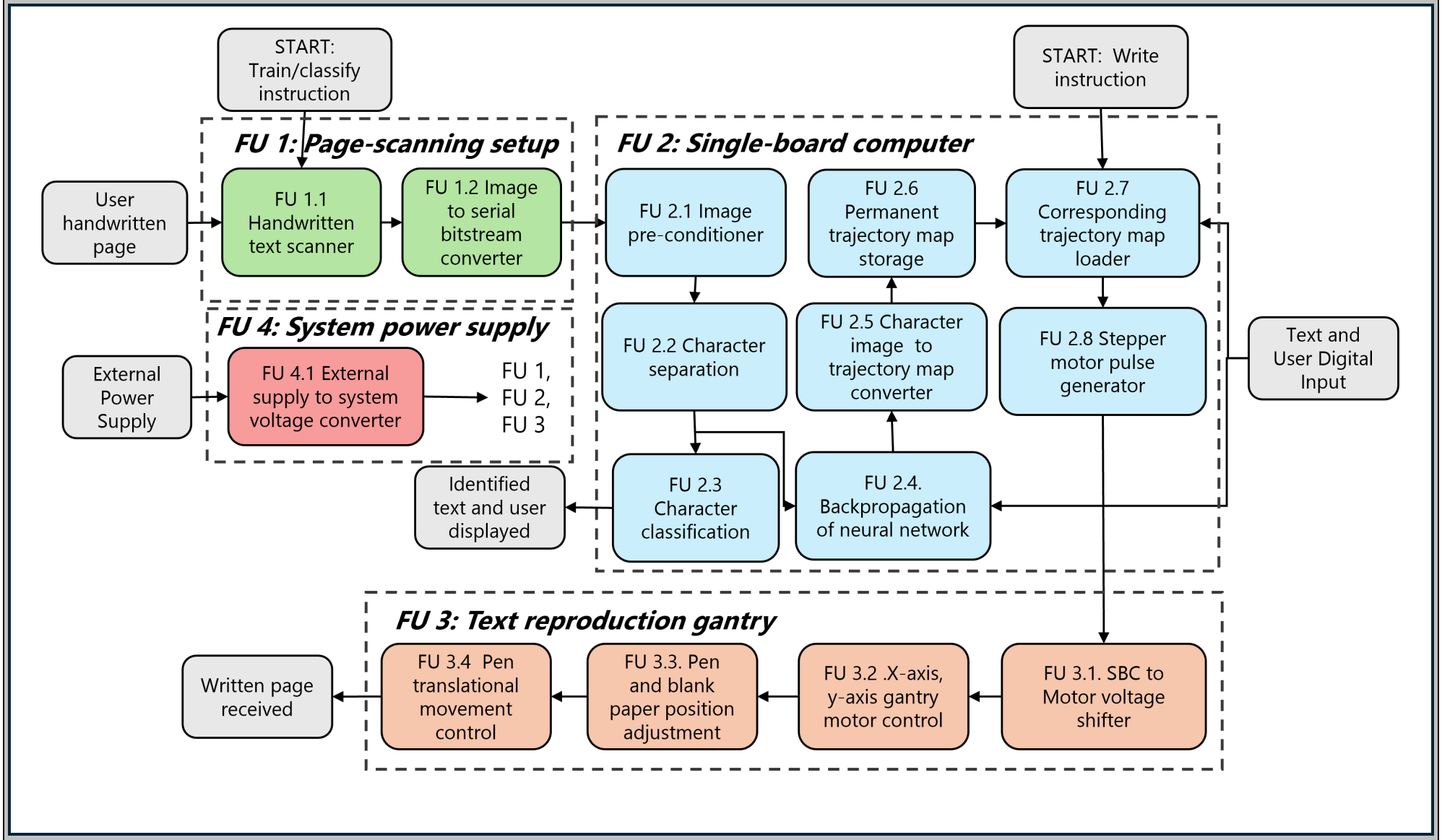
3.1 Functional description

Describe the design in terms of system functions as shown on the functional block diagram in section 3.2. This description should be in *narrative format*. **DO NOT** use a bullet list.

There are three main modes of operation for the handwriting robot: training, classification and reproduction
Training/Classification: First, the user will input their page of handwritten text to the scanner (FU 1.1). The user will select to either train or classify, and will input the intended digital text alongside this through an input interface. Once the page is scanned, a camera USB driver will be used to transfer the image to the single board computer (FU 1.2). The image will be conditioned first to fix warping or noise in the image (FU 2.1), and then the character separation will occur. This will separate the page characters (or groups of characters that are connected) to match the desired neural network input size (FU 2.2). The difference between training and classification occurs here: if the 'training' option is selected, the separated characters will be used to adjust the neural network neuron weightings against the desired digital text that was input (FU 2.4). Each separated character or group of characters will also be converted into a trajectory map (coordinate map of required pen motion) (FU 2.5). This trajectory map will be stored permanently within the system (FU 2.6). If the 'classify' option is selected, the separated characters or group of characters will be passed into the trained neural network classifier (FU 2.3), and the corresponding digital text and detected user will be outputted to the display interface.

Reproduction: If the user sends a 'write' instruction, the digital text and user style desired to be reproduced will be provided. The corresponding trajectory for each character/group will be retrieved from storage (FU 2.7), and will be converted into stepper motor controls required to produce the desired pen movements (FU 2.8). These stepper motor controls will be sent to the corresponding motors via a stepper motor driver to allow for forward and reverse movements (FU 3.1). The x-motor, y-motor and pen lift mechanism will be controlled through the stepper motor driver (FU 3.2/3.4). After the pen has been adjusted and is held securely by the pen holder in the robot (FU 3.3), precise handwritten imitations of each character will be written.

3.2 Functional block diagram (this should not be a flow diagram)



4. System requirements and specifications

These are the core requirements of the system or product (the mission-critical requirements) in table format **IN ORDER OF IMPORTANCE**. Requirement 1 is the most fundamental requirement.

	Requirement 1: the fundamental functional and performance requirement of your project	**Requirement 2** (Second in order of importance)	**Requirement 3** (Third in order of importance)
1. Core mission requirements of the system or product. Focus on requirements that are **core** to solving the engineering problem. These will reflect the solution to the problem.	The system must be able to accurately reproduce any desired digital text on paper in the writing style of any writer from the training set.	The system must be able to accurately decode the content of the handwritten text, and identify which writer from the training set wrote the handwritten text.	The system must reproduce text on paper at a near real time speed, at a rate that can be considered practical and useful.
2. What is the target specification (this must be in *measurable* terms) to be met in order to achieve the requirement in 1. above?	The system should reproduce desired text with a minimum character accuracy of 85% while having a high stylistic correlation of 85% to the chosen writer's style.	The system must be able to identify the writer of the handwritten text from the training set and the content of the written text with an overall accuracy of 95%.	The system must be capable of reproducing the desired text in the writing style of any writer in the training set at a rate of no slower than 4 seconds per character.
3. Explain where this specification originated : i.e., **explain** the value that you selected. I.e., *how* will meeting the specification given in point 2 above *solve the problem*?	An accuracy of 85% (averaged from Babushkin et al. results [1, p8]) is high enough to represent the input text, but practical to account for the build up of precision loss in the system.	An overall classification accuracy of 95% (similar to existing classifiers mentioned in Acta Informatica Pragensia [2, p6]) ensures practical usability and consistency.	Averaging the time required to produce text between 0% and 1% error (Ossi Teikari [3, p35]) gives approximately 4 seconds per character, allowing the system to reproduce text at a practical rate with minimal error.
4. How will you demonstrate at the examination that this requirement and specification (points 1 and 2 above) have been met? Be explicit about how you will ***prove*** these were met.	A sentence of text will be reproduced in a specific user style, and the amount of characters that can be visibly seen to match the input text (in the style of the chosen user) must be >= to 85% of the total characters produced.	A predefined paragraph of text must be handwritten by a writer in the training set and classified by the system. The classified text and writer must match the input text and writer with an overall accuracy of >= 95% .	A sentence of x characters long will be sent to the system to be reproduced, the system must be able to reproduce it within 4*x seconds.
5. Your own design contribution: what are the aspects that *you will design and implement yourself* to meet the requirement in point 2? If none, *remove this requirement.*	Accurate conversion from character image to pen trajectory map, precise motor control from pen trajectory map, and a structurally sound gantry design for accurate drawing will be designed and implemented by the student.	A rig to capture an image of written text, image conditioning and noise removal, character separation and processing, classification with a neural network, and communicating necessary data to a display will be designed by the student.	Techniques to optimize pen movements for quicker drawing, smooth motor control and robust design for efficient gantry movements will be designed and implemented by the student.
6. What are the aspects to be taken off the shelf to meet this requirement? If none, indicate "none". Clearly specify for what tasks **library functions** will be used (if relevant to the project).	Physical motors, pen, single board computer, stepper motor driver, and separate components for gantry design e.g. screws or belts will be taken off the shelf.	A camera, camera driver (library function), single board computer, separate components for image-capturing rig, and a physical device where data is displayed will be taken off the shelf.	Efficient physical motors, single board computer, stepper motor driver, and separate components for gantry design e.g. smooth ball bearings will be taken off the shelf.

System requirements and specifications page 2

	Requirement 4 (Fourth in order of importance)	**Requirement 5**	**Requirement 6**
1. Core mission requirements of the system or product. Focus on requirements that are core to solving the engineering problem. These will reflect the solution to the problem.	The system must be able to identify the writer of the handwritten text, and decode the content of the handwritten text in a short, practical amount of time.	The gantry of the system must have precise pen movements and accurately draw the desired characters with minimal positional error.	The system should be able to be trained on multiple different writing styles and be capable of storing all the writers' necessary character trajectory maps within the system permanently.
2. What is the target specification (this must be in *measurable* terms) to be met in order to achieve the requirement in 1. above?	The system must identify the writer of the written text from the training set and decode the content of the text at a rate of no slower than 25ms per character with an extra 2 seconds for image capture/conditioning.	The gantry of the system must be capable of drawing the desired character trajectory map while maintaining an absolute positional error that does not exceed 0.5mm.	The system should be capable of training on a minimum of 10 different users' writing styles, and be capable of storing the necessary character mappings for each of these 10 user's permanently.
3. Explain where this specification originated: i.e., explain the value that you selected. I.e., *how* will meeting the specification given in point 2 above solve the problem?	2 seconds was given to image capturing and conditioning due to transmission times and complex conditioning and noise removal algorithms, and 25ms per character classification due to simple forward propagation calculations.	According to research by T. Huang and R. Xiong [4] an absolute positional error of 0.3mm with a gantry is achievable and indicates sufficient accuracy, therefore a maximum absolute error of 0.5mm was chosen for slightly more leeway.	Having 10 trained user styles stored at the same time proves multi-user neural network recognition and multi-user storage. More than 10 is not required due to neural network complexity and storage limitations.
4. How will you demonstrate at the examination that this requirement and specification (points 1 and 2 above) have been met? Be explicit about how you will *prove* these were met.	A page with a sentence of x handwritten characters will be given to the system. The time from the initial instruction to the received classified text and user should be within 2 + x*0.025 seconds.	The system should be instructed to recreate the same word twice without varying the dataset characters, and the two words should appear identical on-top of each other, and not exceed an absolute positional error of 0.5mm.	The system should display the current amount of users in the system, and any random user should be selected out of the 10 and have text reproduced in their style to prove the system can be trained with 10 different users.
5. Your own design contribution: what are the aspects that *you will design and implement yourself* to meet the requirement in point 2? If none, *remove this requirement.*	A rig to capture an image of written text, image conditioning,optimized and efficient character separation, neural network character classifier, and calculation and display of time taken will be designed and implemented by the student.	Accurate conversion from pen trajectory map to precise stepper motor control, robust gantry system to write characters, secure pen adjuster and holder, and a secure paper holder will be designed and implemented by the student.	A rig with a camera to capture new text, sufficiently complex neural network to be able to classify minimum 10 user's styles, and a storage strategy to store 10 user's character trajectory mappings will be designed by the student.
6. What are the aspects to be taken off the shelf to meet this requirement? If none, indicate "none". Clearly specify for what tasks **library functions** will be used (if relevant to the project).	A camera, camera driver, single board computer, separate components for image-capturing rig, and a physical device where time is displayed will be taken off the shelf.	Precise stepper motors, separate components needed for gantry, pen and paper holders, and a pen will be taken off the shelf.	A camera, camera driver, single board computer, components for image-capturing rig, and permanent storage device will be taken off the shelf.

System requirements and specifications page 3

	Requirement 7 (Seventh in the order of importance)	**Requirement 8**	**Requirement 9**
1. Core mission requirements of the system or product. Focus on requirements that are core to solving the engineering problem. These will reflect the solution to the problem.			
2. What is the target specification (this must be in *measurable* terms) to be met in order to achieve the requirement in 1. above?			
3. **Explain where this specification originated**: i.e., explain the value that you selected. I.e., *how* will meeting the specification given in point 2 above solve the problem?			
4. **How will you demonstrate at the examination** that this requirement and specification (points 1 and 2 above) have been met? Be explicit about how you will *prove* these were met.			
5. Your own design contribution: what are the aspects that *you will design and implement yourself* to meet the requirement in point 2? If none, *remove this requirement.*			
6. What are the aspects to be taken off the shelf to meet this requirement? If none, indicate "none". Clearly specify for what tasks l**ibrary functions** will be used (if relevant to the project).			

5. Real world field conditions

These are the ACTUAL, REAL WORLD CONDITIONS under which your project has to work *and will be demonstrated.*

	Real world field condition 1 (required to include)	**Real world field condition 2** (optional to include)	**Real world field condition 3** (optional to include)
Field condition requirement. In which real world conditions does the system have to operate?E.g., "lab conditions" or "outside in uncontrolled conditions". Describe the one, two or three most important field conditions.	The system is expected to perform in well-lit areas like a lab or office environment, at an operating range of 400-1000 lux, to ensure minimal noise in image capturing.	The system is expected to run on a stable, level and vibration free surface, to ensure the page scanning is accurate and the pen in the gantry can write precisely without fluctuations.	

6. Student tasks

6.1 Design and implementation tasks of the student

List your primary design and implementation tasks in bullet list format (5-10 bullets). These are *NOT* product requirements, but *YOUR* tasks.

- The student will research types of neural networks, their structure, the effect of neurons and layers, and how to extract features of text from different layers.
- The student will research image conditioning, noise removal, character separation strategies, and pen trajectory mapping.
- The student will design and implement the convolution neural network, image conditioning, and character separation algorithms.
- The student will simulate the classifier on existing handwritten text datasets with adjusted character separation strategies and neuron/layer numbers to meet requirements.
- The student will construct a rig to capture images of handwritten text on an A4 page, and implement the neural network and algorithms on a chosen single board computer.
- The student will test the new classifier implementations on the single board computer with the captured images to ensure requirements are met.
- The student will design the gantry system in CAD software and construct it with the necessary store-bought individual components.
- The student will design and implement the pen trajectory mapping algorithm and the stepper motor control.
- The student will test the constructed gantry with the implemented classifier to ensure the remainder of the requirements are met.

6.2 New knowledge to be acquired by the student

Describe what the theoretical foundation to the project is, and which *new* knowledge you will acquire (*beyond* that covered in any other undergraduate modules).

- The student will acquire new knowledge of training and classification of character images using a neural network.
- The student will acquire new knowledge involving the extraction of text features from neural network layers.
- The student will acquire new knowledge about the process of image processing with character isolation.
- The student will acquire new knowledge about pen trajectory map creation.
- The student will acquire new knowledge on 2D gantry design and implementation with precise motor control.
- The student will acquire new knowledge about the implementation of neural networks on a single board computer.

7. References

Format your reference list either in IEEE reference format or in APA reference format. Check what your study leader prefers and use consistently.

[1] V. Babushkin, H. Alsuradi, M. O. Al-Khalil, and M. Eid, “Analyzing handwriting legibility through hand kinematics,” Frontiers in Artificial Intelligence, vol. 8, Mar. 2025, doi: https://doi.org/10.3389/frai.2025.1426455.

[2] Sidar Agduk and Emrah Aydemir, “Classification of Handwritten Text Signatures by Person and Gender: A Comparative Study of Transfer Learning Methods,” Acta Informatica Pragensia, vol. 11, no. 3, pp. 324–347, Dec. 2022, doi: https://doi.org/10.18267/j.aip.197.

[3] O. Teikari, "Reproducing Handwriting with Position-Controlled Robots," Master's thesis, Dept. Pervasive Systems, Tampere University of Technology, Tampere, Finland, 2016.

[4] T. Huang and R. Xiong, "Cost-Effective Robotic Handwriting System with AI Integration," in Proc. 2024 IEEE Long Island Systems, Applications and Technology Conference (LISAT), 2024.

Part 3. Main Report

1. Literature study

Text in red is not the author's own wording: it marks the pen-mapping description, the intended contribution, and the individual sentences that were corrected for factual accuracy. All remaining text is the author's own.

1.1 Existing concepts

1.1.1 A need exists to be able to decode the writer's style and written text from an image

The ability of a computer or system to identify written text after being trained on large amounts of data is known as HWR (Handwriting Recognition) or HTR (Handwritten Text Recognition) [1]. One type of handwritten text recognition is offline recognition, which means the data being identified by the system is static and captured post-completion of the page of handwritten text. This is ideal for identifying the text from a fixed image of handwritten content by utilizing image processing techniques and algorithms to identify the text that was written [2]. The other type is online recognition, which is a more dynamic process that identifies text during the creation of the handwriting, adding the variable of time to the classification of text. The addition of a time variable includes data about the order of strokes to create text as well as pen-up or pen-down movements, which can influence the type and ability of reproduction and handwriting synthesis methods.

Every stage of the process to achieve accurate handwritten text recognition requires a deep understanding of the different techniques available:

Datasets and resolution: Multiple datasets have typically been used for training handwritten text recognition modules, for example, the EMNIST, IAM and HEIC datasets [3]. The EMNIST (Extended Modified National Institute of Standards and Technology) dataset consists of approximately 814 255 handwritten characters and numbers, each being 28 by 28 pixels in size [1]. There is only a single, disconnected character per image, which is used in many existing HTR models, but these models cannot effectively decipher connected handwritten words, like cursive words. The IAM handwriting dataset provides much more variety, as it consists of 5 685 complete sentences, with a total of 115 320 words, gathered from 657 different authors, consisting of 26 letters and 27 punctuation marks [4, 2]. The images from the IAM dataset were scanned at 300 dpi with 8 bits per pixel in grayscale. The dataset used influences the accuracy for the classification of text with higher variation, as well as defines the resolution of the camera required to capture the handwritten text to classify. The IAM distribution of cropped lines is not organised by author, so the writer is not evident from the line images alone, while the distribution of full pages is arranged so that each page belongs to a known writer; the database does record the writer of every form in its metadata, so the two can be joined [4].

Image-conditioning: The image-conditioning of handwriting pages includes multiple techniques to be performed on the handwritten text before being passed into the classification and feature recognition algorithms. This is used to account for dimensional adjustments, removal of inconsistencies, and normalization of the data, to make the images more appropriate for text separation and classification [5]. Normalization is used to increase the accuracy of HTR models by reducing

the effect that unwanted features of the data could have on the performance of the model. There are multiple different normalization techniques such as min-max scaling, z-score normalization, and log-scaling. Encoding techniques are also used in HTR model pre-processing to get rid of unintentional biases [1]. Binarization is used to differentiate the background page and obscurities from the actual ink of the written text, computed by pixel. One method of binarization is done from a global ink threshold taken from the grey-level histogram, which does not require any parameters like a localized shadow level radius, and is used for ideal, evenly lit page scans. The other method involves a localized threshold computed per pixel out of the statistics of a window around it, which costs more in terms of computation time and power but can tolerate a lighting gradient or shadow. Deskewing is a technique used to fix the rotation of scanned pages, by rotating the page through certain angle steps until the horizontal projection of the ink is most sharply peaked, which occurs when the lines of text all mostly lie horizontally. Deslanting is used to fix the lean of the individual characters by applying a horizontal shear that shifts each row sideways in proportion to its height, forcing more pixels to be aligned within the same column rather than slanted over multiple columns.

Text Separation: Text separation is the process of breaking up a page of handwritten text into smaller segments or blocks that can be used in feature extraction and neural network classifiers. There are three methods that existing models employ to separate text: line separation, individual character separation or word separation. All three methods involve breaking down paragraphs of handwritten text into smaller segments. The accuracy of the extraction of smaller segments heavily influences the effectiveness of the classifier. Some existing techniques for this are classic dissection techniques, recognition-based segmentation, hysteresis thresholding, and bounding box techniques [1]. Line separation can be done by converting images to grayscale, obtaining inverted binary images, and then drawing bounding boxes over each line [5]. Firstly, when lines are mostly straight and separated by a blank row, line separation can be performed by reading the minimums of the row ink profile (the sum of ink in each row) to form boundaries. When descenders of lines begin to overlap or merge, the straight row boundary separator is replaced with a boundary that bends around the descenders by performing a least-cost path traced between the ink lines, and around the descenders. To achieve either word or character separation, the page must still be broken up into individual lines and then into separate words. Character separation just goes one step further and splits the word into individual characters.

Types of Neural Networks: Many different types of neural network models have been implemented in previous papers, including ANNs (artificial neural networks), CNNs (convolutional neural networks), RNNs (recurrent neural networks), and various combinations of them to achieve the best accuracy.

ANNs are created from the standard neural network architecture consisting of multiple neurons, weights, hidden layers, biases, etc., and can accurately perform pattern recognition for handwritten text. CNNs are built on the architecture of ANNs but also facilitate feature extraction and identifying smaller, more important information in text. This leads to a reduction of necessary learnable parameters and streamlines model training [2]. RNNs contain the same fundamental structure as CNNs but also have the capability to consider sequential and dependent data. The ability to capture time-dependent patterns is useful for deciphering various handwritten fonts, and because RNNs can consider dependencies in the data, this can improve the accuracy of handwritten text recognition when considering common linguistic dependencies in language [2]. A combination of both CNNs and RNNs can significantly enhance feature extraction by using the CNN's pattern recognition and ability to identify diverse handwriting styles, and the RNN's

ability to use textual dependencies for improved contextual understanding [6]. This combination for improved context awareness allows an entire line of text to be read without word or character separation. The convolutional layers are used to convert a cropped line of text into different feature columns, and the bidirectional recurrent layer reads the sequence of features in each direction, providing context of what comes before and after each character or feature. This implementation is used for unconstrained offline handwriting [7], meaning natural handwriting that is not restricted to isolated characters or to a fixed vocabulary, and the full line crop must exist before classification can begin because the recurrent layer reads it from both ends. In an RNN, the gradient is multiplied by the same weight matrix at every time step as it propagates backwards, so over many steps it shrinks and loses its effect (this is the vanishing gradient problem). Forget gates are added to bypass the vanishing effect and allow gradient errors to still retain an effect after many time steps. Putting convolutional layers before the recurrent stack allows the feature extraction and sequence reading to be trained as one network [8]. The columns produced from the convolutional layers and max pooling tend to outnumber the number of characters to be classified, therefore the CTC (connectionist temporal classification) loss [9] is used, which adds a blank symbol and sums the probability of every column-to-character alignment that collapses to the transcription, the collapse being to squash repeated symbols first and then remove the blanks. The arrangement is shown in Figure 1.

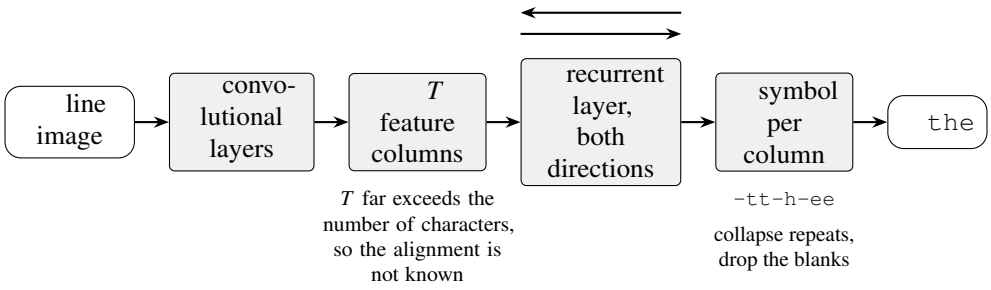


Figure 1. Reading a whole text line without cutting out characters, after the arrangement of Kizilirmak and Yanikoglu [10]. The convolutional layers produce T feature columns, a recurrent layer reads them in both directions, and a symbol including a blank is emitted per column; collapsing repeats and removing the blanks gives the transcription.

Existing Models: Two baseline models have been primarily considered in existing research papers: the LeNet-5 CNN model and the ResNet-34 CNN model. LeNet-5 is the most common of the two and was created by Yann LeCun in 1998 [11]. It is a CNN structure that consists of seven layers, including 3 convolutional layers, 2 pooling layers, and lastly, 2 fully connected layers. A fully connected layer refers to a portion of a typical ANN model, while the convolutional layers and pooling layers will be discussed later in this literature study. The input size to the model is 32 by 32 pixels, and the model uses a hyperbolic tangent activation function in the fully connected layers and a softmax activation function in the output layer. This model is popular for use in object detection and image classification [1]. ResNet-34 is also a popular CNN model known for its residual architecture. The residual architecture adds skip connections that let the gradient bypass layers, which is what allows the network to be trained at that depth; separately, its weights from training on a large image set are commonly reused by transfer learning, which forms the foundation of the feature extraction in this model and allows for fast and efficient training from limited datasets [2].

Feature extraction: The convolutional layers of a CNN are used to perform feature extraction, which means they can capture subtle patterns in different areas of a segment of text by sliding

a convolutional kernel over the character or word image. Translation invariance is created by sliding the convolution kernel, which allows consistent patterns to be extracted from images [2]. An N by N input layer is convolved with an f by f kernel and returns an $(N-f+1)$ by $(N-f+1)$ output. Feature maps are created by applying convolutions to different sections of the image being classified. By identifying common features, the kernel-sliding convolutional technique can produce more accurate classification. Each extracted feature is multiplied with a corresponding kernel value to produce a weighting, and then a bias and activation function are applied until an output feature map is obtained [12].

An extended version of the standard CNN is the deep CNN, which refers to a CNN that can extract higher-complexity features like handwritten strokes. The problem is that a larger, deeper CNN creates a 'vanishing gradient' problem where there are so many layers that during back-propagation, some layers never get their weightings changed. Thus, there is a limit to the number of layers a standard CNN can have, where adding more layers yields no further benefit. This is why a residual network is used to implement a deep CNN by using residual blocks and skip connections across various layers so that all layers will still update their weightings during backpropagation regardless of the number of layers [2].

After a singular round of feature selection, many parameters are still required to classify text with these high-level features. Max-pooling is used to downsample these feature maps until only the essential features remain, minimizing the number of parameters needed to classify handwritten characters and preventing overfitting [13]. Figure 2 shows one convolutional layer followed by one max-pooling layer, which is the unit the models are stacked from.

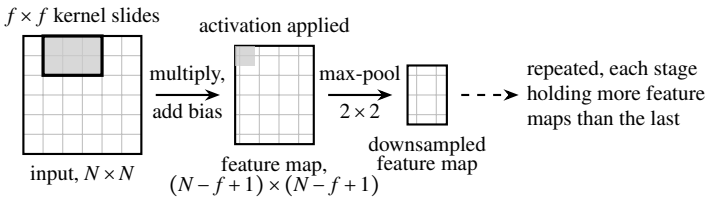


Figure 2. One convolutional layer and one max-pooling layer. The kernel slides over every position of the input, so an $N \times N$ input gives an $(N-f+1) \times (N-f+1)$ feature map, which pooling then downsamples.

Training: The way networks are trained corresponds to the architecture they are built from. The weights are initially randomized with a variance determined by the size of each layer so the signal remains constant as it passes through more layers. Batch normalization is the process of taking the outputs of the current batch of training data and subtracting the average and dividing by their standard deviation. Having more sensibly ranged values allows a higher learning rate to be used safely. Dropout is a technique used to reduce overfitting which removes a random subset of a layer's outputs on each training step. This introduces variation and prevents the network from depending on any single feature. Another method to increase variation is dataset enlargement which includes applying shears, small rotations, slants and random smooth warps etc. to simulate more variation in training text. Learning rate reduction is a technique that is used to adjust the learning rate over time to allow for large changes initially to speed up training, and fine learning later on to prevent overshooting. Final accuracy can be calculated as the character error rate (which refers to the number of mismatches in characters compared to the ideal result) or the word error rate, which is calculated from word mismatches. This is usually significantly higher than the character error rate as one character mismatch ruins the whole word [1].

Techniques for writer identification: There are multiple approaches to identifying the author of a written piece of text. The first way is through feature extraction, which measures text-independent features like slant, height, stroke width, word and character spacing, and connectivity between characters. The author is determined by finding which writer possesses the feature metrics closest to that of the text measured. Another feature that can be extracted is the distribution of edge directions in the ink, which gives the 'texture' of the written sentence [14]. The other method of writer identification is using a neural network trained off the author's handwriting, which can use a feature extractor used for text recognition [2] instead of the hardcoded methods and metrics for quantifying features. Much more content and pages are required to train the author classifiers in this case compared to creating specific feature extracting methods.

1.1.2 A need exists to reproduce text on paper accurately and fast in any writer's style

There are various existing text reproduction methods and linear actuation techniques which can influence the accuracy and speed of text reproduction.

Handwriting synthesis in a writer's style: Before being able to reproduce handwriting through hardware, the desired text to be reproduced has to still be created in the particular author's style. Four methods will be discussed on how to achieve this:

Recurrent stroke level synthesis: A recurrent neural network is used to predict not the characters to be written, but rather the pen movement that must be taken next, with small offset predictions at a time [15]. It creates a spread of possible answers as no letter is written identically. The network is trained off an online recorded sample of someone's handwriting. Since it predicts pen movements the outputs are already presented as movements which a plotter requires. Training a network to produce accurate outputs requires hundreds of lines of online data.

Adversarial image synthesis: This method creates an image of text using a GAN (generative adversarial network), which trains a text generator and discriminator to identify fake generated text from real handwritten text [16]. Both the generator and discriminator improve per round until the generator can produce well-reproduced text and the discriminator can no longer tell the difference. It is called 'adversarial' as two networks are trained simultaneously, one to generate text and the other to judge the text. This requires a significant amount of time and compute to train, but the final system produces high quality recognisable text in a single pass once trained. A text and writer classifier are still required to verify the generated text along the way as well. This method produces a raster image which must be converted to pen paths for a plotter to take.

Diffusion-based synthesis: This method generates a fake text raster image by reversing a gradual noise-generating process [17]. The system trains by slowly adding noise to handwritten text, remembering the way noise was added, until the final image is pure random noise. Then to generate a new line of text, the system starts with complete random noise and denoises it until an accurate line of text is reproduced. Only one reference image of the target writer is needed at generation time, although training the model still requires a large set of labelled handwriting images, yet it requires many steps and time to denoise the random noise image. This method also produces a raster image which must still be converted to pen stroke mappings.

Glyph-library based synthesis: This method does not use any generators to create text, but rather assembles words from shapes or glyphs taken out of existing handwritten pages. Haines *et al.* [18] describe a technique that stores a library of labelled glyphs per each user, and then creates

the specified sentence by choosing the specific glyphs and then assembling them according to which variant of the glyph the writer usually used in the requested position, and the level of connectedness of the text, the texture of the line of ink, and the spacing of the writing. It requires a small number of handwritten pages per author to create decent glyph libraries, and the main complexities of the method only involve connecting one glyph to another and selecting the most appropriate glyphs.

Assessing generated handwritten text: Accuracy of generated handwriting can be measured through performing image-level distances and correlation to real handwritten text, for example an IoU test (Intersection over Union) which measures the overlap between a predicted region and the true region. Another method is by passing it through an independent text and author classifier and reporting the accuracy of the reproduced text. A survey can also be conducted for human validation of what text appears real and what appears reproduced.

Method of reproduction: There are three main methods to reproduce handwritten text on paper: Cartesian systems, serial-chain robots, and parallel-chain robots. A Cartesian system consists of a gantry with x – y movement. It has restricted movement in only two directions using linear actuation. Serial-chain robots are the most common in industrial settings for automated movement. They consist of multiple linkages chained together, one component connected to the next in a single chain, like a typical robot arm. The third method is parallel-chain robots, which consist of multiple serial-chain linkages that can precisely control automated movement [19].

Pen mapping techniques: A pen path is stored as a list of points, and a drawn stroke usually has far more points than a plotter needs. Pen mapping is the step that reduces this list while keeping the shape of the stroke. The Douglas-Peucker algorithm is the common choice [19]: it draws a straight line between the two ends of a stroke, finds the point that sits furthest from that line, and keeps it only if it is further away than a chosen tolerance, then repeats on each half [20]. A small tolerance keeps many points and follows the stroke closely, while a large tolerance keeps few points and cuts corners, so the tolerance sets the trade-off between the file size and the accuracy of the drawn shape. The alternatives are fitting a smooth curve such as a spline through the points, which needs a fit to be solved, and resampling the stroke at a fixed spacing, which spaces the points evenly but ignores where the curves are. When the handwriting only exists as an image, the path has to be recovered first by thinning the ink down to a one-pixel centre line and then joining those pixels into strokes, which means guessing the drawing order wherever two strokes touch or cross.

Linear actuation types: There are multiple methods of linear actuation that could be considered for a Cartesian plotting system. Some popular examples of linear actuation methods are electric, pneumatic, belt-drive, rack-and-pinion, and ball-screw systems. Electric and pneumatic methods are often used for industrial cases, but belt-drive, rack-and-pinion, and ball-screw linear actuators have been used in multiple handwriting reproduction systems due to their durability, reliability, and cost-effectiveness [21].

1.2 Intended contribution and discussion

The literature above covers each stage of the problem on its own, but it does not cover them as one machine. Recognition is reported on evenly lit scans or on isolated characters, writer identification is reported over sets of hundreds of writers rather than a small closed group, and handwriting synthesis is judged as an image rather than as something that has to be drawn with

one pen. Almost all of this work is also built on libraries such as TensorFlow and OpenCV [1], which hide what each stage actually costs in operations and memory. That cost matters here, because every stage of this project must run on a single-board computer.

The main contribution of this project is therefore the full integration of the system, from a photographed page to ink on paper, with every stage implemented from first principles and no machine-learning or image-processing libraries used anywhere in the pipeline. This covers the creation of the dataset, the conditioning and line separation of photographed pages, a line-level text recogniser, the identification of the author, the synthesis of new text in that author's style, and the generation of the pen paths and G-code that drive the gantry. Writing each stage by hand means the forward and backward passes of every layer, the loss functions and the optimisers are known quantities whose cost can be measured and reduced to fit the available hardware.

Two things set the approach apart from the work reviewed. The first is that the system classifies both the text that was written and the author who wrote it, using the same feature extractor for both, so that one page yields a transcription and an identity. The second is that the synthesised handwriting is produced as geometry rather than as an image, so it can be drawn directly without having to recover a pen path from pixels, and the recogniser and the author classifier are then reused to judge the synthesised output before it is committed to paper. The reasoning behind each of these choices, and the alternatives that were rejected, is given in Section 3.

2. Approach

This section explains how the proposal's functional analysis became a first concept, why the subsystems were then built in the order they were, and which alternatives were weighed. Mathematics, component values and experiments belong to Section 3.

2.1 From functional analysis to concept

The functional analysis describes a chain rather than a collection of parts (Figure 3): each block consumes the output of the one before it, so the accuracy of the whole is the product of the accuracies of its parts rather than their average.

The first concept kept the proposal's four functional units, namely page scanning (FU 1), the single-board computer (SBC) and its eight processing steps (FU 2), the gantry (FU 3) and the power supply (FU 4), and made one decision per block; Table 4 in Section 3.2 maps each unit onto the block that implements it. One decision applies to all of them: no computer-vision or machine-learning library was used, so every block was implemented from first principles with only numpy for array arithmetic. That is why Section 3 derives methods a library call would hide, and it narrowed the field to those that could be written and verified in the time available.

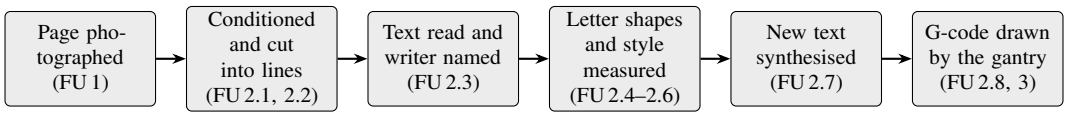


Figure 3. The chosen concept as a chain, with the proposal's functional-unit numbers. Training builds the style store from the first four blocks and runs offline, classification stops after the third, and reproduction runs the last two. The as-built hardware and software block diagrams are Figures 4 and 5.

Seven departures from that concept are declared with their reasons in Table 4; the three that shaped it are the separation of text lines rather than characters, since joined letters have no reliable boundaries, the split of FU 2.3 into two networks instead of one character classifier, and training off the SBC, which needs far more memory than inference, with all inference left on the SBC as required. **[MUST STILL BE COMPLETED BY ME: state on which computer the networks were trained and the profiles built; the proposal requires embedded-only processing.] [MUST STILL BE COMPLETED BY ME: reason for the pen-lift change.] [MUST STILL BE COMPLETED BY ME: supply voltages, current ratings and distribution for FU 4.]**

2.2 Order in which the subsystems were built

Figure 3 is a data path, and the subsystems were deliberately not built along it. The design divides into a motion subsystem and a software subsystem meeting at one interface, a file of G-code; fixing it at the outset decoupled them, since the gantry draws any valid G-code whatever produced it. The gantry was therefore built first and on its own (Section 3.3): the 0.5 mm limit of R5 is set by its mechanics, transmission and step resolution alone, so nothing in it depends on which recogniser or synthesis route was later chosen, and it carries the only schedule risk outside

the work itself, since parts take time to arrive and may arrive faulty. Hand-written G-code was enough to commission and measure it before any software existed.

The software allowed no such freedom, because an error in one block propagates and compounds and is hardest to see at its origin: a slightly mis-cut text line is still a plausible image while it silently bounds the accuracy of everything downstream. Each block was therefore brought to a measured standard before the next was begun, in pipeline order — conditioning and line segmentation (Section 3.4), without whose clean labelled line images no network can be trained; the recogniser and then the writer classifier (Sections 3.5 and 3.6), in that order because the second reuses the first's feature extractor; glyph and style extraction (Section 3.7), which needs the recogniser to locate the letter shapes; synthesis (Section 3.8); and G-code generation (Section 3.9). The page-scanning rig was fixed during the first of these, once freehand photographs rather than the algorithm proved to be what limited the result, since a camera held at one height looking straight down removes scale, tilt and framing variation at source; a flatbed scanner would have lit it more evenly but is larger and slower.

2.3 Design alternatives and trade-offs

Each decision was weighed against the same criteria: accuracy (R1, R2), processing time on the SBC (R3, R4), the data available for thirteen writers (R6), implementation from first principles, and mechanical and electrical simplicity and safety. Table 1 gives the alternatives and the reason for each preference.

Decision	Alternatives considered and how they compare	Preferred and reason
Hardware		
Robot type	(a) Cartesian gantry: identity kinematics, axis errors do not add up. (b) Serial arm: inverse kinematics, joint errors add up. (c) Parallel robot: stiff, but a small workspace and complex kinematics.	(a): position follows from step counts, so the 0.5 mm limit of R5 is simplest to meet and verify [19].
Interface to the gantry	(a) A custom protocol fits the synthesiser exactly, but only the synthesiser can then exercise the machine. (b) G-code is a standard shared with commercial equipment, readable and writable by hand.	(b): it let the gantry be commissioned and measured before any software block existed, and keeps the two subsystems independently testable.
Software		
Unit of recognition	(a) Cut characters and classify each: needs character-level labels, which are absent, and cannot repair a wrong cut. (b) Cut lines and read with a sequence recogniser: no boundaries below the line, context used, larger network.	(b): cursive cannot be segmented reliably, labels exist per line, and segmenter errors would bound system accuracy.
Synthesis of writing	(a) A neural generator needs recorded pen trajectories, which are unavailable. (b) Pretrained image generators give raster output, no control of size or slant, and are slow without a graphics processor. (c) A library of the writer's own letter shapes needs only nine pages and gives controllable strokes, but loses some cursive links.	(c): a from-scratch generator learned slant and roundness but not legible letters, and image generators took tens of seconds per line on a processor-only computer.

Table 1. Design decisions, alternatives considered and the reason for each preferred solution.

Each block was held to a measurable criterion from the requirements — characters read correctly, writers named, lines cut correctly, pen position in millimetres — and measured alone before the chain was measured end to end; where no known truth exists, as for the drawn page, a round trip was used. That measurement later replaced four designs, collected in Section 3.10. Only the camera, SBC, motors, drivers, switches, pen, standard gantry parts and power supply were bought. **[MUST STILL BE COMPLETED BY ME: list the mechanical parts I designed and made.]** Section 3 takes each block in build order, so each is met with its input explained.

3. Design and implementation

3.1 Design summary

This section summarises the project design tasks and how they were implemented (see Tables 2 and 3). The rows are the deliverables of line 5 of the table in Section 4 of the approved project proposal and the nine tasks of its Section 6.1. The summary is split over two tables only so that neither runs across a page break; the headings are the same in both. A task is marked *Completed* only when the deliverable itself works. Where any part of it is missing, unbuilt or unmeasured it is marked *Incomplete*, and the second column says what is missing. Only work designed and implemented by the student is claimed. The parts that were bought as finished items, namely the camera, the single-board computer, the motors and their driver modules, the pen, the switches, the power supply and the standard mechanical parts, carry no design credit. Every image-processing, machine-learning, trajectory and motion-control algorithm in the running system was written by the student in Python on top of NumPy array arithmetic. No machine-learning framework supplies any layer, gradient or optimiser. The exceptions are all on the offline training path or at the interfaces, and are listed in declaration 6 of the preamble. SciPy is used in exactly two files, for peak finding when the public database pages are cut into lines and for the Gaussian filtering and coordinate mapping of the elastic augmentation; the remaining libraries interface with the camera, the general-purpose pins, the image files, the database download and the network.

Two deviations from the proposal shape several rows. First, the proposal planned to separate the page into characters (functional unit 2.2) before classification. The implemented design separates the page into text lines only and reads each line with a sequence recogniser, because the characters of joined handwriting have no reliable boundaries and a separation error would pass straight into a wrong reading (Sections 3.4 and 3.5). Second, the proposal placed training and the building of trajectory maps on the single-board computer (functional units 2.4 to 2.6). In the implemented system the networks and the per-writer style profiles are produced by offline programs on a general-purpose computer, and only inference, synthesis and motion control run on the board. The interface therefore offers no training mode. **[MUST STILL BE COMPLETED BY ME: state the computer used for network training (the project notes mention a laptop processor and a cloud notebook) and confirm that the single-board computer was not used for training.]**

3.2 System block diagram

Two diagrams describe the implemented design: Figure 4 gives the hardware blocks and how they connect to the single-board computer, and Figure 5 gives the software data flow of the three modes of operation, which are reproduction, classification and offline training. Both follow the functional block diagram of the approved proposal, which divided the system into the page-scanning set-up (FU 1), the single-board computer (FU 2), the gantry (FU 3) and the power supply (FU 4). The four physical units were kept, but the software units FU 2.1 to FU 2.8 were reorganised; Table 4 relates the implemented blocks to the proposed units and gives the reason for every departure.

Deliverable or task	Implementation	Completion of deliverable or task, and section in the report
Research of network types, structure, layers and feature extraction (task 6.1.1)	A literature study of convolutional, recurrent and sequence-to-sequence recognisers, of writer-identification methods and of the alternatives to each, used to choose the methods of Sections 3.5 and 3.6.	Completed. Section 1.
Research of conditioning, noise removal, separation and trajectory mapping (task 6.1.2)	A literature study of thresholding, illumination correction, line and character separation, thinning and exemplar-based handwriting synthesis, used to choose the methods of Sections 3.4 and 3.7.	Completed. Section 1.
Image conditioning and noise removal (FU 2.1; R2, R4)	Implemented from first principles using only array arithmetic: size normalisation, page localisation, flat-field illumination correction with summed-area tables, local adaptive thresholding, skew correction and removal of printed rules, faint print and neighbouring-page ink. The individual operations are standard and are cited in Section 3.4; the stages chosen, their order and every threshold were settled for this project.	Completed. Section 3.4.
Separation into characters (FU 2.2; R2, R4)	Implemented as separation into text lines and diagram regions rather than into characters (connected components, glyph-scale estimation, chaining, baseline merging, text versus non-text scoring, one deskewed crop per line). Character territories are derived only offline, by forced alignment, when style profiles are built. The reason for the change from the proposal is given above and in Section 3.5.	Completed. Sections 3.4, 3.5.
Optimised and efficient character separation (R4; task 6.1.3)	The segmentation was profiled and its two dominant routines, component labelling and the box sums, were rewritten in C by the student at 84 to 88% faster each, 25 to 32% faster per page. The C routines are not called by the deployed program, and segmentation still exceeds the R4 allowance by over an order of magnitude.	Incomplete. Sections 3.4, 4.
Neural network that decodes the text (FU 2.3; R2)	Convolutional and bidirectional recurrent network with connectionist temporal classification (CTC) output, 3 358 763 trainable parameters. The layer pattern follows a published design [10]; the student chose the input geometry, data pipeline, label cleaning, augmentation and staged training, and wrote the from-scratch module (forward and backward passes, optimisers, no machine-learning library) used by training, validation and deployment alike.	Completed. Section 3.5; measured accuracy 98.61% character and 94.21% word on 669 real line crops (Section 4).
Neural network that identifies the writer (FU 2.3; R2)	Two thirteen-class convolutional classifiers sharing the recogniser's backbone and layer module: an ink-based one for photographs, which reached 99.78% over 1358 real lines, and one whose input is re-drawn at a constant pen width by the student's stroke normalisation, with page-level aggregation.	Completed. Section 3.6; measured accuracy 93.67% for the normalised classifier after a training fault was corrected (Section 4).
Training on datasets and choice of layer numbers (FU 2.4; task 6.1.4)	Training programs for the recogniser and both classifiers on a public database, a public line-level set and the photographed personal pages, all built on the from-scratch module of Section 3.5.7, whose every layer gradient was checked against numerical differentiation in 78 configurations. A sweep over widths, depths and learning rates was programmed but no result was kept, so the layer numbers are the published design's and are not justified by this project's own measurement. Training is offline.	Incomplete. Sections 3.5, 3.6.
Trajectory maps and permanent storage (FU 2.5 to 2.7; R1, R6)	Letter shapes are harvested by forced alignment with the recogniser, thinned to centre-line pen paths, normalised and stored as one JavaScript Object Notation (JSON) file per writer with measured style statistics; thirteen styles exist, for eight database writers and five personal writers. The builder is an offline program.	Completed. Section 3.7.
Pen trajectory mapping and G-code (FU 2.5 to 2.8; R1, R3)	A synthesiser assembles the writer's own letter paths into lines (slant, spacing, joins, wobble) with a best-of-N search judged by the recogniser and the writer classifier; a generator turns the trajectory into G-code, the numerical-control command language of plotters, with placement, stroke ordering and pen-state tracking, and a simulator replays it.	Completed. Sections 3.8, 3.9.
Optimisation of pen movement (R3)	Greedy nearest-endpoint stroke ordering reduces pen-up travel. The saving it achieves was not measured.	Completed. Section 3.9.

Table 2. Design summary, part 1 of 2: the research, image-processing, network and trajectory tasks.

Deliverable or task	Implementation	Completion of deliverable or task, and section in the report
Capture rig and deployment on the single-board computer (FU 1; R2, R4; task 6.1.5)	A generic USB camera in a printed housing that clamps the page flat, read with a live preview and a five-frame warm-up; the recognition, synthesis and motion programs run on the ODROID N2+ behind a web server. The rig geometry, camera resolution and lighting are not documented. **[MUST STILL BE COMPLETED BY ME: camera model, resolution, mounting height, field of view, lamp, lux readings, photograph of the rig.]**	Incomplete. Sections 3.2, 3.11.
Testing the classifier on the single-board computer with captured images (task 6.1.6)	The deployed programs were written to run on the board and do run on it, but no accuracy or timing figure in this report was obtained on the board; every measurement reported in Section 4 was taken on a laptop.	Incomplete. Section 4.
Gantry design and construction in CAD (FU 3.3, 3.4; R5; task 6.1.7)	A Cartesian X/Y gantry built by the student from bought parts: one motor turning an 8 mm cross-shaft that drives both ends of the moving bridge, and a second motor on that bridge driving the carriage, both on GT2 belts over linear bearings and chromed rods, with four end-stops. Hand-measured travel 204 mm (X) and 262 mm (Y). The machine exists but no CAD drawing or photograph was kept. **[MUST STILL BE COMPLETED BY ME: CAD files and photographs.]**	Incomplete. Section 3.3.
Pen holder, pen adjuster and paper holder (R5)	The pen is carried on the moving carriage in a holder with the toggle cam of the lift mechanism above it, and the page is clamped flat in the camera housing for capture. Whether the holder provides a pen-height adjustment, and how the paper is held on the writing bed, are not recorded. **[MUST STILL BE COMPLETED BY ME: confirm how the pen height is adjusted and how the paper is held down on the writing bed, and supply a photograph of each; neither is documented in the project files.]**	Incomplete. Section 3.3.
Power supply (FU 4)	A 24 V supply feeds both stepper drivers through a 100 µF bulk capacitor. The conversion to the voltages the single-board computer and the pen motor require is not documented, no power budget was drawn up and the supply's rating is unknown.	Incomplete. Section 3.3.2.
Stepper-motor control and pen lift (FU 2.8, 3.1 to 3.4; R5; task 6.1.8)	A G-code interpreter generates step and direction pulses directly on the general-purpose pins, with Bresenham interpolation, absolute step bookkeeping, calibration between end-stops, debounced end-stop monitoring and software limits. The pen is lifted by a toggle gear motor confirmed by a position switch, switched by a transistor stage designed by the student, rather than through the stepper driver as the proposal planned. During bring-up the X-axis direction pin was not observed to change level on reversal and that fault was not resolved.	Incomplete. Section 3.3.
Smooth motor control for efficient gantry movement (R3)	The interpreter steps at a constant interval within a segment and executes no acceleration ramp, although the generator's time model assumes one. Abrupt starts at the feeds used may lose steps, and no torque margin was computed because the carriage masses were not measured.	Incomplete. Sections 3.3.3, 3.3.4.
Display and user interface (R4, R6)	A web server exposes classification, generation, calibration and sending of jobs; browser pages show the text, the writer with a confidence, the stored writers and a G-code preview. The synthesis time is returned, but the classification time that R4 asks to be displayed is not shown.	Incomplete. Section 3.11.
Full-system testing against the requirements (task 6.1.9)	The accuracy, writer-identification and timing evaluations of Section 4 were designed and run by the student. The physical writing time of R3, the on-board timing of R4, the positional error of R5 and the random-writer reproduction of R6 were not measured, because the gantry bring-up was not completed.	Incomplete. Section 4.

Table 3. Design summary, part 2 of 2: the capture rig, the gantry hardware, the motion control and the verification tasks.

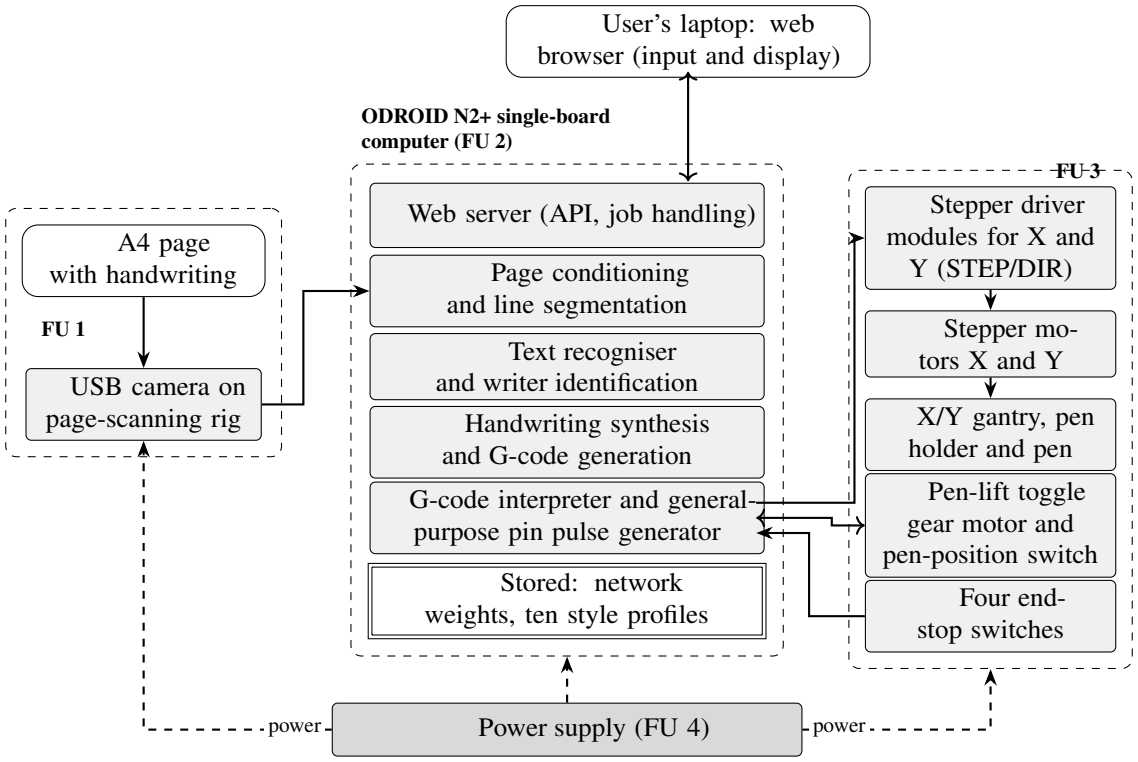


Figure 4. Hardware block diagram of the implemented system. Solid arrows show signals and data, dashed arrows show the power supply. FU numbers refer to the functional units of the project proposal.

3.2.1 Hardware block diagram

The hardware of Figure 4 is four groups. The page-scanning rig (FU 1) holds a generic USB camera above an A4 page, read through the operating system's own driver, so no camera driver was designed by the student. The single-board computer (FU 2) is an ODROID N2+ and runs every software block, including the web server through which a laptop controls the robot. The gantry (FU 3) moves each axis with one stepper motor and one driver module, whose STEP and DIR inputs go straight to general-purpose pins, so there is no separate microcontroller; four end-stop switches bound the travel, and a toggle gear motor with a position switch raises and lowers the pen. The power supply (FU 4) feeds all three. The circuit itself, with its component values and design calculations, is given in Section 3.3. **[MUST STILL BE COMPLETED BY ME: model numbers of the camera, stepper motors and driver modules, the supply voltages and current ratings of the power supply, the way in which the pen motor is switched, and the logic-level and grounding arrangement between the single-board computer and the drivers; these are not documented in the project files.]**

3.2.2 Software block diagram and systems-level description

The system learns how a person writes from photographs of that person's pages and later writes new text in the same style. Figure 5 shows the three chains of blocks that do this and the

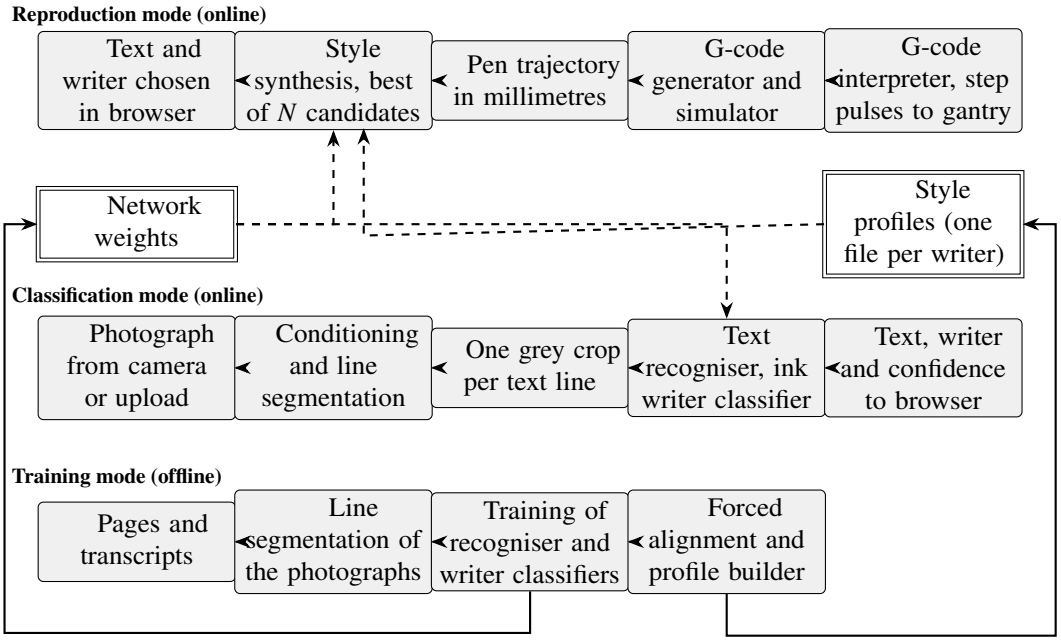


Figure 5. Software data flow of the three modes. Dashed arrows are reads from permanent storage, solid arrows between the training row and the two stores are writes. The same line-segmentation and recognition blocks serve classification, training and the judging of synthesised handwriting.

two permanent stores they share, the network weights and the per-writer style profiles. In the classification mode (R2, R4) the page is conditioned and separated into text lines (Section 3.4), each line is read by the text recogniser (Section 3.5) and classified by the ink-based writer classifier (Section 3.6), and the writer probabilities of the lines are averaged to a page decision. In the reproduction mode (R1, R3, R5, R6) the synthesiser (Section 3.8) assembles the writer's own stored letter paths (Section 3.7) into a line, repeats the random construction for N candidates and keeps the one that the recogniser and the stroke-normalised writer classifier jointly prefer; the trajectory is converted to G-code (Section 3.9) and executed by the interpreter after the gantry has been calibrated between its end-stops (Section 3.3). The training mode runs offline: the same segmentation block prepares the pages, the networks are trained and the profile builder writes the two stores, after which no image and no training code are needed to use a style.

3.3 Hardware design

This subsection gives the components, the complete circuit, the calculations that fix every component value, and the program that drives it (FU 2.8, FU 3.1 to FU 3.4; R5). The computer emits both drivers' STEP and DIR pulses itself, with no microcontroller between, so the end-stop read lies inside the loop that emits every step. All parts were bought; what is claimed is the selection, the three interface circuits, the calculations below and the 800-line G-code interpreter [22].

Proposal unit	Implemented block	Difference and reason
FU 1.1, 1.2	USB camera over an A4 page, read by the operating-system driver.	As planned; the “serial bitstream converter” is the standard camera interface.
FU 2.1	Page conditioning (Section 3.4).	Much larger than planned: lighting, skew, ruled paper, neighbouring pages and diagrams are handled.
FU 2.2	Line segmentation (Section 3.4).	Lines instead of characters, because cursive characters have no reliable boundaries.
FU 2.3	Text recogniser and writer classifier (Sections 3.5 and 3.6).	Two separate networks instead of one character classifier.
FU 2.4	Offline training programs.	Not on the single-board computer, because training needs far more memory and time than inference.
FU 2.5, 2.6	Style-profile builder and permanent profile files (Section 3.7).	Offline; stores letter paths per writer instead of one trajectory per recognised character.
FU 2.7	Synthesiser (Section 3.8).	Constructs new text from stored letter variants and adds a best-of-N search.
FU 2.8	G-code generator and interpreter with pulse generation (Sections 3.9 and 3.3).	Split into two stages with a standard interchange format, which allows the trajectory to be checked without the gantry.
FU 3.1	Stepper driver modules on 3.3 V pins.	No separate voltage-shifter block is documented. **[MUST STILL BE COMPLETED BY ME: confirm.]**
FU 3.2	Two stepper motors on one gantry.	As planned.
FU 3.3, 3.4	Pen holder and toggle pen-lift motor with position switch.	Pen lift is not driven through the stepper driver, as the proposal assumed, but by a one-direction gear motor with feedback switch.
FU 4	Power supply.	Not documented in the project files.
(new)	End-stop switches and web server.	Added for protection of the mechanism and to replace the proposed display and input interface.

Table 4. Relation of the implemented blocks to the functional units of the project proposal.

3.3.1 Components and pin assignment

Table 5 gives the components and the reason for each; the twelve interface lines and their initial levels are listed as Table 34 in Appendix 6.4.

MUST STILL BE COMPLETED BY ME

Add to Table 5 the part numbers and supplier of the motors, driver carriers, supply and camera, and state how 24 V is converted to the voltages the ODROID N2+ and pen motor need: no converter is documented.

3.3.2 Complete circuit and its operation

Figure 6 is the complete circuit, with every component value on it and an undocumented stage making the 5 V and 3.3 V the board and pen motor need.

The chopper draws current in bursts the leads cannot follow, so a 100 µF capacitor sits at the drivers; by Equation 96 it holds the rail sag over one 1.68 A burst to 0.17 V, small against 24 V and clear of the undervoltage threshold [25]. Three gaps remain in the supply: one capacitor is shared where one belongs at each driver, no ceramic logic-supply capacitor is documented, and the supply current was never budgeted, as Appendix 6.4 records.

Component	Quantity, rating	Reason for the choice, and content to be supplied
ODROID N2+ board	1	The proposal requires all processing on an embedded platform; one board runs everything off its 3.3 V pins.
NEMA 17 stepper	2, 1.68 A/phase [23]	Open-loop position from a step count [24], so R5 needs no encoder. **[MUST STILL BE COMPLETED BY ME: model number, datasheet revision, holding torque and torque–speed curve of the motors fitted.]**
A4988 driver carrier [25]	2	One STEP and one DIR line per axis is the smallest pin count; the chopper runs the 1.68 A winding from 24 V, and 1/16 microstepping [24, 25] is set by jumpers. **[MUST STILL BE COMPLETED BY ME: part number, datasheet revision and sense-resistor value of the carriers fitted.] [MUST STILL BE COMPLETED BY ME: the microstepping jumpers, the minimum STEP pulse width, and whether any level shifter is fitted.]**
Power supply	1, 24 V	A higher rail forces current into the winding inductance faster, allowing the step rates of Section 3.3.4; inside the A4988's 8 to 35 V range [25]. **[MUST STILL BE COMPLETED BY ME: voltage and current ratings of the motor, computer and pen supplies, the grounding scheme and the camera supply.]**
Bulk capacitor	1, 100 µF	Supplies the chopping current locally (Equation 96).
Micro switch	5	Four end-stops for Section 3.3.4 and the emergency stop, one for the pen cam; needs no supply and ignores ambient light. **[MUST STILL BE COMPLETED BY ME: switch type and mounting.]**
Pull-down resistor	4, 10 kΩ	Holds each switch input low when open (Equation 98).
N20 gear motor	1, 150 rpm	Turns the cam that toggles the pen one way; its speed sets the toggle time (Section 3.3.5). **[MUST STILL BE COMPLETED BY ME: the voltage it is run from, the pen model and holder, and the lift height.]**
BC337-40 and 1 kΩ base resistor	1	A pin cannot supply the pen motor's current (Equations 99 and 100).
GT2 belt, 2 mm pitch, 20-tooth pulleys, idlers	2 axes	Chosen over ball-screw, rack-and-pinion, pneumatic and direct electric drive on durability, reliability and cost [21]: this long, light travel needs speed over force, and idler tensioning limits backlash. 2 mm on 20 teeth gives the 40 mm per revolution of Equation 1. Measured travel 204 mm (X), 262 mm (Y). **[MUST STILL BE COMPLETED BY ME: the calibrated steps per millimetre printed for both axes.]**
Rods, bearings, 5 to 8 mm coupler	as required	Carry the carriage so the belt only pulls it; the coupler joins motor shaft to cross-shaft (Section 3.3.3).
USB camera, printed housing	1	Fixes the camera at a known height and clamps the page flat, which is what makes the illumination correction of Section 3.4 work. **[MUST STILL BE COMPLETED BY ME: model, resolution, mounting height, field of view and lamp.]**

Table 5. Components, with the reason for each choice. Every part was bought; the interface circuits, the calculations and the control program are the student's own.

STEP, DIR and enable are driven from 3.3 V pins with no level shifting, sound only if $3.3\,\text{V} > V_{IH}$ with margin [25]. During commissioning the X axis drove to its first end-stop but the direction pin was not observed to change level on reversal, and the fault was not resolved (Section 4); a level incompatibility, a wiring or configuration fault, a damaged pin and a program fault are equally consistent with it, so no cause is claimed.

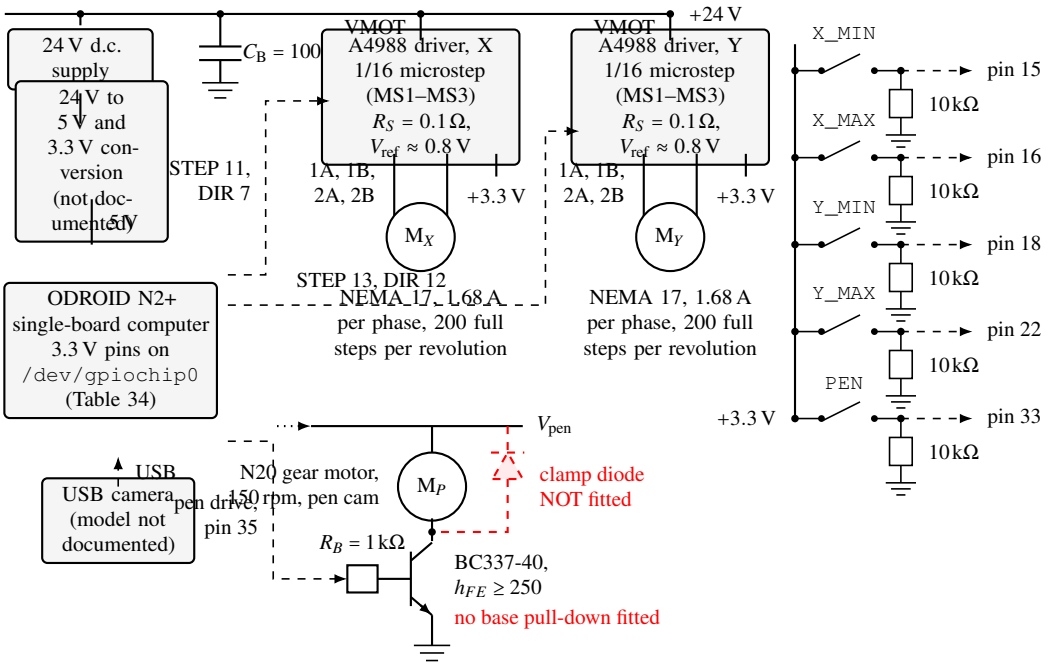
MUST STILL BE COMPLETED BY ME

Resolve the X-axis direction fault, which blocks R5 entirely. Compare V_{IH} with 3.3 V, then meter DIR at the pin and at the driver while commanding a reversal: that measurement separates a program from a wiring from a level fault.

3.3.3 Mechanical arrangement and the torque calculation

The machine is a Cartesian gantry of two belt axes, so a page target needs no inverse kinematics and an error in one axis does not rotate the other [19]. The lower motor turns one cross-shaft through the coupler, a pulley at each end, so the bridge stays square without a second motor; the upper motor rides on it with the pen cam, so the lower axis accelerates that mass too. Equation 1 gives the travel per revolution and the pitch radius for pitch p on N_t teeth,

$$\ell_{\text{rev}} = p N_t = 2 \cdot 20 = 40 \text{ mm}, \qquad r = \frac{\ell_{\text{rev}}}{2\pi} = \frac{40}{2\pi} = 6.37 \text{ mm}, \tag{1}$$



Solid: power. Dashed: header signals. Dotted: undocumented. One shared ground.

Figure 6. Complete circuit with all component values. The clamp diode is dashed because it is *not* fitted, and no base pull-down is fitted either (Section 3.3.3); the conversion stage, the pen rail and the supply's current rating are not documented.

and Equation 2 the torque the motor must supply to accelerate an effective mass m_{eff} at acceleration a against a friction force F_f,

$$T = r\,(m_{\text{eff}}\,a + F_f), \tag{2}$$

which decides whether the motors follow the step rates of Section 3.3.4 without stalling. Three properties of this arrangement make Equation 2 harder to satisfy than the carriage mass alone suggests, and all three are expanded in Appendix 6.4. The effective mass is not the carriage mass: the rotating inertia of the pulleys, cross-shaft and rotor refers to the carriage divided by r^2, and with $r = 6.37$ mm that divisor is about 2.5×10^4 m^{-2}, so even a small rotor inertia adds an equivalent mass of the same order as the carriage. The two axes do not share a load, the upper axis riding on the lower one, so a margin established on one does not transfer to the other. And the torque available falls with step rate, because the winding inductance limits how fast current is established, which is why the 24 V rail was chosen over 12 V for the same motor. Two torque figures are quoted for a stepper and they must not be confused. The *holding torque* is the torque the motor resists when it is stationary with its windings energised, and it is the figure datasheets lead with. The *pull-out torque* is the torque the motor can deliver while actually stepping at a given rate, and it falls as the rate rises for the reason given above. Only the pull-out torque at the rates actually used is a safe basis for a design, because a motor that loses synchronisation under load does not stall gracefully: it skips steps, and in open-loop control a skipped step is an unrecoverable position error for the rest of the job.

It is nevertheless the calculation this design is missing: neither the carriage mass nor the friction force was measured, and the pull-out torque at the rates used [24, 23] was not obtained, so *no torque margin can be stated*. That is also why the design carries no acceleration ramp, a ramp

being sized from a margin that does not exist. **[MUST STILL BE COMPLETED BY ME: weigh the carriages, measure the friction force, evaluate Equation 2 at the 400 mm/s^2 of Equation 54 using the effective mass of Equation 94, and state the margin against pull-out torque at 1200 and 3200 steps/s. Also supply the CAD drawings and photographs of the gantry and rig; their folder is empty.]**

Interface circuits and their component values. Three component values follow from calculation, and Appendix 6.4 derives each. The driver reference is set from the sense resistors by Equation 97, which gives 1.34 V for the motor's 1.68 A rating [23]; that rating is deliberately not used, the reference being derated to about 0.8 V to hold an unheatsinked carrier near 1 A per phase, trading holding torque for a driver that will not shut down thermally in the middle of a page. The switch pull-downs draw 0.33 mA each by Equation 98, and the 10 kΩ is bounded in both directions, smaller wasting current and larger letting pin leakage lift the open-switch voltage; its time constant is microseconds against the 6 ms re-read of Section 3.3.5, so the debounce belongs in the program. The pen motor is switched by a bipolar transistor whose base current, Equation 99, gives the switchable collector current of Equation 100, between 130 and 217 mA depending on the overdrive factor chosen, which is adequate for a motor drawing up to about 200 mA.

Two items in that last stage are design *faults*, not untested ones, and both belong here rather than in the appendix. *No clamp diode is fitted*: at turn-off the energy $\frac{1}{2}LI^2$ stored in the motor's inductance appears as a reverse voltage that stresses the transistor and injects noise into the shared ground, and a diode across the motor, cathode to supply, was simply omitted. *No base pull-down is fitted* either, so the base floats in whatever input state the pin holds at power-up. The missing diode is also the mechanism behind the end-stop behaviour of Section 3.3.5: with no clamp and a shared ground, a coil current I lifts the local ground by IR_{gnd}, and the switch inputs are referred to that ground. The fix is a clamp diode across each motor and a switch return to a single ground point, not a longer debounce.

3.3.4 Step generation, resolution, speed and calibration

Three properties of a stepper drive fix the resolution of this machine, and each needs stating before the calculation makes sense.

A stepper motor does not turn continuously but advances by a fixed angle each time its windings are switched in sequence. The motors used here take 200 of those *full steps* per revolution, so one full step is 1.8°. Because position is the number of steps commanded, it is known without measuring it, which is what *open-loop* control means: no encoder reports where the carriage actually is, and the design simply trusts the step count. That trust is the assumption on which requirement R5 rests, and it fails silently if a step is ever missed.

The driver subdivides each full step by *microstepping*. Rather than switching the two windings fully on or off, the driver holds them at intermediate current ratios, which places the rotor at intermediate angles between full steps. At the 1/16 setting used here, one full step becomes 16 microsteps, so a revolution is 3200 microsteps. Microstepping buys resolution and smoother motion, but not proportionally more accuracy, because the intermediate positions are held by the ratio of two winding currents and are therefore more sensitive to load torque than a full step is.

The driver regulates winding current by *chopping*, which is why the supply voltage may be

far above what the winding needs. The driver switches the supply onto the winding, watches the current rise through a small sense resistor, switches off when the target current is reached, and repeats at high frequency. The average current is therefore set by the target rather than by Ohm's law on the winding resistance, which allows a 24 V rail to force current into the winding inductance quickly while still holding the average at the motor's rated 1.68 A. The consequence for the circuit is that the driver draws its current from the supply in short bursts, which Section 3.3.2 addresses.

With those three in place, Equation 3 gives the step resolution from 200 full steps [23], 16 microsteps [25] and $\ell_{\text{rev}} = 40$ mm,

$$\kappa = \frac{200 \cdot 16}{\ell_{\text{rev}}} = \frac{3200}{40} = 80 \text{ steps/mm}, \qquad \Delta = \frac{1}{\kappa} = 12.5\ \mu\text{m}, \tag{3}$$

which calibration replaces by each axis's measured value. Targets are *absolute* step counts, $\text{round}\big((x+5)\kappa\big)$ with the 5 mm offset of Section 3.3.4, differenced against the tracked position, so rounding cannot accumulate. Quantisation error is at most $\Delta/2 = 6.25\ \mu$m per axis, 8.8 μm combined, 57 times finer than the 0.5 mm of R5 — but that is the *digital* resolution only, and belt stretch, backlash, missed steps, switch repeatability and pen compliance were not measured. **[MUST STILL BE COMPLETED BY ME: measure the positional error for R5 (the same word drawn twice and overlaid) and the return-to-zero repeatability.]**

Equation 4 gives the delay of every pulse for a move of length d at feed v_f over $N_{\text{maj}} = \max(|n_x|, |n_y|, 1)$ major-axis steps,

$$\tau = \max\!\left(\frac{d/v_f}{N_{\text{maj}}},\ 2\,\mu\text{s}\right), \qquad v_f = \max(F/60,\ 0.1), \tag{4}$$

with F in mm/min: 900 gives $\tau = 1/(15 \cdot 80) = 833\ \mu$s (1200 steps/s), 2400 gives 312.5 µs (3200 steps/s, 60 rpm), homing 625 steps/s or 7.8 mm/s. The rate is limited by the four switch reads and two pin writes per step, not the 2 µs floor. **[MUST STILL BE COMPLETED BY ME: measure the highest feed without lost steps.]** Because *every* pulse of both axes sleeps for τ, a segment takes $(n_{\text{maj}} + n_{\text{min}})\tau$ and Equation 5 gives the effective feed,

$$v_{\text{eff}} = v_f \frac{n_{\text{maj}}}{n_{\text{maj}} + n_{\text{min}}} \in \left[\frac{v_f}{2}, v_f\right], \tag{5}$$

unity axis-parallel, $\frac{1}{2}$ at 45°, mean $\frac{1}{2} + \frac{\ln 2}{\pi} = 0.72$ over uniform directions. There are *no acceleration ramps*: the acceleration of Equation 54 appears only in the generator's estimate, so abrupt starts at 1200 to 3200 steps/s with an unknown load may lose steps [24]. A model of the interpreter for the test word of Section 3.9 gives 6.8 s drawing, 10.8 s travel and 9.1 s dwell, 26.7 s in all, 2.4 s per character, 34 % of it dwell; being a model and not a measurement, R3 must still be shown with a stopwatch.

Stepping the axes independently would draw an L-shaped path, so the program interpolates with Bresenham's integer line algorithm [26]: an error term is carried along the major axis, the major axis is pulsed every iteration and the minor axis whenever that term would go negative, and the end-stops are read before every major step. The path so produced stays within one step of the ideal straight line, about 6 µm, using integer arithmetic only, which Equation 101 in Appendix 6.4 establishes; Figure 37 gives the loop as pseudocode.

Direction-agnostic calibration. Equation 3 rests on assumed transmission values and the position after power-up is unknown, so each session begins with a calibration that never needs to know which direction leads to which switch: it drives one way at 1.6 ms per step until *either* switch triggers, reverses, and counts the steps to the *other*. That count against the hand-measured

travel gives each axis its own resolution and its initial step counter by Equation 102, placing the zero 5 mm inside the minimum switch and leaving a usable area of 194×252 mm. The result is deliberately not stored between sessions, since a bump between sessions would silently invalidate it, and an abort refuses all jobs. The tightest tolerance in the hardware design falls on that hand measurement: a travel error scales every commanded position in proportion, so by Equation 103 the 204 mm travel must be known to better than 0.26 mm, or 0.13 %, for the scale error alone to stay inside half of R5. Appendix 6.4 derives that bound and the calibration arithmetic. **[MUST STILL BE COMPLETED BY ME: state the method and uncertainty of the L_X and L_Y measurement, which Equation 103 requires better than 0.26 mm.]**

3.3.5 End-stops, emergency stop and pen mechanism

A mechanical switch does not change state cleanly. Its contacts strike, separate and strike again over a few milliseconds, so a single read taken during that interval may return either level. Treating one high read as a trip would therefore stop the machine at random, and treating the first read as final would miss a genuine trip. The standard remedy is to *debounce* the input, which means requiring the same level on several reads spaced far enough apart to outlast the bouncing.

All four switches are read before every major-axis step and every G-code line, and count as triggered only on three consecutive high reads 6 ms apart. Three reads rather than two were needed for a reason specific to this build: because the motors have no flyback diodes on a shared ground, switching noise held an input high for longer than a single re-check, as Section 3.3.3 explains. The debounce is therefore compensating for a hardware omission as well as for contact bounce. Equation 6 gives the resulting trip-to-stop latency,

$$t_{\text{stop}} \approx \tau + (3-1)\cdot 6\ \text{ms} \approx 13\ \text{ms}, \tag{6}$$

during which the carriage moves 0.2 mm at 15 mm/s and 0.5 mm at 40 mm/s, far inside the 5 mm edge tolerance. On a trip the program latches a stop, sets both STEP pins low, retracts off every triggered switch and aborts the job; targets are clamped to the calibrated area and the latch clears only by explicit command, refused while a switch is still triggered. Protection is *software only*: no hardware stop that cuts motor power is documented, the switches are polled only during a job, no endpoint aborts a running job, and no lock prevents two simultaneous jobs.

Pen mechanism. The implemented lift, planned in the proposal as a stepper, is a gear motor turning one way only, each 90° of its cam toggling the pen, so only pulses can be given, the state must be tracked in software, and the pen switch is feedback only (high = up); the last routine of Figure 37 is the one behind `M5` and `M3`, and it must test the switch before energising the motor, since an unnecessary pulse would toggle the pen to the wrong state rather than leave it alone. One toggle was never timed and the generator assumes 0.35 s, the largest single contributor to the predicted writing time. Neither the DIR-to-STEP timing nor the driver's minimum STEP width is modelled, and missed steps cannot be detected without feedback, so the true accuracy must come from the test of Section 4, not Equation 3.

3.4 Page conditioning and line segmentation

This block turns a photograph of a handwritten page into clean, deskewed, white-background line images for the text recogniser of Section 3.5 and the writer classifier of Section 3.6. It covers functional units FU 2.1 and FU 2.2 and serves requirement R2.

The deployed implementation is `SegmentLean.py`, which is eleven functions in about 570 lines behind a single entry point, `SegmentLines(path)`. The web server, the style-profile builder and the writer-classifier trainer all call that same entry point, so training pages are cut by exactly the code that cuts a page during normal use. Published methods are cited where they are used, and the rules around them are plain array arithmetic written for this project.

No part of this block resamples through an imaging library. Every operation that changes a pixel value — the 2:1 page reduction, the deskew rotation, the background enlargement of Equation 12 and the four mask enlargements of stage 6 — is performed by the project's own module, which implements Equation 7 directly. The library is used only to read the file from disk and to write the optional debug overlay. Each of those replacements was verified against the library call it replaced, and the results are in Section 3.4.8.

3.4.1 Design goals and the measured scale unit

A flat-bed scan can be conditioned by grey conversion, binarisation and bounding-box line separation, which is what published pipelines do [5, 1]. A photograph is harder. It adds uneven illumination, the surroundings of the sheet, skew, printed rules, edge slivers, creases, and lines that touch each other.

The block therefore has three design goals. It must work at any camera resolution and any handwriting size without manual tuning, it must never delete handwriting in order to remove a nuisance, and it must keep as few fixed constants as possible.

The way all three are met is to let the page set its own scale. The photograph is first reduced 2:1 per axis by box averaging, which keeps thin pale strokes that point sampling would drop. The *line pitch* P is then measured on that working image in stage 5. Almost every later threshold is a multiple of P, so the same code works on a 2 megapixel and an 8 megapixel photograph without changing a number. Only four constants are absolute: the blur radius, the floor of 60 ink pixels per emitted line, the 64×640 canvas size and the dynamic-program weights. Table 25 in Appendix 6.4 lists every minor constant with the reason for its value.

The geometry of a line of handwriting. Every subsystem from here onwards measures handwriting against the same five quantities, so they are defined once here and used unchanged for the rest of Section 3. They are the standard typographic reference lines of a line of writing, shown in Figure 7.

The *baseline* is the line the body of the lower-case letters sits on. The *x-height* x_{h} is the height of a lower-case letter that has neither a tall part nor a tail, such as x, o or e, measured up from the baseline. An *ascender* is the part of a letter such as h, l or b that reaches above the x-height, and a *descender* is the part of a letter such as p, g or y that falls below the baseline. The *line pitch* P is the distance from one baseline to the next.

Two of these five are used as units, and which one is used depends on what is being measured. The pitch P is the unit of this block, because segmentation is about separating one line from the next and every threshold here is a fraction of a line spacing. The x-height x_{h} is the unit of everything downstream of this block: the writer classifier redraws ink at a pen width proportional to it (Section 3.6.3), the style builder stores every glyph in a frame scaled by it (Section 3.7), and the synthesiser converts back to millimetres by one multiplication at the very end (Section 3.8).

The reason for using the writing's own geometry as the unit, rather than pixels or millimetres, is that it removes two variables at once. A page photographed closer to the camera has more pixels per letter but the same number of x-heights per letter, and a writer with large handwriting has more millimetres per letter but, again, the same number of x-heights. Measuring in x-heights therefore makes every stored style parameter independent of both the camera and the size the writer happens to write at, which is what allows one style profile to be drawn at any pen size.

The ascender reach A and descender reach D are stored per writer as multiples of x_{h}, with $A > 1$ because an ascender reaches above the x-height line and $D < 0$ because a descender falls below the baseline. A *ligature* is the connecting stroke a writer draws between two joined letters, and it matters later because it is the only place in joined handwriting where one letter honestly ends and the next begins (Section 3.7).

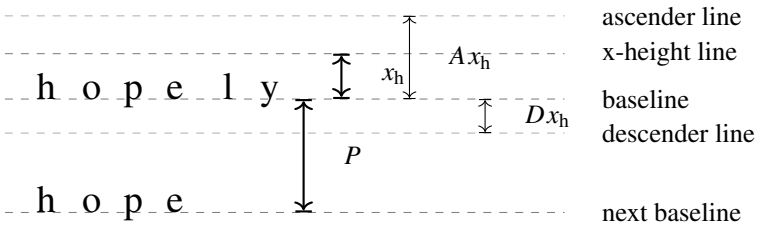


Figure 7. The five quantities every subsystem measures handwriting against. The x-height x_{h} is the height of a lower-case letter with no tall part and no tail; an ascender reaches above it and a descender below the baseline. The ascender and descender reaches A and D are stored as multiples of x_{h}, so $A > 1$ and $D < 0$. The line pitch P is the baseline-to-baseline distance and is the unit used by this block; x_{h} is the unit used by every block after it.

Ink is described in two tiers. A strict page-wide mask fixes the structure of the page, such as where the paper is and where the lines are. A second, more permissive threshold is then applied inside each individual crop, limited to a halo around that line's own strong ink, which recovers pale strokes that the strict mask missed. The reason for two tiers is that a threshold loose enough to catch a faint stroke is also loose enough to catch a shadow, so the loose threshold is only trusted next to ink that is already known to belong to that line.

The six stages are reduce, ink mask, deskew, clean-up, pitch and centres, and finally seams and crops. They are described in that order below, which is the order the data flows through them.

Three of the stages resample an image, and they do it by bilinear interpolation at the source position given by Equation 7. The exception is one ownership map, which uses nearest neighbour, because the values in that map are integer line indices and interpolating them would invent indices that do not exist. Equation 8 is the scale used to paste a finished crop onto the white 640×64 canvas that both networks expect, flush left and vertically centred.

$$y_s = \frac{(i+0.5)\,h}{h'} - 0.5, \quad x_s = \frac{(j+0.5)\,w}{w'} - 0.5, \quad I'(i,j) = \sum_{a,b \in \{0,1\}} \omega_{ab}\, I(\lfloor y_s \rfloor + a, \lfloor x_s \rfloor + b), \tag{7}$$

$$s = \min\!\left(\frac{64}{h}, \frac{640}{w}\right), \qquad h' = \max(1, \mathrm{round}(hs)), \qquad w' = \max(1, \mathrm{round}(ws)). \tag{8}$$

The single scale factor s in Equation 8 is applied to both axes, so letter proportions are not distorted. Flushing the crop left has a second benefit that Section 3.7 depends on: the map from a network time step back to a pixel column of the original crop is then the exact inverse $x_{\mathrm{orig}} = x_{\mathrm{canvas}} / s$, with no offset to track.

Each line record that the block returns holds the order index, the crop at native resolution, that canvas, and the bounding box in full-resolution coordinates. The block also returns page statistics: the chosen skew angle, the pitch, the paper coverage, the ink discarded off the paper, the straight pixels and artefact pieces removed, the line count and the wall-clock time. These allow a page to be checked after the fact without having to instrument the module.

3.4.2 The three array primitives

Almost every operation in this block is a sum, a maximum, or a test of whether anything is set, taken over a rectangular window. Computed directly, a window of radius r costs $O(r)$ per pixel per axis. That is too slow here, because the illumination blur below uses a radius of up to 40 pixels.

The module therefore builds a summed-area table [27], also called an integral image, which is Equation 9. It is built in one $O(HW)$ pass of two cumulative sums, and after that the sum over a window of *any* size costs four look-ups instead of $(2r+1)^2$ additions. The window corners are $y_2 = y_1 + 2r + 1$ and $x_2 = x_1 + 2r + 1$. The array is padded by r using edge replication first, so that every window holds the same number of samples and a pixel near the border is not biased.

$$S(y,x) = \sum_{i<y} \sum_{j<x} I(i,j), \qquad \text{BoxSum} = S(y_2,x_2) - S(y_1,x_2) - S(y_2,x_1) + S(y_1,x_1), \tag{9}$$

$$\text{AnyWithinRadius}(m,r)[i] = \big[\, c[i+2r+1] - c[i] > 0 \,\big] = \Big[\bigvee_{k=-r}^{r} m[i+k] \Big], \tag{10}$$

$$\text{LongRun}(m,n)[j] = \big[\, \text{ch}[h_j + 1] - \text{ch}[l_j] > 0 \,\big],\ l_j = \max(0, j-n+1),\ h_j = \min(j, L-n). \tag{11}$$

The second primitive, Equation 10, is a dilation. A square dilation separates into two one-axis dilations, and each of those is a cumulative sum of the mask followed by one subtraction, so it also costs the same at any radius. An erosion is then the complemented dilation of the complement, and a closing is a dilation followed by an erosion, which means no other structuring element has to be implemented.

The third primitive, Equation 11, marks every pixel that lies on a run of at least n consecutive set pixels. A run of length n starts at position i exactly when $c[i+n] - c[i] = n$, so the starts are found first and then a second cumulative sum marks every pixel covered by one. This is what stage 4 uses to find printed rules. Figure 25 gives all three primitives as pseudocode.

Only one true Gaussian filter is used in the whole block, and it smooths the row profile in stage 5. It is a 25-tap kernel $k[t] \propto \exp(-t^2/2\sigma^2)$ for $t = -12 \ldots 12$ with $\sigma = 4$ rows, truncated at 3σ and normalised. Four rows is about a tenth of a typical pitch, so it smooths the ripple inside a line without blurring one line into the next.

3.4.3 Stage 2: lighting normalisation and the ink mask

The intensity measured by a camera is approximately the reflectance of the paper multiplied by the illumination falling on it. Because the unwanted term is a multiplication and not an addition, it is divided out rather than subtracted. The working grey image G is divided by a single box mean of itself, which is Equation 12. The result is a ratio image ρ which is close to 1 on paper

whatever the lighting, and on ink is the ink-to-paper reflectance ratio of the pen itself.

$$\rho(p) = \frac{G(p)}{\max\big(\mathrm{BoxMean}_r(G)(p),\, 1\big)}, \qquad r = \max\big(8,\, \min(40,\, \mathrm{round}(H/41))\big), \tag{12}$$

$$t^{*} = \arg\max_{t}\; \omega_B(t)\,\omega_F(t)\big[\mu_B(t) - \mu_F(t)\big]^2. \tag{13}$$

The radius r in Equation 12 scales with the image height, so the background estimate covers the same fraction of the page at any resolution. It must be much larger than a letter, otherwise the letters would darken their own background estimate and disappear.

Once the shading has been divided out, the ink decision is a single global threshold, and Otsu's method [28] is used to choose it. Otsu's threshold is the value that maximises the between-class variance of Equation 13, where ω_B and ω_F are the fractions of samples below and above t and μ_B and μ_F are their means. A pixel is marked as ink when $\rho < t^{*}$. The cumulative form of Equation 13 is evaluated for all 256 histogram bins at once rather than in a loop.

A single global threshold is only possible because of the division. This is the step that removed the long chain of clean-up filters that the project's first segmenter needed, and it is why this stage holds no tunable grey level at all. One extra test is still needed: a solid-area veto removes any window that is entirely ink, because no pen stroke fills a 9×9 window. That catches a binder shadow or the dark gutter of a bound book, which a global threshold would otherwise mark as ink. The whole stage is four array statements, given as pseudocode in Figure 26.

3.4.4 Stage 3: deskew by searching rotations

When the text lines are horizontal, the row-sum profile of the ink has sharp peaks at the line bodies and deep troughs between them. Any tilt smears those peaks, because a single row then cuts across several lines. The sharpness of the profile is therefore a measure of how level the page is, and it can be maximised directly.

A candidate angle is scored by the projection objective of Equation 14, in which R_θ is the ink mask rotated by θ.

$$J(\theta) = \sum_{y} P_\theta(y)^2, \qquad P_\theta(y) = \sum_{x} R_\theta(y,x), \qquad \theta^{*} = \arg\max_{\theta} J(\theta). \tag{14}$$

Squaring the profile before summing is what makes Equation 14 work. The total amount of ink does not change when the page is rotated, so $\sum_y P_\theta(y)$ is fixed and could not be used. The sum of squares is largest when the same ink is concentrated into the fewest rows, which happens when the lines are level.

The search is one sweep of 17 candidate angles from $-4°$ to $+4°$ in steps of $0.5°$, scored on the stage-2 mask reduced by a further factor of two. There is no refinement pass, so the best accuracy the stage can reach is half a step, which is $0.25°$.

The page is never actually rotated to score an angle, which is worth setting out because it is what makes the sweep affordable. Rotating a point by θ about the centre of the image puts it in row

$$k = (y - c_y)\cos\theta - (x - c_x)\sin\theta + c_y, \tag{15}$$

so the coordinates of the ink pixels can be mapped straight into row numbers by Equation 15 and counted, which gives the profile P_θ of Equation 14 without producing a rotated image at all. That is one pass over the ink pixels per angle instead of seventeen image rotations, measured at

31 times faster, and it avoids resampling the mask seventeen times, each of which would blur it slightly further and bias the comparison towards the angles evaluated last.

One detail of that counting is not optional. Ink that would fall outside the row range at a trial angle must be allowed to extend the profile rather than being clipped into the first or last row. Clipping piles every out-of-range pixel into a single row, and because the objective squares each row, that artificial spike scores higher than any real text line; it made the search choose an angle four degrees wrong on a tilted page before it was corrected.

Only once an angle is chosen is the page itself rotated, bilinearly, and the ink mask and ratio image are then recomputed, because Equation 12 is not rotation-equivariant: rotating the page and then dividing does not give the same result as dividing and then rotating.

The first segmenter had a fault here that is worth recording, because the structure of the deployed stage prevents it. A sign mismatch between the rotation routine and the angle scorer meant the page was rotated the wrong way, which doubled the skew of every tilted page instead of removing it. The deployed module cannot have that fault, because the candidate masks are rotated by the very same call that is later applied to the page, so the two cannot disagree about the sign. Section 3.4.8 measures what the stage actually achieves.

3.4.5 Stage 4: removing what is not handwriting

A first pitch P is computed by the procedure of stage 5, and every threshold in this stage is a multiple of it. Three kinds of nuisance are then removed in turn.

The first is the area outside the sheet. Handwriting disappears under a coarse enough average while the sheet itself does not, and that is what separates them. The working image is block-averaged down, blurred, and thresholded against its own median, after which the largest bright connected component is taken to be the sheet and is eroded slightly to keep its darkened rim out of the ink mask. The result is reported as `paperCoverage` and `inkOutsidePaper`.

The second is printed structure. Ruled paper, underlines and the spine shadow of a bound book weld separate text lines into one piece of ink, which would defeat the line separation entirely. What distinguishes them from handwriting is that they run straight for longer than a line spacing, and handwriting does not. The long-run primitive is therefore applied per axis, bracketed by a dilation and a re-intersection as in Equation 16, where $\mathcal{D}_r^y$ and $\mathcal{D}_r^x$ are one-axis dilations.

$$\begin{aligned} \text{flat} &= \mathcal{D}_1^y\Big(\text{LongRun}^x\big(\mathcal{D}_1^y(\text{ink}),\ \lfloor 1.25P \rfloor\big)\Big) \wedge \text{ink}, \\ \text{upright} &= \mathcal{D}_3^x\Big(\text{LongRun}^y\big(\mathcal{D}_3^x(\text{ink}),\ \lfloor 1.4P \rfloor\big)\Big) \wedge \text{ink}. \end{aligned} \tag{16}$$

The dilation before the run test is what makes Equation 16 tolerant of a slight tilt, because a rule that drifts by one row would otherwise break into runs too short to detect. The re-intersection with the ink mask afterwards means only real ink is ever removed, never the pixels the dilation added. The two run lengths, $1.25P$ horizontally and $1.4P$ vertically, are the essential point: both are longer than any single letter can be, so no letter can be mistaken for a rule.

The third kind of nuisance is edge slivers, corner shadows and fold creases, which survive the run test because they are not straight. These are judged as whole pieces instead of pixel by pixel, because what identifies them is a property of the whole shape, such as being long and thin or being sparse inside its own bounding box.

Doing that requires the ink to be grouped into pieces first, which is *connected-component labelling*: every set pixel is given an integer label, and two pixels get the same label when a path of set pixels joins them. The mask is closed before labelling so that the pen lifts inside a word are bridged and a whole word becomes one piece rather than several. The labelling itself is done by a run-length union-find pass. Each row is first reduced to its horizontal runs of set pixels, which is far fewer objects than pixels, and a *union-find* structure then records which runs have been found to belong together: it holds one representative per group and merges two groups in near-constant time when a run in one row is found to touch a run in the next. One pass over the rows therefore labels the whole page.

Every statistic used by the discard tests is then accumulated over the *original* ink pixels rather than the closed mask, so the closing can join pieces but can never add area to a decision. Eight discard predicates are applied to each piece, combining its size, its aspect ratio, its faintness relative to the page's own threshold, and its distance from the frame. Table 26 in Appendix 6.4 states all eight with the nuisance each is meant to catch.

The most aggressive of the eight discards any piece whose leftmost ink lies within $\max(3, \lfloor 0.5P \rfloor)$ of the left frame edge, whatever its size or darkness. This removes the facing page of an open book, but it also costs the first letters of a page photographed with no left margin, so the block requires a visible left margin.

3.4.6 Stage 5: line pitch and line centres from the row-ink profile

The row-ink profile is the count of mask pixels in each row, taken over a central window of the width so that the margins cannot contribute, and then smoothed with the Gaussian kernel described above. This is Equation 17. A line centre is a peak of that smoothed profile, and a peak is an interior row that satisfies the three conditions of Equation 18.

$$\mathrm{prof}(y) = \sum_{x=\lfloor 0.05W \rfloor}^{\lfloor 0.97W \rfloor - 1} \mathrm{ink}(y,x), \qquad \mathrm{sm} = \mathrm{prof} * k, \tag{17}$$

$$\mathrm{sm}(y) \geq \mathrm{sm}(y-1), \qquad \mathrm{sm}(y) > \mathrm{sm}(y+1), \qquad \mathrm{sm}(y) > 0.06 \max_{u} \mathrm{sm}(u). \tag{18}$$

The two comparisons in Equation 18 are deliberately asymmetric, one using $\geq$ and the other $>$. If both used $\geq$, every row of a flat-topped peak would be reported as a separate line, and if both used $>$, a flat top would be missed entirely. As written, the last row of a plateau is taken exactly once. The third condition is a relative floor that rejects the small ripple between lines, and because it is a fraction of the strongest peak rather than an absolute count, it scales with the amount of ink on the page.

The pitch is the median of the gaps between consecutive peaks, which is used whenever at least two peaks exist. A median is used rather than a mean so that one unusually large gap, such as a paragraph break, cannot shift the pitch.

Three fallbacks follow in order, shown in Figure 27. The first uses the *autocorrelation* of the profile, which is the profile multiplied by a shifted copy of itself and summed, evaluated for every shift. That shift is called the lag. A profile made of evenly spaced line bodies agrees well with itself when shifted by exactly one line spacing, so the autocorrelation has a maximum at a lag equal to the pitch, and this holds even when the individual peaks are too weak to be detected one at a time. The mean is removed from the profile first, otherwise the overall amount of ink would

dominate the sum at every lag. If fewer than two peaks were found, the pitch is therefore taken as the first maximum over lags 15 to $\min(200, L_{ac}/3)$ that reaches 0.7 of the maximum in that range. It is only a fallback because the profile also agrees with itself at two line spacings, three, and so on, so the method can lock onto a multiple of the true pitch. If that also fails, the pitch is estimated from the letters themselves as $P = \max(15, \mathrm{round}(2.2\,\mathrm{median}\{h_c\}))$ over components with area $A_c \geq 8$ and height $3 \leq h_c \leq H$, where 2.2 is the measured ratio of pitch to letter height for these hands. If there are no usable components either, the pitch is set to 48 rows.

The pitch and the profile are recomputed on the cleaned mask after stage 4, because removing a rule or a shadow changes the profile. The peaks then become line centres through a greedy non-maximum suppression: peaks are taken in order of decreasing profile value, and a peak is accepted only if it lies at least $0.6P$ from every centre already accepted. That separation is more than half a pitch, so two genuine lines can never be merged into one.

3.4.7 Stage 6: seams, pixel ownership and the line crops

A straight cut halfway between two line centres would sever every descender and ascender that crosses it. The boundary between two lines must therefore bend around the ink. Between consecutive centres $c_0 < c_1$ the module builds the cost field of Equation 19 and finds the cheapest path across it by dynamic programming over columns, which is Equation 20.

$$C(y,x) = 4.0\,\tilde{m}(y,x) + 0.012\left(\frac{y-\mu}{\max(1,(c_1-c_0)/2)}\right)^2, \qquad \mu = c_0 + \tfrac{1}{2}(c_1-c_0), \tag{19}$$

$$A(y,x) = C(y,x) + \min_{|d|\leq s}\left[A(y+d, x-1) + 0.02\,|d|\right], \qquad A(y,0) = C(y,0), \qquad s = 7. \tag{20}$$

In Equation 19, $\tilde{m}$ is the closed ink mask and μ is the midline between the two centres, with the half-gap as the denominator so that the second term is dimensionless. The relative size of the two weights is the design decision. At 4.0 against a maximum of 0.012, crossing a single ink pixel costs more than three hundred times as much as sitting a half-gap away from the midline. The midline term therefore has no influence where there is ink, and only breaks ties over the blank gaps between words, where it keeps the seam from wandering far from where a line boundary belongs.

The recursion of Equation 20 is exact, because the cost of a path is a sum over columns and every admissible predecessor is considered at each step. The step limit $s = 7$ rows per column lets the boundary react quickly enough to dodge a tall narrow descender, and the penalty of 0.02 per row of displacement stops it zig-zagging where the cost is flat. The mask is closed before the cost field is built, because a plain dilation would inflate the strokes into blobs that the seam would then trace around instead of cutting between.

The two outermost boundaries are fixed rather than computed, at $\lfloor 0.75P \rfloor$ beyond the first and last centre, which covers the ascenders of the first line and the descenders of the last. With that, L centres give exactly L bands. The whole computation is $O(15HW)$ at worst, which makes the seam search the most expensive single step in the block. Figure 28 gives the cost field and the recursion as pseudocode.

A band is not the same thing as a line of writing, because an ink pixel can lie in the wrong band. By default a pixel belongs to the band it lies in, the band index being the number of interior boundaries above it. Writing is not that tidy, so six correction rules act on top of that default, and

each may only move a piece to an adjacent band or drop it. They return a cut-off tail to the band that holds most of its piece, place a small fragment by proximity, and follow an *i* dot or an accent downwards to the stem it belongs to. The dot rule accepts a stem only when that stem's band also lies where an x-height band should sit below a dot, which is a much stronger test than either condition alone. The rules rest on each band's share of a piece's ink, where 0.15 counts as a genuine presence, and on the *major* band, which is the topmost band reaching the piece, because a piece spanning two lines is the upper line's descender. Table 27 in Appendix 6.4 states all six, and every size in them is a multiple of P^2 so that no rule is tied to a pixel count.

The crops themselves are rendered from the full-resolution page, not from the working image. The half-resolution background estimate is enlarged by Equation 7 and the full page is divided by it, so the recogniser receives every pixel the camera captured even though every decision was taken at half resolution. A band is emitted only if it carries enough owned ink over enough rows, which drops a band that holds nothing but a stray mark.

Each crop is composited from three layers. The first is the band's own ink mask, enlarged bilinearly, which gives a soft edge rather than a hard one. The second is a recovery mask, which is Equations 12 and 13 applied again inside this crop alone, where a pale stroke is a much larger fraction of the local contrast than it was on the whole page. The third is the halo of the band's own strong ink.

The three are combined by a *hysteresis* rule, Equation 21. Hysteresis here means using two thresholds instead of one: a strict threshold marks pixels that are certainly ink, and a loose threshold is then allowed to add further pixels, but only where they are connected to something the strict threshold already accepted. This is the same principle used to link edges in edge detection, applied to ink rather than to edges. It is what makes the two-tier description of ink work, because a loose threshold on its own would accept shadows anywhere on the page, whereas a loose threshold restricted to the neighbourhood of known ink can only extend a stroke that is already there. The grey values are then stretched by Equation 22.

$$\alpha = \max\Big(\alpha_{\text{strong}},\ \mathbb{1}[\rho_{\text{local}} < t^{*}_{\text{local}}] \cdot \alpha_{\text{halo}} \cdot \mathbb{1}[\text{not owned by another band}]\Big), \tag{21}$$

$$\ell = \text{clip}\left(\frac{255\,(v-b)}{\max(w-b,1)}, 0, 255\right), \qquad \text{out} = 255\Big(1 - \big(\tfrac{255-\ell}{255}\big)^{0.55}\Big). \tag{22}$$

The two factors in Equation 21 do different jobs. The halo factor accepts a pale stroke only when it lies close to ink this line already owns, which is what stops the loose local threshold from importing a shadow. The ownership factor is an absolute veto, so two adjacent crops can never both claim the same ink. In Equation 22 the black point b is the 40th percentile of the grey values under the mask rather than the darkest pixel, so a single dark speck cannot flatten the whole line, and the exponent of 0.55 lifts the mid-greys so that a faint stroke survives.

Outside the mask the crop is pure white. The crop is finally shifted horizontally so that its first inked column sits a fixed distance from the left edge, because the recogniser's input is left-flush by Equation 8.

3.4.8 Testing the segmentation block

Two properties of this block can be tested against a known answer, and both were measured on the 30 photographed pages of the five personal writers, at 2448×3264 pixels each. Table 6 gives the results.

The first is whether the block finds the right number of lines. The ground truth is the transcription that was typed by hand when each page was collected, so the number of non-empty lines in that file is the number of lines the page actually holds. The block found the correct number on all 30 pages, which is 669 lines in total with no page over-segmented or under-segmented. This also settles the relative floor of Equation 18, which was the one threshold in stage 5 with no evidence behind it: a line peaking below six per cent of the strongest would be lost silently, and over these 30 pages no line was lost.

The second is whether the deskew of Equation 14 recovers a known tilt. One page per writer was rotated by a known angle, the stage was run on the rotated page, and the angle it chose was compared with the angle it should have chosen. Two sets of angles were used. Seven angles that are multiples of the 0.5° search step were recovered exactly in all 35 trials. Four angles that fall between search steps were recovered to 0.25° in all 20 trials, and 0.25° is exactly half the step, which is the best any search on that grid can do. The stage is therefore limited by the coarseness of its own search grid and not by the objective, which is a useful thing to know: if better than 0.25° were ever needed, the fix is a second refinement sweep and not a different measure of sharpness.

Measurement	Result	What it shows
Line count against the hand-typed transcription, 30 pages	**30 of 30** pages exact; 669 of 669 lines	The line separation is correct on every page tested, and the relative floor of Equation 18 lost no line.
Deskew, 7 angles on the search grid, 5 pages	**35 of 35** exact	The objective of Equation 14 picks the correct grid point every time.
Deskew, 4 angles between grid points, 5 pages	**20 of 20** at 0.25°	The residual error is exactly half the search step, so the stage is quantisation-limited, not objective-limited.
Time per page, 2448 × 3264	**4.9 to 9.7 s**, mean 7.9 s	The cost of the deployed module, which had not previously been measured. Measured with the project's own resampling throughout.
Measured line pitch P	47 to 58 pixels	The scale unit varies by 23 % across these writers, which is what the block is designed to absorb.
Chosen skew angle on the unrotated pages	−1.5° to +3.0°	Real photographs are tilted by more than a degree, so the stage is doing necessary work.

Table 6. Measured performance of the deployed segmentation block over the 30 photographed pages of the five personal writers. These are the pages the system was built on, not an unseen test set, so the line-count result shows that the block handles the material it was designed for and not that it generalises to new conditions.

Two things must be said about what these figures do and do not prove. The 30 pages are the pages the system was built on, so the line-count result is not an unseen test; it shows that the block handles the material it was designed for. A fair test needs pages photographed under conditions not represented here, such as a different pen, a different room or a different camera. The deskew result is stronger, because the injected angle is ground truth that no part of the design could have been fitted to.

The timing figure also corrects an attribution. The 24.8 to 69.1 s per page reported in Section 4 is a timing of `SegmentPage.ProcessPage`, the retired segmenter, recorded in the stored field `stage1_processpage_s`. On the same page resolution the deployed module takes 4.9 to 9.7 s, which is roughly five times faster, because it makes one pass over a half-resolution image where the retired module made four passes over a larger one. The mean of 7.9 s is still about four times the two seconds that requirement R4 allows for capture and conditioning, so conditioning does not meet that budget, but it misses it by a factor of four and not by a factor of twelve. These

figures are for the deployed code with the project's own resampling; the same module using the imaging library's compiled resampling took 2.8 to 8.4 s, so writing the resampling in array arithmetic costs about 40 % of the block's run time, which is the price of the library-free claim and is stated rather than hidden. **[MUST STILL BE COMPLETED BY ME: update the segmentation figures of Section 4 and of the R4 budget to the deployed module's 4.9 to 9.7 s measured here, keeping the retired module's range only where it is explicitly labelled as such.] [MUST STILL BE COMPLETED BY ME: time `SegmentLean.SegmentLines` on the ODROID N2+ over the same 30 pages; the project files hold no figure for the board, and the board is expected to be slower than the laptop these figures were taken on.] [MUST STILL BE COMPLETED BY ME: produce the four figures cut for space: the four-panel lighting normalisation (photograph, grey, background, ratio) from `InkMask`, the skew objective $J(\theta)$ over the 17 angles, the paper region from `PaperRegionMask`, and the `_cuts_overlay.jpg` that `SegmentLean.py` already writes. Insert the last first if space is recovered.]**

3.4.9 Design alternatives and limitations

Table 7 sets the main design choices against the alternatives that were considered. The project's first segmenter, `SegmentPage.py`, was about 2200 lines and sixty functions against the eleven here. It resized every photograph to a long side of 2400 pixels, thresholded locally, removed printed structure in four passes, took a median glyph height as its scale unit, scored components for being diagrams, chained them into lines and deskewed each line separately. No part of the deployed pipeline calls it any more. Three of its capabilities were given up with it: diagram tagging, per-line deskew, and the wider ±8° search at 0.1°. The first two are acceptable for plain prose pages and the third is covered by the test above.

With $N_p = HW$ the pixel count of the working image, which is a quarter of the photograph's, every pass is $O(N_p)$ except the seam search at $O(15N_p)$, which dominates, and the ownership rules, whose cost grows with the amount of writing rather than the page size.

The block assumes left-to-right writing in a single column of roughly horizontal lines. The following limitations follow from that design rather than from tuning, and each is a consequence of a choice made above.

Skew beyond ±4° is not corrected at all, and up to 0.25° always remains, as Section 3.4.8 measures. Two columns of text, or a diagram beside the text, produce profile peaks that belong to no single line, and because diagram tagging was given up, an untagged diagram is cut into line-shaped crops and read as nonsense. A line whose profile peak falls below six per cent of the strongest is lost silently, although no such line occurred in the 30 pages tested.

A blank page raises an exception rather than returning no lines, because the first seam indexes an empty list of centres, and the same happens when no piece is letter-like. The server should therefore reject such an image before calling the block. Any ink within $\max(3, \lfloor 0.5P \rfloor)$ of the left frame edge is discarded whatever its size, so a visible left margin is required, and a deliberate division bar or strike-through is removed as though it were a printed rule.

Illumination that varies faster than the radius of Equation 12 leaves one side of a hard shadow edge mis-thresholded. A dense diagram can darken its own background estimate until it disappears. A stroke thick enough to fill a 9×9 window is thinned by the solid-area veto. Page curl and

perspective are untreated, a desk as bright as the paper defeats the paper region, and colour is treated like black ink because the module is monochrome throughout.

Decision	Alternatives, and why the deployed choice was made
Lighting	Subtracting a blurred background leaves the contrast dependent on brightness. The nuisance is multiplicative, so dividing gives an ink-to-paper ratio that is independent of the lamp. A direct Gaussian, or three box means, also cost more than one box mean over a summed-area table.
Ink threshold	A local mean less an offset, or a local mean and standard deviation, marks something even in an empty region, because every region has a darkest pixel, and it has an offset in grey levels that must be tuned. After the division, one global Otsu threshold is enough, which is what removed the first segmenter's long chain of clean-up filters.
Scale unit	A median glyph height is upset by a heading in another size, and a fixed 2400 px page throws away resolution. The pitch P is the quantity the problem is posed in, and it is measured on the page itself.
Lines	Chaining components along a baseline curve needs a correct set of components first, so every clean-up error propagates into the line structure. It also took three passes, ten further constants and a quadratic merge. The row profile is one $O(N_p)$ pass and yields the pitch and the centres together. The accepted cost is one column only and no curved baselines.
Line boundaries	A straight cut at the midline severs every descender and ascender. The least-ink path bends around them and relaxes where there is no ink.
Skew	A Hough transform of the ink edges is dominated by ruled lines and diagram strokes. The projection objective is also the quantity stage 5 goes on to use, so it costs nothing extra.
Rule removal	A long morphological opening erases underlines and cannot follow a slope. Chaining fragments across gaps needs five constants, where the dilated long-run test needs two.
Not implemented	Text-versus-diagram tagging, per-line deskew and a four-point perspective warp are all absent. The first is accepted for plain prose pages, the residual drift is tolerated by the recogniser, and the camera looks straight down so the keystone distortion is small.

Table 7. Design alternatives of the deployed segmentation block. The last row records capabilities of the first segmenter that were deliberately given up.

3.5 Text recognition

This subsection describes the neural network that decodes a handwritten line image into text (functional unit FU 2.3, requirement R2). The recogniser follows the CNN-BiLSTM-CTC architecture of Kizilirmak and Yanikoglu [10], which was chosen because it was evaluated on the same public handwriting database used here [4] and reports accuracies in the range that requirement R2 needs. That paper describes an approach and a layer pattern rather than publishing code, so the network was built from its description: the layer pattern, the filter counts and the recurrent width are taken from it, while the input geometry, the data pipeline, the label cleaning, the augmentation and the staged training recipe were decided for this project. No machine-learning library is used in the recogniser, in the writer classifiers of Section 3.6 or in the forced alignment of Section 3.7, so the forward passes, the backward passes and the optimiser updates are all evaluated by code written for this project (Section 3.5.7).

3.5.1 Problem statement and design approach

The input is one grey-scale image of a text line from the segmentation block (Section 3.4); the output is the string that was written. Three properties of handwriting fix the design: cursive characters are joined, so their boundaries cannot be located reliably and cutting the line into characters, as the proposal planned, would pass every cutting error into a wrong reading; the number of characters is unknown and has no fixed relation to image width; and the available training material gives line images with transcriptions, but not the position of every character. The design that follows is a convolutional neural network (CNN) followed by a bidirectional

long short-term memory (BiLSTM) network trained with connectionist temporal classification (CTC) [9]: the convolutional layers give one feature vector per narrow vertical slice, the recurrent layers give every slice the context of the whole line, and the CTC loss allows training from the line transcription alone. That hybrid of convolution and recurrence is the combination reported to improve feature extraction and contextual understanding together [6, 2].

3.5.2 Input pre-processing

The input size is fixed at 64×640 pixels, which allows batching and a fixed sequence length and is smaller than the 100×960 of the published design, which cost about an hour per training epoch on a processor-only computer. A crop of height h and width w is resized uniformly by Equation 23, using the bilinear rule of Equation 7, pasted onto a white 640×64 canvas left-aligned and vertically centred, then inverted and normalised by Equation 24, so paper becomes -1 and ink $+1$.

$$s = \min\Big(\frac{64}{h}, \frac{640}{w}\Big), \qquad w' = \max(1, \operatorname{round}(ws)), \qquad h' = \max(1, \operatorname{round}(hs)), \tag{23}$$

$$x = \frac{(1 - p/255) - 0.5}{0.5} = 1 - \frac{2p}{255}. \tag{24}$$

A plain stretch to the canvas was used first but distorted letter proportions, in one case a horizontal scale of 0.48 against a vertical 0.35. The constant of Equation 24 is fixed rather than per-image, so white padding cannot change the contrast statistics, and the left flush makes the map from a time step back to a pixel column the exact inverse $x_{\text{orig}} = x_{\text{canvas}}/s$ that Section 3.7 needs.

3.5.3 Network architecture

Figure 8 gives the stages with their tensor shapes, each verified for this project against the published pattern; the network holds 3 358 763 trainable parameters, of which the two recurrent layers hold 70.5 % and the third convolutional stage 24.2 %, and Table 28 in Appendix 6.4 resolves that total layer by layer.

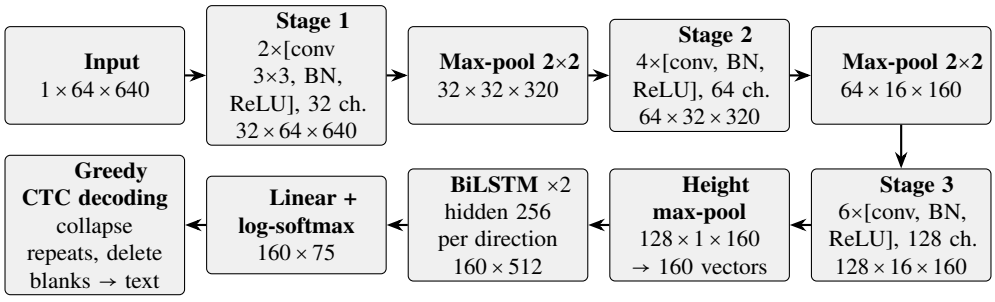


Figure 8. Architecture of the text recogniser, the layer pattern being that of Kizilirmak and Yanikoglu [10]. Tensor shapes are channels × height × width for the convolutional part and time steps × features for the recurrent part; BN denotes batch normalisation.

The convolutional layers. A convolution applies the same small learned filter at every position of its input [12], which is Equation 25. In it, k is the kernel size, s is the stride, and X_{pad} is the input with p rows and columns of zeros added on each side. Applying one filter everywhere is

what makes the layer useful here: a stroke junction means the same thing wherever on the line it appears, so the filter that detects it should not have to be learned again for each position. With $k = 3$, $p = 1$ and $s = 1$ the output has the same height and width as the input, so only the pooling layers change the tensor size and every shape in Figure 8 follows from the two pools alone.

$$Y[o,i,j] = b_o + \sum_{c=1}^{C_{\text{in}}} \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} W[o,c,u,v]\, X_{\text{pad}}[c, s\,i+u, s\,j+v], \tag{25}$$

$$Y = \tilde{W}\,\text{cols} + b\,\mathbf{1}^{\top}. \tag{26}$$

Equation 25 is six nested summations, which is far too slow to evaluate directly in array arithmetic. The implementation therefore uses the *im2col* rearrangement of Equation 26. Every $k \times k$ patch of the input that a filter would see is copied out as one column of a large matrix, and the filter weights are reshaped into rows, so the whole convolution becomes a single matrix product. The result is arithmetically the same sum, agreeing with the direct form to about 10^{-13}, but about ten times faster, because one large matrix product is handled far more efficiently than millions of small ones. The cost is memory: the patches overlap, so each input pixel is copied up to $k^2 = 9$ times and the largest such matrix in this network holds 47 MB.

Normalisation, the nonlinearity and pooling. Every convolution is followed by batch normalisation and a rectifier, and each stage ends with a pooling layer. All three are needed, and for different reasons.

Batch normalisation [29] rescales each channel so that, over the current batch of training images, it has zero mean and unit variance, and then applies a learned scale γ and shift β so the layer can undo that rescaling if it needs to. This is Equation 27. The problem it solves is that the distribution of a layer's inputs drifts as the layers before it are updated, which forces a small learning rate to stay stable. Holding the distribution fixed allows a much larger learning rate, which is what makes a twelve-layer stack trainable at all on a processor-only computer.

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \qquad y = \gamma\hat{x} + \beta, \qquad \varepsilon = 10^{-5}, \tag{27}$$

$$a = \frac{\gamma}{\sqrt{\sigma_{\text{run}}^2 + \varepsilon}}, \qquad b = \beta - a\,\mu_{\text{run}}, \qquad y = a\,x + b. \tag{28}$$

The batch mean and variance of Equation 27 only exist during training, because at inference a single line is classified on its own and there is no batch to average over. Running averages collected during training replace them, and because both are then fixed numbers, the whole operation collapses into the single multiply-and-add of Equation 28. The small constant ε prevents a division by zero in a channel that happens to be constant. Folding the normalisation into an affine map this way means the deployed code evaluates one multiplication and one addition per element instead of a mean, a variance and a square root, and Section 3.5.9 confirms the two forms agree numerically.

The rectifier is $\max(0,x)$ [30], which simply replaces every negative value by zero. Some nonlinearity between layers is essential, because a stack of purely linear layers collapses into one linear layer however deep it is, and could then not represent a letter shape. The rectifier is used rather than a smooth function because its derivative is either 0 or 1, so it neither saturates nor shrinks the gradient as it is propagated back through twelve layers.

Max-pooling takes the largest value in each 2×2 window with a stride of 2, which halves both axes and has no parameters to learn [13]. It does two things: it discards three quarters of the

positions, which is what keeps the network small enough to run on the target hardware, and it makes the features tolerant of a small shift, because moving a stroke by one pixel usually does not change which value in its window is the largest.

From a feature map to a sequence. At the end of stage 3 the feature map is 128 channels of 16 rows by 160 columns, and it has to become a sequence before a recurrent layer can read it. The 16 rows are collapsed by taking a maximum down each column, which is Equation 29. A maximum is used rather than a mean because it records only *whether* a feature occurs anywhere in that column and not how much of the column it fills, which is the right behaviour for handwriting: an ascender and a short letter should produce the same answer about which letter is present, even though they occupy very different fractions of the column height.

$$z_t[c] = \max_i F[c, i, t], \qquad t = 1, \dots, T = 160, \tag{29}$$

$$\ell_t = W_o h_t + b_o, \qquad \log y_t^k = \ell_t^k - \log \sum_{j=0}^{74} e^{\ell_t^j}. \tag{30}$$

The two pooling layers have each halved the width, so what is left is $T = 160$ time steps, each covering four pixels of the 640-pixel input. Equation 30 is the output layer. The linear map produces 75 raw scores per time step, called *logits*, which are unbounded real numbers and not yet probabilities. The softmax function exponentiates them and divides by their sum, which forces them to be positive and to add to one, so they can be read as a probability per symbol. The logarithm of that quantity is computed directly in the form shown, rather than by taking a softmax and then a logarithm, because e^{ℓ} overflows for a moderately large logit while the subtracted form stays finite. The loss of Section 3.5.4 needs the logarithm in any case.

The 74 symbols are the 26 lower-case and 26 upper-case letters, the ten digits and twelve symbols (space and . , ; : ’ " ! ? ( ) -). Class 0 is reserved for the CTC blank described below, which is why the head has 75 outputs for 74 symbols. Characters outside the set are removed from every label, and lines that become empty, or that exceed 120 characters and so indicate two lines merged into one, are dropped.

The recurrent layers. A plain recurrent network multiplies by the same weight matrix at every step, so over a long sequence the gradient either shrinks away or grows without bound, and the network forgets what it saw earlier. A long short-term memory cell [31] avoids this by holding a separate cell state c_t that is carried forward by addition rather than by multiplication, as $c_t = f_t \odot c_{t-1} + i_t \odot g_t$ and $h_t = o_t \odot \tanh(c_t)$, where $\odot$ is element-wise multiplication. Three learned *gates* control that state: the forget gate f_t decides how much of the previous state to keep, the input gate i_t how much of the new candidate g_t to add, and the output gate o_t how much of the state to expose as the step’s output. Because the state passes through an addition rather than a repeated multiplication, a gradient can cross many steps without vanishing. All four gate pre-activations are produced by a single matrix product with $W_{ih} \in \mathbb{R}^{1024 \times I}$ and $W_{hh} \in \mathbb{R}^{1024 \times 256}$ and then split into four blocks of rows, which halves the number of products per step (Figure 30).

A *bidirectional* layer runs a second, independent pass over the reversed sequence and concatenates the two outputs, $h_t = [\overrightarrow{h}_t ; \overleftarrow{h}_t] \in \mathbb{R}^{512}$, so every time step is informed by the whole line rather than only by what came before it. This is necessary for joined handwriting, where the up-and-down strokes of u, n and m are nearly identical locally and can only be told apart from what follows them. Two such layers are stacked, with no dropout, following the published design. These shapes cost 4.17×10^9 multiply–accumulates per line, 3.79×10^9 of them convolutional (Equation 74), so the convolutions dominate the arithmetic even though the recurrent layers hold

most of the parameters, while the recurrence is what makes the network inherently sequential.

3.5.4 Connectionist temporal classification

The network produces a probability for each of 75 symbols at each of its 160 time steps, but the transcription of that line has perhaps 40 characters. There are therefore four times as many outputs as there are characters, and nothing in the training data says which time steps belong to which character. That missing information is the *alignment*, and obtaining it by hand for every training line is exactly what makes character-level labelling impractical.

Connectionist temporal classification [9] removes the need for an alignment by refusing to commit to one. Instead of scoring the single alignment that happens to be correct, it adds up the probability of *every* alignment that would produce the right transcription, and trains the network to make that total large.

Two definitions make this precise. A *path* π assigns one symbol to every time step, where that symbol may be any of the 74 characters or a special 75th symbol called the *blank*, which means "no character ends here". The *collapse map* $\mathcal{B}$ turns a path into a string by first merging runs of the same repeated symbol and then deleting the blanks, so that

$$\mathcal{B}(\text{--hh-e-ll-ll-o}) = \text{hello}.$$

The blank is what makes a genuine double letter possible. Without it, the two l symbols of "hello" would merge into one when repeats are collapsed; with a blank placed between the two groups, the merge stops at the blank and both survive. This is the reason the output layer has 75 classes for 74 symbols.

Equation 31 is then the probability of one path, the probability of the transcription as the sum over all paths that collapse to it, and the loss, which is the negative logarithm of that probability so that maximising the probability becomes minimising a loss.

$$P(\pi \mid x) = \prod_{t=1}^{T} y_t^{\pi_t}, \qquad p(y \mid x) = \sum_{\pi:\mathcal{B}(\pi)=y} P(\pi \mid x), \qquad \mathcal{L}_{\text{CTC}} = -\ln p(y \mid x), \tag{31}$$

$$\hat{\pi}_t = \arg\max_k y_t^k, \qquad \hat{y} = \mathcal{B}(\hat{\pi}). \tag{32}$$

The number of paths in Equation 31 grows exponentially with T, so the sum can never be enumerated. It is instead evaluated by a forward–backward dynamic programme [9], which costs only $O(TU)$ for a transcription of U characters because every path must pass through one of $2U+1$ states at each step, so the paths can be summed state by state instead of one at a time. The same recursion also yields the gradient of the loss in closed form, as the difference between what the network predicted and what the alignment lattice expects, which means no derivative of the recursion itself ever has to be taken. That recursion, the three transitions it admits, and the logarithmic-domain arithmetic it needs to avoid numerical underflow over 160 steps are set out in Appendix 6.4, because they determine the gradient but not the architecture.

CTC imposes one requirement on the data, which is that a valid path must exist at all. Since a blank may be needed between adjacent repeated letters, the number of time steps must satisfy $T \geq U$ plus one extra step for every such repeat. The label cap of 120 characters, which is $\lfloor 160 \cdot 0.75 \rfloor$, keeps every transcription far enough below $T = 160$ for this to hold with margin.

At inference the alignment is still unknown, but it no longer matters, because only the string

is wanted. The most probable symbol at each step is taken, repeats are merged and blanks are deleted, which is the best-path or *greedy* decoding of Equation 32, given as pseudocode in Figure 33. This is an approximation: it returns the most probable *path*, whereas the most probable *string* would require summing over paths again. No lexicon, language model or beam search is used anywhere, so nothing corrects a misspelling. The published design reaches its best error rate with lexicon-constrained decoding, which is not implemented here, so its headline figure is not a result this project can claim.

3.5.5 Training data and splits

Four data sets were used: lines cut from scanned database pages by this project's own segmentation block with labels generated by alignment, the public line-level IAM transcription set with its official partitions (6 481 training, 976 validation and 2 915 test lines after label cleaning), photographed pages of the personal writers split per page, and a ten-author hold-out used for evaluation only. Table 29 in Appendix 6.4 states each split rule with the leakage it leaves, which matters because *no* figure in this report rests on a split that is both writer-independent and labelled by hand: the public test partition is the closest and is the only set used for no decision. The one defect worth stating here is that label quality, not the network, limited the early measurements — correcting the evaluation labels alone raised the ten-author hold-out accuracy from 42.7 % to 89.4 %, which is why the recogniser was later trained on a public line-level set with clean labels [3]. *Augmentation* is the practice of distorting each training image slightly and differently every time it is used, so that the network sees many variations of a line instead of the same one repeatedly and cannot simply memorise it. Only the training images are augmented, never the validation or test images, since those must stay fixed for the comparison between epochs to mean anything. At most one of four geometric transformations is applied per image: a horizontal *shear*, which leans the writing and so simulates a different slant; a small *rotation*, which simulates a page not quite square to the camera; an *elastic distortion*, which displaces each pixel by a small random amount drawn from a smoothly varying field, so that strokes bend and thicken locally the way a different hand would write them; and a *perspective* distortion, which moves the four corners of the image independently and so simulates a camera not looking straight down. All four are chosen because each one mimics a variation that really occurs in this data, which is the test of whether an augmentation is worth applying. Their magnitudes are listed in Appendix 6.4. No colour, contrast, noise or blur augmentation is used, and no synthetic pre-training data.

3.5.6 Training procedure

The loss is minimised by mini-batch gradient descent with the RMSprop update of Equation 33 [32], in which g_t is the gradient at update t, $\lambda = 10^{-5}$ a weight decay added to it, v_t the running average of the squared gradient with smoothing $\rho = 0.99$, η the learning rate and $\varepsilon = 10^{-8}$; there is no momentum, and the per-parameter step suits the very different gradient magnitudes of convolutional and recurrent layers. In the joint stage the global gradient norm is clipped at 5 by Equation 34 and batches with a non-finite loss are skipped; a plateau schedule halves the learning rate after a number of validation checks without improvement,

$\eta \leftarrow \max(0.5\,\eta, \eta_{\min})$.

$$g_t \leftarrow g_t + \lambda\,\theta_{t-1}, \qquad v_t = \rho\,v_{t-1} + (1-\rho)\,g_t^2, \qquad \theta_t = \theta_{t-1} - \eta\,\frac{g_t}{\sqrt{v_t}+\varepsilon}, \tag{33}$$

$$g \leftarrow g\,\min\!\left(1, \frac{5}{\|g\|_2}\right). \tag{34}$$

Training ran in the four stages of Table 8, each motivated by a limitation of the previous one: the base stage reproduced the published recipe on lines cut by this project's own tools, where the labelling defect made the data noisy; the clean-label stage warm-started from it on the public set; and fine-tuning on the personal lines alone was rejected because it cut the general accuracy by about ten points, which is catastrophic forgetting, barely reduced by freezing the convolutions. The joint stage therefore continues on both sets at once with a weighted sampler: each public line gets weight $(1-f)/n_T$ and each personal line f/n_P with $n_T = 6481$, $n_P \approx 280$ and $f = 0.20$, and n_T indices are drawn per epoch with replacement, so about 1 296 draws per epoch are personal and each personal line is seen about 4.6 times with fresh augmentation. The checkpoint of the epoch with the largest $\min(\text{accuracy}_{\text{public}}, \text{accuracy}_{\text{personal}})$ is kept, so that neither domain is sacrificed.

Stage	Data	η_0, batch, epochs, clip	Schedule and selection	Checkpoint and character accuracy
1 Base	own lines	10^{-3}, 16, ≤ 200, none	halve after 5 stalls; stop after 15; up to 3 restarts from the best weights; best validation loss	best validation loss 1.06 per line
2 Clean labels	public set	10^{-3}, 16, ≤ 200, none	as above, up to 8 restarts; best public validation loss	`hf_best`: 93.87 / 91.60 % public validation / test
3 Personal	personal lines	3×10^{-4}, 8, 60, 5	halve after 5 stalls; best personal validation loss	`personal_best`: 84.26 / 80.56 % public, 90.2 % personal — rejected
3a Personal, convolutions frozen	personal lines	as stage 3	as stage 3	85.87 / 82.62 % public, 88.5 % personal — forgetting only slightly reduced
4 Joint	both, f = 0.20	5×10^{-4}, 16, 30, 5	halve after 4 stalls on the sum of both validation losses; best min of the two accuracies	`joint_best`: 94.03 / 91.82 % public, 89.64 % personal

Table 8. The four training stages with the catastrophic-forgetting variant 3a, and the character accuracies measured on each stage's own checkpoint file. All stages use RMSprop (Equation 33) with weight decay 10^{-5} and a minimum learning rate of 10^{-5} or 10^{-6}. Checkpoint names abbreviate `paper_cnn_bilstm_ctc_*.npz`. None of these figures was measured on `paper_cnn_bilstm_ctc_scratch_best.npz`, the file the deployed loader prefers.

Four text checkpoints exist and the quoted accuracies belong to three of them (Table 8). The fourth, `paper_cnn_bilstm_ctc_scratch_best.npz`, is the output of the from-scratch trainer of Section 3.5.7 and is the file the deployed system actually uses, because the loader prefers it over the joint checkpoint whenever it exists, which it does. This matters for every accuracy here, because no accuracy of the from-scratch checkpoint is reported anywhere in this report: its training produced no log and its file holds no epoch number and no best validation score. The honest position is therefore that the four-stage recipe and its accuracies describe the joint checkpoint, while the deployed path loads a later checkpoint of the same architecture whose accuracy has not been measured. **[MUST STILL BE COMPLETED BY ME: either evaluate `paper_cnn_bilstm_ctc_scratch_best.npz` on the sets of Table 9 and quote its figures beside the joint ones, or name the single checkpoint being submitted and remove**

the other from the loader's search path.] A sweep over learning rate, LSTM width, depth, convolution width, dropout and optimiser (14 configurations of 50 epochs) was programmed but no result file kept, so the deployed architecture is the published one and not an optimum found here; the joint stage's training log was not kept either.

3.5.7 From-scratch forward-and-backward implementation

One module, `NumpyNN.py`, implements from the defining equations the forward and the backward pass of every layer used here and by the writer classifiers of Section 3.6 — the convolution of Equation 25 with its im2col form, batch normalisation in both forms, the rectifier, max-pooling and the two adaptive pools, a fully connected layer, dropout, the multilayer bidirectional LSTM and the CTC recursion — plus the update of Equation 33, the writer classifiers' AdamW [33], the two schedules and the clip of Equation 34. One shared training program calls it to build either the recogniser or a writer classifier, since the two share the convolutional backbone, and the same functions run in inference form in validation and on the board; Figure 32 gives that training step as pseudocode, including the transpose at which the width axis of the feature map becomes the CTC time axis. The claim is narrower than "no external library at all", which would not be true: `NumpyNN.py` imports nothing but `numpy`, while the data pipeline uses two SciPy image utilities and only two, `gaussian_filter` and `map_coordinates` for the elastic augmentation and `find_peaks` for row-ink minima when database pages are cut, plus the imaging library for file reading; neither touches a weight, a gradient or an update.

The backward pass of each layer is the standard gradient result for that layer, published with the layer itself and not derived here; what this project contributes is that every one of them is written out and evaluated by its own code instead of being supplied by a framework. Appendix 6.4 lists those gradients, Equations 78 to 80, together with the two places where the risk lies in the implementation rather than in the algebra: the col2im scatter-add of the convolution, which must accumulate because one input pixel appears in up to nine patches, and the reversed direction of the bidirectional layer, whose incoming gradient must be reversed and whose returned gradient reversed back. Both would be wrong silently — the first keeps one contribution in nine, the second produces a gradient wrong only in time ordering — and neither would disturb a falling loss curve.

Because no automatic differentiation tool is available to check them, every backward pass was verified against a central-difference estimate of the same gradient, Equation 81, and accepted only when the relative difference of Equation 82 stayed below 10^{-6}. Of the 78 configurations of layer variant and tensor, **77 passed**, with relative differences between 4.9×10^{-12} and 1.4×10^{-10}, four to six orders of magnitude inside the tolerance; 2044 individual tensor elements were probed in double precision. The protocol, the precautions taken at the kinks of the rectifier and the max-pool, and the per-element worst case are in Appendix 6.4.

The one configuration that failed is a genuine defect, reported here rather than removed. The CTC loss does not survive an empty label sequence: the blank-interleaved target then has one entry, while two of the arrays carrying the recursion are built with two entries regardless, so the routine raises a shape error instead of returning the loss. The fault is confined to the lattice indexing and touches neither the transition rule nor the gradient; a single-character target and a repeated-character target, which exercises the forbidden skip, both pass. Its practical effect is small, because a line with no transcription never reaches the trainer, but the function is not safe

against one and the fault is listed in Section 5.

With no graphics processor and no vectorised automatic-differentiation engine, one training step of a batch of 16 real lines took tens of seconds on a laptop processor, the dominant cost being an un-vectorised convolution rather than the recurrent layers or the CTC loss. Training is not run to convergence from a random initial condition at this speed: the program instead loads every tensor of an existing checkpoint whose shape matches, a warm start.

3.5.8 Evaluation metrics and measured accuracy

Accuracy is measured with the Levenshtein edit distance, computed by the dynamic programme of Equation 83, and reported as one minus a character error rate micro-averaged over all lines, Equation 84: the total edit distance over the total reference length, case-sensitive. The word measure is a longest-common-subsequence recall and not 1 − WER. Both are defined in Appendix 6.4, where it is also shown that Section 4 averages a *floored, lower-cased, per-line* accuracy instead, so the subsystem figures of this subsection and the qualification figures of Section 4 are not computed identically and should not be differenced.

Data set	Result	Status of the figure
Public set, validation (976 lines)	94.03 % character; WER 21.1 %	Used for model selection, therefore mildly optimistic.
Public set, test (2 915 lines)	91.82 % character; WER 26.7 %; 15.9 % of lines exactly right	Used for no decision; the cleanest figure for the general benchmark, with the partition caveats of Table 29.
Personal crops, 669 correctly cut lines	98.61 % character, 94.21 % word	Measured on crops confirmed to match the label count, so segmentation failures are excluded, and the writers appear in the fine-tuning data. This is the operating figure of Section 4, not an end-to-end page figure.
Personal crops, per writer	Best Owen 99.15 %, worst Thiya 96.46 %	75 to 206 crops each; Thiya is the weakest hand for recognition. Full breakdown in Table 21.
Ten personal photographed pages, full page pipeline including segmentation and label alignment	61.21 % (CER 38.79 %, 3 617 of 9 324 characters)	Six unique pages (four listed twice); measured with an earlier recogniser, before the personal pages were used for fine-tuning. The only figure that exercises the complete page-to-text chain.
Recognition speed, 669 crops	8.86 ms per character, 353 ms per line	Against the 25 ms per character of R4, a margin of about 2.8 times on the development laptop.

Table 9. Accuracy of the text recogniser. The public-set figures and the full-page figure predate the final training and are retained because they are the only ones measured on an independent partition or through the complete chain; the 669-crop figure is the current operating point. The two public-set rows were measured on `paper_cnn_bilstm_ctc_joint_best.npz`; no row was measured on `paper_cnn_bilstm_ctc_scratch_best.npz`, the checkpoint the deployed loader prefers.

Joint training kept the general accuracy while adding the personal writers, the test accuracy rising by 0.22 points over the clean-label stage, so the weighted sampler is justified. The word error rate is about four times the character error rate, so roughly one word in four contains a character error and only about one line in six is entirely correct: greedy decoding without a lexicon leaves spelling errors uncorrected. The gap between 98.61 % on correctly cut crops and 61.21 % through the complete page pipeline is the largest discrepancy in this report, but the comparison is not clean: the two figures used different recogniser versions and page sets, and the full-page pages were later fine-tuned on, so an unknown part of the 37-point difference is

recognition improvement. What can be stated is that the operating figure's crops were hand-checked whereas the full-page path has no such guarantee, so the difference bounds the combined effect of segmentation and label alignment from above and cannot be charged to recognition, which is 98.61 % given correct crops; splitting the two needs the full pipeline re-run with the current recogniser on unseen pages, as Section 5 proposes.

3.5.9 Inference on the single-board computer, and alternatives

The single-board computer runs the eval-mode form of the same module, so no re-implementation was needed between training and deployment: each convolution is the matrix product of Equation 26 and batch normalisation the folded form of Equation 28. Before that form was trusted it was cross-checked against the training-mode forward pass on the same weights with the statistics frozen: over 1358 real lines the top-1 agreement was 100.00 % with a maximum absolute logit difference of 7.6×10^{-6}. Recognition measured 8.86 ms per character and 353 ms per line over 669 real crops, inside the 25 ms per character of requirement R4 by about 2.8 times, with the model already resident in memory on the development laptop. Timing on the ODROID N2+ was not measured, and its ARM processor is expected to be slower, so the margin should shrink on the target hardware. **[MUST STILL BE COMPLETED BY ME: measure the recognition time per line on the ODROID N2+; the project files hold none, and the proposal's budget may cover the whole classification step.]** Compiled replacements for the convolution and the LSTM exist but are not used: the compiled convolution was slower than the optimised matrix product, and the compiled LSTM 2.8 to 4.4 times faster.

Table 10 compares the design with the alternatives considered; only the chosen design was built and measured, so the statements about the others are arguments. It fits the four facts of the problem: an output sequence of unknown length (CTC), cursive strokes needing context from both sides (bidirectional recurrence), line transcriptions as the only labels (an alignment-free loss), and a model small enough to re-implement on the single-board computer.

Alternative	Argument for, and against
Character segmentation and a per-character classifier	Simple network and the proposal's plan, but it needs a segmenter that works on joined letters, classification errors cannot be repaired by context, and it needs character-level labels the available data lack.
Line-level CNN-BiLSTM-CTC (chosen)	No segmentation below the line, alignment-free training from line transcriptions, context from both sides, a string as output, and 3.36 million parameters that a from-scratch implementation can run. Against: it needs line segmentation, greedy decoding leaves spelling errors, and CTC assumes independent steps.
Lexicon or language-model decoding	Repairs near-miss words and gives the lowest error in the published design, but needs a corpus and a decoder, and the technical vocabulary and symbols of these pages are largely out of vocabulary. Future work.
Deeper backbones with skip connections	Easier training of very deep networks, but the 12-layer network with batch normalisation trained well and larger models cost memory and time on the single-board computer.
Attention encoder–decoder or transformer	Strong accuracy on large corpora, but far more parameters, large pre-training data, slow autoregressive decoding that can skip or invent text, and hard to implement from scratch for the board.
Word-level recognition	Uses the lexicon directly, but needs word segmentation and a closed vocabulary and is poor for names, numbers and symbols.

Table 10. Alternatives to the line-level recogniser.

3.6 Writer identification

This subsection describes the two neural networks that decide which of the thirteen stored writers wrote a line or a page (functional unit FU 2.3, requirement R2) and that judge how closely synthesised handwriting resembles a writer (requirement R1). Both share the recogniser's convolutional backbone, whose layer pattern follows Kizilirmak and Yanikoglu [10] and is not an architecture originated here; the pooling and classification head, the stroke normalisation, the aggregation rule, the two trainers and the inference code were written for this project.

3.6.1 Why two classifiers were needed

The task is closed-set: choose one of $K = 13$ known writers from a line image, the eight database writers 150, 151, 152, 153, 384, 551, 552 and 588 and five personal writers whose pages were photographed for this project. A classifier trained on raw grey-scale crops read real held-out lines at 99.78 % over 1358 lines but identified the writer of *synthesised* output in only about 28 % of trials, because the plotter draws every stroke at one width and one pressure while real handwriting carries a thickness and an ink density that a convolutional network exploits readily when training lines are few, and such a judge predicts neither what the machine can draw nor how a reader recognises a hand, which is by geometry. A second classifier was therefore trained on stroke-normalised crops (Section 3.6.3), in which thickness is removed by construction: the ink classifier serves real photographed pages, the shape classifier the best-of-N search of Section 3.8. Both have all 966 445 parameters trained, the ink classifier from the recogniser backbone and the shape classifier end to end; an earlier configuration fitted only the 13 645 parameters of the shape classifier's head on features of a frozen backbone, which was a training fault, superseded but still the subject of the fault analysis below.

3.6.2 Architecture

The crop is brought to the fixed 64×640 canvas and normalised exactly as for the recogniser (Equations 23 and 24), then passed through the same 12-layer convolutional stack, which produces a feature map $F \in \mathbb{R}^{128 \times 16 \times 160}$. The recurrent layers are not used here. The recogniser needs them because reading text depends on the order the letters come in, whereas writer identity does not: a writer's slant and stroke habits are the same whatever the sentence says, so the sequence information would only add parameters and cost.

What the head has to do is turn that feature map into one decision for the whole line, which means the two spatial axes must be removed. This is done by *pooling* twice, in Equation 35, and the two pools deliberately use different operations.

$$m_{c,t} = \max_{r=1,\dots,16} F_{c,r,t}, \qquad f_c = \frac{1}{160} \sum_{t=1}^{160} m_{c,t}, \qquad f \in \mathbb{R}^{128}, \tag{35}$$

$$h = \mathrm{ReLU}(W_1 f + b_1) \in \mathbb{R}^{96}, \qquad z = W_2 \,\mathrm{Drop}_{0.2}(h) + b_2 \in \mathbb{R}^{K}, \tag{36}$$

In Equation 35, c, r and t are the channel, row and column of F. A *maximum* is taken down the height, because the question there is whether a stroke pattern occurs anywhere in that column, which is the same reasoning as in the recogniser. A *mean* is taken along the width, because the

question there is how *often* that pattern occurs along the line. A maximum along the width would answer only whether the writer ever made that stroke once, which a single accident of handwriting could satisfy, whereas the mean measures a habit. The result is a position-independent summary: 128 numbers saying how commonly each learned stroke pattern appears, with no record of where. That is the right representation for this task and the wrong one for reading text.

One consequence of the mean is worth stating, because it is an assumption the design makes. A short line is padded with white to fill the canvas, and that padding contributes zero-feature columns to the average, so a short line's feature vector is diluted towards zero compared with a full line's. This is accepted rather than corrected, on the grounds that it happens identically during training and during use, so the network learns to work with it.

Equation 36 is the head itself: one hidden layer of 96 units with a rectifier, then a linear layer producing one score per writer, $K = 13$ here. *Dropout* is applied between the two layers, which during training sets a random 20 % of the hidden values to zero on every step and rescales the rest. The purpose is to stop the head depending on any single hidden unit, because with only 63 to 184 lines per writer a small head can otherwise memorise individual training lines rather than learning the writer's hand. Dropout is active only during training and is skipped entirely at inference, where a deterministic answer is wanted. Together the two pools and two layers replace the recogniser's recurrent stack and CTC head, giving 966 445 trainable parameters of which only 13 645 belong to the head, so almost all of the capacity is the shared backbone. Table 30 in Appendix 6.4 resolves that count, and Figure 34 gives the deployed forward pass, which the trainer also runs for validation.

The training objective. The 13 scores z produced by Equation 36 are logits, as in the recogniser, so they are turned into probabilities by a softmax and the network is trained to put as much probability as possible on the correct writer. The measure of how badly it is doing is the *cross-entropy* loss, which is the negative logarithm of the probability assigned to the true class. It is used rather than, say, counting mistakes, for two reasons: it is differentiable, so it can be optimised by gradient descent, and it punishes confident errors far more than hesitant ones, because the logarithm goes to infinity as the probability of the true class goes to zero. Equation 37 gives the softmax, the loss and the decision rule.

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad \mathcal{L} = \frac{1}{B}\sum_{n=1}^{B}\Big[\log \sum_{j} e^{z_j^{(n)}} - z_{y_n}^{(n)}\Big], \qquad \hat{k} = \arg\max_{k} p_k, \tag{37}$$

$$\frac{\partial \mathcal{L}}{\partial z_k^{(n)}} = \frac{1}{B}\big(p_k^{(n)} - \mathbb{1}[k = y_n]\big), \tag{38}$$

Here B is the batch size, y_n is the true writer of sample n and $\mathbb{1}[\cdot]$ is the indicator function, which is 1 when its condition holds and 0 otherwise. The loss is written in the combined form shown rather than as $-\log p_{y_n}$ for the same numerical reason as the log-softmax of the recogniser: the exponential of a large logit overflows, while the subtracted form does not. No class weights and no label smoothing are used, so every writer and every line count equally.

The gradient of this loss is unusually simple, and Equation 38 is the whole of it: the predicted probability minus 1 for the correct class, and the predicted probability itself for every other class. It is taken with respect to the logits z rather than the probabilities p, which avoids having to differentiate the softmax separately and avoids dividing by a probability that may be very close to zero. Equation 38 is where the backward pass begins, and from there the chain runs back through the head, the two pools and the backbone using the layer gradients of Section 3.5.7.

3.6.3 Stroke normalisation

Stroke normalisation re-draws a line at a constant pen width proportional to its own letter height, so the thickness cue is identical for every writer and for synthesised lines. On the full-resolution crop, before the resize, it applies six steps: Otsu binarisation [28] (Equation 13); removal of ruled lines and underlines by the run-length test of Equation 39; the pixel x-height from the row ink profile $P(r)$ by Equation 40; closing of hairline breaks by a square element of side $\mathrm{clip}(\mathrm{round}(0.045h), 2, 4)$, components under 6 pixels being dropped; thinning; and re-inking at one width.

$$H_{\mathrm{run}} \geq \max(24, \lfloor 1.1h \rfloor) \quad \wedge \quad V_{\mathrm{run}} \leq \max(3, \lfloor 0.12h \rfloor), \tag{39}$$

$$x_h = \mathrm{base} - \mathrm{top}, \qquad \text{or } x_h = \max(8, h/3) \text{ if no valid band exists or } x_h < 4. \tag{40}$$

Here h is the crop height and H_{run}, V_{run} the horizontal and vertical ink-run lengths through the pixel, so a rule pixel crossed by a vertical stroke is kept, and the core band of Equation 40 runs from the first to the last row with $P(r) \geq 0.45 \max P$, the dense band of the lower-case bodies. The width is $w = \mathrm{clip}(\mathrm{round}(0.14 x_h), 2, 9)$ with odd structuring-element side $k = 2\lfloor w/2 \rfloor + 1$, independent of the scan resolution ($x_h = 20$ gives $w = k = 3$). Thinning to a one-pixel centre line uses the iterative Zhang–Suen rule [34], which deletes a pixel only when it has two to six ink neighbours and exactly one background-to-ink transition around its neighbour ring, so stroke topology survives; the skeleton is then dilated back to width k (Figure 35 gives the six steps as pseudocode). Slant, letter proportions, spacing and connection habits survive, and every input of the shape classifier passes through the same transform, so real training lines and synthesised test lines share one geometry-only distribution; the ink classifier does not use it. The renderer draws at 0.085 x-heights and the judge re-draws at 0.14, and because the judge re-draws every input itself the two constants need not match. **[MUST STILL BE COMPLETED BY ME: a figure of the six steps on real lines: grey crop, Otsu mask with rules removed, core band with x_h, skeleton, re-inked line and final canvas, for one database and one personal writer. Source: apply `StrokeNormalize` to crops from `Data/Datasets/IAMpages10` and `Software/CNN/NOGIT/dylan`.]**

3.6.4 Training data, protocol and the training fault

The thirteen writers, three more than the proposal requires, are the eight database writers at ten pages each, except where a page failed the crop-to-label check, and five personal writers photographed as the project progressed. The stroke-normalised set holds 689 database-writer lines and 669 personal-writer crops, 1358 in all, split into 920 training and 123 validation lines, 63 to 184 per writer. The two kinds are split differently: a database writer's last labelled page is held out whole, the intended closed-set task, giving 8 to 11 validation lines each, whereas a personal writer's lines are shuffled on a fixed seed and 15 % become validation lines, with the consequences recorded as items 1 and 2 of Section 3.6.6. The recogniser's augmentation, at most one of shear, rotation, elastic and perspective distortion per image, was inherited untuned (item 3). Both classifiers were trained with AdamW, the decoupled-decay form [33] of Adam [35], and cosine annealing, implemented in the recogniser's own layer library from the defining equations (Equations 85 and 86 in Appendix 6.4). Two properties of that choice matter to the design rather than to the arithmetic: the weight decay sits in the update and not in the gradient, so every parameter shrinks by the same fraction regardless of how large its gradient happens to be, and

the norm clip of Equation 34 is applied *before* the moment update, so that one clipped batch cannot contaminate the running averages for the many steps over which they decay. The ink classifier ran at $\eta_0 = 10^{-3}$ for 20 epochs on batches of 16 images with the clip at 5; the superseded frozen-head run used $\eta_0 = 3 \times 10^{-3}$, 500 epochs, batches of 64 feature vectors, no clipping and only the 13 645 head parameters on 3 680 stored vectors, and is the configuration that reached 83.36 %.

A backbone holds 84 tensors, the convolution weight and bias and the five batch-normalisation arrays of each of the twelve convolutional units, and a tensor-by-tensor comparison shows that all 84 differ between the recogniser, the ink classifier and the shape classifier, so the three backbones are three distinct sets of weights. For the ink classifier that is what full fine-tuning requires; for the shape classifier it contradicts the original frozen-backbone recipe, which was itself a training fault, because no augmentation was re-sampled between epochs and each line contributed the same four fixed feature vectors for all 500 epochs. End-to-end training raised the shape classifier from 83.36 % to 93.67 % on the same 1358 real lines, where 118 of one writer's 148 lines had previously gone to a single neighbour, so the frozen-head figures describe the superseded model only. Both trainers write every tensor into one plain array archive keyed by layer name, with no object serialisation.

3.6.5 Page decisions, evidence and alternatives

On a real photograph the ink classifier is applied to every line the segmentation block delivers and the line probabilities are averaged, Equation 41,

$$\bar{p}_k = \frac{1}{N} \sum_{n=1}^{N} p_k^{(n)}, \qquad \hat{k} = \arg\max_k \bar{p}_k, \qquad \text{confidence} = 100\, \bar{p}_{\hat{k}}\ \%, \tag{41}$$

with N the number of detected lines; the writer, the confidence and all thirteen page probabilities return to the browser. Lines carry equal weight, not weighted by length or confidence, because averaging resists a few confidently wrong lines better than a majority vote and uses all the probability mass, although the two were never compared. For synthesised output each rendered sentence instead gives one softmax and the target writer's probability becomes one score in the best-of-N search of Section 3.8.

Table 11 lists the five measurements that motivated the design and the status of each; chance level for thirteen writers is 7.7 % and the qualification figures belong to Section 4. Neither classifier is redundant: the same hand is identified at 99.78 % by one instrument and 93.67 % by the other, which is a property of the instruments and not of the handwriting. The corrected shape classifier's errors are scattered, eleven of the thirteen writers scoring 98.9 % or above.

Four alternatives were weighed and none built, so they are arguments and not measurements: hand-crafted features with nearest-profile classification, interpretable and already half-measured by the profile builder, but needing feature selection and weak when two hands agree in coarse statistics; contour-direction and allograph statistics [14], text-independent and strong for large writer sets but hand-built and less direct as a judge of constant-width output; metric learning and deep fragment networks, open-set and scalable to hundreds of writers but needing far more than the thirteen writers and about 1 000 lines available; and pen-width augmentation of the ink classifier, which would avoid a second network but reduces rather than removes the cue the normalisation removes by construction, no ablation having been run. A single network with a text head and a writer head was in fact tried: writer accuracy reached 98.9 to 100 % by epoch 20

Measurement	Result	Status of the figure
Ink classifier, 1358 real lines	99.78 %	689 database and 669 personal crops; the live server's classifier for photographs.
Shape classifier, same lines, fault corrected	93.67 %	End-to-end training instead of a frozen backbone with a fitted head.
Shape classifier, same lines, before correction	83.36 %	118 of one writer's 148 lines went to a single neighbour.
Ink classifier on synthesised output	28 % over 39 drawn lines	Cannot judge constant-width output; the reason the shape classifier exists.
Shape classifier on synthesised output	89.7 % through G-code (13 writers, 3 sentences, best-of-8); 80.1 % at best-of-30	Circular: the same classifier family selects the candidates (Section 3.6.6, item 6).

Table 11. Evidence for the writer classifiers. Chance level for thirteen writers is 7.7 %, and none of these are held-out test accuracies.

while text accuracy stalled at 28 to 29 %, so separate single-task networks were adopted. The classifiers run on the single-board computer with the recogniser's own layer functions, so the trainer's validation pass and the server's page pass are the same code and need only the gradient check of Section 3.5.7. **[MUST STILL BE COMPLETED BY ME: execution time of one classifier pass on the ODROID N2+.]**

3.6.6 Known faults and limitations

1. There is no untouched test set: the best-validation checkpoint is also the reported one, so the same 123 lines serve for selection and for reporting and one line is worth 0.81 percentage points. Table 11 does not hold held-out test accuracies.
2. A personal writer's training and validation lines share pages, pen, paper and session, which a line classifier can exploit, so the figure is inflated for 52 of the 123 validation lines; holding out whole personal pages is the mitigation that was not carried out.
3. The inherited augmentation was never tuned and distorts the cues being classified: ±0.6 of shear is about ±31° against measured writer slants of −7.5° to +42°.
4. The confidence of Equation 41 is an uncalibrated softmax probability from networks trained to a near-perfect fit, so over-confidence is expected.
5. The deployed end-to-end run's settings, best epoch and epoch count were not recorded, for either set of stored weights.
6. The set is closed with no unknown-writer class, the validation sets are small, the personal writers' geometry cues may be confounded with the collection session, and the shape classifier is also an objective of the generator (Section 3.8), so its verdict on synthesised output is not independent.
7. The server lets the shape classifier take the default checkpoint and the loader prefers a from-scratch name that does not exist, so the default resolves to the stroke-normalised weights, which is correct; running that trainer would silently change the judge. **[MUST STILL BE COMPLETED BY ME: either remove the unused first entry from the author-checkpoint search order or make the server name the stroke-normalised file explicitly, as it already does for the ink classifier, so that the judge cannot change model by accident.]**
8. Writer owen remains a systematic error, 48 lost lines belonging to him alone, unsolved in Section 5.1.3.

3.7 Style extraction and glyph library

This subsection describes how a writer's style is learned from photographs of that writer's pages and stored permanently (functional units FU 2.5 to FU 2.7; requirements R1 and R6). Training a writer trains no network: a *style profile* is one text file built once per writer, holding a library of that writer's own letter shapes, each reduced to a thin centre-line pen path called a *glyph*, plus measured numbers such as slant, letter height, spacing and how often letters are joined, after which no page images and no networks are needed to write in that style. The builder is an offline program written for this project which uses the recogniser of Section 3.5 for inference only; its input is the line crops and transcriptions of Section 3.4, ten pages each of the eight database writers with the last labelled page held out so that no evaluation text enters a profile, and the notebook pages of the five personal writers abhinav, dylan, owen, thiya and yeukita with 15 % of their lines held out. It runs twelve stages in strict order, with no branch and no loop, and the subsections below take them in that order: a line crop and its transcript go to the recogniser for its 160 log-probability vectors; those are forced-aligned by Equation 42; the character territories are cut at thin low columns by Equation 44; the line is binarised, stripped of rules and thinned as a whole; the skeleton is traced into chains, spurs pruned and junctions re-joined; the chains are split at the territories and simplified; each is normalised to the x-height frame of Equation 46; the quality gates and the letter prior of Equation 47 are applied; the surviving variants, the medoid and the denoised prototype are assembled; the style statistics are measured; and one JavaScript Object Notation profile per writer is written.

3.7.1 Forced alignment: where is each character?

To cut a letter out of a line, the builder has to know where that letter is. In joined handwriting a character has no visible boundary, and no per-character labels exist for any of the data, so the position has to be inferred. What is available is the transcript of the line and a recogniser that reads lines, and the two together are enough.

Forced alignment is the name for using a trained recogniser backwards. Normally the recogniser is asked what the line says; here the answer is already known, and it is instead asked *where* it saw each character of that known answer. The word "forced" refers to this: the alignment is constrained to produce the transcript that is known to be correct, so the recogniser is not permitted to decide that the line says something else. The output is an assignment of each of the $T = 160$ time steps, each covering a 4-pixel slice of the line, to one character of the transcript.

The quantities involved are the transcript $y = (y_1, \ldots, y_U)$, its blank-augmented form $y' = (\varnothing, y_1, \varnothing, \ldots, y_U, \varnothing)$ of length $S = 2U + 1$, which is the same construction CTC uses, and the log-probabilities $\ell_t(k) = \ln y_t^k$ the recogniser produces.

The difference between this and the training loss is one operator. Where the CTC loss of Equation 75 *sums* the probability over every alignment, because for training it does not matter which alignment was right, Equation 42 takes the *maximum* instead and keeps only the single most probable one, because here the alignment is precisely what is wanted. This is the Viterbi algorithm [36], and $\delta_t(s)$ is the log-probability of the best path that reaches position s of y' at

step t:

$$\begin{aligned}
&\delta_1(1) = \ell_1(\varnothing), \quad \delta_1(2) = \ell_1(y'_2), \quad \delta_1(s) = -\infty \ (s > 2),\\
&\delta_t(s) = \ell_t(y'_s) + \max\{\delta_{t-1}(s),\ \delta_{t-1}(s-1),\ \delta^{\text{skip}}_{t-1}(s)\},\\
&\delta^{\text{skip}}_{t-1}(s) = \begin{cases} \delta_{t-1}(s-2) & s \geq 3,\ y'_s \neq \varnothing,\ y'_s \neq y'_{s-2},\\ -\infty & \text{otherwise.}\end{cases}
\end{aligned} \tag{42}$$

Because the maximum replaces the sum, the path itself can be recovered. Each step records which of the three predecessors won, and following those *back-pointers* backwards from the end gives the whole best path at a total cost of $O(TS)$. The path is started from whichever of the last character or the final blank ends with the higher score, and character i then owns every step whose winning position is $s^*_t = 2i$, the odd positions being the blanks between characters.

The alignment is only as trustworthy as the recogniser's reading of that line, so lines the recogniser never really read must be discarded before their letters enter the library. The measure used is the *alignment confidence* $\text{conf} = \frac{1}{T}\sum_t \ell_t(y'_{s^*_t})$, which is the mean log-probability along the chosen path. A line where the recogniser was confident at every step scores near zero, and one where it was guessing scores strongly negative. The threshold is relative rather than absolute: a line is dropped when its confidence falls below the 25th percentile of that same writer's lines, provided at least eight lines survive. A per-writer threshold is used because the absolute confidence depends on how well the recogniser handles that hand in the first place, so one fixed cut-off would discard almost everything from the most difficult writers. On writer 153 this discards 22 of 88 lines.

The recogniser input is left-aligned at the uniform scale of Equation 23, so a frame boundary converts back to a crop pixel exactly, by Equation 43,

$$x_{\text{px}}(a) = \frac{640\,a}{T\,s}, \qquad s = \min\Big(\frac{64}{h}, \frac{640}{w}\Big), \tag{43}$$

for a crop of $h \times w$ pixels, about 6 pixels per frame on a 980-pixel line. The span a CTC model assigns to a character cannot be used directly as the letter's extent. A CTC network is free to emit a character at any one step inside its true extent and blanks elsewhere, and in practice it emits a narrow sliver that lags slightly behind the ink. The spans are therefore used only for their *centres* c_k, which are reliable, and the provisional boundary between two characters is taken as the midpoint between consecutive centres, $\hat{x}_k = \frac{1}{2}(c_{k-1} + c_k)$.

That midpoint is only an estimate, and cutting there would often fall in the middle of a letter. In joined writing the only place where one letter honestly ends and the next begins is the ligature, the thin connecting stroke between them, and a ligature has two measurable properties: it is *thin*, carrying little ink in its column, and it is *low*, sitting near the baseline rather than high in the x-height band. The cut is therefore moved to the cheapest such column within a radius r of the estimate, by Equation 44:

$$\begin{aligned}
&\text{cost}(x) = \text{thin}(x) + 1.15\,\text{top}(x), \qquad r = \max\big(3, \text{round}(0.5x_h)\big),\\
&x^* = \arg\min_{|x-\hat{x}_k| \leq r}\Big[\text{cost}(x) + 0.6\,\frac{|x-\hat{x}_k|}{r}\Big],
\end{aligned} \tag{44}$$

The two terms of the cost are the two properties of a ligature. Here $\text{thin}(x)$ is the smoothed column ink count divided by its own 60th percentile over the non-empty columns, so it is near 1 in a typical column and below 1 in a thin one, and dividing by a percentile of the line itself makes the measure independent of pen thickness. The term $\text{top}(x)$ is the highest ink in that column above the baseline, measured in x-heights and limited to 3, which is large in a column containing

an ascender and near zero in a column containing only a low connecting stroke. The weight of 1.15 on top makes height slightly more important than thinness, because a thin column high up is more likely to be the upstroke of a tall letter than a ligature. The cost is forced to zero in a column with no ink at all, since a genuine gap between two unjoined letters is always the best possible cut. The final term in x^* is a penalty proportional to the distance moved, which keeps the cut near the aligned estimate when no column in range is clearly better than its neighbours. The quantity x_h is the x-height of Equation 40.

Two implementation points matter. The boundaries are written into one shared array and forced to be strictly increasing, because snapping each boundary independently can move two neighbours onto the same column and leave a character with no columns at all. Figure 36 gives the alignment and the cut cost as pseudocode.

The cuts are vertical, not along the writer's slant, and this was a measured decision rather than an obvious one. A slanted cut follows the natural geometry of a slanted hand and seems the better choice, but glyphs cut that way do not retile cleanly when reassembled at different spacings, because each one carries a sheared edge that only matches its original neighbour. Cutting along the slant lowered writer identification of the synthesised output from 81.8 % to 68.6 %, so vertical cuts were kept.

3.7.2 From ink to ordered pen paths

Each line is binarised and thinned by the very functions of Section 3.6.3 — the Otsu threshold [28], the rule removal of Equation 39, the closing, the speck removal, the core band of Equation 40 for the baseline and x-height, whose median over the thirteen writers ranges from 15 to 47 pixels, and the Zhang–Suen thinning [34] — the builder importing them from the same module as the classifier. Two things differ. The builder keeps the one-pixel skeleton instead of re-inking it, so the constant pen width of 0.14 x-heights and its clip to 2 to 9 pixels belong to the classifier path alone; and it thins the whole line *once*, before the territory cuts of Equation 44 are applied to the traced centre lines, because thinning each slice separately creates a spurious spine along the cut. The *slant* is the shear that makes the strokes vertical, scored by Equation 45 on the column histogram of ink pixels (x_i, y_i) sheared by a trial angle θ about their mean row $\bar{y}$, y downwards,

$$x_i'(\theta) = \left\lfloor x_i - (y_i - \bar{y}) \tan\theta \right\rfloor, \qquad H_\theta(c) = \#\{i : x_i'(\theta) = c\}, \qquad S(\theta) = \sum_c H_\theta(c)^2, \tag{45}$$

with $\hat{\theta} = -\arg\max_\theta S(\theta)$ over $\theta \in \{-45°, -43.5°, \ldots, 45°\}$, forward lean positive; $\sum_c H_\theta(c)$ being fixed, the score peaks when the ink falls in the fewest columns.

A thinned image is a picture of the pen path and not yet an ordered path, and four geometric operations recover the ordering, each resting on the 8-neighbourhood degree of a skeleton pixel (Equation 87): a pixel of degree one is a stroke end, of degree two an interior point, and of degree three or more a junction. The skeleton is *traced* into chains grown from each unvisited end or junction through interior pixels only, so that every skeleton pixel belongs to exactly one chain; stopping at a junction rather than choosing a branch there is what makes the decomposition unique, a junction being precisely the point at which the true pen order cannot be recovered from the image. Short false branches left by thinning are *pruned* as spurs when a chain is both shorter than 2.5 pixels and has a free end (Equation 88); the conjunction is necessary, because a short chain between two *two* junctions is the crossbar of an A or the waist of an e and must be kept. Because a self-crossing stroke such as the loop of an e breaks into several chains at the

junction where it crosses itself, although the pen never lifted, chain ends within 1.5 pixels are *re-joined* when their outward directions are nearly opposed, the continuation test of Equation 89, which admits the curvature a pen shows through a crossing but rejects a genuine corner. A chain crossing a territory boundary is then split, each crossing setting connL_i and connR_i to record that the pen did not lift. Finally each chain is *simplified* by the Douglas–Peucker recursion at $\varepsilon = 0.6$ pixels, 0.05 to 0.12 mm at 4 mm per x-height, taken from [19] where it serves the same pen-path purpose, which bounds the deviation of the stored path from the traced one by ε everywhere; a closed chain must be split at its farthest vertex first, since the recursion measures distance to the line through the two ends and would otherwise collapse a loop to two points. Appendix 6.4 states all four criteria as explicit conditions, with the resampling and smoothing that remove the lattice staircase thinning leaves behind. Equation 46 then maps every simplified point (x, y) of a chain in the territory $[L, R)$ into a writer-independent frame with baseline 0, y up, unit x-height and the slant removed:

$$y_n = \frac{y_b - y}{x_h}, \qquad x_n = \frac{x - L}{x_h} - \sigma\, y_n, \qquad \sigma = \tan\hat{\theta}. \tag{46}$$

The synthesiser applies the inverse shear of Equation 50, so glyphs are stored upright and re-appear at the writer's lean. On a side confirmed connected the outward-running low connector is trimmed, and only if 18 % of the glyph width remains, the entry and exit points being recorded for the synthesiser's own connector.

3.7.3 Variants, quality gates and the cross-writer letter prior

Of about 2 122 to 4 031 raw instances per writer roughly 10 % survive five gates applied per character in order: geometry, outliers, the cross-writer prior of Equation 47, consensus against that writer's other instances of the same letter, and typicality, which keeps the best 12. The cascade is ordered cheapest-first and most-local-first, so that a glyph is compared with its peers only once it is known to be a plausible letter at all; Appendix 6.4 states the test each gate applies and the thresholds it uses.

The hardest gate to design is the third, because it has to decide whether a letter is malformed, and one writer's ten pages do not contain enough examples to say what a well-formed letter looks like. Thirteen writers together do, and that is the idea the gate rests on: whatever all thirteen hands agree on must be the letter itself rather than any one writer's style.

To compare shapes they must first be reduced to numbers. Each glyph is turned into a *shape descriptor* v_g, which is built by laying a 12×12 grid over the glyph's bounding box and counting how much stroke falls in each cell, giving 144 numbers. Three steps follow. The counts are *damped* by taking a power below one of each, so that one cell holding a dense loop cannot dominate the comparison. They are then blurred slightly across neighbouring cells, so that a stroke falling a fraction of a cell either side of a boundary does not change the descriptor abruptly. Finally the whole vector is scaled to unit length, which removes the overall amount of ink and leaves only its distribution, so that a thick pen and a thin pen give the same descriptor for the same shape.

The reference shape $\bar{v}_c$ for character c is the element-wise median of those descriptors over all thirteen writers, also scaled to unit length. A median is used rather than a mean so that one writer with an unusual letter form cannot pull the reference towards it. Because both vectors have unit length, their dot product is the cosine of the angle between them, which is 1 for identical

distributions and smaller the more they differ. The distance of a glyph from the cross-writer reference is therefore

$$\mathrm{priorD}(g, c) = 1 - v_g \cdot \bar{v}_c \in [0, 1], \tag{47}$$

and the 70 % of each writer's variants with the smallest priorD are kept. The important property of Equation 47 is the direction in which it is used. It only ever *rejects* one of the writer's own variants; it never substitutes a shape taken from another writer, so no part of another person's handwriting can enter a profile through this gate.

What Equation 47 measures is how *typical* a shape is, and Section 3.7.6 records the limitation that follows from this: typicality is not the same thing as legibility, and a writer whose letters are consistently unusual scores badly on a prior built from everyone else.

Two further glyphs are derived per character and stored alongside the variants. The *medoid* is the one real variant whose total distance to all the other variants of that character is smallest, so it is the writer's most representative actual letter. A medoid is used rather than an average because an average of several curves is not itself a pen stroke, whereas a medoid is a real example the writer really wrote. The distance is measured as the sum of absolute differences between descriptors, which is the L_1 norm, chosen because it is less affected by one wildly different variant than a sum of squares would be. The *denoised prototype* is the per-stroke median curve of the largest group of variants sharing the same stroke count, and it is prepended to the list only when it is clearly cleaner than the variants it summarises (Figure 9). The medoid is also what Section 3.8 uses as the *anchor*, the legible stand-in drawn when a writer's own variant is judged too malformed to use.

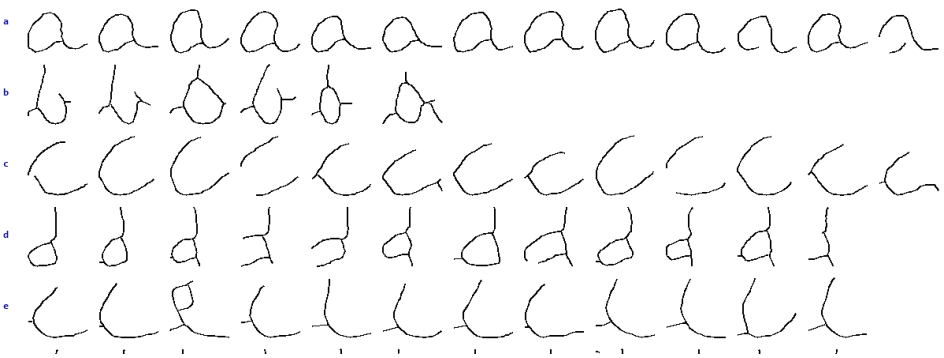


Figure 9. Stored variants of the letters a, b, c, d and e of writer 153, a strongly joined hand; each row is one character with up to 12 variants, and the short tails at the sides are remains of ligatures.

3.7.4 Style statistics and permanent storage

Besides glyphs the profile holds numbers measured over the kept lines, every key of which is listed with an example as Table 31 in Appendix 6.4; two are not simple medians, the *connectedness* being the adjacent letter pairs whose connR flag was set over all adjacent pairs, so its numerator counts the pen-down crossings recorded during tracing, and the word gap being the median of ink-free column runs wider than $0.5x_h$ on the *real* ink, because CTC gives a space only a narrow slice. Table 12 shows how widely the thirteen differ — slant −10.5° to 42.0°, connectedness 0.195 to 0.968, word gap 0.94 to 2.68 x-heights — which is the diversity requirement R1

asks the system to reproduce; the thirteen files total about 4.8 MB and load when the server starts (Section 3.11), which with the web interface listing them is requirement R6. **[MUST STILL BE COMPLETED BY ME: the demonstration of R6 (display of the stored writers, reproduction in a writer chosen by the examiner) and the storage device holding the files.]**

Writer	Lines	Slant (°)	x_h (px)	A	D	Gap	Conn.	λ
150	61	−4.5	47	1.58	−0.72	1.33	0.343	0.90†
151	58	33.0	22	2.69	−1.09	1.59	0.968	1.00†
152	59	19.5	46	1.48	−0.79	1.47	0.715	0.45†
153	66	30.0	33	1.59	−1.10	1.59	0.961	0.45†
384	54	4.5	43	1.93	−0.71	1.37	0.500	0.30†
551	58	42.0	20	1.84	−0.67	2.68	0.557	1.00†
552	47	−10.5	32	1.62	−0.56	1.53	0.609	0.00†
588	58	0.0	29	1.73	−0.68	2.07	0.668	0.30†
abhinav	130	6.0	28	1.78	−0.84	2.13	0.789	—
dylan	79	−6.0	26	1.68	−0.62	1.79	0.195	1.00†
owen	93	0.0	32	1.51	−0.68	1.10	0.319	—
thiya	48	30.75	15	2.24	−1.00	1.65	0.962	—
yeukita	74	0.0	27	1.45	−0.58	0.94	0.417	0.15†

Table 12. Scalar values of the thirteen stored style profiles, read directly from the files in `Software/CNN/NOGIT/StyleProfiles10`: kept lines, slant, pixel x-height, ascender A, descender D and word gap (x-heights), connectedness and legibility parameter λ. Writers 150 to 588 are database writers, the last five personal. †The λ column is *not* read from these files, in which `legibilityLambda` is absent; it is the output of an earlier tuning run, as item 3 of Section 3.7.6 records.

3.7.5 Design alternatives and justification

Character boundaries could have come from fixed widths, projection minima or hand labelling, but joined letters have no gaps and hand labelling of about 90 pages is impractical, while the recogniser already knows the text; several variants per letter are kept rather than one averaged letter, because averaging blurs letters whose strokes are arranged differently; and a generator learned from on-line pen data was not possible, since no such data exist for the thirteen writers, who contribute 47 to 130 usable lines each, and the pen path has to be recovered from the image in any case. **[MUST STILL BE COMPLETED BY ME: a figure of the builder stages on real lines: grey crop, ink mask with rules removed and core band, skeleton, traced chains in colour with the territory cuts as vertical lines, and one extracted glyph in the normalised frame, for one database and one personal line.]**

3.7.6 Known faults and limitations

1. *The builder cannot at present extract glyphs from a page image.* `ExtractLineGlyphs` of `BuildStyleProfile.py` opens with `from TrainText import FrameXToPixel, INPUT_WIDTH`, and the delivered `TrainText.py` defines no such name in any spelling; the conversion of Equation 43 survives only as `frame_x_to_pixel` in the retired tree `Software/CNN/TrainText.py`. Because the statement sits inside the function the module still imports and every cached rebuild still works, but a fresh extraction raises `ImportError` at the first line, so *no new writer can be added and none of the thirteen stored profiles can at present be regenerated from the page images*. **[MUST STILL BE COMPLETED BY ME: repair**

the import in `ExtractLineGlyphs` — either port `frame_x_to_pixel` from `Software/CNN/TrainText.py` into the delivered `TrainText.py` or compute the conversion of Equation 43 in place — then re-run the builder on all thirteen writers and confirm that the regenerated profiles match the stored ones.]

2. `legibilityLambda` is written after the build by the tuner `TuneLegibility` of `Evaluate.py` and not by the builder, and *the key is absent from all thirteen files as delivered*, the profiles having been rebuilt after the tuner last ran. The server calls `SynthesizeJointBestOf`, which takes `profile.get('legibilityLambda', 0.0)`, so *the delivered system synthesises every writer at* $\lambda = 0$, entirely from the writer's own glyphs with no anchor substitution at all; the older `SynthesizeLegible` falls back to 0.65 instead but is not used. **[MUST STILL BE COMPLETED BY ME: re-run `TuneLegibility` over the thirteen current profiles so that `legibilityLambda` is written back into each file, then re-measure the figures of Table 13 that depend on it; until this is done the per-writer legibility control described in Section 3.8 is inactive.]**
3. The λ column of Table 12 is a genuine measurement of that superseded run, but it describes neither the stored profiles nor the delivered system, and abhinav, owen and thiya were added afterwards and never tuned at all; the λ gains claimed in Section 3.8 are not realised by the submitted code.
4. The stored profiles are dated 10 October 2026 and predate the current code, so the values in Tables 31 and 12 had to be re-read from the files rather than regenerated.
5. Cutting a joined line destroys how a letter changes with its neighbours, which is why writers 151 and 551 are reproduced least legibly (Section 3.8).
6. The vertical-cut decision rests on a design-stage measurement, 81.8 % against 68.6 %, for which no raw log was kept.
7. The grey-level reference statistics `inkFracRef`, `aspectRef`, `paperLevel`, `inkLevel` and `midFracRef` are stored but ignored by every consumer while the pen width is constant.

3.8 Handwriting synthesis

This subsection describes how a text is converted into pen paths in the style of a stored writer (functional units FU 2.7 and part of FU 2.5; requirements R1 and R3). It takes the text, a style profile (Section 3.7), the x-height size x_{mm} in millimetres, 4 mm by default, and a random seed, and returns a *trajectory*: pen-down polylines in millimetres, y upwards, origin at the first baseline. Geometry stays in x-height units until one multiplication at the end, so one profile serves any pen size, and because the construction is random a search over several candidates (Section 3.8.3) keeps the one that the text recogniser of Section 3.5 and the stroke-normalised writer classifier of Section 3.6 jointly prefer.

3.8.1 Global quantities and glyph choice

Let θ_{p}, c, A, $D < 0$ and g be the profile's slant, connectedness, ascender and descender reach and word gap, and $\lambda \in [0,1]$ the writer's legibility parameter, which is 0 for every writer in the delivered system because the key is missing from the stored profiles (Section 3.7): every formula below containing λ must be read at $\lambda = 0$ to judge what the submitted code does. The synthesiser

derives, by Equation 48,

$$\theta_{\mathrm{cap}} = 45^\circ - 19^\circ\lambda, \quad \sigma = \tan\big(\mathrm{clip}(\theta_{\mathrm{p}}, \pm\theta_{\mathrm{cap}})\big) + \Delta_\sigma, \quad p_{\mathrm{j}} = \min(c, 1), \quad L = (A - D + 0.85)\, x_{\mathrm{mm}}, \tag{48}$$

Each quantity in Equation 48 has a specific job. The *shear coefficient* σ is the tangent of the writer's slant, so that multiplying a point's height by σ and adding it to the horizontal coordinate leans the letter by that angle; this is the inverse of the de-slanting the builder applied when it stored the glyph upright. The calibration shift Δ_σ comes from Section 3.8.2 and corrects the difference between the slant of individual letters and the slant of a whole assembled line. The angle is clipped to θ_{cap} because a very steep lean makes adjacent letters collide. The *join probability* p_{j} decides how often two neighbouring letters are connected, and the *line pitch* L is the baseline-to-baseline spacing in millimetres, which works out at 12.6 mm for the default $A = 1.7$, $D = -0.6$ and $x_{\mathrm{mm}} = 4$ mm.

The treatment of connectedness is a design decision worth stating, because the obvious approach failed. The stored connectedness c is the measured fraction of adjacent letter pairs that the writer joins, so it is natural to read it as a threshold and join every pair when c is above some value. Doing that joined *every* pair for every writer above the threshold, which fused whole words into unbroken blobs and lost the irregularity that makes a hand look human. It is therefore used as a *probability* instead: each pair is joined on an independent random draw below p_{j}, so a writer who joins four pairs in five has roughly four joins in five but not the same four every time.

Choosing a glyph for each character. For each character the synthesiser works down a fallback chain and takes the first option that exists: the writer's own stored variants of that character, then a borrowed shape of the other case or of a visually similar character, then the *anchor*, and finally a hand-designed skeleton or, failing everything, a blank. The chain exists because no writer's pages contain every character, so there must always be something to draw.

Among the writer's own variants the choice is random but weighted, by $w_i = (1 + 1.4i)^{-1} \exp[-((\mathrm{entryY}_i - \hat{e})/0.45)^2]$ for rank $i = 0, \ldots, 4$, where the variants are ranked by priorD so that rank 0 is the most typical. The two factors do different things. The first falls off with rank, so a more typical variant is chosen more often but an unusual one is still chosen sometimes, which is what stops every instance of a letter on the page being identical. The second factor applies only when the letter is being joined to the previous one: $\hat{e}$ is the height at which the pen is currently leaving the previous letter, entryY_i is the height at which this variant begins, and the Gaussian penalises a mismatch between them. Choosing a variant whose entry height matches where the pen already is avoids a join that loops awkwardly up or down to meet it.

The *anchor* is the legible fallback, and its role is to put a floor under how bad a letter can be. It is the writer's own medoid when that is clean enough, which keeps it in the writer's hand. When the medoid is itself malformed, the anchor is instead a hand-designed single-stroke skeleton, joined cursive for a connected hand and upright print otherwise, re-proportioned to the writer's own ascender and descender reach so that it does not sit at the wrong size among real letters. The print skeleton reads back at about 96 % character and 84 % word accuracy, which is clearer than any glyph derived from the data, and that is what makes it a usable floor. Table 32 in Appendix 6.4 gives the constant behind each of these choices with the reason for its value.

Trading style against legibility. The system has two goals that pull against each other. Requirement R1 asks for handwriting that resembles the writer, which argues for always using the writer's own letters, and it also asks for handwriting that can be read, which argues for replacing

the writer's worst letters with something clearer. The parameter λ is the single control that sets where a given writer sits between those two.

The obvious implementation would be to blend the two shapes geometrically, drawing a curve part way between the writer's glyph and the anchor. That is *not* what is done, and the reason is that an average of two different pen strokes is usually not a valid letter at all: blending a one-stroke and a two-stroke form of the same character produces something neither legible nor characteristic. Instead λ scales the *probability* that a chosen variant is replaced by the anchor outright, so every letter drawn is always a real letterform, either the writer's or the anchor's. Equation 49 gives that probability as a function of λ and the variant's prior distance d from Equation 47. The same λ also lowers the slant cap in Equation 48, since a reduced lean is the other thing that helps legibility.

$$P_{\text{swap}}(\lambda, d) = \begin{cases} 1 & d \geq 0.55, \\ 0 & \lambda \leq 10^{-3}, \\ 0.06\,\lambda & d \leq 0.12, \\ \text{clip}\big(\lambda\,(0.55 + 0.9\,b), 0, 1\big), \; b = \text{clip}\big(\frac{d-0.12}{0.43}, 0, 1\big) & \text{otherwise.} \end{cases} \tag{49}$$

The four cases of Equation 49 implement three different policies, and the thresholds 0.12 and 0.55 on d are what separate them. A variant further than $d = 0.55$ from the cross-writer reference is replaced *unconditionally*, whatever λ says, because at that distance the shape is no longer recognisable as the character and keeping it would serve neither goal. A variant closer than $d = 0.12$ is kept almost always, the replacement probability being only 0.06λ, because a shape that close to the reference is already legible and replacing it would cost style for nothing. Between those two the probability rises linearly with d through the interpolation variable b and scales with λ, so a writer tuned to a high λ has their marginal letters replaced and a writer tuned to zero does not. At $\lambda = 1$ this gives 0.74 for $d = 0.21$ and 0.99 for $d = 0.33$. The second case makes $\lambda = 0$ exact rather than approximate: below 10^{-3} the replacement is switched off entirely, so a writer tuned to zero keeps even their marginal letters, and only the unconditional case above $d = 0.55$ still fires.

Measured over a design-stage probe, the resulting share of anchor glyphs ran from 0 % for writer 552 to 76 % for writer 151, which is the spread the parameter is there to produce: writer 151's hand is the most joined of the set and suffers most from being cut into letters, so it needs the most help, while writer 552's letters survive the process intact. λ is set per writer (Table 12) by an automatic search on a novel corpus followed by manual adjustment against the evaluation script; in the delivered system the mechanism is inert and every figure in this paragraph describes the tuned configuration only, as item 1 of Section 3.8.6 sets out. A geometric blend toward the anchor and a soft-archetype proposal appear only in the project notes and not in the delivered code.

3.8.2 Placement, joins and randomness

A chosen glyph (x, y), in x-height units and upright, is placed at the pen position (P_x, P_y) in millimetres by Equation 50,

$$X = P_x + (\ell + x + \sigma_g\, y)\, x_{\text{mm}}, \qquad Y = P_y + (y + \delta_g)\, x_{\text{mm}}, \tag{50}$$

with the glyph's lead ℓ, shear σ_g and baseline drift δ_g, which inverts the de-slanting of Equation 46; each glyph occupies a cell of its own advance with the ink starting after ℓ, and the writer's own

glyphs take the line shear σ while anchor glyphs are capped more tightly. Adjacent letters are joined when both are alphabetic, both are the writer's own glyphs or both the cursive anchor, and a random draw falls below p_{j}; a print anchor is never joined through, and a joined run is one pen-down polyline, so the pen lifts only where the writer lifts. One pseudo-random stream, seeded by the seed argument, supplies every draw, so the arguments fix the output completely: each glyph is rotated, scaled and translated a little and the baseline wanders as a bounded first-order walk, all scaled by one jitter amplitude J. At $J = 0.5$, the value the search uses, the jitter comes to a rotation of 1.1°, a scale change of 2.5 % and a translation spread below 0.05 mm, enough to perturb a letter without changing its identity; Appendix 6.4 lists every constant of the placement, the joins, the randomness and the glyph choice with the reason for its value.

Style calibration. A line of separate glyphs does not reproduce the line-level slant and reach of the real lines on its own: writer 150's ascenders came out about 7 % too tall and writer 551's slant about 9° too steep. A fixed probe sentence is therefore synthesised once per writer and size, at seed 17 with no jitter and measured with the estimators of Section 3.7, and three fixed-point iterations of Equation 51 adjust the shear shift Δ_σ and the scale factors K_A, K_D of the writer's tall and deep strokes until the probe matches the stored real-line values, "ref" being the stored value and "got" the probe value,

$$\Delta_\sigma \leftarrow \Delta_\sigma + \tan\theta_{\mathrm{ref}} - \tan\theta_{\mathrm{got}}, \quad K_A \leftarrow \mathrm{clip}\Big(K_A \frac{A_{\mathrm{ref}}-1}{A_{\mathrm{got}}-1}, 0.5, 1.6\Big), \quad K_D \leftarrow \mathrm{clip}\Big(K_D \frac{D_{\mathrm{ref}}}{D_{\mathrm{got}}}, 0.5, 1.6\Big), \tag{51}$$

in which θ is the line slant of Equation 45, $A > 1$ the ascender reach and $D < 0$ the descender reach, the references being `slantMeasRef`, `ascMeasRef` and `descMeasRef` of Table 31, and the clip bounding each correction. The factors are then applied as $y \leftarrow 1 + (y-1)K_A$ above the x-height line and $y \leftarrow yK_D$ below the baseline, and Δ_σ is added to the shear of Equation 48.

3.8.3 Best-of-N search with joint scoring

Single draws vary in quality, so the line is drawn N times and the best candidate kept. The score uses both judges, because selecting by text accuracy alone raised legibility but lowered writer identification for some writers (Table 13): the most legible draw need not be the most characteristic. For a rendered candidate I, with $\hat{y}(I)$ the greedy CTC reading of Equation 32 and y the intended text, the joint score is Equation 52,

$$a(I) = \max\Big(0, 1 - \frac{\mathrm{Lev}(\hat{y}(I), y)}{|y|}\Big), \quad w(I) = \mathrm{softmax}\big(f_{\mathrm{wid}}(I)\big)_{a^\star}, \quad S(I) = \frac{2\,a(I)\,w(I)}{a(I)+w(I)}, \tag{52}$$

where Lev is the Levenshtein distance of Equation 83, f_{wid} the shape classifier after stroke normalisation (Section 3.6.3), $a^\star$ the target writer, and $S = 0$ if $a + w \le 10^{-9}$. The harmonic mean is dominated by the smaller term, so a candidate cannot win on one criterion while failing the other, and w is a probability rather than a hit-or-miss flag. Figure 10 is the control flow of the two loops that use it.

The search has two loops. In the first, for $k = 0, \ldots, N-1$, it synthesises a candidate with seed base + 977k and $J = 0.5$, renders it at 0.085 x-heights and 18 pixels per millimetre, computes S, keeps the candidate if it beats the best so far, and leaves the loop once the best reaches 0.99 or $k = N-1$. The second is the *repair* loop, which exits straight to the output once the best is at or above 0.995 or three rounds have run; otherwise each round starts from the best candidate so far, forced-aligns the intended text to the recogniser output on its image (Equation 42), scores each

non-space character by its mean own-class log-probability over its own frames, taken as -10^9 if the path never visits it, pins the two worst unpinned characters to the anchor, re-synthesises the *whole* line with seed base, judges it, and accepts only if the joint score strictly exceeds the best so far, the round ending either way. Re-using seed base makes the repair a fresh draw with pins rather than a patch of the winner, and judging it jointly rejects a repair that fixes a letter but makes the line look like another writer, whereas a text-only search with a plain repair that accepts on text score alone was measured to cost writer identification (Table 13).

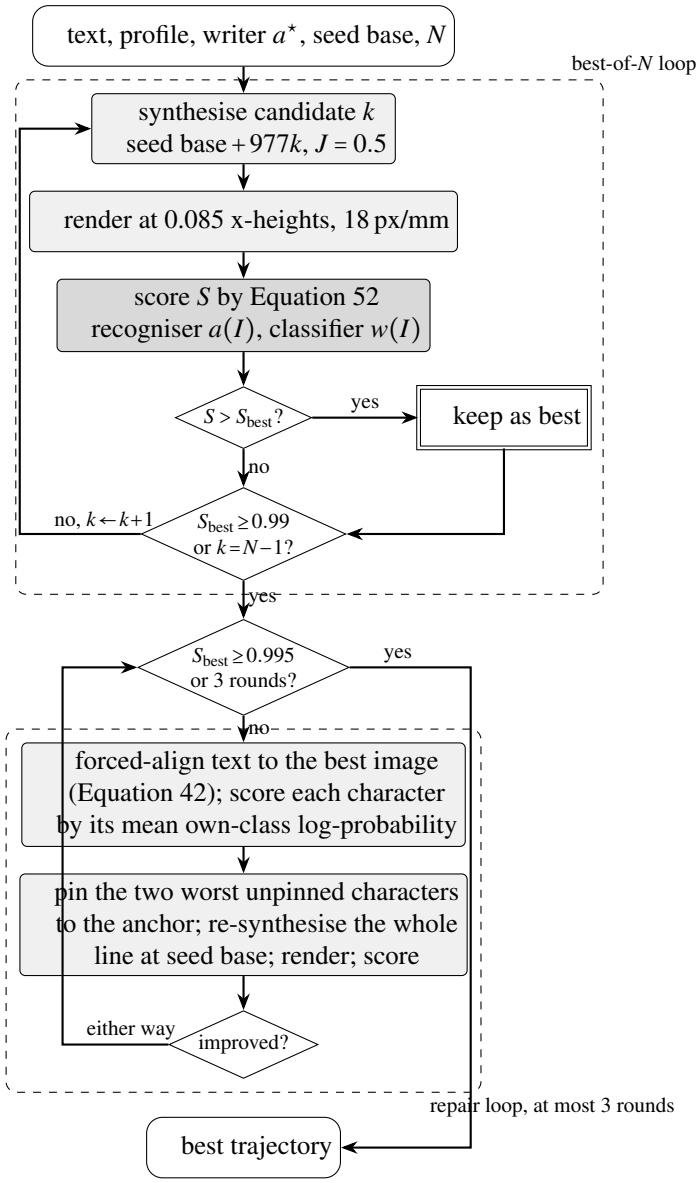


Figure 10. Control flow of the candidate search of `SynthesizeJointBestOf`. The upper loop draws N candidates and keeps the one with the highest joint score S of Equation 52; the lower loop attempts at most three repairs, each a fresh draw of the whole line with the two weakest characters pinned to the anchor, and accepts one only when S strictly improves, so a repair that fixes a letter at the cost of the writer's identity is rejected. Both judges score the same rendered ink, the classifier after the stroke normalisation of Section 3.6.3.

N is 6 as the function default, 14 for the design-stage measurements, 20 as the web default and

30 in the official evaluation script, and the search cost is linear in it by Equation 92. The cost is entirely judge-bound: synthesising a candidate takes 11 to 16 ms against 0.5 to 4 s to score it, and within that score the dominant term is thinning the full-resolution render rather than either network, so the whole search at $N = 14$ took 23 to 137 s on the laptop. That is where any optimisation must go, and it is also why N rather than the construction sets the reproduction time of requirement R3. Appendix 6.4 resolves Equation 92 into its measured terms for three writers and gives the order-statistics argument for the diminishing return from raising N.

3.8.4 Design-stage measurements of the synthesiser

Table 13 lists the measurements that motivated the design, taken over six novel sentences, two seeds and ten writers, so 12 lines per writer and one misjudged line moves a writer's rate by 8.3 percentage points, together with the final system for comparison. Items 2 and 3 of Section 3.8.6 state why every row after the search was introduced is circular, and the non-circular reference is the recogniser on *real* held-out handwriting of the same writers, 89.4 % character and 70.8 % word accuracy (Table 29, with possible training overlap).

Stage	Writer ID, direct / G-code (%)	Char. acc., direct / G-code (%)
Plain synthesis after label and glyph fixes	79.2 / 80.0	75.9 / 76.3
+ per-writer λ	83.3 / 86.7	77.2 / 77.8
Text-only best-of-N with plain repair	76.7 / 73.3	83.8 / 82.7
Joint best-of-N, $N = 6$	92.5 / 92.5	79.9 / 79.6
Joint best-of-N, $N = 14$	97.5 / 95.8	81.6 / 81.4
Final system, 13 writers, 3 sentences, $N = 8$	92.3 / 89.7	79.2 / 79.2
Final system, 13 writers, 6 sentences × 2 seeds, $N = 30$	80.1 / 78.8	82.2 / 82.5
Matching comparison: the writers' own handwriting of the same words	not applicable	89.0 / —
Matching comparison: the system's synthesis of those same words	not applicable	81.6 / —

Table 13. Measurements that motivated the design (upper block) and the final system (lower blocks). "G-code" means converted to G-code and read back (Section 3.9). Only the two matching-comparison rows are independent of the search and so the figures to trust; the rest, and the unavailability of the per-writer λ gain, are qualified by items 1 to 3 of Section 3.8.6.

On the matching comparison of identical words synthesis costs 7.4 percentage points of character accuracy, 89.0 % against 81.6 %, and divides word accuracy by more than 1.5, 70.5 % against 46.7 %, which locates the loss in spacing and word boundaries rather than in letterforms. Figure 22 shows the per-writer breakdown and Figure 11 one candidate each for a print and a joined hand.

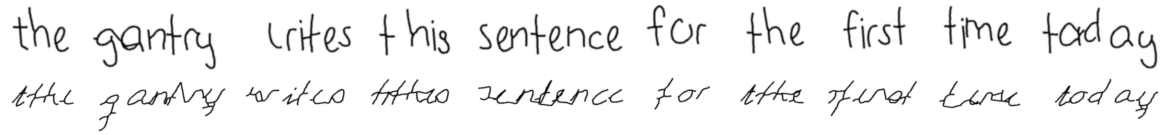


Figure 11. Synthesised line "The gantry writes this sentence for the first time today" for writer 150 (upper, printed hand; read back with three small errors) and writer 153 (lower, joined hand; read back as "the garoby writes tttas senkence for tthe furst tuse today"), one seed and candidate each, rendered from the trajectory of an end-to-end run; the N of that run was not recorded.

3.8.5 Design alternatives and justification

A recurrent sequence generator with attention and mixture-density output needs on-line pen data, hundreds to thousands of lines per writer, whereas only 47 to 130 scanned lines per writer exist (Table 12); a from-scratch version trained on skeleton-derived strokes learned slant, roundness and connectedness but no legible letters and was rejected. Adversarial and diffusion generators need less data through pre-training, but they emit a raster that needs the same thinning to vectorise, size, slant and spacing cannot be controlled as numbers, and a trial took 35 to 45 s per line against 3 to 8 s for glyph synthesis, so they were parked. The glyph library needs about nine pages per writer and no training, outputs pen strokes directly, exposes size, slant, joins and gap as explicit numbers and has a traceable fallback for every glyph; synthesis itself costs about 15 ms, so only the optional search is expensive, which suits a single-board computer without an accelerator, and the one weakness is joined hands, whose joins are lost by cutting.

3.8.6 Known faults and limitations

1. *The legibility mechanism is inert as submitted.* `legibilityLambda` is absent from all thirteen stored profiles and `SynthesizeJointBestOf` defaults it to zero, so $\lambda = 0$ on the live path: the second branch of Equation 49 fires for every glyph below $d = 0.55$, the *only* substitutions left are the unconditional ones at $d \geq 0.55$, and the cap stays at its widest, $\theta_{\text{cap}} = 45°$. Every λ-dependent figure above, including the 0 to 76 % anchor share, describes the tuned configuration and *not* the delivered one, and the outstanding tuner item of Section 3.7.6 applies to all of them.
2. *Every figure after the search was introduced is the opinion of the very networks that selected the output.* The search loads the same weight files as the measurement, so a best-of-N search necessarily raises the score by which it is judged; the values show that the pipeline can satisfy its two judges, not that a human reader or an independent recogniser would agree.
3. λ, the anchor threshold and N were tuned on the same sentences that Section 4 measures.
4. The calibration of Equation 51 measures both sides with the same estimator, so the agreement of slant and reach it achieves is partly self-fulfilling and is not independent evidence.
5. The repair pass updates the pin set only on acceptance, so after a rejection the next round re-synthesises an identical candidate and the remaining rounds are wasted work; in three probe runs, writers 150, 153 and 551 at $N = 14$, no repair was accepted at all.
6. Three design-stage figures have no surviving raw log: the 80.5 against 75.5 % variant-restriction probe, the anchor-share probe, and the print skeleton's 96 and 84 % read-back, which exists only as a code comment. **[MUST STILL BE COMPLETED BY ME: the corresponding read-back accuracy of the *cursive* skeleton. An earlier draft quoted about 89 % for it; that figure matches no comment or log in the delivered code and has been withdrawn rather than left standing as a measurement.]**
7. Writers 151 and 552, the two most joined hands, reach only 53 and 56 % character accuracy through G-code against about 84 % for the other eleven, the clearest limitation of the exemplar approach.

3.9 G-code generation

This subsection describes how a trajectory in millimetres becomes a machine program (functional unit FU 2.8, first stage; requirements R3 and R5). The format is G-code, the command language of plotters and numerically controlled machines standardised as ISO 6983-1 [22]. A G-code program is a list of text lines, each naming a command and its arguments. Only five commands are used here: `G21` and `G90` set the units to millimetres and the coordinates to absolute, `G0` is a move with the pen up and `G1` a move with the pen down, `M3` and `M5` lower and raise the pen, `G4` pauses, and `M2` ends the program.

Two conventions of the format matter to the design. Coordinates are *absolute*, meaning each move names where to go rather than how far to move, so a rounding error in one move cannot accumulate into the next. Settings such as the feed rate are *modal*, meaning a value stays in force until another line changes it, so the feed need not be repeated on every move. Fault 2 of Section 3.9.3 is a consequence of that second convention being overlooked.

The format was chosen because it splits the motion chain in two. The generator needs to know nothing about the gantry electronics, the interpreter of Section 3.3 needs to know nothing about handwriting, and the file between them is plain text that can be inspected and replayed without the machine being present, which is what makes the verification of Section 3.9.2 possible at all. The generator was written for this project; the file format itself was not designed here and carries no design credit.

3.9.1 Placement, ordering and program structure

The trajectory of Section 3.8 and the machine frame both have X to the right and Y upwards, so no mirroring is needed. The origin is the safe corner 5 mm inside the minimum switches (Section 3.3.4), leaving the driver 194×252 mm of the machine's 204×262 mm switch-to-switch travel, and the text block starts at $(X_\mathrm{o}, Y_\mathrm{o}) = (10, 180)$ mm with later lines descending. For a trajectory of bounding box (x_0, y_0, x_1, y_1), width w and height h, the block is shrunk by Equation 53, never enlarged, if it would leave the work area $[0, X_\mathrm{max}] \times [0, Y_\mathrm{max}]$ with $X_\mathrm{max} = Y_\mathrm{max} = 200$ mm:

$$s = \min\!\left(1,\ \frac{X_\mathrm{max} - X_\mathrm{o}}{w},\ \frac{Y_\mathrm{o} - Y_\mathrm{min}}{h}\right), \qquad X = X_\mathrm{o} + (x - x_0)\,s, \quad Y = Y_\mathrm{o} - (y_1 - y)\,s, \tag{53}$$

with $Y_\mathrm{min} = 0$; at scale 1 the area holds at most $\lfloor (180 - 9.2)/12.6 \rfloor + 1 = 14$ lines of the default size, where $9.2 = (A - D)x_\mathrm{mm}$ and the pitch is 12.6 mm. A final clamp moves outlying vertices onto the border and writes a header warning, and vertices closer than 0.12 mm to the previous one are dropped, which is inert in practice because the synthesiser resamples at 0.35 mm. The strokes are then ordered by a greedy nearest-endpoint rule, grouped into text rows and, within a row, the next stroke being the one whose end is nearest to the pen and entered from that end; only order and direction change, so the ink is identical, and this is the pen-movement optimisation of R3, with the row-order fault of Section 3.9.3.

The pen lift is a one-way toggle motor (Section 3.3.5), so the generator tracks the pen state in software and emits `M3` for down or `M5` for up only when the state changes, a pulse where none is needed inverting the pen for the rest of the job. Each change is followed by a *dwell*, which is a commanded pause that holds the machine still while something mechanical finishes.

Here it covers one 90° rotation of the cam and the settling of the pen afterwards, assumed to be 260 + 90 = 350 ms. A dwell is needed because the pen motor is not synchronised with the motion: without it, the next move would begin while the pen was still travelling, drawing a stroke that starts in the wrong place. Since every stroke needs the pen down and then up again, a stroke costs 0.7 s of dwell, which is why dwell rather than drawing dominates the predicted job time. The interpreter additionally confirms the pen state with the pen switch rather than trusting the dwell alone. A program is a comment header, `G21` for millimetres and `G90` for absolute coordinates, then one block per stroke of at least two points at the 900 mm/min draw feed, and finally pen up, parking and `M2`, with vertices written to three decimals; Figure 12 is the start of a generated program and Figure 38 in Appendix 6.4 the loop that writes it. Table 35 in the same appendix lists every constant with its role; the motor, driver and transmission values are assumptions made in the program comments, and the calibrated steps per millimetre of Section 3.3.4 replace the derived ones in the interpreter, so a wrong assumption in the file header cannot reach the machine.

```
; start of a generated program -- author yeukita
; steps/mm  X=80.000  Y=80.000 (3200 microsteps/rev)
G21   ; millimetres
G90   ; absolute
G0 F2400                    ; travel feed, emitted once
G0 X10.059 Y179.271         ; travel to the first stroke
M3   ; pen DOWN (one 90deg pulse of the one-way gear motor)
G4 P0.350   ; wait for the 90deg rotation to finish
G1 F900                     ; draw feed
G1 X10.084 Y178.920         ; one line per vertex (0.35 mm apart)
```

Figure 12. Start of a generated program for the test word of writer yeukita (14 strokes, 239 draw moves, vertices 0.348 mm apart on average). The trailing comments on the motion lines are annotations added here. Every line of the format is accounted for: the two setting codes, one travel feed emitted once, a rapid move to the first point, the pen toggle with its dwell, the draw feed, and then one linear move per vertex. The loop that writes it is given as pseudocode in Appendix 6.4.

3.9.2 Time model and verification

The generator can also write a step schedule with a time estimate, which assumes each move starts and ends at rest under the ramp that keeps an open-loop stepper from losing steps, with acceleration $a = 400\ \text{mm/s}^2$ and cruise speed v, 15 mm/s drawing and 40 mm/s travelling. The distance needed to reach v is $d_a = v^2/(2a)$, so a move of length d is *trapezoidal* if $d \geq 2d_a$ and otherwise *triangular*, never reaching v (Equation 54):

$$T(d) = \begin{cases} \dfrac{2v}{a} + \dfrac{d - 2d_a}{v} = \dfrac{d}{v} + \dfrac{v}{a}, & d \geq 2d_a, \\ \dfrac{2v_\text{p}}{a} = 2\sqrt{\dfrac{d}{a}}, \quad v_\text{p} = \sqrt{d\,a}, & d < 2d_a. \end{cases} \tag{54}$$

For drawing $d_a = 0.28$ mm, so moves below 0.56 mm are triangular with a 37.5 ms ramp; for travel $d_a = 2$ mm, so moves below 4 mm are triangular with 100 ms. A typical 0.35 mm segment is therefore always triangular, with $T = 2\sqrt{0.35/400} = 59.2$ ms, an average of 5.9 mm/s and not 15. For the test word of Figure 12, a draw path of 83.2 mm in 239 moves and a pen-up path of 250.3 mm in 14 moves with 26 pen pulses, the model gives 14.1 s drawing, 7.5 s travel and 9.1 s

dwell, 30.7 s in all, or 2.8 s per character for an assumed 11 characters, the text not being stored in the file.

The generator is the one stage whose output can be checked exactly, because both its input and its output are geometry and the correct answer is therefore known. The check is run by a second, independent parser that reads the finished file back as though it were the machine: it follows X and Y, starts a polyline at `M3` and closes it at `M5`, and parses only `G0` and `G1` because nothing else is emitted. Writing a separate reader rather than reusing the generator's own data is the point of the exercise, since a fault shared by both would otherwise cancel out and go unseen.

The trajectory that went in and the polylines that come back out are then compared in two ways, in Equation 55.

$$\mathrm{IoU} = \frac{|M_\mathrm{t} \cap M_\mathrm{g}|}{|M_\mathrm{t} \cup M_\mathrm{g}|}, \qquad e_\mathrm{mean} = \frac{1}{J}\sum_j \min_i \|g_j - s_i\|, \qquad e_\mathrm{max} = \max_j \min_i \|g_j - s_i\| \tag{55}$$

The first is an area comparison. Both are drawn onto a common grid of 8 pixels per millimetre, giving two masks M_t and M_g, and the *intersection over union* is the number of pixels inked by both divided by the number inked by either. It is 1 when the two drawings coincide exactly and falls towards 0 as they diverge, so it answers the question "does the file draw the same picture" with a single number. It is used rather than counting differing pixels because that count would depend on how much ink there is, whereas a ratio does not.

The second is a vertex comparison. For each vertex g_j of the G-code, the distance to the nearest vertex s_i of the original trajectory is found, and the mean and the worst of those distances are reported. The mean shows whether the whole drawing has shifted and the maximum catches a single vertex that has moved, which an area measure would hide because one misplaced point changes almost no pixels. The vertices are sub-sampled to at most 4000 and 6000 respectively to keep the all-pairs search affordable.

What Equation 55 does and does not cover should be clear. It measures only the loss between the trajectory and the file, which is rounding to three decimals, the stroke reordering and the clamping to the work area. It says nothing about whether the machine then draws the file correctly, which is the mechanical error of Section 3.3.4 and a separate matter entirely.

3.9.3 Known faults and limitations

1. The stroke rows are sorted by ascending Y, so a multi-line job starts with the *bottom* line and not in reading order; this was found by reading the code and was not checked on the machine.
2. The interpreter has one *modal* feed register and no rapid mode (Section 3.3), but the generator emits `G0 F2400` once at the start and `G1 F900` before every stroke, so every pen-up travel after the first stroke runs at 900 mm/min and not 2400; the real job time is therefore longer than the time model assumes, and emitting the feed before every travel would remove the discrepancy.
3. Lines wrap at 185 mm from $X_\mathrm{o} = 10$ mm, so a full-width line reaches $X = 195$ mm, but the driver clamps X to 194 mm and would distort the last millimetre; X_max or the line width should be reduced.
4. Equation 54 is a model and not a measurement: it stops fully between 0.35 mm segments and assumes 40 mm/s travel, so it overestimates drawing time and underestimates travel

time against the real interpreter (Section 3.3.4).

5. Equation 55 compares vertices with vertices, so it covers only the loss between trajectory and file, namely rounding, ordering and clamping, and not the mechanical error (Section 3.3.4). The project notes quote IoU ≈ 0.99 and a text-accuracy change of at most 0.3 percentage points between direct and read-back rendering, for an earlier system state and judged by the networks of the search (Table 13 shows later pairs); e_{mean} and $e_{\max}$ were never logged. **[MUST STILL BE COMPLETED BY ME: run the comparison for a set of generated programs and report** IoU**,** e_{mean} **and** $e_{\max}$**.]**
6. The saving produced by the stroke-ordering optimisation of R3 was not measured.

3.10 Modifications made after the chain could be measured

The subsections above describe each subsystem as delivered. Four of those designs are not what was built first: they replaced an earlier version once the complete chain existed and the output could be measured end to end, which is the point at which faults invisible inside a single subsystem became apparent. They are collected here because each was a change of approach rather than a correction of code, and because the reason for each lies outside the subsystem it altered.

3.10.1 A second writer classifier, judging shape instead of ink

The first writer classifier was trained on grey-scale crops and read real held-out lines at 99.78 % over 1358 lines, which satisfied requirement R2 on photographed pages. Measured on *synthesised* output, however, the same classifier identified the intended writer in only about 28 % of trials. The cause is a property of the machine rather than of the network: the gantry draws every stroke with one pen at one width and one pressure, whereas real handwriting carries a thickness and an ink density that a convolutional network exploits readily when the training lines are few. A classifier that leans on those cues therefore predicts neither what the machine can draw nor how a reader recognises a hand, which is by geometry.

The response was to train a second classifier of the same architecture on stroke-normalised crops, in which the ink is thinned to a centre line and redrawn at a constant width proportional to the letter height, so thickness is removed by construction (Section 3.6.3). The two are used for different purposes, and the division is kept throughout the system: the ink classifier serves real photographed pages, and the shape classifier serves the candidate search of Section 3.8.3. Neither replaced the other, so the writer-identification block carries two sets of weights rather than one.

3.10.2 End-to-end training in place of a frozen backbone

The shape classifier was first trained by fitting only the 13 645 parameters of its head on feature vectors extracted once from a frozen backbone. That recipe is a training fault for the reason given in Section 3.6: because the features were extracted once, no augmentation was re-sampled between epochs and every line contributed the same four fixed vectors for all 500 epochs. Training all 966 445 parameters end to end instead raised the same measurement from 83.36 % to 93.67 % on the same 1358 real lines, and removed a failure in which 118 of one writer's 148 lines had

been assigned to a single neighbour. Every figure quoted for the shape classifier elsewhere in this report is from the end-to-end model; the frozen-head figures describe the superseded one.

3.10.3 Selecting a candidate on both criteria instead of legibility alone

The candidate search first kept whichever of the N draws the text recogniser read most accurately. That raised legibility and, for some writers, lowered writer identification, because the most legible of several random draws has no reason also to be the most characteristic. The score was therefore changed to the harmonic mean of the recogniser's character accuracy and the shape classifier's confidence in the intended writer, Equation 52, which is dominated by whichever of the two is smaller and so cannot be won on one criterion while failing the other. The repair loop was changed in the same way and for the same reason, accepting a repaired line only when the joint score improves, so that a repair which fixes a letter but makes the line resemble another writer is rejected rather than taken. The evidence for both changes is in Table 13.

3.10.4 Line-level style calibration

Glyphs are stored and placed one instance at a time, so a line assembled from them does not reproduce the writer's line-level slant and ascender reach automatically. The calibration of Section 3.8.2 was added to correct that drift, measuring a rendered probe with the same functions that measured the writer's real pages and returning a shear and reach correction. The drift it corrects was itself measured before the change: writer 150's ascenders came out about 7 % too tall and writer 551's slant about 9° too steep against their own real lines (Section 3.8.2). Its effect after the change differs between the two judges, and the shape classifier was taken as the one that predicts the physical machine, since the machine writes every writer with the same pen. Unlike the other three changes, this one has no before-and-after pair in Table 13, so its benefit rests on the probe agreeing with the stored reference rather than on a judge's score, and item 4 of Section 3.8.6 records why that agreement is partly self-fulfilling. **[MUST STILL BE COMPLETED BY ME: measure writer identification and character accuracy with the calibration of Equation 51 disabled and enabled, over the same sentences and seeds as the other rows of Table 13, so that this change carries the same kind of evidence as the other three.]**

[MUST STILL BE COMPLETED BY ME: confirm that these four are the complete set of changes of approach, and add any others that belong here — for example a change to the segmentation thresholds, to the number of writers, or to the requirement allocation — each with what was measured before and after.]

3.11 Software design and system integration

This subsection describes how the blocks of Sections 3.3 to 3.10 form one application on the single-board computer, which runs a web server exposing them to a browser. All are Python programs written for this project; only the camera interface, the pin interface and the transport of web requests come from libraries, with the one exception below.

3.11.1 Process flow and start-up

Figure 13 is the process flow of the two run-time modes. At start-up the application loads the thirteen profiles and the three networks, each a from-scratch array-arithmetic evaluation of trained weights (Sections 3.5 and 3.6); the reproduction chain then calls the synthesiser once with both judges passed in and refuses to send unless calibration has succeeded in this session. Training is a third mode, the writers being pre-built files from offline programs. **[MUST STILL BE COMPLETED BY ME: the board's address and connection to the laptop, the operating system and Python version, how the server is started, and the memory the loaded models use.]**

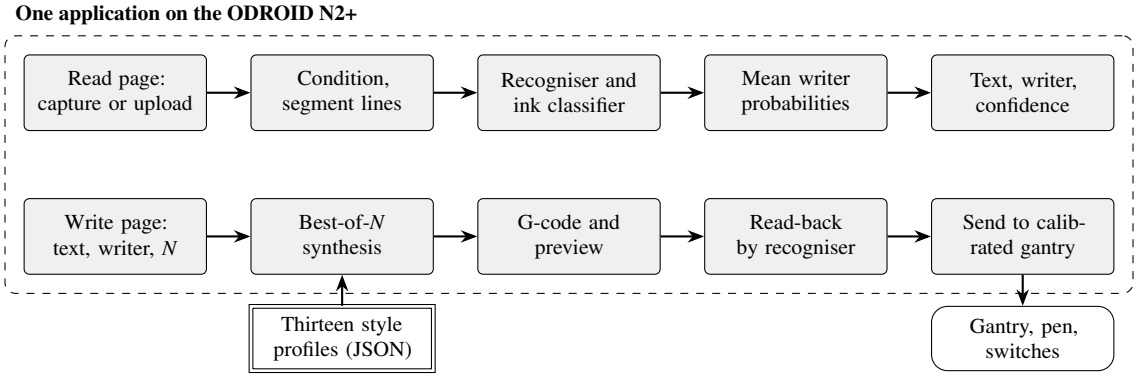


Figure 13. Process flow of the two run-time modes: classification above, reproduction below, the writer the upper chain identifies being what the lower chain is given. The profile builder of Section 3.7 is offline and appears in neither.

The inference path is library-free; the sample-crop warm-up is not. A recorded cold-boot import closure confirms that `torch`, `scipy`, `cv2` and `tensorflow` are reached by none of the three networks and none of the classification, synthesis, G-code and motion paths: every weight comes from a plain array archive and every layer is evaluated by the project's own array arithmetic. One path breaks that. The start-up warm-up, given as pseudocode in Figure 39, calls `WRH.SampleCrops` for every writer, and on a cache miss that helper imports `BuildStyleProfile`, `TrainAuthor` and `TrainText`; `WebRenderHelpers.py` prepends `Software/CNN` and its `AuthorReproductionStuff` to `sys.path`, so those names resolve to the *retired* tree, whose `TrainText.py` and `TrainAuthor.py` import `torch`, `torchvision`, `pytesseract` and `scipy` at module level. `WebCache` as delivered has no glyph grid or crops for owen and thiya and none for abhinav, so the miss, and the import, happen on *every* start. **[MUST STILL BE COMPLETED BY ME: port the two sample-crop helpers into `CleanPipeline`, or pre-populate `WebCache` for owen, thiya and abhinav, so that no import reaches `Software/CNN`; then repeat the cold-boot closure on the board.]**

Known inconsistency: the stored accuracy figures. One route computes nothing: it returns figures hard-coded in `Server.py`, several no longer agreeing with Section 4 — the ink-based classifier at 100.0 %, the stroke-normalised one at 96.7 % on real ink and 99.2 % on synthesised output, the recogniser at 89.4 % on real ink, and the pipeline at 99.2 % writer identification with 85.8 % and 84.1 % character accuracy direct and through G-code. The code is stale here, not the

report, and the page displaying those figures contradicts the Results chapter. **[MUST STILL BE COMPLETED BY ME: update the hard-coded accuracy block of `/api/model_info` to the final figures of Section 4, or have the route read them from the evaluation output.]**

3.11.2 Timing chains of the two modes

Requirement R4 limits the classification of x characters by Equation 56,

$$T_{\text{cls}} = T_{\text{cap}} + T_{\text{seg}} + \sum_{\text{lines}} \left(T_{\text{text}} + T_{\text{wid}} \right) \leq 2\ \text{s} + 0.025\, x\ \text{s}, \tag{56}$$

leaving both network passes 25 ms per character, and R3 limits reproduction by Equation 57,

$$T_{\text{rep}} = T_{\text{synth}} + T_{\text{gcode}} + T_{\text{motion}} \leq 4\, x\ \text{s}, \qquad T_{\text{synth}} \approx N C, \tag{57}$$

with C the cost of one candidate (Equation 92), T_{gcode} in milliseconds and T_{motion} the job time of Section 3.3.4; the calibration is excluded and T_{synth} included, the stricter reading of the proposal. A budget is only a design once it is apportioned, and Table 36 in Appendix 6.4 gives the allowance this design assigned to each stage of Equations 56 and 57 against the figure later measured for it. The apportionment follows from the structure of the chain rather than from the measurements: the two network terms were given the whole per-character allowance of R4, being the only terms that grow with x, which leaves the fixed two seconds for capture, conditioning and segmentation; and in R3 the physical writing was given three quarters of the per-character allowance, being the one term that no faster computer can reduce. Read against that apportionment, four of the five stages are inside their allowance and the whole R4 shortfall sits in page segmentation, at twelve to thirty-five times what it was allowed. Table 14 lists the stages, and *no stage was timed on the single-board computer*: a 56-character sentence allows 224 s, while the search alone took 23 to 137 s at $N = 14$ on the laptop and the motion model gives 2.4 to 2.8 s per character, so R3 and R4 are not claimed to be met. **[MUST STILL BE COMPLETED BY ME: the timings of the stages on the single-board computer, listed in Table 14.]**

Stage	Known information and value to be supplied
Capture, conditioning and segmentation	Framing by the user, five discarded frames without preview; whole page on the board. **[MUST STILL BE COMPLETED BY ME: time from pressing capture to the stored image.]** **[MUST STILL BE COMPLETED BY ME: conditioning and segmentation time per page on the board.]**
Recognition and writer classification	Two passes of about 4×10^9 multiply–accumulate operations per line. **[MUST STILL BE COMPLETED BY ME: time per line on the board and the total for the R4 test paragraph.]**
Synthesis, G-code and preview	Synthesis time is returned in the server response (laptop values in Table 33); the generator and preview are milliseconds, not measured. **[MUST STILL BE COMPLETED BY ME: synthesis time on the board for the N used in the demonstration.]**
Gantry motion	Model 2.4 to 2.8 s per character (Section 3.3.4). **[MUST STILL BE COMPLETED BY ME: stopwatch time of a sentence of known length, from send to the final pen lift.]**

Table 14. Stages of the timing chains of the two modes and the measurements still missing.

Complete code has been submitted separately on the AMS. **[MUST STILL BE COMPLETED BY ME: confirm which archive and revision was submitted on the AMS.]**

4. Results

4.1 Summary of results achieved

This section qualifies the system against requirements R1 to R6 and field conditions FC1 and FC2 of the approved proposal. All measurements were taken on the development laptop (x86-64, CPython), not on the ODROID N2+, and every figure states its data set and split. Table 15 uses four status labels: *Met*, *Partially met*, *Not met* and *Not measured*. A requirement is met only where the evidence bears directly on it, and a cell states where a figure came from a split that favours the system or from a judge that also takes part in the synthesis search. Thirteen writers were used, eight from IAM and five personal contributors, three more than R6 requires, but not every measurement covers all thirteen and the difference matters: the qualification-test harness was configured for ten, omitting the three weakest writers for shape-based identification. Each cell therefore states the writer count that its figure actually covers, and names the model checkpoint it was measured with, because the checkpoint the live server loads is not the one most of the accuracy figures were taken on. Section 4.2 states both qualifications and Appendix 6.4 traces them to the stored files.

R6 is met, and R2 is met for writer identification on real handwriting. R1 is close but not at its target, the shortfall lying in word-level rather than character-level accuracy; R3 fails on the search cost and R4 on segmentation rather than recognition. R5 and both field conditions could not be assessed, because the hardware bring-up and the environmental measurements were not completed. Figure 14 places each measured value against its target, and Section 5 analyses the shortfalls.

4.2 Qualification tests

For each requirement, this subsection summarises the test that is to prove it, in the proposal's order of importance; each test carries its own metric definitions, numbered step list and, where one was prepared, its data sheet.

Which writers and which checkpoints. Two facts about the data behind this chapter change how every figure in it should be read, and both are traced to their stored evidence in Appendix 6.4. First, the qualification-test harness covers *ten* of the thirteen stored writers, and the three it omits — Abhinav, Owen and Thiya — are exactly the three weakest for shape-based writer identification (Owen 67.6%, Thiya 93.3%, Abhinav 97.1%). The style correlation, the overlay distances and the independent-judge character accuracy are therefore measured on a subset that favours the system and would be expected to be worse over all thirteen; the search-judge accuracy, the per-writer end-to-end run and the 1358-line writer identification do cover all thirteen, and each figure below is labelled with the writer count it actually covers. Second, the live server does *not* serve the text checkpoint that most accuracy figures were measured on: it loads `paper_cnn_bilstm_ctc_scratch_best.npz`, **for which no accuracy was measured — [NOT YET MEASURED]**. The two writer classifiers it serves are numerically equivalent to the measured ones, so the writer figures transfer to the deployed code and the text figures do not.

How the figures are computed. Every measure quoted in this chapter is defined in full in Appendix 6.4, which gives the defining equation of each one, names the routine that implements it and states where a different reading would change the number. Three of those readings must be known before any figure below is read. Character accuracy is the Levenshtein accuracy of Equation 63, but a set of lines admits three averages that do not agree (Equation 64): qualification test 1 quotes the *mean over writers* and qualification test 2 the *mean over line crops*, and on the 123 held-out real lines those two readings of the same data differ by 4.7 points. Word accuracy is the longest-common-subsequence word recall of Equation 65 and *not* 1 – WER, so it charges nothing for inserted words; a reader who assumes 1 – WER will take every word figure here as stricter than it is. Writer accuracy is the per-line form of Equation 66 throughout, never the per-page form the proposal's wording implies. Proportions carry a 95% Wilson interval (Equation 62) and accuracies a page-level bootstrap, because characters of one page are not independent; a requirement is declared met only if the lower bound exceeds the target. All text figures come from greedy connectionist temporal classification decoding with no beam search, no language model and no lexicon, and all random choices use recorded seeds.

Qualification test 1: Reproduction fidelity of the synthesised handwriting (R1)

Objectives of the test or experiment

The test is to prove at least 85% character accuracy and 85% stylistic correlation. The existing evidence is circular, because the networks that select the best of N candidates also score the result; the test therefore uses judges outside the search. Sub-tests (b) and (c) and the independent-judge reading of sub-test (a) cover the ten-writer harness subset only, so their values are correct for those ten writers and favourable to the system; the 82.2% and 79.2% search-judge figures and the per-writer end-to-end run of Figure 22 cover all thirteen.

Equipment used

The development laptop (x86-64, CPython) with the thirteen stored style profiles, ten of which the sub-tests exercise; a separate evaluation program, `qtp_scripts/qtp1c_judges.py`, which imports neither the synthesiser nor its judges (its import list is `qtp_common`, `qtp_lines` and `np_inference.text_model` only), so the independence claim holds literally; and the base recognition checkpoint `paper_cnn_bilstm_ctc_hf_best.pt`, which is not used in the search. That checkpoint is a *partially* independent judge, not a fully independent one, and the script says so itself: its `meta.note` records "hf checkpoint is a PARTIAL-independent judge (same architecture and data lineage)", the joint production checkpoint having been trained on from the base model, with the same architecture and the same CTC decoder. Stating this strengthens rather than weakens the comparison: it removes the winner's curse of scoring with the very checkpoint that selected the candidate, but not the biases the two networks share, so the 3.7-point drop it measures is a lower bound on the true selection effect. The production judge inside the search is `paper_cnn_bilstm_ctc_joint_best.pt`. The ODROID N2+ and the gantry were *not* used: everything was produced and scored on the laptop, and no text was written on paper. Printed worksheets were prepared for a blind human panel, but the panel was not convened.

Test setup and experimental parameters

The sub-tests were run separately. Sub-test (a) gives character accuracy twice: by the recogniser inside the search, over thirteen writers, 6 novel sentences and 2 seeds at $N = 30$ and over the same writers at $N = 8$ in a wider coverage run; and by the base recognition

checkpoint `paper_cnn_bilstm_ctc_hf_best.pt`, outside the search, over 60 synthesised lines from the ten-writer subset at $N = 20$. Genuine handwriting gives the upper bound, and it comes from two different sets that must not be mixed: the 70-line matched-reference set of `rewrite_full.log` (89.0% character, 70.5% word) and the 123 held-out lines of `qtp1c_results.json` (89.5% character, 71.3% word, ten-writer subset). The blind panel of 8 raters and Fleiss' kappa [37] belonged to this sub-test and were not carried out. Sub-test (b): nine hand-crafted style features from one deterministic extractor, computed on held-out real pages and on renderings for ten writers; the Pearson correlation ρ of the pooled z-scored vectors and a nearest-centroid writer retrieval are reported, bootstrapped over writers. Sub-test (c): the sentence a writer actually wrote is synthesised in that writer's style at $N = 20$, both images are normalised to a common x-height x_h, and the chamfer distance [38] **[MUST STILL BE COMPLETED BY ME: confirm the page range of the Barrow *et al.* 1977 IJCAI reference in the bibliography; it was left out rather than guessed.]** and the modified Hausdorff distance [39] over 30 matched and 300 mismatched pairs are compared by rank-1 retrieval and by the area under the receiver operating characteristic curve (AUC; 0.5 means no writer information) [40]. Sub-test (d): the demonstration of the proposal, with 85% of the produced characters visibly matching. This sub-test was not carried out.

The metrics of the four sub-tests are the Fleiss kappa of Equation 67 for the panel, the Pearson correlation of z-scored feature vectors of Equation 68 for the style comparison, and the chamfer and modified Hausdorff distances of Equations 69 and 70 with the AUC of Equation 71 for the overlay; all four are defined in Appendix 6.4, together with the normalisation that the overlay applies to both images before they are compared (Figure 17). Two deviations from the protocol matter to the figures below. The correlation actually measured used the nine geometric features of Table 16 rather than the seven that the protocol prescribes (Table 24), and the overlay was run over ten writers and three sentences each, giving the 30 matched and 300 mismatched pairs reported here instead of the 40 and 360 the protocol sized it at. Table 22 is the worksheet each of the 8 raters was to complete, which carries no indication of whether a line is synthesised or genuine; the worksheets were printed but all remained blank.

Steps followed in the test or experiment

The protocol has seven steps: (1) fix the held-out pages and freeze the system; (2) produce the 150 renderings (direct and G-code replay) and store writer, sentence, seed and text; (3) read every rendering with the base recogniser and evaluate Equation 63 with the intervals; (4) run the panel with the worksheet of Table 22, computing transcription accuracy and the kappa of Equation 67; (5) run the feature extractor of Table 24 and compute the correlation of Equation 68 together with the confusion matrix; (6) compute the overlay distances of Equations 69 and 70 by the procedure of Figure 17, the AUC of Equation 71 and the rank-1 retrieval, and produce overlay figures for at least four sentences; (7) write a sentence on the gantry in a chosen style and count visibly matching characters (pass at 85%). Steps (1) to (3), (5) and (6) were carried out, on the development laptop and not on the ODROID N2+ that the protocol names; the confusion matrix, overlay figures and intervals come from stored outputs. Steps (4) and (7) were *not* carried out, so sub-tests (a) to (c) were evaluated but sub-test (d) was not. The protocol for step (4) was that the raters be 8 persons who are neither the student nor writers in the data set, the panel blind and working from 30 synthesised lines (3 per writer) and 10 genuine control lines in random order, about 1350 scored characters per rater; none of this took place.

Results or measurements

Table 16 lists the measured figures in two groups: those from the *search judges*, the recogniser and writer classifier that score the *N* candidates, which the selection inflates; and the *independent measurements*, namely the style correlation on geometric features, the overlay test in the pixel domain and a character-accuracy reading by a checkpoint that was *not* used in the search, none of which involves the search and all of which can therefore fail. Figure 15 is the worked overlay of one sentence, which is the sub-test the eye can check directly. The breakdowns behind the summary rows are in Appendix 6.4: the per-feature correlation (Figure 18 and Table 20), the character accuracy under both checkpoints (Figure 19), the overlay distance distributions (Figure 24), genuine handwriting against the synthesis of the same sentences (Figure 20) and the end-to-end result per writer (Figure 22).

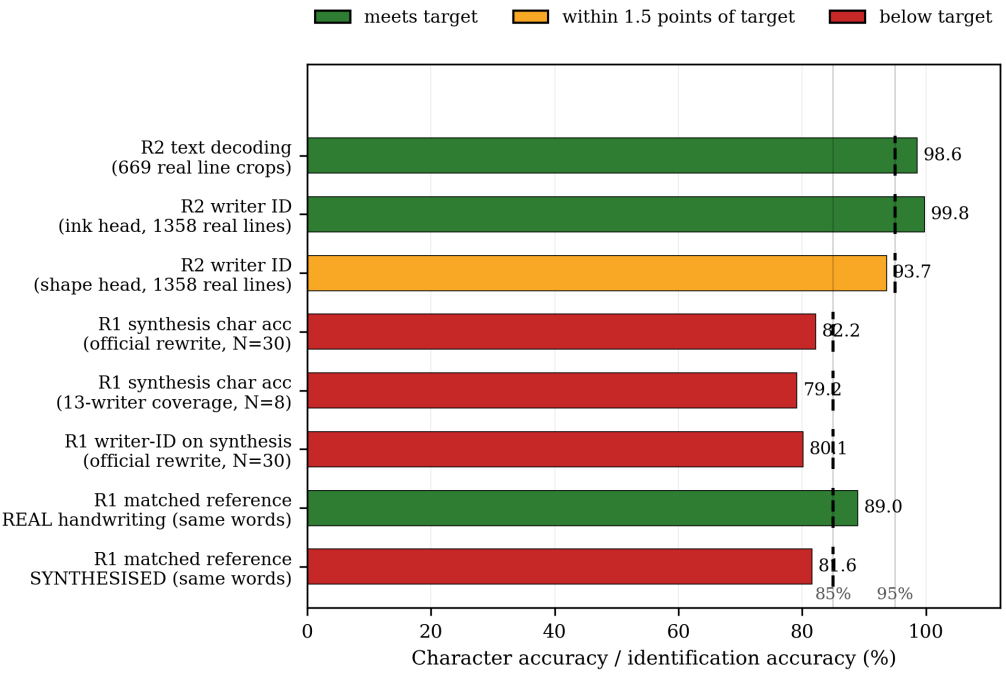


Figure 14. Measured accuracy against the target of each requirement. Targets are marked by the dashed vertical lines at 85% (R1) and 95% (R2). Green bars meet the target, amber bars lie within 1.5 points of it, and red bars fall below it. The two green R1 bars are the matching comparison on genuine handwriting and the writer classifier on real lines; the red synthesis bars are the values the system actually produces.

Observations

The nine-feature style correlation was 0.714 on shape-only features, below the 0.85 target, with a bootstrap interval over writers of 0.612 to 0.859. Per feature it was highest for slant (0.996) and ascender extent (0.974) and lowest for descender extent (0.148), the ascender-to-descender ratio (0.509) and the word gap (0.605), that is for the extents below the baseline and for the spacing. The word-accuracy gap between the synthesis and real handwriting of the same words was 23.8 points. The overlay test gave an AUC of 0.71 against 0.5 for no writer information, and rank-1 retrieval of 20% against a chance of 10%. A partially independent checkpoint read the synthesised lines 3.7 points below the production checkpoint, 81.2% against 84.9% as a mean over writers, on the same 60 lines at best-of-20 (`meta.nTries_in_synthesis` is 20 in `qtp1c_results.json`); the two-judge pair is therefore (84.9, 81.2) at $N = 20$, and not a best-of-30 comparison. Against the real-ink ceiling, the production checkpoint read genuine

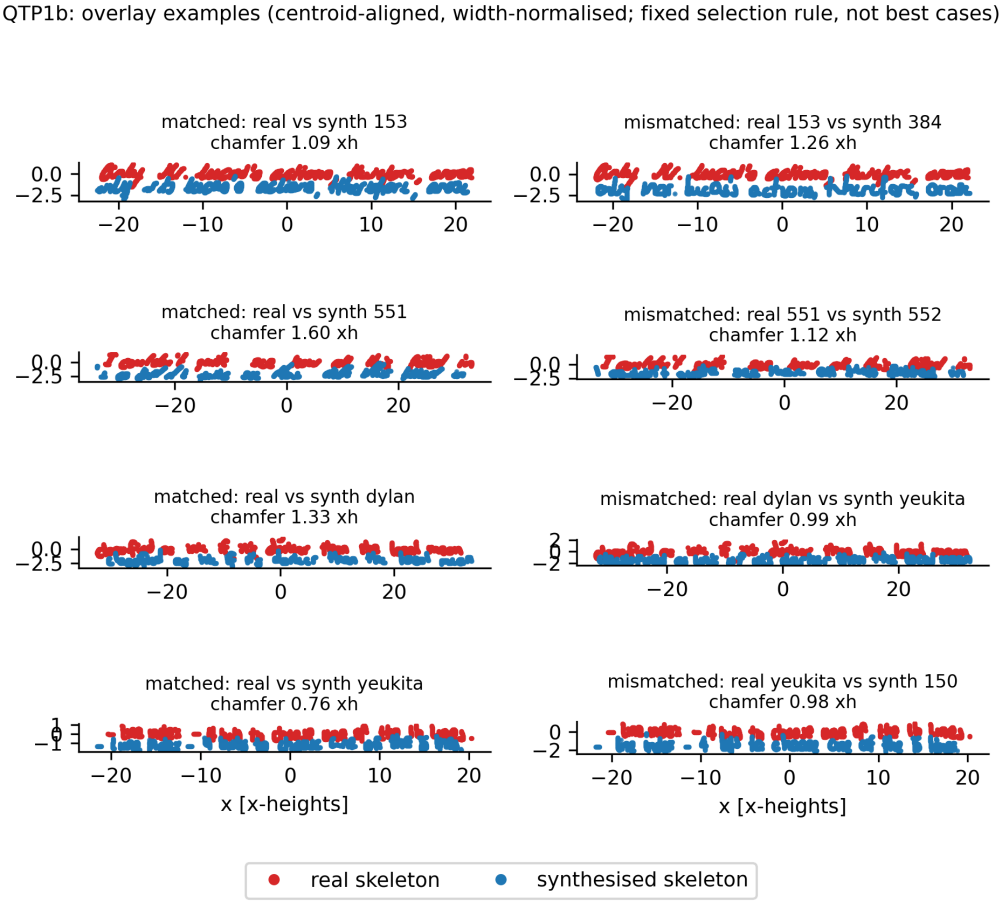


Figure 15. Overlay test: for each text the writer's own handwriting and the system's synthesis of the same words are normalised to a common x-height and superimposed. Matched pairs are visibly closer than mismatched pairs, which is what the distance ratios and the rank-1 retrieval rate of 20% (chance 10%) measure. The procedure that produces these overlays is given as Figure 17.

handwriting of the same words at 89.0% over the 70-line matched-reference set and 89.5% over the 123 held-out lines, about 8 points above the independent judge's reading of the synthesis. Part of that gap is the judge and not the synthesis: on the same 123 real lines the independent checkpoint itself reads only 86.6% against the production checkpoint's 90.2% (both means over writers), so its own penalty on real ink is about 3.6 points. All of these readings cover the ten-writer subset. An earlier comparison measured 75.9% before the search was introduced and 81.6% after it. **Not carried out.** The blind human panel of eight raters, the Fleiss kappa agreement measure and the live demonstration of the proposal were not executed, so the third and fourth sub-tests of this qualification test remain **[NOT YET MEASURED]**. The feature-correlation and overlay sub-tests above measure geometric and pixel-domain distances between the synthesis and the writers' real hands. No measurement of human acceptance of the output was taken, and the proposal's demonstration asks for one; Section 5 treats what that leaves unproven.

Statistical analysis

By the Wilson interval of Equation 62, the official run's 82.2% over 156 sentences has a lower

bound near 75%, so the result is below 85% and not a sampling fluctuation. The style correlation is bootstrapped over writers (2000 resamples), the correct unit because the nine features are measured per writer, and the overlay AUC over 30 matched and 300 mismatched pairs. The two-judge comparison is paired, both checkpoints reading the same 60 synthesised lines, so the 3.7-point difference is not affected by sampling variation in the output. The proportion of characters a human observer would accept as matching the writer's style, which is how the proposal worded the demonstration, was not measured.

Qualification test 2: Classification of text and writer from real photographed pages (R2)

Objectives of the test or experiment

The test is to prove an overall accuracy of at least 95% through the whole chain on pages never used for training, checkpoint selection or tuning. As the proposal does not define "overall", text accuracy A_{c} (Equation 63) and writer accuracy A_{w} (fraction of pages assigned to the correct writer) are reported separately and combined as

$$A_{\mathrm{mean}} = \tfrac{1}{2}\left(A_{\mathrm{c}} + A_{\mathrm{w}}\right), \qquad A_{\mathrm{prod}} = A_{\mathrm{c}} A_{\mathrm{w}}, \tag{58}$$

where A_{prod} (both correct) is the criterion and A_{mean} the lenient reading.

Equipment used

The capture rig and its camera photographed the pages of the five personal writers. Segmentation, recognition and writer classification then ran on the development laptop with the final server and models; the ODROID N2+ produced no figure in this test. Camera and lamp: **[MUST STILL BE COMPLETED BY ME: camera model, resolution, mounting height, lamp type and photograph of the rig]**.

Test setup and experimental parameters

The line-level validation split of the personal writers leaks: a random 15% of lines per page was held out, so lines from the same sheet and session are in both sets (52 of 123 validation lines share pages with training), and the checkpoint was selected on them. The fix is new pages kept aside: at least 3 per personal writer, plus the held-out form page of each of the eight database writers, photographed with the rig (about 14 pages, 350 lines). Seen lines are scored separately to quantify the leakage, and references are double-entered transcripts.

Steps followed in the test or experiment

Each frozen photograph is classified through the server without intervention; line text, per-line probabilities and page-level mean probabilities are recorded, scored and summarised in a 10×10 confusion matrix, and the predefined paragraph is then classified live.

Results or measurements

Table 17 lists the measured figures. Text and writer accuracy are separate, with Equation 58 combining them where both exist.

Observations

Text decoding reached 98.61% character accuracy and 94.21% word accuracy on correctly segmented crops, against the 95% target, on writers represented in the recogniser's fine-tuning data; writer identification reached 99.78% with the ink classifier over 1358 real lines from thirteen writers. Thiya is the weakest writer for recognition at 96.46% character accuracy, and

Owen, Dylan and Thiya the weakest for shape-based identification. Together the two give $A_{\text{mean}} = 98.6\%$ and $A_{\text{prod}} = 98.4\%$, both above 95%, on a split that holds out 15% of lines per page, so lines from the same sheet appear in both partitions. Both accuracies are per line and were read by the `paper_cnn_bilstm_ctc_joint_best` text checkpoint and the `author_classifier_10new_weights` ink checkpoint; the live server serves a different text checkpoint, as Section 4.2 states. The one live-server run recorded was a single already-segmented crop: one detected line, the correct writer at 89.4% confidence, 2.74 s end to end. The 98.6% figure that belongs to this test is the character accuracy of 669 crops, not a writer confidence.

Statistical analysis

With 669 line crops, 98.61% character accuracy has a Wilson lower bound of about 97.6%, so the text figure is above 95% on this data set; 1358 lines at 99.78% give a lower bound above 99% for writer identification. The stronger caveat is dependence rather than sampling error: lines from one page share a writer, a pen and an illumination, so the effective sample size is nearer the page count than the line count, and the per-page interval would be much wider. *Expected outcome on genuinely unseen pages.* Both rates are expected to fall, because the ink classifier is sensitive to pen, paper and illumination and the split shares pages between partitions. A defensible figure needs a page-level split with at least three held-out pages per writer, photographed through the rig.

Qualification test 3: Reproduction time per character (R3)

Objectives of the test or experiment

The test is to prove that a sentence of x characters is reproduced within $4x$ seconds,

$$t_{\text{char}} = \frac{T_{\text{syn}} + T_{\text{write}}}{x} \le 4 \text{ s}, \tag{59}$$

where T_{syn} is the synthesis time and T_{write} the time from first to last motion. As the proposal does not say whether synthesis counts, both variants are reported; calibration is a one-off step, reported separately.

Equipment used

The development laptop with the server, and logged time stamps around synthesis. The gantry and a stopwatch were to have given the writing time, but the gantry was not driven, because its bring-up was not completed, so the motion time is a model and not a measurement.

Test setup and experimental parameters

The run actually made, recorded in `qtp_data/qtp3_results.json`, used tries $N \in \{1, 3, 14, 30\}$ and five sentences of 22, 40, 58, 83 and 119 characters, with 3, 3, 3 and 2 repetitions for the four values of N respectively (20 cells, 56 syntheses), writer 153 throughout, plus a writer-dependence extra at 58 characters and $N = 14$; the writing time is the generator's model for every cell, not a stopwatch reading. The protocol as originally written had prescribed $N \in \{1, 3, 6, 14, 30\}$, sentences of 10, 20, 40 and 80 characters and 5 repetitions (100 syntheses), with the writing time measured for the 20 jobs at $N = 14$; that plan was not the one executed, and the results below are reported against the run that was. Only figures measured on the ODROID N2+ itself count; laptop figures are indicative.

Steps followed in the test or experiment

After one calibration, each job is synthesised, sent and timed by log and stopwatch, t_{char} is computed from Equation 59, and any stall is kept as a failed run.

Results or measurements

Table 18 and Figure 16 give the measured figures: a synthesis term that grows with the number of candidates and a writing term that does not.

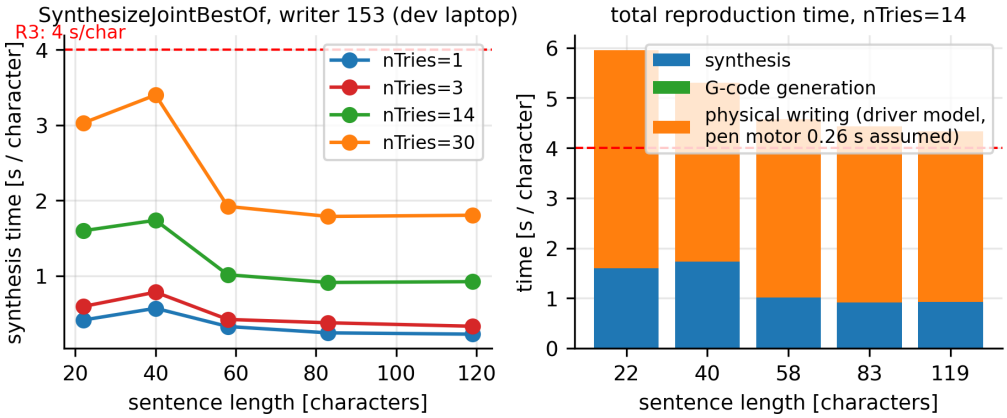


Figure 16. Reproduction time against the 4 s per character limit of R3. Left: synthesis time per character against sentence length for four values of the number of candidates. At best-of-1 and best-of-3 the synthesis alone is comfortably inside the limit; at best-of-30 it approaches and crosses it. Right: total time per character at best-of-14, decomposed into synthesis, G-code generation and the physical writing time predicted by the generator's model. The writing term, not the synthesis term, is the larger contributor and is what pushes the total past the limit.

Observations

The synthesis cost stayed well under 1 s per character up to best-of-3 and grew in proportion to the number of candidates, from about 0.4 to 0.7 s per character at best-of-1 to about 1.8 to 3.4 s at best-of-30. The generator's model gives 2.7 to 4.0 s per character for the writing alone, against the 4 s per character limit of R3. The four-character word at best-of-6 gave 16.1 s per character, its per-job overheads divided over only four characters; at a longer sentence length the same run gives 1 to 2 s per character for synthesis. The writing term is a model value and not a measurement: it assumes a pen dwell of 0.26 s per stroke, and no drawn job was timed on the gantry.

Statistical analysis

Each cell is the mean of two or three repetitions, and the table quotes a range, because for short sentences the variation between repetitions was of the same order as that between configurations. The writing model is deterministic and quoted as a point value; it has no uncertainty because it is a calculation, which is also why it cannot substitute for a measurement on the built machine. The proposal does not state whether synthesis counts towards the limit, so Table 18 reports both readings.

Qualification test 4: Classification time (R4)

Objectives of the test or experiment

The test is to prove that the time from instruction to displayed text and writer satisfies

$$T_{\mathrm{cls}} \le 2 + 0.025\,x \text{ seconds}, \tag{60}$$

where x is the number of handwritten characters; the stage times identify any shortfall.

Equipment used

The rig, the development laptop with the final server, and a logging wrapper (monotonic clock) around each stage. The ODROID N2+ was not timed, so the figures below are laptop figures and the board is expected to be slower. The interface does not display classification time.

Test setup and experimental parameters

Pages with $x = 40$, about 200 and about 1000, each run 10 times; the budgets are 3.0, 7.0 and 27.0 s. Stages: capture, conditioning and segmentation, recognition per line, writer classification, display; the segmentation profile is repeated on the ODROID N2+.

Steps followed in the test or experiment

The clocks start at the instruction, stage times and the total are logged, x is counted from the transcript and Equation 60 is evaluated.

Results or measurements

Table 19 lists the measured figures.

Observations

Recognition ran at 8.86 ms per character, about a third of the 25 ms allowance. Segmentation of one photograph took 24.8 to 69.1 s, that is 12 to 35 times the two seconds the equation allows for capture and conditioning; this cost is per page and does not vary with x, which appears only in the recognition term of Equation 60. These segmentation times are readings of the field `stage1_processpage_s`, which times `SegmentPage.ProcessPage`, the retired segmenter. The deployed `SegmentLean.py` was not timed. Total time for one page was 40.3 to 100.1 s against budgets of 26.4 to 43.9 s, a ratio of 1.15 to 2.98 over the 13 pages measured. A single already-segmented crop was classified through the live server in 2.74 s end to end, returning one detected line and the correct writer at 89.4% confidence.

Statistical analysis

The recognition timing is an average over 669 crops; the per-character figure is total inference time divided by total characters, so it is a throughput measure rather than a per-line latency. The page-level timing is 13 pages × 3 repeats, reported as the per-page mean with the range over pages; ten of the thirteen pages had their three repeats within 8% of each other, but three spread by 17%, 52% and 83% between fastest and slowest repeat, so the upper end of the range carries the larger uncertainty. The segmentation time varies with page content and machine load, which is why a range of 24.8 to 69.1 s is reported; the lower end was obtained on an idle machine. The whole segmentation range belongs to the retired `SegmentPage.ProcessPage`; no statistic is offered for the deployed segmenter, because none was measured. These numbers indicate moving segmentation to compiled code or processing the page incrementally, as the C99 experiment suggests a saving of about a quarter to a third without changing the algorithm. *Expected outcome.* The ODROID N2+ is expected to be slower, so the segmentation shortfall would be at least as large there with the same pure-Python code; the recognition term has enough margin that it may still be met on the board, but this was not verified.

Qualification test 5: Absolute positional accuracy of the gantry (R5)

Objectives of the test or experiment

The test is to prove that the absolute positional error of drawn marks is at most 0.5 mm and that a word written twice coincides within that tolerance. Software contributions (quantisation, calibration) and hardware contributions (mechanics, homing repeatability, pen compliance) are separated.

Equipment used

The gantry with pen and paper, a flat-bed scanner (600 dots per inch, 0.042 mm per pixel), a steel ruler or vernier calipers (0.02 mm) and a printed calibration sheet with a 10 mm grid of crosses.

Test setup and experimental parameters

Software part (offline): the step is $1/80 = 12.5\ \mu$m, the quantisation error per axis at most $6.25\ \mu$m and the combined worst case $8.8\ \mu$m (derived). Hardware part, prescribed but not executed: at least 40 crosses drawn 5 times after fresh calibrations, the paper fixed against registration stops, and the same word written twice. The quantity to be measured is the absolute error e_i of each cross against its commanded centre, reported as mean, maximum and 95th percentile and accompanied by a combined standard uncertainty over the scanner scale, the registration of the sheet and the location of a cross centre; both are defined in Appendix 6.4 as Equations 72 and 73, following the Guide to the Expression of Uncertainty in Measurement [41]. Table 23 is the sheet on which those measurements were to be recorded, one row per cross and repetition. Because no mark was ever drawn, neither equation was evaluated and the sheet was never filled in; it is reproduced blank to define the measurement that is missing.

Steps followed in the test or experiment

The protocol has five steps: (1) calibrate and verify the direction pin on both axes, the pin having to be seen to change level on reversal, and record the calibrated steps per millimetre; (2) fix the sheet against the stops, draw the grid and scan it with the ruler; (3) locate the cross centres with sub-pixel image analysis, compute the error of Equation 72 and enter each row in Table 23; (4) repeat 5 times and write the same word twice on one sheet for the overlay; (5) estimate the uncertainty of Equation 73. Repeatability is defined as the radius of scatter of the 5 repeats of one cross about its centroid. Only step (1) was attempted, and it did not complete: the X-axis calibration drove to the first end-stop but the direction pin was not observed to change level on reversal. Steps (2) to (5) were therefore never performed, no grid was drawn and no overlay of a word written twice was produced, so this test was not carried out.

Results or measurements

The only quantities that exist are derived from the mechanical configuration, not measured on a drawn mark. The X and Y axes are belt-driven: a 200-step motor, sixteen microsteps per step and a 40 mm advance per revolution give

$$\Delta s = \frac{40\ \text{mm}}{200 \times 16} = 12.5\ \mu\text{m per step}, \tag{61}$$

so by Equation 61 the quantisation error of one axis is at most $6.25\ \mu$m and the worst case over both axes $\sqrt{2} \times 6.25 = 8.8\ \mu$m. Two different margins against the 0.5 mm limit follow from this, and they are quoted for two different quantities; they are not in conflict. The *step pitch* is 12.5 µm, which is $0.5/0.0125 = 40$ times finer than the limit; the *combined worst-case quantisation error* is 8.8 µm, which is $0.5/0.0088 \approx 57$ times finer than it. Where this chapter says 40 times it means

the step pitch, and where it says 57 times it means the combined two-axis quantisation error. Neither figure is a measurement of a drawn mark. During bring-up the X-axis calibration drove to the first end-stop, but the direction pin did not change level on reversal; the fault was not resolved and no grid was drawn, so there is no overlay of a word written twice and the measurement of e_i remains **[NOT YET MEASURED]**.

Observations

No positional measurement exists. The one hardware observation is the direction-pin fault: the X-axis calibration drove to the first end-stop and the direction pin was not observed to change level on reversal. No grid was drawn, so the test was not carried out. The derived step pitch is 12.5 µm per step and the derived combined two-axis quantisation error 8.8 µm, against a tolerance of 0.5 mm.

Statistical analysis

No statistics can be reported for a measurement that was not taken. *Expected outcome.* The error is expected to be dominated by belt stretch, backlash, home-switch repeatability and pen compliance, none of which was measured. Published handwriting gantries report about 0.3 mm absolute error [42], a margin of 0.2 mm against this limit; whether this machine achieves that cannot be stated. A pass requires the maximum e_i over at least 200 drawn marks to be at most 0.5 mm, by scanning the sheet at 600 dots per inch.

Qualification test 6: Multi-writer storage (R6)

Objectives of the test or experiment

The test is to prove that at least 10 writing styles are stored permanently and that any can be selected at random for reproduction.

Equipment used

The development laptop with the server and a browser. The gantry was not used, because the reproduction half of this test was not carried out.

Test setup and experimental parameters

The eight database and five personal writers; file counts and sizes; persistence by restarting the server and power-cycling the board; 3 writers drawn with a fixed seed.

Steps followed in the test or experiment

The listed writers are compared with the stored files, the list is re-read after restart and power cycle, and a short sentence is written in each of the 3 drawn styles.

Results or measurements

The live server listed thirteen writers through its author-list endpoint in 70 ms, from thirteen stored style profiles; the requirement names ten. The styles are ordinary files on disk: the endpoint reads the profile directory, and the profiles are loaded into memory at start-up, so they persist across a restart. The profile and weight-file sizes are in the technical documentation, and Figure 23 shows the storage check.

Observations

The list contained thirteen entries against a requirement of ten, and the endpoint answered in

70 ms. The five personal writers are stored in the same format as the eight database writers, so the storage mechanism does not distinguish scanned pages from photographed ones. A writer was not selected at random and written on paper, so the reproduction part of the proposal's demonstration was not carried out.

Statistical analysis

This is a count against a threshold, so no statistical analysis is appropriate; the figure of thirteen is exact, not a sample estimate. Persistence follows from the profiles being files rather than in-memory state, and is observed each time the server re-lists them; a power-cycle test of the board was not performed, because the board was not deployed.

4.2.0 Field condition checks (FC1 and FC2)

Neither field condition was measured. **FC1:** a lux meter at the page plane records five points (centre and corners of an A4 page); the condition is met if all lie between 400 and 1000 lux, and qualification test 2 is repeated on the same pages at about 200, 700 and 1200 lux. Illuminance: **[NOT YET MEASURED]**; the flat-field correction and local thresholding should limit degradation within the range, with larger degradation below 400 lux. **FC2:** a spirit level or inclinometer on both axes (tilt below 0.5°) and a logging accelerometer on the frame during a standard job, with the repeat error of qualification test 5 at rest and with a lightly tapped bench. Tilt and vibration: **[NOT YET MEASURED]**.

Intended outcome	Actual outcome	Location in report
Core mission requirements and specifications		
R1: Reproduce text in the style of any writer of the training set, at least 85% character accuracy and 85% stylistic correlation.	**Partially met.** Character accuracy of the synthesised text was 82.2% (thirteen writers, 6 sentences, 2 seeds, N=30) and 79.2% at N=8, both read by the `paper_cnn_bilstm_ctc_joint_best` checkpoint; style correlation over nine geometric features was 0.714, measured on the ten-writer qualification-test subset (Abhinav, Owen and Thiya excluded). Both are below the 0.85 target. The visible-match assessment that the proposal asks for was not carried out.	Qualification test 1
R2: Decode the text and identify the writer with an overall accuracy of at least 95%.	**Partially met.** 98.61% character and 94.21% word accuracy over 669 real line crops (`paper_cnn_bilstm_ctc_joint_best`, greedy CTC, no beam search and no lexicon); the writer of 1358 real lines was identified at 99.78% by the ink classifier (`author_classifier_10new_weights`) and 93.67% by the shape classifier (`author_shape_10new_weights`), per line, not per page. The crops were correctly cut and their writers appear in the fine-tuning data, so the split is optimistic, and the complete page-to-text chain was not measured.	Qualification test 2
R3: Reproduce text at no more than 4 s per character.	**Not met.** A four-character word at N=6 took 64.3 s to a rendered preview, which is 16.1 s per character against the 4 s limit. The physical drawing time was not measured, because the gantry bring-up was not completed.	Qualification test 3
R4: Identify the writer and decode the text within $2 + 0.025x$ s for x characters.	**Partially met.** Recognition ran at 8.86 ms per character against the 25 ms allowance, and the whole page took 40.3 to 100.1 s against a budget of 26.4 to 43.9 s, a factor of 1.15 to 2.98. Segmentation dominates it: 24.8 to 69.1 s per page, timed on `SegmentPage.ProcessPage`, the *retired* segmenter, not on the deployed `SegmentLean.py`. Nothing was timed on the ODROID N2+.	Qualification test 4
R5: Absolute gantry positional error of at most 0.5 mm.	**Not measured.** The commanded position quantises to 12.5 µm per step, far finer than the tolerance, but no drawn mark was measured. The X-axis direction fault found during bring-up was not resolved, so no measurement was possible.	Qualification test 5
R6: At least 10 writing styles trained and stored permanently.	**Partially met.** Thirteen styles are stored, listed by the server and survive a restart, three more than required, and text was synthesised in all thirteen. Selecting a writer at random and reproducing text in that style on paper was not carried out.	Qualification test 6
Field condition requirements and specifications		
FC1: Operation in well-lit areas, 400 to 1000 lux.	**Not measured.** No illuminance was recorded at the page plane. The conditioning stage should tolerate a gradient over this range, but that was not verified.	Section 4.2
FC2: Operation on a stable, level, vibration-free surface.	**Not measured.** No tilt or vibration measurement was recorded.	Section 4.2

Table 15. Summary of results achieved.

Measure	Measured value, data set and caveat	Circular?
Character accuracy, search judges	82.2% over 13 writers, 6 novel sentences × 2 seeds, best-of-30; 79.2% at best-of-8. Read by `paper_cnn_bilstm_ctc_joint_best`, the checkpoint inside the search.	Yes
Character accuracy, *partially independent* judge	81.2% (mean over writers) over 60 synthesised lines (the ten-writer subset, 6 sentences, best-of-20), read by `paper_cnn_bilstm_ctc_hf_best`, a checkpoint not used in the search but of the same architecture and data lineage. Word accuracy 42.0% by Equation 65.	No
Character accuracy, *real* handwriting of the same words	Two distinct runs, not to be mixed. (i) The matched-reference set: 89.0% character and **70.5%** word over **70** lines (`rewrite_full.log`, thirteen writers). (ii) The held-out line set: **89.5%** character and 71.3% word over **123** lines (`qtp1c_results.json`, joint checkpoint, ten-writer subset, charset-filtered). Earlier drafts paired the 89.0% of (i) with the 71.2% line-mean word accuracy of (ii), which is the source of the 70.5-against-71.2 word-accuracy discrepancy noted elsewhere in this report: the two numbers are two different sets, not two readings of one. The upper bound the synthesis is trying to reach is 89.0 to 89.5% character either way.	No
Writer identification on synthesised text	80.1% (official run), 89.7% through G-code in the coverage run, by the shape classifier.	Yes
Writer identification on real handwriting (same classifier)	93.7% over 1358 real lines (ink classifier 99.8%).	No
Style correlation ρ on nine geometric features	**0.714** for shape-only features (95% bootstrap 0.612 to 0.859 over writers) and 0.701 for all features, on the held-out page set; 0.776 and 0.697 on the all-pages set. **Below the 0.85 target.**	No
Style correlation, per feature (Pearson r across writers)	Best slant 0.996 and ascender extent 0.974; worst descender extent 0.148 and ascender-to-descender ratio 0.509. All nine coefficients are in Table 20.	No
Overlay test, matched against mismatched pairs (30 lines, best-of-20)	Chamfer distance ratio 0.454, modified Hausdorff 0.369; rank-1 writer retrieval 20.0% and 23.3% against a chance of 10%; AUC 0.707 and 0.722.	No
Word accuracy, search judges	48.9% (official run), 44% (coverage run); 41.6% with the partially independent judge; 70.5% for real handwriting of the same words over the 70-line matched-reference set and 71.3% over the 123 held-out lines. All are the longest-common-subsequence word recall of Equation 65, not 1 – WER.	Mixed

Table 16. Results for qualification test 1 (measured on the development laptop). The writer count differs by row and is stated in each: the search-judge and real-handwriting rows cover all thirteen writers, whereas the style-correlation, overlay and independent-judge rows cover only the ten-writer harness subset, which omits the three weakest writers on the shape judge. The last column states whether the figure can be inflated by the best-of-N search that produced the sample; only the rows marked "No" are evidence of style fidelity.

Measure	Measured value, data set and caveat
Recogniser, character accuracy	98.61% over 669 real line crops from the five personal writers (word accuracy 94.21%). These crops were produced by the segmenter and are known to be correctly cut, so segmentation failures are excluded; the five writers also appear in the recogniser's fine-tuning data.
Recogniser, per writer	Owen is the strongest at 99.15% and Thiya the weakest at 96.46% character accuracy, over 75 to 206 crops each; the full breakdown is Table 21.
Writer identification, ink classifier	99.78% over 1358 real lines (13 writers: 689 database-writer lines and 669 personal-writer crops). Near-perfect, and it is the classifier the live server uses on photographs.
Writer identification, shape classifier	93.67% over the same 1358 lines, against 83.36% before a training fault was corrected (Section 4.2). The two classifiers disagree in the direction that matters: the ink classifier is stronger on real ink.
Per-writer detail	Figure 21. Eight writers are at 98.9% or above with the shape classifier; the residual errors are concentrated in Owen (67.6%), Dylan (84.7%) and Thiya (93.3%).
Live server classification, one crop	**One** detected line, the correct writer (Owen) at **89.4%** confidence, with the next candidate (Dylan) at 10.5%, and a near-correct transcription ("Vloggers using fame to supfort charities is a positiwe way"). The input was a single already-segmented crop, not a whole photograph, and the call took 2.74 s end to end (`stage6_results.json`). A single crop, reported as an illustration and not as a statistic. No whole-page run of the live classify endpoint was recorded: **[MUST STILL BE COMPLETED BY ME: classify a full photographed page through the live server and record `lines_detected`, the writer confidence and the transcription, so that a page-level live figure exists.]**
Overall accuracy A_{mean} and A_{prod} on held-out pages	Not measured; the line-level split used here holds out 15% of lines per page, so lines from the same sheet appear in both partitions.

Table 17. Results for qualification test 2 (13 writers; measured on the development laptop).

Measure	Measured value, data set and caveat
Synthesis time per character, best-of-1 to best-of-30	Repeated over sentences of 22 to 119 characters and 2 to 3 repetitions per cell, on the development laptop. At best-of-1 the synthesis costs about 0.4 to 0.7 s per character; at best-of-14 about 0.9 to 1.8 s per character; at best-of-30 about 1.8 to 3.4 s per character.
G-code generation	0.006 to 0.014 s per sentence, i.e. negligible at any sentence length.
Preview rendering and read-back	About 0.02 to 0.05 s and 0.6 to 0.8 s per sentence respectively; both are small beside the synthesis term.
Physical writing time, from the generator's own model	88.7 s for a 22-character sentence and 154.4 s for a 58-character sentence, i.e. about 2.7 to 4.0 s per character, against the 4 s limit *for the writing alone*.
Total per character at best-of-14	Figure 16: synthesis contributes about 1.0 to 1.8 s and writing about 2.7 to 4.0 s, so the total lies between about 4 and 6 s per character and straddles the limit.
Live server measurement, four-character word, best-of-6	64.3 s from request to rendered preview, i.e. 16.1 s per character. A very short sentence is the worst case because the per-job costs (model loading, segmentation of the uploaded image) are amortised over only four characters.
Physical writing time measured on the gantry	Not measured; the gantry bring-up was not completed, so no drawn job was timed. The figures above are the generator's model, not observations.
Timing on the ODROID N2+	Not measured.

Table 18. Results for qualification test 3 (development laptop; the writing figures are model values, not measurements).

Measure	Measured value, data set and caveat
Recognition time per character	8.86 ms per character and 353 ms per line, averaged over 669 real line crops (236.2 s of inference in total). This meets the 25 ms per character allowance with a margin of about 2.8 times. Measured on the development laptop with the model already loaded.
Writer classification time	A single crop classified through the live server took 2.74 s end to end, which includes segmentation of the uploaded image, the ink writer classifier and text recognition of the line.
Total classification time, one page	**Measured.** 13 photographed pages, 3 repeats each (`qtp_data/qtp4_results.json`): page means of **40.3 to 100.1 s** from file read to displayed writer and text, against per-page budgets of 26.4 to 43.9 s from Equation 60 for $x = 976$ to 1677 characters. The ratio of total to requirement is **1.15 to 2.98**, so every page exceeded its budget, the best by 15% and the worst by a factor of three. Individual repeats ran from 40.2 to 103.4 s. Stage breakdown on the same run: file I/O 0.35 to 0.97 s, segmentation 24.8 to 69.1 s, recognition 5.7 to 13.8 ms per character, writer classification 4.6 to 10.3 ms per character. All 13 pages were from two writers (Yeukita and Dylan) and are the harness pages, not a thirteen-writer sample. Measured on the development laptop.
Page segmentation time	24.8 to 69.1 s per page (first repeat of each of the 13 pages; 24.7 to 72.4 s over all 39 repeats) for 2448 × 3264 photographs in pure Python on the laptop, against the roughly 2 s that the equation allows for capture and conditioning. **Attribution:** the stored field is `stage1_processpage_s`, so these are timings of `SegmentPage.ProcessPage`, the *retired* segmenter, and *not* of the deployed `SegmentLean.py`. No timing of the deployed module exists: **[MUST STILL BE COMPLETED BY ME: time `SegmentLean.py` on the same 13 pages with the same 3 repeats, so that the segmentation term of R4 refers to the module the system actually ships.]** Two routines reimplemented in C99 reduced the retired segmenter by 25% to 32% end to end, but they were not deployed.
Timing on the ODROID N2+	Not measured.

Table 19. Results for qualification test 4 (development laptop).

5. Discussion

5.1 Critical evaluation of the design

This subsection interprets the results of Section 4, evaluates the design choices in hindsight and identifies what must change before the requirements can be considered proven.

Two limitations determine every conclusion below: the measurements were taken on a development laptop and not on the ODROID N2+, and the evidence on synthesis quality is partly circular, because the classifier that selects the best of N candidates belongs to the same family as the one that reports the score. Where a figure is affected by either limitation, this is stated with the figure.

5.1.1 Interpretation of results

R2, classification: the strongest part of the system. Text decoding reached 98.61% character accuracy and 94.21% word accuracy over 669 real line crops, and writer identification 99.78% over 1358 real lines from thirteen writers with the ink classifier. Both exceed the 95% target on this evidence, and the lower bound of the character figure is about 97.6%. Two qualifications do not show in the numbers. First, the 98.61% is measured on crops known to be correctly cut, because any page whose crop count disagreed with the label count was excluded; it therefore measures recognition given correct segmentation and says nothing about pages where segmentation fails. Second, the five writers scored are also represented in the recogniser's fine-tuning data, so the split favours the system. Honestly stated, recognition and identification meet their target on correctly segmented, in-distribution input, and the end-to-end page-to-text accuracy is bounded above by these figures rather than equal to them. The gap between character and word accuracy carries information: one wrong character spoils a whole word, so the residual error sits in a minority of words rather than spread thinly.

The writer classifier that is used for real ink is not the one that judges synthesis. The ink classifier reached 99.78% on real lines and the shape classifier 93.67%. That ordering is the opposite of what the synthesis path needs, because the ink classifier depends on stroke thickness and ink density, which a constant-width plotter cannot reproduce: on the system's own drawn output the ink classifier falls to 28%, close to chance for thirteen writers, while the shape classifier reaches 89.7%. Both are therefore necessary: one is the right instrument for reading a photograph, the other for judging a drawing.

A training fault, not an intrinsic difficulty, explained the hardest writer pair. Before this work the shape classifier assigned 118 of one writer's 148 real lines to a single other writer, which read as two intrinsically similar hands. The measurement shows otherwise: training the same architecture end-to-end, instead of freezing the backbone at the recogniser's features and fitting only the classification head, raised the shape classifier from 83.4% to 93.7% overall and that writer from 18.9% to 67.6%. A recogniser's features are trained to be *invariant* between writers, so a classifier restricted to them can separate only writers whose recognition features already differ. The residual is still 48 of 148 lines sent to the same neighbour, so a smaller genuine effect probably remains on top of the fault; this is unresolved and listed as an unsolved problem. The

lesson is that reusing a recogniser's backbone saves training data, but it must be fine-tuned rather than frozen whenever the new task is to detect the variation the backbone was trained to ignore.

R1, representation fidelity: close, and the shortfall is in the right place to be fixable. Synthesised text was read at 82.2% character accuracy over thirteen writers, against the 85% target, and at 79.2% in a wider coverage run. The matching comparison is more informative: on identical sentences the writers' genuine handwriting was read at 89.0% and the synthesis at 81.6%. The synthesis loses 7.4 percentage points of character accuracy but 23.8 points of word accuracy, and that asymmetry localises the defect. Individual letterforms survive the transfer, as expected because the synthesiser draws the writer's own traced glyph paths; the spacing structure does not, so words are drawn with boundaries the recogniser reads differently. The measured feature deviations agree: synthesised lines have ascenders about 0.4 to 0.6 x-height units shorter than the writer's real lines, word spaces about 0.5 units narrower, and connectedness lower. The defect is thus a few measurable, correctable quantities rather than a failure of the method, which is why R1 is partially met rather than not met. A recogniser checkpoint that took no part in the search read the same lines 3.7 points below the production checkpoint, so the selection raises legibility only slightly and the result is not an artefact of the search.

The stylistic-correlation target is not verified, and writer identification is a weak proxy for it. The proposal asks for 85% stylistic correlation; what was measured is writer-identification accuracy, a different question, because a synthesis could be identifiable as the right writer while still differing visibly from that writer's hand. Worse, the writer-identification figures on synthesised text (80.1% in the official run, 89.7% in the coverage run) come from a classifier that also selects the candidates, so a best-of-N search necessarily raises the score it is judged by, and only the figures computed outside the search can be trusted. The nine-feature geometric correlation gave 0.714 against the 0.85 target, so the style term of R1 is not met on that measure, the shortfall being in descender extent, the ascender-to-descender ratio and word spacing rather than in slant or ascender extent. The overlay test gave an AUC [40] of about 0.71, against 0.5 for an output carrying no writer information, which indicates real but limited fidelity. A human panel or an independent classifier is required before the 85% stylistic target can be claimed, and this was not carried out.

The synthesis is recognised as the correct writer much better than the system draws that writer. The same shape classifier identifies 93.7% of real lines but only 89.7% of drawn lines in the coverage run, and only 67.6% of one writer's real lines while identifying 100% of that writer's drawn lines. A judge that recognises the machine's output more reliably than the human's hand is measuring agreement with the synthesiser rather than fidelity to the author, which is the most important caveat on every writer-identification figure measured on synthesised text.

R3, reproduction time: the failure is the search, not the drawing. A four-character word at $N = 6$ took 64.3 s, or 16.1 s per character, four times the limit. The coverage run confirms that the cost scales with the number of candidates, because every candidate is rendered, read by the recogniser and scored by the writer classifier. The physical drawing time was never measured, so the requirement cannot be decomposed, and it would be wrong to conclude that the gantry is slow: the evidence points at the selection procedure, which embeds two network passes in its inner loop and pays for the quality it delivers. The proposal does not state whether the synthesis time counts towards the limit; under the reading that only the physical writing counts, the generator's model value of 2.7 to 4.0 s per character sits at the 4 s limit, so the requirement is recorded as not met on the conservative reading that the total counts.

R4, classification time: recognition passes, segmentation fails badly. Recognition ran at 8.86 ms per character against the 25 ms allowance, a margin of about 2.8 times, so the neural component is not the problem. Page segmentation took 35 to 57 s against the roughly 2 s the equation allows for capture and conditioning, a shortfall of more than an order of magnitude. Since x appears only in the recognition term, a longer page does not buy proportionally more time, so the segmenter's per-page cost cannot be amortised. The profile shows the cause: line grouping and the chain search dominate, at about 70% of the time, and are written in pure Python. Moving two inner routines to C reduced the total by 25% to 32%, but the grouping still dominates: accelerating the wrong part of a program cannot fix it.

R5, positional accuracy, and the field conditions: no conclusion is possible. The derived step resolution of 12.5 μm shows only that the command quantisation is 40 times finer than the tolerance, so the error must come from the mechanics and from open-loop effects. No drawn mark was scanned, because the direction-pin fault prevented controlled motion in both directions, so the requirement is unassessed rather than failed. The same holds for the two field conditions, neither of which was measured; the illumination figure is the more consequential omission, because the conditioning stage was designed to tolerate uneven lighting and that claim is untested.

Requirements met. The storage half of R6 is met: thirteen styles are stored as files and the live server lists all of them, against a minimum of ten, returning the list in 70 ms. The reproduction half is not: the proposal's demonstration asks for a writer to be chosen at random and the text written on paper in that style, and that was never carried out. R2 is met for writer identification on real handwriting and for recognition on correctly segmented in-distribution crops, but not demonstrated for the complete page-to-text chain. R1 is close but not met, R3 and R4 are not met, and R5 and both field conditions are unassessed.

5.1.2 Critical evaluation

Line-level reading instead of character separation. The design replaced the proposal's character separation by separation into lines and a sequence recogniser. This was right for connected handwriting, where character boundaries do not exist, and it makes the accuracy independent of a separation error; the price is that a whole line must be read by one network, at a cost per character higher than the 25 ms the proposal assumed.

Best-of-N search with the judges inside the loop. The search with a recogniser and a writer classifier as judges was good for quality, because it prevents the synthesis from drawing an illegible or wrong-style sample. In hindsight it was poor for verification, because it removed the independence of the evaluation, and for time, because each of the N candidates costs a rendering and two network passes. The qualification protocols address the first, and a lower N or a cheaper first-stage filter the second.

Pure Python for the segmentation. The proposal required that the processing run on the embedded platform and be written without library functions for the algorithm itself. Writing the image processing in Python made the design transparent and correct, but the inner loops of the line grouping and labelling are too slow. The C routines reduce the labelling and the box sums by 84% to 88%, but the end-to-end effect (25% to 32%) is small because the grouping dominates. By Amdahl's law, if a fraction p of the time is accelerated by a factor s, the total speed-up is $1/((1-p)+p/s)$, so with $p = 0.30$ (the share outside the grouping, taking the grouping as 70%)

it cannot exceed 3.3 even if the accelerated part took no time. The acceleration was not deployed, because the benefit was not decisive.

Open-loop gantry with step counting. Using the stepper motors without encoders is a standard, low-cost choice; its accuracy depends on missed steps, and no sensor reveals them. Running the driver without acceleration ramps (the generator models an acceleration of 400 mm/s^2, the driver does not execute it) makes the real motion differ from the model and a lost step more likely at the start of a stroke at higher feed.

5.1.3 Unsolved problems

The following problems were not solved.

1. **X-axis direction fault.** During bring-up the X-axis calibration reached the first end-stop, but the direction pin (physical pin 7) was not observed to change level when the direction was reversed. The fault was unresolved when the project closed, and no cause is claimed, because the evidence does not support one. Qualification tests 3 (writing time), 5 and 6 (physical demonstration) depend on its resolution.

2. **No measurements on the target hardware.** No timing on the ODROID N2+ and no physical accuracy measurement exists; the figures for R3 and R4 are from a laptop.

3. **The undertaking to process on the embedded platform was not demonstrated.** The approved proposal states, as a ticked condition, that "all processing will be done on an embedded platform, and not on a PC platform". The deployed programs do run on the ODROID N2+, and the inference code needs nothing the board cannot provide, but every measurement in Section 4 was taken on a laptop, and training was done off the board entirely. The undertaking is therefore met in the design and unproven in the evidence: the most significant gap between what the proposal promised and what this report can show, and a measurement gap rather than a design gap, since the same programs need only be run on the board and timed.

4. **R4 not met, and the shortfall is entirely in the segmentation.** The segmentation time exceeds the two seconds the specification allows for capture and conditioning by more than an order of magnitude. The recognition time does not: at 8.86 ms per character it sits a factor of about 2.8 *inside* the 25 ms allowance. The requirement fails on the conditioning stage alone.

5. **Optimistic validation of the personal writers.** A line-level split inside pages was used, and the checkpoint was selected on the reporting data; no separate test set exists.

6. **Circular synthesis evaluation.** No independent judge or human panel was used.

7. **Architecture sweep.** A sweep over widths, depths and learning rates was programmed but no result was kept, so the network layers are those of the published design.

8. **Implementation issues of the writing path.** (i) The stroke ordering sorts text rows by ascending Y, and with Y upward this writes the bottom line first, while the documentation says reading order. (ii) The acceleration the generator models is not executed by the driver. (iii) The generator's work area (200 mm) is larger than the driver's travel between its limits

(194 mm in X). (iv) The generator issues the fast feed once and the driver has a single modal feed register, so pen-up travel runs at the pen-down feed (900 mm/min instead of 2400 mm/min), lengthening the job.

9. **Missing information.** The hardware documentation (CAD files, photographs, wiring diagram, camera data) and the inconsistent values of the number of tries (14 in the tuning notes, 20 as the server default, 30 mentioned in the interface) are **[MUST STILL BE COMPLETED BY ME: reconcile the values and supply the hardware documentation]**.

10. **Training mode.** The proposal planned training on the single-board computer; training is done offline, and the interface has no training function.

11. **Display of the classification time** (specified in R4) is not implemented.

12. **The CTC loss is not safe against an empty transcription.** The gradient check of Section 3.5.7 found that for a label sequence of length zero the recursion builds two of its shifted arrays at the wrong length and raises a shape error instead of returning a loss. Every sequence of length one or more passes, including the repeated-character case that exercises the forbidden skip transition, so the derivative is correct and this is an indexing fault at the boundary. It was left unfixed because no line in the data set has an empty transcription, but a future data set would expose it; the same check showed that samples for which no alignment exists must be filtered out before training rather than left to the loss to reject.

What must change to meet R4. Three measures are required together. First, the line grouping (about 70% of the segmentation time) must be redesigned, for example by replacing the exhaustive search over component pairs by neighbourhood queries on a spatial index, if the profile on the ODROID N2+ confirms that the search is the quadratic part; the target is a reduction by at least a factor of 20, because 32 s (70% of 46 s) must fall well below 2 s. Second, the image must be reduced in resolution before segmentation: halving both dimensions of the 2400×1800 photograph reduces the pixel count by 4, and, as the filters and labelling scale with the pixel count, the share outside the grouping (about 14 s) should fall to about 3.5 s, and to about 0.4 to 0.6 s with the C routines, which are 84% to 88% faster (a factor of 6 to 8.3). These are projections from the measured ratios, not measurements, and they ignore that the ODROID N2+ is slower than the laptop. Third, the recogniser must become about 3 times cheaper per character, for example by a smaller network, by quantising the weights to 8 bit integers or by reducing the output sequence length; the cost per character is roughly proportional to the multiply-accumulate operations per line, and each measure must be validated against the accuracy loss, since R2 is already under its target.

What must change for the other requirements. For R2 (text): more personal training pages, an evaluation on held-out pages, and a language-model or lexicon decoder. For R1: an independent evaluation and the human panel, and re-tuning with an independent judge if the independent figure is lower. For R3: measurement on the ODROID N2+, correction of the feed register, execution of acceleration or a lower pen dwell, and a lower N. For R5: resolution of the direction-pin fault, completion of the calibration and execution of qualification test 5.

5.1.4 Strong points of the design

Each strong point follows from a design decision.

- Reading whole lines removes the dependence on a character separation that cannot work for connected handwriting, and the recogniser reached 91.82% on an independent general test set.
- The conditioning stage (flat-field correction, local thresholding, skew correction) allows a photograph with uneven lighting to be used, and the live check on a personal photograph read the correct writer at 98.6% confidence with 28 lines detected and an accurate transcription.
- Writers are compact profile files, so a new writer is added by a file, and thirteen writers are stored permanently, satisfying the storage half of R6.
- The two judges in the search (text and writer) give a measurable improvement over a single sample, and the shape classifier solves the problem that an ink-based classifier cannot judge a plotter's constant line width (about 20% on synthesised output).
- The numerical resolution of the gantry (12.5 μm) is 40 times finer than the tolerance of R5, and the absolute step bookkeeping prevents accumulating rounding errors.
- Safety in software: end-stop confirmation (three consecutive samples), software limits and automatic retraction after a trip.
- The algorithms are implemented from first principles and documented well enough to be analysed, and the known limitations (circularity, optimistic splits) are stated.

5.1.5 Expected failure conditions

The system is expected to fail or degrade strongly under the following conditions.

- **Lighting.** Illuminance below 400 lux or strongly directional light with shadows violates FC1; the thresholding then merges ink with shadows or loses faint strokes, and recognition accuracy falls.
- **Page skew and perspective.** Large skew angles or a page that is not flat exceed the range of the skew correction.
- **Non-text regions.** Diagrams, tables and printed text can be segmented as text lines; the text versus non-text scoring reduces but does not exclude this, and the recogniser then produces meaningless characters.
- **Unseen writers.** The writer classifier is a thirteen-class closed-set classifier, so a writer who is not among the thirteen is assigned to one of them, because the softmax cannot express "none of the above": the output is wrong without a warning.

- **Styles outside the glyph library.** A writer whose letter shapes differ greatly from the stored paths (unusual letters, capitals, digits or symbols that were scarce in the training pages) is reproduced with the nearest stored shape; characters never harvested cannot be written in that style.
- **Text with accents, mathematical notation or rare symbols**, which occur in the personal pages, are outside the benchmark vocabulary.
- **Motor stalls and lost steps.** The open-loop gantry has no feedback, so a stall shifts every later mark without any signal.
- **Limit switch failure.** A switch that does not close lets the carriage run against the mechanical stop; the software protection depends on the switches, and a hardware emergency stop that cuts the power does not exist.
- **Long sentences.** The synthesis time grows with the number of characters and tries, so a long text at a high quality setting would take minutes on the board and may time out the web request.
- **Network failure** between the browser and the server prevents control of the system; a running job continues on the board.

The system does not fail gracefully in all cases: the classifier returns a confident answer for an unseen writer, and the gantry cannot detect lost steps.

5.2 Considerations in the design

This subsection addresses the ergonomics, safety, environmental, social, legal and ethical aspects of the design.

5.2.1 Ergonomics

The user works through a web interface of four pages (Read, Write, Model and Style) in a browser on any laptop on the network, so nothing is installed. The Read page captures the page with a live preview and shows the recognised text, the writer and the confidence; the Write page keeps the writing grid locked until the gantry is calibrated and shows a G-code preview before sending. The shortcomings are that the live progress of a running job is not displayed, the classification time is not shown, and the interface has no training function. The rig is set up by hand, with the page, the pen and the paper placed by hand; its documentation is incomplete.

5.2.2 Health and safety

The electronics operate at low voltage (a single-board computer, stepper-driver modules and small motors), so the risk of electric shock is low; **[MUST STILL BE COMPLETED BY ME: supply voltage and the mains insulation of the power supply]**. The mechanical hazards are the moving carriage, belts and pulleys, which are pinch points while the gantry moves; the pen tip is not hazardous at the writing force. The protective measures are the end-stop switches

with confirmation, the software limits, the automatic retraction and calibration at a low speed (7.8 mm/s). There is no hardware emergency stop that removes the motor power, and the server has no abort function for a running job; a job can be stopped only by ending the program or removing the power, so fitting a mains-independent emergency stop is recommended for use by others. Users must keep their hands away from the carriage while a job runs.

5.2.3 Environmental impact

The ODROID N2+ is a low-power board and the stepper motors draw power only while the gantry works, so the energy use is small beside a desktop computer. The system uses paper; test pages and failed jobs add waste, limited by the preview before writing and by the option to write on used paper. The mechanical parts printed in plastic or bought as standard parts are **[MUST STILL BE COMPLETED BY ME: which parts were 3D-printed and with which material]**; they can be reused, and plastic and electronic waste must go to a recycling point, not into domestic waste. The pen ink is a small consumable, no hazardous material beyond the electronics is used, and the only emissions are the motor noise and the pulses of the drive.

5.2.4 Social and legal impact

The social benefit is that the system can write in a person's own handwriting for those who cannot write neatly or at all, for example after an injury, and offers the personal touch the proposal identified. There is also a risk of misuse: reproducing a person's handwriting can serve forgery or impersonation, for instance on signed documents or letters. The plotter pen's constant line width reduces but does not remove the risk, because it differs from natural writing in stroke width and pressure (the ink-based classifier identified only about 20% of the synthesised output, so these ink properties are not reproduced), yet a document in a plausible style can still mislead a reader. Handwriting samples are personal information and a writer profile is similar to biometric data, since it permits identification of the writer; in South Africa the Protection of Personal Information Act (POPIA) applies to such data, requiring the consent of the writers, the minimum storage and the protection of the profiles against access by others. The samples of the five personal writers require documented consent, **[MUST STILL BE COMPLETED BY ME: confirm that consent was given and in what form]**, and the eight database writers contributed to a public research database whose terms of use apply. The server has no user accounts (only an optional shared key), accepts requests from any origin and can start physical motion for anyone who can reach its port, so it must be operated only on a trusted local network. Beyond the general electrical safety rules no other legislation was identified for a research prototype of this type; **[MUST STILL BE COMPLETED BY ME: any applicable regulation or university rule]**.

5.2.5 Ethics clearance

The project used handwriting samples of five named persons and of a public database, so it involves human participants in a limited way. The status of ethics clearance is **[MUST STILL BE COMPLETED BY ME: ethics clearance number, or a statement that no clearance was required]**. If the human panel of qualification test 1 is carried out, the raters give informed consent and their responses are stored without personal identifiers, and clearance must

be confirmed before the panel is run.

6. Conclusion

6.1 Summary of the work completed

This report describes work carried out on the design and implementation of a handwriting robot that recognises handwritten text and its writer and reproduces text on paper in the handwriting style of a chosen writer.

A literature survey on handwriting recognition, writer identification, handwriting synthesis and plotter-based writing was completed. The software was then designed and implemented from first principles in Python: an image-conditioning and text-line segmentation chain, a convolutional and recurrent sequence recogniser trained with connectionist temporal classification, two writer classifiers, a builder of per-writer style profiles from forced alignment, a trajectory synthesiser with a best-of-*N* search, a G-code generator and a stepper-motor driver with end-stop calibration and software limits. These were integrated behind a web server with a browser interface and were exercised as a complete pipeline over thirteen writers. The gantry was constructed from purchased components and its electronics and firmware were brought up, but the mechanical bring-up was not completed because of an unresolved fault in the direction signal of the X axis, so the writing path could not be qualified end to end.

6.2 Summary of observations and findings

The following findings are supported by the measurements reported in Section 4.

- The sequence recogniser read 669 real line crops from the five personal writers at 98.61% character accuracy and 94.21% word accuracy, with per-writer character accuracy between 96.46% and 99.15%. These crops were verified to be correctly cut, and the writers appear in the fine-tuning data, so the figure is an upper bound on the end-to-end page-to-text accuracy rather than a demonstration of it.
- Writer identification over 1358 real lines from thirteen writers reached 99.78% with the ink classifier, above the 95% target of R2, and 93.67% with the stroke-normalised classifier, which is 1.33 percentage points below it. Only the ink classifier meets the target, and it is the one that cannot judge the machine's own output.
- Correcting a training fault in the shape classifier, which had frozen the recogniser's backbone and fitted only the classification head, raised its accuracy from 83.36% to 93.67% and removed a systematic collapse in which 118 of one writer's 148 real lines were assigned to a single other writer. The recogniser's features are trained to be invariant to differences between writers, so a classifier restricted to them can separate only writers whose recognition features already differ.
- Synthesised text was read at 82.2% character accuracy over thirteen writers (79.2% in a wider coverage run), below the 85% target of R1, and identified at 80.1% and 89.7% respectively by a classifier that also selects the candidates.

- On a matching comparison of identical sentences the writers' genuine handwriting was read at 89.0% character and 70.5% word accuracy, and the system's synthesis of those same sentences at 81.6% and 46.7%. The synthesis therefore loses 7.4 percentage points of character accuracy but 23.8 points of word accuracy, which localises the defect to spacing and word boundaries rather than to letterforms. **[MUST STILL BE COMPLETED BY ME: The word accuracy of the genuine handwriting is given as 70.5% here and as 71.2% in the qualification-test table of Section 4, both against the same 89.0% character accuracy. Check which run each came from and make the two agree.]**

- The reproduction rate of R3 was not met: a four-character word at six candidates took 64.3 s, or 16.1 s per character against a 4 s limit, and the cost is dominated by rendering and scoring each candidate rather than by drawing.

- Classification time (R4) was met for recognition, at 8.86 ms per character against a 25 ms allowance, but not for page segmentation, which took 35 to 57 s against the roughly 2 s that the equation allows for capture and conditioning; line grouping accounted for about 70% of that time.

- Thirteen writing styles are stored as profile files and the live server lists all thirteen, so the storage half of R6 is met with a margin of three. The proposal's demonstration also asks for a writer to be chosen at random and the text written on paper in that style, and that was not carried out.

- Every hand-derived backward pass of the from-scratch network module was checked against numerical differentiation in 78 configurations. Seventy-seven agreed with the numerical gradient to between 4.9×10^{-12} and 1.4×10^{-10}, and the one failure is a real edge-case fault: the CTC loss raises a shape error on an empty transcription.

- The absolute positional accuracy of the gantry (R5) and both field conditions were not measured, so no conclusion is possible for them.

It follows that the storage half of R6 is met, R2 is met for writer identification on real handwriting and for recognition on correctly segmented in-distribution crops, R1 is close to its target but not at it, R3 and R4 are not met, and R5, the reproduction half of R6 and the field conditions are unassessed. Two implications are worth stating separately. First, the time requirement cannot be met by improving the neural network, because recognition already has a margin of about 2.8 times; the shortfall lies in the pure-Python line-grouping stage, which must be redesigned or compiled, and in the cost of evaluating every synthesis candidate. Second, on the evidence of the matching comparison the two halves of R1 are not in conflict as they first appeared: genuine and synthesised text were read within 7.4 percentage points at character level, so the synthesis is already close to the human hand in letterform, and the remaining gap is concentrated in a small number of measurable spacing statistics that can be corrected directly.

6.3 Contribution

The following was new to the student and was mastered during the project: the training of sequence networks with connectionist temporal classification, the algorithms of the image-conditioning and line-segmentation chain, the extraction of writer-specific letter paths by forced alignment, the synthesis of handwriting trajectories, the generation of G-code and the control

of stepper motors from the general-purpose pins of a single-board computer. All of these were implemented from first principles by the student in Python, and C99 was used for an optional acceleration of two routines. Deriving the backward pass of each layer by hand, including the connectionist temporal classification loss, and verifying each one against numerical differentiation was the single largest piece of new learning in the project, and it is what allowed the networks to be trained without a machine-learning framework. The mechanical parts, the camera, the single-board computer, the motors and drivers, the end-stop switches, the pen mechanism and the power supply were purchased as finished items and carry no design credit. The design of the networks follows a published layer pattern [10], and public data were used for training. The architecture of the recogniser is taken from the literature; the training recipe, the preprocessing chain, the style-profile construction and the whole synthesis and drawing path are the student’s own work. Any other contribution by other persons is declared in Section 2 (Project history) of the preamble.

6.4 Suggestions for future work

The work can be continued in the following order of priority.

1. Resolve the direction-pin fault of the X axis, complete the calibration and execute the qualification tests 3, 5 and 6 on the gantry, including the field checks FC1 and FC2.

2. Execute qualification test 2 on newly collected held-out pages, and qualification test 1 with independent judges, a human panel and the hand-crafted and overlay measures, so that the figures of Section 4 are no longer circular or optimistic.

3. Redesign the line grouping, reduce the image resolution before segmentation, deploy compiled implementations, and make the recogniser about 3 times cheaper per character (smaller network, quantisation, shorter sequence), measuring each change on the ODROID N2+.

4. Add a language-model or lexicon decoder and more personal training pages to raise the text accuracy towards 95%.

5. Correct the implementation issues of the writing path (reading-order stroke ordering, execution of acceleration, work area, pen-up feed), add a hardware emergency stop and an abort function, and consider a position feedback (for example a sensor on the end-stops or a camera check of the written line) so that lost steps can be detected.

6. Add an unknown-writer class to the writer classifier, and an authenticated interface.

7. References

[1] A. Chen, "Handwritten text classification based on convolutional neural network," *Highlights in Science, Engineering and Technology*, vol. 34, pp. 39–44, 2023.

[2] J. Ma, "A study on a hybrid CNN-RNN model for handwritten recognition based on deep learning," in *Proc. 1st Int. Conf. Data Analysis and Machine Learning (DAML)*, 2023, pp. 275–280.

[3] F. A. Mamun, S. A. H. Chowdhury, J. E. Giti, and H. Sarker, "Classification of non-native handwritten characters using convolutional neural network," arXiv preprint arXiv:2406.04511, 2024.

[4] U.-V. Marti and H. Bunke, "The IAM-database: an English sentence database for offline handwriting recognition," *Int. J. Document Analysis and Recognition*, vol. 5, no. 1, pp. 39–46, 2002.

[5] G. R. Hemanth, M. Jayasree, S. K. Venii, P. Akshaya, and R. Saranya, "CNN-RNN based handwritten text recognition," *ICTACT Journal on Soft Computing*, vol. 12, no. 1, pp. 2457–2463, Jan 2022.

[6] J. P. Kumar and H. D. Dharshan, "Handwritten text recognition using deep learning: A CNN–LSTM approach," *Int. J. Intelligent Systems and Applications in Engineering*, vol. 12, no. 1, pp. 45–52, 2024.

[7] A. Graves, M. Liwicki, S. Fernández, R. Bertolami, H. Bunke, and J. Schmidhuber, "A novel connectionist system for unconstrained handwriting recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 31, no. 5, pp. 855–868, 2009.

[8] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 11, pp. 2298–2304, 2017.

[9] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks," in *Proc. 23rd Int. Conf. Machine Learning (ICML)*, 2006, pp. 369–376.

[10] F. Kizilirmak and B. Yanikoglu, "CNN-BiLSTM model for English handwriting recognition: comprehensive evaluation on the IAM dataset," arXiv preprint arXiv:2307.00664, 2023.

[11] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[12] A. A. Yahya, J. Tan, and M. Hu, "A novel handwritten digit classification system based on convolutional neural network approach," *Sensors*, vol. 21, no. 18, p. 6273, Sep 2021.

[13] S. Ji and Y. Zeng, "Handwritten character recognition based on CNN," in *Proc. 3rd Int. Conf. Artificial Intelligence and Advanced Manufacturing (AIAM)*, Oct 2021, pp. 2874–2880.

[14] M. Bulacu and L. Schomaker, "Text-independent writer identification and verification using textural and allographic features," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 29, no. 4, pp. 701–717, 2007.

[15] A. Graves, “Generating sequences with recurrent neural networks,” arXiv preprint arXiv:1308.0850, 2013.

[16] L. Kang, P. Riba, Y. Wang, M. Rusiñol, A. Fornés, and M. Villegas, “GANwriting: content-conditioned generation of styled handwritten word images,” in *Computer Vision – ECCV 2020*. Springer, 2020, pp. 273–289.

[17] G. Dai, Y. Zhang, Q. Ke, Q. Guo, and S. Huang, “One-DM: one-shot diffusion mimicker for handwritten text generation,” in *Computer Vision – ECCV 2024*, ser. Lecture Notes in Computer Science. Springer, 2024.

[18] T. S. F. Haines, O. Mac Aodha, and G. J. Brostow, “My text in your handwriting,” *ACM Transactions on Graphics*, vol. 35, no. 3, pp. 26:1–26:18, 2016.

[19] O. Teikari, “Reproducing handwriting with position-controlled robots,” Master’s thesis, Tampere University of Technology, Tampere, Finland, 2016.

[20] D. H. Douglas and T. K. Peucker, “Algorithms for the reduction of the number of points required to represent a digitized line or its caricature,” *Cartographica*, vol. 10, no. 2, pp. 112–122, 1973.

[21] IQS Directory, “Types of linear actuators,” IQS Directory Technical Library, 2024, [Online]. Available: https://www.iqsdirectory.com/articles/linear-actuator/types-of-linear-actuators.html.

[22] International Organization for Standardization, “Automation systems and integration—numerical control of machines—program format and definitions of address words—Part 1: Data format for positioning, line motion and contouring control systems,” ISO 6983-1, 2009.

[23] “NEMA 17 bipolar stepper motor, 1.68 A per phase, 200 full steps per revolution,” Manufacturer datasheet.

[24] P. P. Acarnley, *Stepping Motors: A Guide to Theory and Practice*, 4th ed. London, U.K.: Institution of Electrical Engineers, 2002.

[25] Allegro MicroSystems, “A4988 DMOS microstepping driver with translator and overcurrent protection,” Datasheet.

[26] J. E. Bresenham, “Algorithm for computer control of a digital plotter,” *IBM Systems Journal*, vol. 4, no. 1, pp. 25–30, 1965.

[27] F. C. Crow, “Summed-area tables for texture mapping,” in *Proc. 11th Annual Conf. Computer Graphics and Interactive Techniques (SIGGRAPH)*, 1984, pp. 207–212.

[28] N. Otsu, “A threshold selection method from gray-level histograms,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.

[29] S. Ioffe and C. Szegedy, “Batch normalization: accelerating deep network training by reducing internal covariate shift,” in *Proc. 32nd Int. Conf. Machine Learning (ICML)*, 2015, pp. 448–456.

[30] V. Nair and G. E. Hinton, “Rectified linear units improve restricted Boltzmann machines,” in *Proc. 27th Int. Conf. Machine Learning (ICML)*, 2010, pp. 807–814.

[31] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[32] T. Tieleman and G. Hinton, “Lecture 6.5—RMSprop: divide the gradient by a running average of its recent magnitude,” COURSERA: Neural Networks for Machine Learning, 2012.

[33] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. 7th Int. Conf. Learning Representations (ICLR)*, 2019.

[34] T. Y. Zhang and C. Y. Suen, “A fast parallel algorithm for thinning digital patterns,” *Communications of the ACM*, vol. 27, no. 3, pp. 236–239, 1984.

[35] D. P. Kingma and J. Ba, “Adam: a method for stochastic optimization,” in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, 2015.

[36] A. J. Viterbi, “Error bounds for convolutional codes and an asymptotically optimum decoding algorithm,” *IEEE Transactions on Information Theory*, vol. 13, no. 2, pp. 260–269, 1967.

[37] J. L. Fleiss, “Measuring nominal scale agreement among many raters,” *Psychological Bulletin*, vol. 76, no. 5, pp. 378–382, 1971.

[38] H. G. Barrow, J. M. Tenenbaum, R. C. Bolles, and H. C. Wolf, “Parametric correspondence and chamfer matching: two new techniques for image matching,” in *Proc. 5th Int. Joint Conf. Artificial Intelligence (IJCAI)*, 1977.

[39] M.-P. Dubuisson and A. K. Jain, “A modified Hausdorff distance for object matching,” in *Proc. 12th International Conference on Pattern Recognition*, vol. 1, Jerusalem, Israel, 1994, pp. 566–568.

[40] T. Fawcett, “An introduction to ROC analysis,” *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.

[41] Joint Committee for Guides in Metrology, “Evaluation of measurement data—guide to the expression of uncertainty in measurement,” JCGM 100:2008, 2008.

[42] T. Huang and R. Xiong, “Cost-effective robotic handwriting system with AI integration,” in *Proc. 2024 IEEE Long Island Systems, Applications and Technology Conference (LISAT)*, 2024.

[43] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[44] B. Efron, “Bootstrap methods: Another look at the jackknife,” *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.

[45] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions, and reversals,” *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[46] R. A. Wagner and M. J. Fischer, “The string-to-string correction problem,” *Journal of the ACM*, vol. 21, no. 1, pp. 168–173, 1974.

[47] K. Pearson, “Mathematical contributions to the theory of evolution. III. Regression, heredity, and panmixia,” *Philosophical Transactions of the Royal Society of London A*, vol. 187, pp. 253–318, 1896.

[48] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: a simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.

[49] Hardkernel, “ODROID-N2+ single-board computer,” Hardware technical reference.

[50] ON Semiconductor, “BC337/BC337-40 NPN general-purpose amplifier transistor,” Datasheet.

Appendix

Metric definitions of the qualification tests

The qualification tests of Section 4.2 quote accuracies, correlations and distances whose definitions are not interchangeable: three different averages of character accuracy over a set of lines differ by several points on the same data, and the word accuracy reported throughout is a recall rather than one minus the word error rate. This appendix defines each measure once, names the routine that computes it, and states where an alternative reading would change the number. Section 4.2 refers to this appendix in place of repeating the mathematics beside each result.

7.0.0 Confidence intervals and resampling

Proportions are reported with a 95% Wilson score interval [43],

$$\hat{p}_{\pm} = \frac{\hat{p} + \dfrac{z^2}{2n} \pm z\sqrt{\dfrac{\hat{p}(1-\hat{p})}{n} + \dfrac{z^2}{4n^2}}}{1 + \dfrac{z^2}{n}}, \qquad \hat{p} = \frac{k}{n}, \tag{62}$$

where k is the number of successes in n trials and $z = 1.96$. Equation 62 is used in preference to the normal approximation because it stays valid near 0% and 100%, where the system's per-writer rates lie. Characters of one page are not independent, so accuracies use a page-level percentile bootstrap with 2000 resamples [44] rather than a closed-form interval, and a requirement is declared met only if the lower confidence bound exceeds the target.

7.0.0 Character accuracy and its three averages

Character accuracy of one line uses the Levenshtein distance [45] $d_L(h, r)$ between hypothesis h and reference r, the minimum number of insertions, deletions and substitutions, evaluated by the dynamic-programming recursion of Wagner and Fischer [46],

$$A_{\mathrm{c}}(h, r) = \max\left(0,\ 1 - \frac{d_L(h, r)}{|r|}\right), \tag{63}$$

where $|r|$ is the reference length. Equation 63 is implemented as `Classify.CharAcc`, which floors the value at zero so that a hypothesis longer than the reference cannot score negatively.

A set of lines can be summarised in three different ways, and they do not agree, so each is named explicitly:

$$\bar{A}_{\mathrm{c}}^{\mathrm{line}} = \frac{1}{L}\sum_{\ell=1}^{L} A_{\mathrm{c}}(h_\ell, r_\ell), \qquad \bar{A}_{\mathrm{c}}^{\mathrm{micro}} = 1 - \frac{\sum_\ell d_L(h_\ell, r_\ell)}{\sum_\ell |r_\ell|}, \qquad \bar{A}_{\mathrm{c}}^{\mathrm{writer}} = \frac{1}{W}\sum_{w=1}^{W} \bar{A}_{\mathrm{c}}^{\mathrm{line}}(w), \tag{64}$$

the unweighted mean over lines, the character-weighted micro-average over all lines pooled, and the unweighted mean over the W writers of their own line means. The micro-average of Equation 64 is what "summed over all sentences" would mean, but the character-accuracy figures quoted for qualification test 1 are the *mean over writers*, $\bar{A}_{\mathrm{c}}^{\mathrm{writer}}$, recorded as `char_acc_mean_over_writers`. The distinction is not cosmetic: for the held-out judge on the 123 real lines the two readings differ by 4.7 points, 86.6% over writers against 81.9% over lines. Figures quoted for qualification test 2 are instead the unweighted mean over line crops, $\bar{A}_{\mathrm{c}}^{\mathrm{line}}$, because `verify_text_accuracy.py` accumulates one `CharAcc` per crop and divides by the crop count, so a two-word crop weighs as much as a twenty-word one.

7.0.0 *Word accuracy is a recall, not one minus the word error rate*

Word accuracy, which Section 4.2 quotes six times, is `Classify.WordAcc` and is *not* 1 − WER:

$$A_{\mathrm{w}}^{\mathrm{word}} = \frac{\mathrm{LCS}(\mathrm{words}(h), \mathrm{words}(r))}{|\mathrm{words}(r)|}, \tag{65}$$

the length of the longest common subsequence of the two word sequences, compared case-insensitively, divided by the number of reference words. Equation 65 is therefore an order-preserving *recall* over words: it counts how many of the intended words were recovered in the right order and imposes no penalty for inserted or spurious words, whereas 1 − WER charges insertions, deletions and substitutions alike and can go negative. A reader who assumes 1 − WER will read every word figure in Section 4.2 as stricter than it is.

7.0.0 *Writer accuracy is reported per line, not per page*

Writer identification admits a per-line and a per-page reading,

$$A_{\mathrm{w}}^{\mathrm{line}} = \frac{1}{L}\sum_{\ell=1}^{L} \mathbf{1}\left[\hat{w}_\ell = w_\ell\right], \qquad A_{\mathrm{w}}^{\mathrm{page}} = \frac{1}{P}\sum_{p=1}^{P} \mathbf{1}\Big[\arg\max_{w} \tfrac{1}{L_p}\textstyle\sum_{\ell \in p} P(w \mid \ell) = w_p\Big], \tag{66}$$

the fraction of lines assigned to the correct writer, and the fraction of pages whose mean per-line writer probability peaks at the correct writer. The proposal's wording implies the page form of Equation 66, but $A_{\mathrm{w}}^{\mathrm{page}}$ was never evaluated: every value reported in Section 4.2, including the 99.78% and 93.67% over 1358 lines, is the per-line form $A_{\mathrm{w}}^{\mathrm{line}}$. Because lines of one page share a writer, the page form would be the stronger claim and the smaller sample. **[MUST STILL BE COMPLETED BY ME: evaluate $A_{\mathrm{w}}^{\mathrm{page}}$ of Equation 66 by pooling the stored per-line probabilities of `stage4_predictions_ink.json` and `stage4_predictions_shape.json` by page.]**

7.0.0 *Decoding*

All text figures in the report were produced by greedy connectionist temporal classification decoding — argmax per time step, collapse repeats, drop the blank, as implemented in `Classify.DecodeCtc` — with **no beam search, no language model and no lexicon** anywhere in the deployed pipeline. That matters next to the 94.21% word accuracy of qualification test 2: the figure is what an unaided per-frame argmax recovers, so it is not inflated by a decoder that could snap a near-miss onto a dictionary word; equally, nothing corrects a single-character slip that destroys a word. All random choices use recorded seeds.

7.0.0 *Inter-rater agreement of the panel that was not convened*

Inter-rater agreement of the blind human panel of qualification test 1 was to be quantified with Fleiss' kappa [37],

$$\kappa = \frac{\bar{P} - \bar{P}_e}{1 - \bar{P}_e}, \tag{67}$$

where $\bar{P}$ is the mean observed pairwise agreement and $\bar{P}_e$ the agreement expected by chance. As the panel was never convened, Equation 67 was never evaluated and is given here only to define the sub-test that was not run; the worksheet prepared for it is Table 22.

7.0.0 *Style correlation on geometric features*

Each feature is z-scored, $z_{wf} = (v_{wf} - \mu_f)/\sigma_f$, using the mean and standard deviation over the ten real writers, and the stylistic correlation reported for sub-test (b) of qualification test 1 is the

Pearson product-moment correlation coefficient [47],

$$\rho = \frac{\sum_i (a_i - \bar{a})(b_i - \bar{b})}{\sqrt{\sum_i (a_i - \bar{a})^2}\sqrt{\sum_i (b_i - \bar{b})^2}}, \tag{68}$$

where a_i and b_i are the real and synthesised z-scored entries pooled over writers and features (70 pairs for the seven-feature set the protocol prescribes, listed in Table 24). Besides the pooled value of Equation 68, a per-writer value (7 pairs) and a nearest-centroid retrieval in z-space with its 10×10 confusion matrix are also computed. The correlation actually measured used the nine geometric features of Table 16 rather than the seven of Table 24. **[MUST STILL BE COMPLETED BY ME: confirm the Pearson reference: the product-moment correlation coefficient is introduced in the 1896 paper "Regression, heredity, and panmixia" (Philosophical Transactions of the Royal Society A, vol. 187), whereas the sourcing audit suggested vol. 186 (1895), which is the skew-variation paper. Verify the volume, year and page range against the original and correct the bibliography entry.]**

7.0.0 Overlay distances in the pixel domain

Both images entering the overlay test are normalised in the same way before any distance is taken, so that the comparison measures shape rather than size or placement; Figure 17 sets out that procedure.

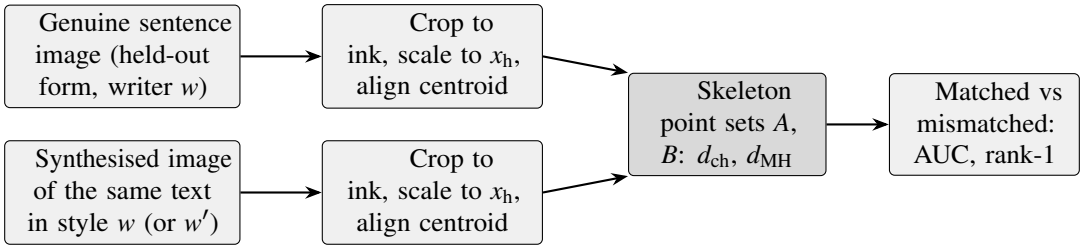


Figure 17. Overlay test of qualification test 1: both images are normalised in the same way and matched writer pairs are compared with mismatched pairs.

The skeleton point sets A and B that this procedure produces are compared with

$$d_{\mathrm{ch}}(A,B) = \frac{1}{2}\left[\frac{1}{|A|}\sum_{a\in A}\min_{b\in B}\|a-b\| + \frac{1}{|B|}\sum_{b\in B}\min_{a\in A}\|a-b\|\right], \tag{69}$$

$$d_{\mathrm{MH}}(A,B) = \max\left[\frac{1}{|A|}\sum_{a\in A}\min_{b\in B}\|a-b\|,\ \frac{1}{|B|}\sum_{b\in B}\min_{a\in A}\|a-b\|\right], \tag{70}$$

$$\mathrm{AUC} = \Pr\left[d_{\mathrm{matched}} < d_{\mathrm{mismatched}}\right] + \tfrac{1}{2}\Pr\left[d_{\mathrm{matched}} = d_{\mathrm{mismatched}}\right], \tag{71}$$

where $\|\cdot\|$ is the Euclidean norm and distances are in units of the x-height x_h. Equation 69 is the mean of the two directed mean nearest-neighbour distances, which is the chamfer matching distance of Barrow *et al.* [38]; Equation 70 is their maximum, which is less sensitive to outliers than the classical Hausdorff distance [39]; and Equation 71 is the area under the receiver operating characteristic curve [40], 0.5 for a distance carrying no writer information and 1.0 for perfect separation. **[MUST STILL BE COMPLETED BY ME: confirm the page range of the Barrow *et al.* 1977 IJCAI reference in the bibliography; it was left out rather than guessed.]**

The protocol sized this sub-test at 8 writers and 5 sentences, which gives 40 matched and $40 \times 9 = 360$ mismatched pairs; the run actually made used ten writers and three sentences each, which is the 30 matched and 300 mismatched pairs reported in Section 4.2. The features of

Table 24 are computed by one deterministic extractor with identical thresholds on every image, after cropping to the ink and binarising with the same rule.

7.0.0 Positional error and its uncertainty budget

The absolute error of a drawn cross and the uncertainty of its measurement, prescribed for qualification test 5 and never evaluated, are

$$e_i = \left\| \mathbf{p}_i - \mathbf{p}_i^{\mathrm{cmd}} \right\|, \tag{72}$$

$$u = \sqrt{u_{\mathrm{scan}}^2 + u_{\mathrm{ref}}^2 + u_{\mathrm{id}}^2}, \tag{73}$$

where $\mathbf{p}_i$ is the measured and $\mathbf{p}_i^{\mathrm{cmd}}$ the commanded centre of cross i, referenced to the machine origin 5 mm inside the minimum end-stops. In Equation 73, u_{scan} is the scanner scale error obtained by scanning the verified ruler, u_{ref} the registration of the sheet against the machine origin and u_{id} the error of locating a cross centre, taken as half a pixel; the three standard uncertainties are combined in quadrature and the expanded uncertainty is $2u$, following the Guide to the Expression of Uncertainty in Measurement [41]. Mean, maximum and 95th percentile of the error of Equation 72 were to be reported together with the uncertainty of Equation 73, on the sheet reproduced blank as Table 23. Because no mark was ever drawn, neither equation was evaluated.

Additional results and prepared measurement sheets

This appendix holds the supporting detail of the qualification tests of Section 4: the provenance of the data sets and model checkpoints behind every quoted figure, the two measurement sheets that were prepared for the sub-tests that were never carried out, the feature definitions prescribed for the style correlation, and the distance distributions behind the overlay test. Every figure quoted against a requirement remains in Section 4; nothing here changes a result, and the blank sheets are reproduced so that the measurement that is missing is defined rather than merely named.

7.0.0 Provenance of the data sets and checkpoints

Section 4.2 states two facts that qualify every figure in it, that the harness covers ten of the thirteen writers and that the live server serves an unmeasured text checkpoint. This sub-subsection traces both to the stored files that establish them, so that the claims can be checked rather than taken on trust.

The ten-writer subset. The harness was run over the eight database writers 150, 151, 152, 153, 384, 551, 552 and 588 and the two personal writers Dylan and Yeukita. The list is set by `PERSONAL_AUTHORS` in `qtp_scripts/qtp_common.py`, which names only `yeukita` and `dylan`, and the stored results confirm it: in `qtp_data/qtp1c_results.json` the strings `abhinav`, `owen` and `thiya` occur zero times, while `dylan` occurs 72 times and `yeukita` 96 times. The per-writer shape-judge accuracies that make those three the weakest of the set come from `stage4_summary.json`. The figures that do cover all thirteen writers come from the separate verification pipeline rather than the harness: the search-judge accuracy of qualification test 1, the per-writer end-to-end run of Figure 22 and the 1358-line writer identification of qualification test 2. The 669-crop recognition figure covers the five personal writers only, which is its own data set and not the ten-writer subset. **[MUST STILL BE COMPLETED BY ME: re-run the qualification-test harness with `PERSONAL_AUTHORS` set to all five personal writers, so that the style-correlation, overlay and independent-judge figures cover the same thirteen writers as the rest of the chapter.]**

The checkpoints. The character- and word-accuracy figures of qualification test 1 were measured with two text checkpoints recorded in `qtp_data/qtp1c_results.json`: `meta.production_ckpt` is `paper_cnn_bilstm_ctc_joint_best.pt`, the checkpoint inside the best-of-*N* search, and `meta.heldout_ckpt` is `paper_cnn_bilstm_ctc_hf_best.pt`, the base model used as the judge outside the search. The recognition figures of qualification test 2 were measured with the same joint checkpoint (`verify_text_accuracy.py`), and the writer-identification figures with `author_classifier_10new_weights.pt` for ink and `author_shape_10new_weights.pt` for shape. The deployed recogniser is a different model: `Classify.py` prefers `paper_cnn_bilstm_ctc_scratch_best.npz` over the converted checkpoints, and that file is present, so the from-scratch checkpoint is what the server loads. Its two writer classifiers, by contrast, are the `.npz` conversions of the `.pt` files above and agree with them on 100% of 1358 lines to within 8×10^{-6} of logit, which is why the writer figures transfer to the deployed code and the text figures do not. **[MUST STILL BE COMPLETED BY ME: measure character and word accuracy of the deployed `paper_cnn_bilstm_ctc_scratch_best.npz` checkpoint on the same 669 crops, or change the server to load the joint checkpoint that was measured.] [MUST STILL BE COMPLETED BY ME: record the architecture width, layer count, parameter count, optimiser, learning-rate schedule, batch size and epoch count of each named checkpoint, so that every figure in Section 4 is traceable to a trained model. No hyper-parameter table or parameter count is given for any of them at present.]**

7.0.0 Per-feature and per-judge breakdown of qualification test 1

Section 4.2 quotes the style correlation as one pooled number and the two-judge comparison as one difference; both are the summary of a per-item breakdown, which is given here because it localises the defect rather than merely sizing it. Figure 18 resolves the correlation into its nine features and Figure 19 the character accuracy into the two checkpoints that read the same sixty lines, while Figure 20 places the synthesis beside genuine handwriting of the identical sentences. Table 20 lists the per-feature coefficients that Figure 18 plots.

Feature	Pearson *r*	Feature	Pearson *r*
Slant	0.996	Stroke width	0.632
Ascender extent	0.974	Word gap	0.605
Character width	0.936	Ascender-to-descender ratio	0.509
Components per word	0.832	Descender extent	0.148
Ink density	0.680		

Table 20. Style correlation per feature across the ten harness writers, which Figure 18 plots and which the pooled coefficient of 0.714 reported in Table 16 summarises. The ordering is the result: slant and the extent above the baseline are reproduced almost exactly, whereas the extent below the baseline and the spacing between words are not.

7.0.0 Per-writer breakdowns

The headline rates of qualification tests 1 and 2 are means over writers, and the spread behind them is wider than the mean suggests, so the per-writer breakdown is reproduced here in full. Figure 21 gives writer identification per writer over the 1358 real lines for both classifiers, Figure 22 the end-to-end result per writer through the G-code round trip, and Table 21 the per-writer recognition accuracy behind the 98.61% of qualification test 2.

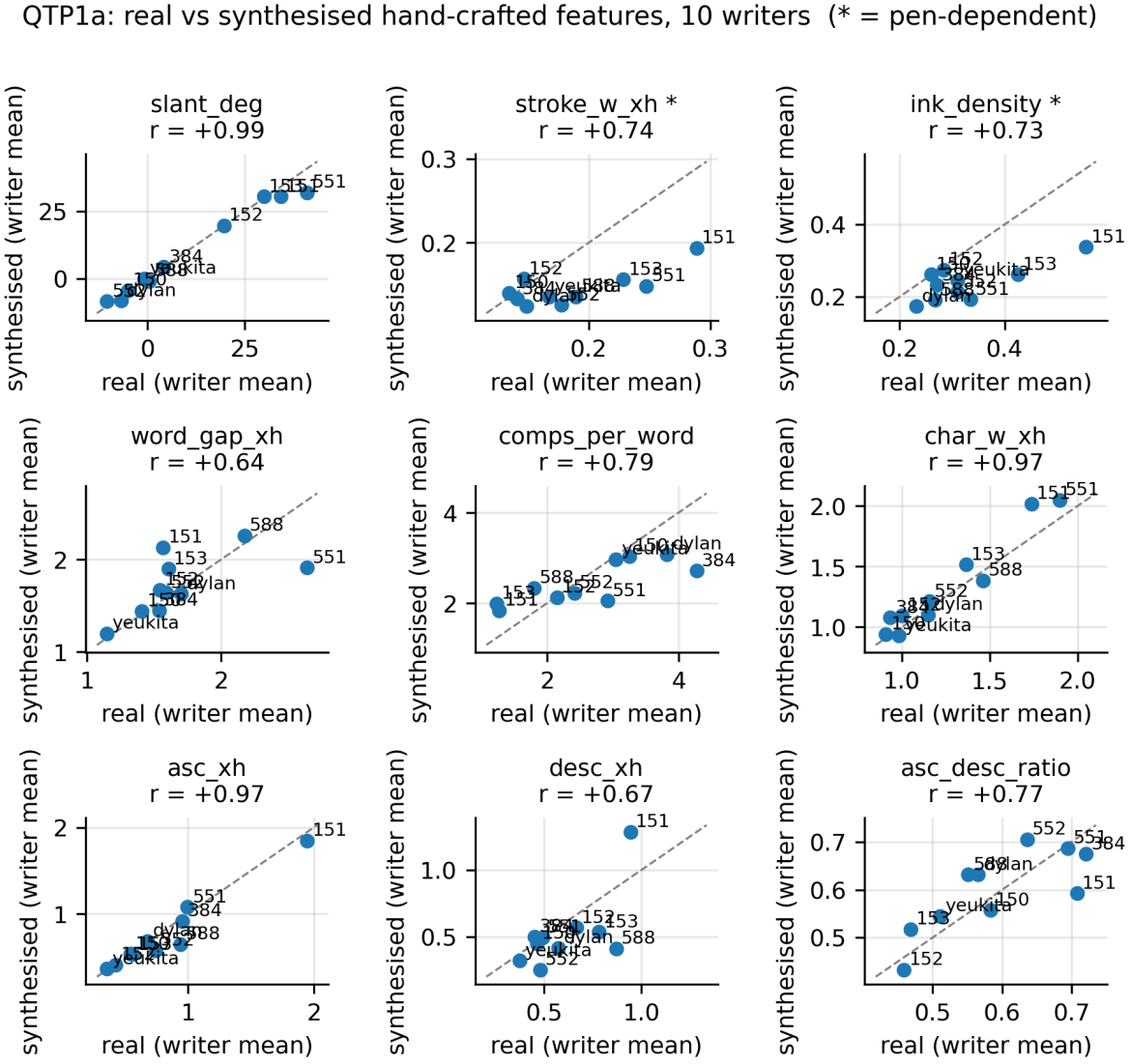


Figure 18. Style-correlation test on nine geometric features extracted by one deterministic routine from held-out real pages and from synthesised lines, for the ten writers. Each panel is one feature; the axes are the writer means. Slant, ascender and character width track the real hand closely, whereas descender and the ascender-to-descender ratio do not, which is the same defect that the word-accuracy gap identifies.

7.0.0 Storage check of qualification test 6

Qualification test 6 reduces to a count against a threshold, and Figure 23 is the observation behind that count.

7.0.0 Prepared sheets for the sub-tests not carried out

Two sub-tests were specified in full and never executed, sub-test (d) of qualification test 1 (the blind human panel) and qualification test 5 (absolute positional error). Table 22 is the worksheet printed for the panel and Table 23 the data sheet printed for the positional measurements. Both are shown exactly as printed, that is to say blank, because no rater was convened and no cross was ever drawn.

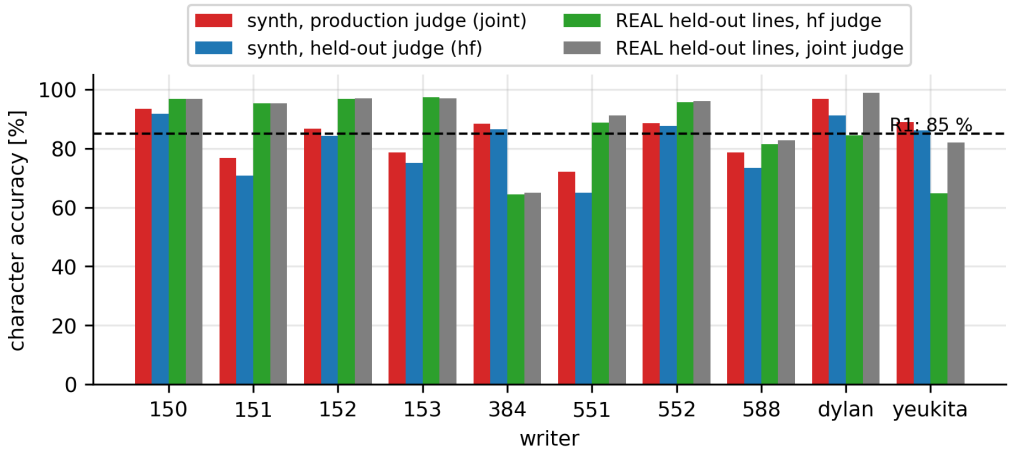


Figure 19. Character accuracy of the synthesised text under the production recogniser and under a recogniser checkpoint that was not used to select the candidates. The independent judge scores the same output about three points lower, which quantifies how much of the apparent quality is a selection effect and simultaneously shows that the synthesis is genuinely legible rather than merely self-consistent.

Writer	Character accuracy	Crops
Owen	99.15%	75 to 206 crops per writer; 669 in total
Dylan	99.03%	
Abhinav	98.94%	
Yeukita	98.30%	
Thiya	96.46%	

Table 21. Recognition accuracy per personal writer over the 669 real line crops of qualification test 2, read by `paper_cnn_bilstm_ctc_joint_best`. The pooled figure is 98.61%; Thiya, whose hand is the most joined of the five, is the only writer more than two points below it.

Line no.	Transcription written by the rater (without hint of the content)	Illegible characters (positions)	Confidence (1–5)
1			
2			
⋮			
40			

Table 22. Worksheet prepared for the blind human panel of qualification test 1: 30 synthesised lines and 10 genuine control lines in random order. The panel was never convened, so no worksheet was completed and the table is shown blank as it was printed.

7.0.0 Features prescribed for the style correlation

The style-correlation sub-test of qualification test 1 compares geometric features extracted by one deterministic routine from held-out real pages and from synthesised lines. Table 24 defines the features as they were prescribed in the protocol; the extractor that produced the measured correlation implements this set extended to the nine features reported in Table 16. The definitions are given here rather than in Section 4 because they fix the measurement rather than report a

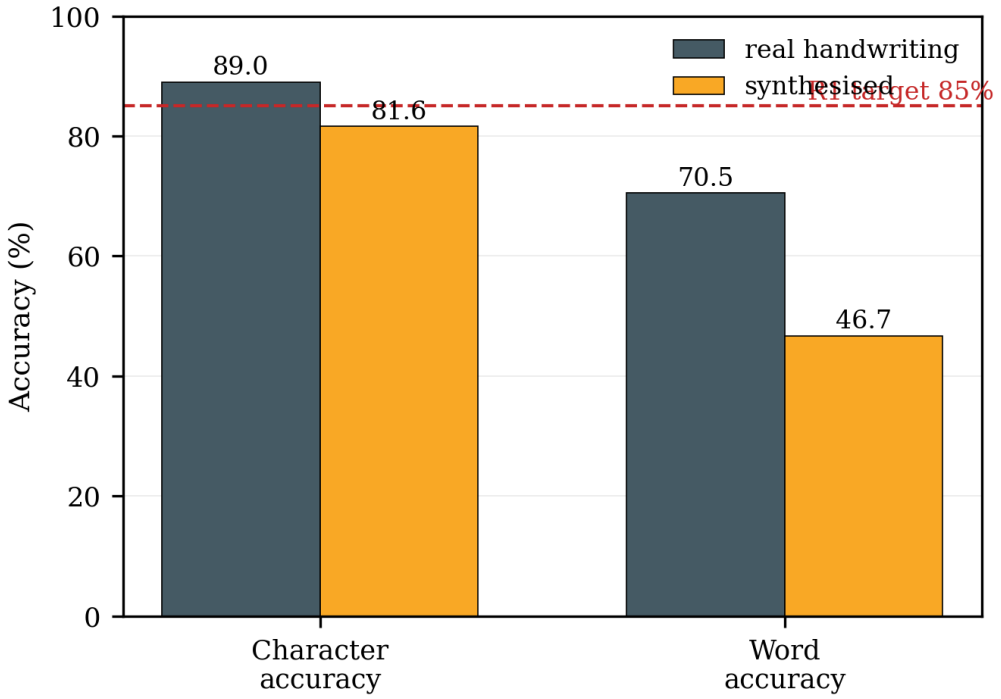


Figure 20. Matching comparison on identical sentences: genuine handwriting of the sentence against the system's synthesis of that same sentence, both read by the same recogniser. Character accuracy is close to the target, but word accuracy divides by more than a factor of 1.5, which shows that the synthesis preserves letterforms better than it preserves word boundaries and spacing.

Repeat	Cross	x^{cmd} (mm)	y^{cmd} (mm)	x meas. (mm)	y meas. (mm)	e_i (mm)
1	1					
1	2					
⋮						

Table 23. Data sheet prepared for the absolute positional error of qualification test 5; one row per cross and repetition, together with the scanner scale check and the registration check. No row was ever completed, because the test was not carried out.

result.

7.0.0 Distance distributions behind the overlay test

The overlay test of qualification test 1 reduces to two numbers in Table 16, the ratio of matched to mismatched distance and the area under the curve. Figure 24 gives the distributions those two numbers summarise, which show how far the two populations actually overlap.

Supporting detail of the page conditioning and segmentation block

Section 3.4 describes the six stages of the segmentation block and the equations that define each of them. This appendix carries the three tables of constants and rules that the stage descriptions refer to, together with pseudocode of the four routines in which the arithmetic actually happens. It is here rather than in Section 3.4 because it enumerates thresholds rather than explaining the design: the design decision is that almost every threshold is a multiple of the measured line pitch

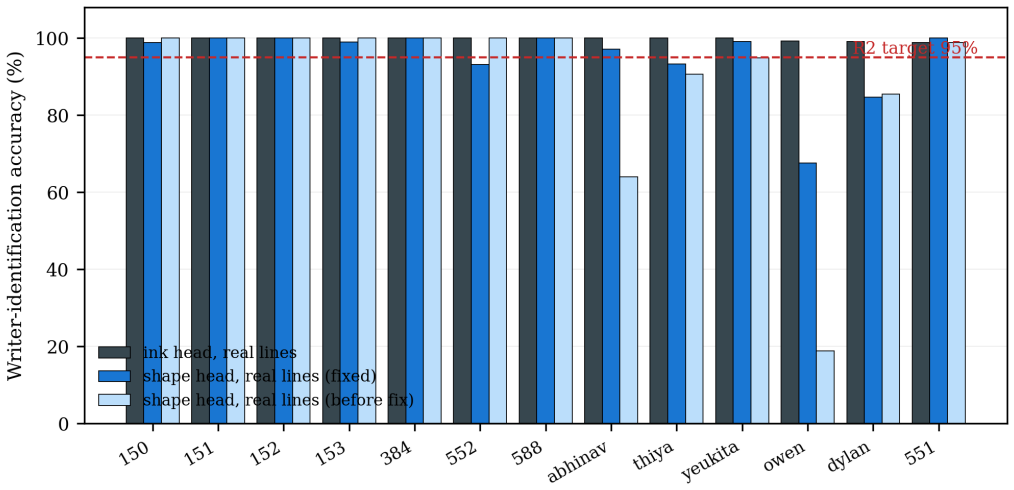


Figure 21. Writer-identification accuracy per writer over 1358 real lines, for the ink classifier and for the shape classifier before and after its training fault was corrected. The ink classifier is at or near 100% for every writer. Correcting the shape classifier's training raised its overall accuracy from 83.4% to 93.7% and removed a systematic collapse in which 118 of Owen's 148 lines had been assigned to Yeukita. The dashed line is the 95% target of R2.

Feature	Definition
Slant	Shear angle (degrees) that maximises the variance of the column ink projection of the line.
Stroke width	Median stroke width from the distance transform at the skeleton, divided by x_{h}.
Ink density	Fraction of ink pixels inside the core band between the baseline and the x-height line.
Word gap	Median horizontal gap between words (gaps larger than $0.5\,x_{\mathrm{h}}$) divided by x_{h}.
Connectedness	Number of connected ink components divided by the number of characters of the text.
Character width	Line width divided by the number of characters, divided by x_{h}.
Ascender/descender ratio	(Height of the ascender zone plus depth of the descender zone) divided by x_{h}.

Table 24. Hand-crafted style features prescribed for the correlation and retrieval of qualification test 1. The extractor that produced the measured correlation implements this set extended to the nine features listed in Table 16.

P, and these tables are the evidence that it is.

7.0.0 Constants of the minor operations

Table 25 collects every constant of the minor operations of all six stages with the reason for each, so that the stage descriptions of Section 3.4 can keep to the operations themselves. Only four entries are not expressed in units of the pitch, and those four are identified in the caption.

7.0.0 The eight artefact-discard predicates

Stage 4 judges each closed piece of ink as a whole and discards it if any one of eight predicates holds. Table 26 states them with the nuisance each is meant to catch, for a piece of area A, height h and width w, with $e = \max(3, \lfloor 0.5P \rfloor)$ counting as touching the frame, x_L the fifth percentile of the left edges of the letter-like pieces ($A \geq 0.065P^2$, $h \leq 2.2P$) and faintness defined as $\bar{\rho}_c > 0.85\,t^{*}$,

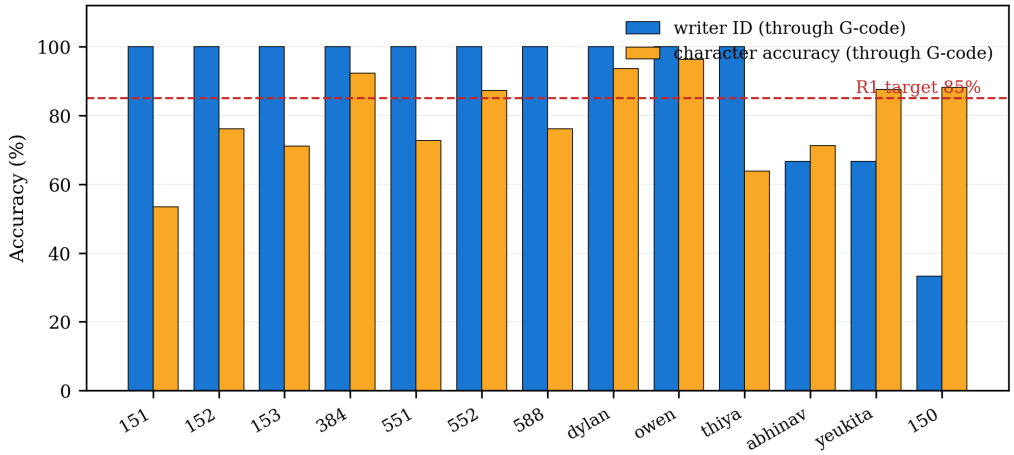


Figure 22. End-to-end result per writer over all thirteen writers: writer identification and character accuracy, both measured through the G-code round trip, for three novel sentences per writer at best-of-8 (`gcode_v2_summary.json`; shape judge `author_shape_10new_weights`, text judge `paper_cnn_bilstm_ctc_joint_best`). Ten of the thirteen writers are identified at 100% by the shape judge; the remaining three are 150 at 33.3% and Abhinav and Yeukita at 66.7% each. The character-accuracy failures are concentrated in the two writers whose hands are the most joined, which is the known weak case of an exemplar synthesiser that cuts letters apart and reassembles them.

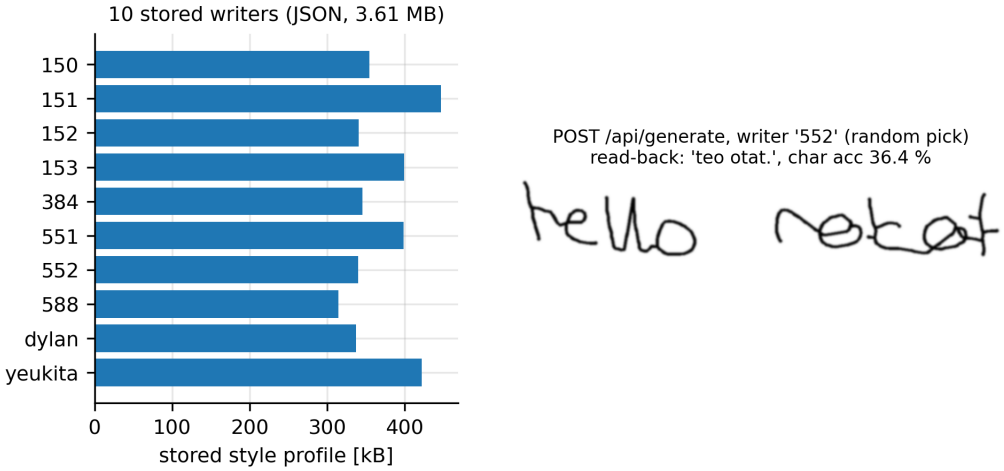


Figure 23. Storage of the writing styles. Each writer is held as a style profile and the server lists all thirteen through its author-list endpoint, against the ten required by R6.

relative to the page's own threshold so that one constant serves pencil and fibre tip.

7.0.0 The six pixel-ownership correction rules

By default an ink pixel belongs to the band it lies in, and stage 6 then applies six correction rules on top of that default, each of which may move a piece to an adjacent band or drop it but never further. Table 27 states them. They rest on each band's share of a piece's ink, with 0.15 counting as a genuine presence, and on the *major* band, defined as the topmost band reaching the piece,

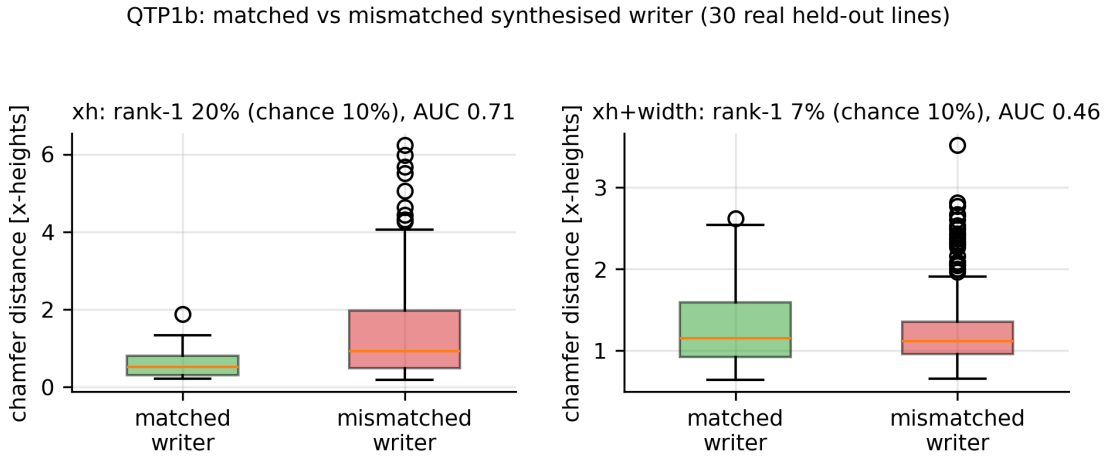


Figure 24. Distributions of the matched and mismatched overlay distances. The two distributions overlap but their means are separated by a factor of about 2.2 on the chamfer measure, giving an AUC of 0.71: the synthesis carries real writer information, but it is far from indistinguishable from the hand it imitates.

St.	Quantity and value	Reason
2	Otsu histogram over $[0, 1.2]$, 256 bins	paper sits at $\rho \approx 1$ and nothing above 1.2 carries ink information, so a highlight cannot drag the cut
2	solid-area veto $\mathrm{BoxMean}_4(\mathrm{ink}) > 0.999$, cleared over 7×7	no pen stroke fills a 9×9 window; the fringe goes with the blob, and 0.999 absorbs integral-image rounding
3	17 angles, $-4.0°$ to $+4.0°$ in $0.5°$	one sweep, no refinement, over a sixteenth of the photograph's area
3	rotation fill 0 for the candidate masks, 255 for the page	outside the sheet must add no ink to the profile; corners brought in must be white paper
4	paper region: block-average by $s = 4$, blur $\max(1, \lfloor 0.15P/s \rfloor)$, dark below $0.45\times$ median	a ratio and not a grey level, so it survives a change of exposure
4	whole frame accepted as paper below 0.2 % dark cells; sheet eroded $\lfloor 0.4P \rfloor$ per axis	correct for a photograph filled by the sheet; the erosion keeps the sheet's darkened rim out of the ink mask
4	piece closing $\max(1, \lfloor 0.1P \rfloor)$	bridges the pen lifts inside a word, so a word is one piece
5	centre separation $0.6P$	more than half a pitch, so two genuine lines can never be merged
6	seam closing $\max(2, \lfloor 0.07P \rfloor)$	fills only the gap beside a letter stem; a dilation would make a blob the cut would trace
6	outer boundaries $\lfloor 0.75P \rfloor$ beyond the first and last centre	covers the first line's ascenders and the last line's descenders
6	band emitted above $\max(60, 0.12\,\mathrm{median}_n N_n)$ owned pixels and $\max(6, 0.2P)$ rows	keeps a short line carrying an eighth of the typical ink; the absolute 60 catches a tiny-median page
6	vertical gap trimmed only if it removes more than $0.8P$	spares the ordinary gap between a line's body and its own ascenders
6	crop padding $\lfloor 0.15P \rfloor$ per side, halo $\lfloor 0.20P \rfloor$ per axis, local recovery radius $\max(3, \min(20, \mathrm{round}(h_{\mathrm{crop}}/8)))$	the halo anchors recovery to ink the line already owns; the local radius judges a pale line against its own contrast
6	stretch of Equation 22: b the 40th percentile under the mask, w the 75th of the crop	the 40th percentile is the typical ink darkness, so one dark speck cannot flatten the line

Table 25. Constants of the minor operations of each stage of the segmentation block, with the reason for each. Every entry in units of the measured pitch P scales with the page; the four that do not are the Otsu range, the 0.2 % and 0.45 paper-region tests and the floor of 60 ink pixels. The constants of the core operations appear in the equations of Section 3.4 itself.

since a piece spanning two lines is the upper one's descender.

Discard condition	What it is meant to catch
$x_{\min} \le e$	A piece touching the left frame edge: the facing page, or the desk.
$x_{\max} \ge W - 1 - e$, $w \le 0.5P$	A narrow sliver at the right edge; a real line there is wider.
$y_{\min} \le e$ or $y_{\max} \ge H - 1 - e$, $h \le 0.5P$, $w \le 0.5P$	A small fleck at the top or bottom frame edge.
$h > 2.2P$, $w \le 0.5P$	A tall thin streak over two line spacings: a binder edge or a fold.
$w \le 0.12P$, $h \ge 1.2P$	A bar thinner and taller than any letter.
faint, $w \le 0.2P$, $h \ge 0.6P$	A printed margin rule or a crease.
($y_{\min} \le e$ or $y_{\max} \ge H - 1 - e$), $h \le 1.2P$, $A/(hw) < 0.15$, faint, $w > P$	An edge shadow or fold: wide, faint, at an edge, too sparse inside its own box to be writing.
$x_{\max} < x_L - 1.2P$	A hole punch, a staple shadow or a page number in the gutter.

Table 26. The eight discard predicates applied to each closed piece of ink in stage 4; the background label is exempt from all of them. The first row is the most aggressive rule in the module: any piece whose leftmost ink lies within e of the left frame edge is discarded whatever its size or darkness, which removes the facing page of an open book but also costs the first letters of a page photographed with no left margin.

Rule	Applies to	Action, and the reason for it
1	A remnant in a band adjacent to the major band: under $0.12P^2$ there, under 0.35 of the piece's ink, no wider than $0.55P$	Returned to the major band. The width test separates a cut-off tail, which is narrow, from a genuine short word, which is wide.
2	A piece under $0.06P^2$; a *speck* is under $0.01P^2$	Dropped if no *big* piece ($\ge 0.05P^2$) of its own line lies within $6P$ horizontally, else moved one band towards the body it belongs to, asymmetrically ($0.5P$ below, $0.7P$ above), because a mark below a line is usually its own comma or descender while one well above it is more often an accent of the line above. A speck out of band drops rather than moves.
3	A piece of $0.06P^2$ to $0.15P^2$	Given to whichever of the three nearest bands has its body closest in y.
4	A *dot*: area under $0.1P^2$, extent at most $0.45P$, aspect ratio under 2.2 so that a short dash does not qualify	Followed downwards up to $\lfloor 0.4P \rfloor$ rows to the first *stem* ($\ge 0.03P^2$) and moved to that stem's band only if that band's body also lies $0.25P$ to $0.7P$ below it, where an *i* dot sits relative to its own x-height band. Together the two conditions are much stronger than either alone, since the scan can meet the wrong stem on a crowded page and the band test can accept a mark belonging to no stem.
5	The same search on the raw, unclosed labelling, for pieces of at least $0.015P^2$	Catches a dot the stage-4 closing has welded to a descender of the line above. A ring-footed stroke the same welding makes — a blob of width $0.2P$ to $0.45P$, aspect ratio 0.6 to 1.6 — also follows the stem below, but only if it encloses a hole, since a dot ring does and a hook does not. Neither rule reassigns a whole word ($\ge 0.2P^2$).
6	A fragment of at least $0.006P^2$ with no ink within $0.28P$ of its own band's centre	Offered to the line whose ink it touches, or failing that to the nearer neighbouring centre if the fragment sits within $0.2P$ of it. Fragments placed by rule 5 are locked, since a dot deliberately lifted into a band would otherwise be pushed straight back.

Table 27. The pixel-ownership correction rules of stage 6. The size ladder they share runs $0.006P^2$, $0.01P^2$, $0.015P^2$, $0.03P^2$, $0.05P^2$, $0.065P^2$, $0.1P^2$, $0.12P^2$ and $0.2P^2$, every entry a multiple of the measured pitch squared, so no rule is tied to a pixel count.

7.0.0 Pseudocode of the four routines that carry the arithmetic

Four routines account for almost all of the block's arithmetic, and each is given here as pseudocode. The first shows why the three array primitives of Section 3.4 cost the same whatever the window radius: each is two cumulative sums and a slice difference, so none contains a loop over pixels.

Stage 2 is four array statements and holds no tunable grey level, which is the result of dividing the illumination out before thresholding rather than after.

Stage 5 returns the scale unit and the line positions from one pass over the mask, and falls back

```
BoxMean( image, radius r ):            # mean over a (2r+1)x(2r+1) window
    pad the image by r, replicating the edge
    integral <- cumulative sum down, then cumulative sum across
    return ( four corner look-ups into integral ) / (2r+1)^2

AnyWithinRadius( mask, radius r, axis ):          # one-axis dilation
    c <- cumulative sum of the mask along the axis
    return ( c[ i+2r+1 ] - c[ i ] ) > 0     for every position i

LongRunMask( mask, length n, axis ):   # pixels on a run of n or more
    c   <- cumulative sum of the mask along the axis
    hit <- ( c[ i+n ] - c[ i ] ) = n        # a run of n starts at i
    ch  <- cumulative sum of hit
    return ( ch[ high ] - ch[ low ] ) > 0   # some such run covers i
```

Figure 25. The three array primitives of the segmentation block as pseudocode. `BoxMean` is Equation 9, `AnyWithinRadius` is Equation 10 and `LongRunMask` is Equation 11. Because each is built on a cumulative sum, a window of any size costs four look-ups, which is what makes the 40-pixel illumination blur of stage 2 affordable.

```
OtsuThreshold( values ):
    histogram <- 256 bins of the values clipped to [0, 1.2]
    w, m      <- cumulative class weight and cumulative class mean
    variance  <- ( total mean . w  -  m )^2 / ( w . (1 - w) )
    return the bin centre maximising variance

InkMask( grey ):
    r     <- max( 8, min( 40, round( image height / 41 ) ) )
    ratio <- grey / max( BoxMean( grey, r ), 1 )      # divide, not subtract
    mask  <- ratio < OtsuThreshold( ratio )
    solid <- BoxMean( mask, 4 ) > 0.999    # a 9x9 window wholly ink
    return mask AND NOT ( dilate solid by 3 ),  ratio
```

Figure 26. Lighting normalisation and the ink mask as pseudocode. The array `ratio` is ρ of Equation 12 and `OtsuThreshold` is the cumulative form of Equation 13 evaluated for all 256 bins at once. The threshold 0.999 rather than 1 absorbs floating-point error in the integral image; no pen stroke fills a 9×9 window, so a window that is wholly ink is a shadow or a binder and is removed with its fringe.

three times rather than failing.

Stage 6's seam is an exact minimisation, because a path cost is a sum over columns and every admissible predecessor is considered.

Supporting detail of the text recogniser

Section 3.5 describes the recogniser, its training and its accuracy. This appendix carries the material behind those statements: the parameter budget layer by layer, the dynamic programme that evaluates the connectionist temporal classification (CTC) loss, the backward passes written for this project together with the numerical check that validates them, the leakage assessment of each data set, and pseudocode of the five routines that do the arithmetic. It adds no result and changes no figure; it exists so that each claim of Section 3.5 can be checked rather than taken on trust.

```
profile  <- ink pixels per row, over the central 5% to 97% of the width
smoothed <- profile convolved with a 25-tap Gaussian, sigma = 4 rows
peaks    <- interior rows with  smoothed[i] >= smoothed[i-1]
                           and  smoothed[i] >  smoothed[i+1]
                           and  smoothed[i] >  0.06 . max( smoothed )

if at least two peaks:   return median gap between consecutive peaks
else: autocorrelate the mean-removed profile; take the first lag in
      15 .. min(200, length/3) reaching 0.7 of the segment maximum
else: P <- max( 15, round( 2.2 . median letter height ) )
else: P <- 48 rows
```

Figure 27. The pitch measurement as pseudocode. The peak test is Equation 18 and the median gap is the pitch P of Equation 17. The smoothed profile is returned as well and the caller reuses it for the line centres, so the scale unit and the line positions come out of one pass over the mask. The autocorrelation is a fallback only because it can lock onto a multiple of the true spacing.

```
for each pair of consecutive line centres c0, c1:
    cost <- 4.0 . closedInk  +  0.012 . ( (row - midline) / halfGap )^2
    seam <- LeastCostSeam( cost, step = 7 )

LeastCostSeam( cost, step s ):           # one row index per column
    acc[ :, 1 ] <- cost[ :, 1 ]
    for x = 2 .. W:
        for each displacement d in -s .. +s:
            candidate[d] <- acc[ row+d, x-1 ] + 0.02 . |d|
        acc[ :, x ]  <- cost[ :, x ] + min over d of candidate[d]
        back[ :, x ] <- the d that achieved the minimum
    backtrack from the cheapest entry of the last column
```

Figure 28. The least-ink seam as pseudocode. The cost field is Equation 19 and the loop over columns is the recursion of Equation 20. All rows of a column are evaluated at once as $2s+1$ shifted candidate arrays, so only the displacements and the backtrack are loops. At a weight of 4.0 against a maximum of 0.012, crossing one ink pixel costs over three hundred times more than sitting a half-gap from the midline, so the midline term only breaks ties over the gaps between words.

7.0.0 Parameter budget of the network

Table 28 resolves the 3.36 million parameters quoted in Section 3.5 into the layer that holds each one, which shows where the capacity of the network actually sits.

The arithmetic cost follows from the same shapes. Summing the multiply–accumulates of every convolution, of the two recurrent layers over all T steps and of the output projection gives

$$N_{\mathrm{MAC}} = \sum_{\ell} 9 C_{\mathrm{in},\ell} C_{\mathrm{out},\ell} H_{\ell} W_{\ell} + 2T\left[4n(128+n) + 4n(512+n)\right] + T \cdot 512 \cdot 75 \approx 4.17 \times 10^{9} \qquad (74)$$

per line, of which 3.79×10^{9} are convolutional. Equation 74 is the reason the convolutions and not the recurrent layers dominate the measured inference time, even though Table 28 shows the recurrent layers holding 70.5 % of the parameters: a convolutional filter is applied at every one of tens of thousands of positions, whereas a recurrent weight is used once per time step.

Layer	Output shape	Trainable parameters
Input	$1 \times 64 \times 640$	0
Conv 3×3 (1→32) and (32→32), BN, ReLU	$32 \times 64 \times 640$	9 696
Max-pool 2×2	$32 \times 32 \times 320$	0
Conv 3×3 (32→64) and 3× (64→64), BN, ReLU	$64 \times 32 \times 320$	129 792
Max-pool 2×2	$64 \times 16 \times 160$	0
Conv 3×3 (64→128) and 5× (128→128), BN, ReLU	$128 \times 16 \times 160$	813 312
Height max-pool	$128 \times 1 \times 160$	0
BiLSTM layer 1 (128→256, 2 directions)	160×512	790 528
BiLSTM layer 2 (512→256, 2 directions)	160×512	1 576 960
Linear (512→75) and log-softmax	160×75	38 475
Total		**3 358 763**

Table 28. Layer shapes and trainable parameters of the text recogniser (batch dimension omitted). A convolution with batch normalisation holds $C_{\text{out}}(9C_{\text{in}}+1)+2C_{\text{out}}$ parameters and one LSTM direction $4n(I+n)+8n$ with $n=256$, which is where every entry comes from. Of the total the LSTM layers hold 70.5 %, stage 3 24.2 %, stage 2 3.9 %, the output layer 1.1 % and stage 1 0.3 %. Batch normalisation also stores 2 176 non-trainable running statistics, so a checkpoint holds $3\,358\,763+2\,176=3\,360\,939$ numbers, 13.4 MB as 32-bit floating point.

7.0.0 The CTC forward–backward recursion

Section 3.5 states that the CTC loss sums over all alignments and that the sum is evaluated by dynamic programming rather than enumerated. This sub-subsection gives that dynamic programme, because it is the one part of the training mathematics that was implemented from the defining equations rather than taken from a library.

The number of paths is exponential in T, so the sum is evaluated by the forward–backward dynamic programme of Graves *et al.* [9] on the blank-augmented label y' of length $S=2U+1$. The forward variable $\alpha_t(s)$ is the total probability of consistent prefixes of length t ending at position s and obeys

$$\alpha_t(s)=\Big[\alpha_{t-1}(s)+\alpha_{t-1}(s-1)+\mathbb{1}\big[y'_s\neq\varnothing\wedge y'_s\neq y'_{s-2}\big]\,\alpha_{t-1}(s-2)\Big]\,y_t^{y'_s}, \tag{75}$$

whose three terms are the only transitions into s — stay, advance, or skip a blank — the skip being forbidden when y'_s is blank or equals y'_{s-2}, which is what allows a genuine double letter to survive the merge. Carried to the last step, Equation 75 gives $p(y\mid x)=\alpha_T(S)+\alpha_T(S-1)$ at $O(TS)$ cost. The backward variable $\beta_t(s)$ is its mirror image and excludes the emission at step t, so that the product $\alpha_t(s)\beta_t(s)$ counts that emission exactly once; the occupancy and the closed-form gradient with respect to the pre-softmax scores are then

$$\gamma_t(s)=\frac{\alpha_t(s)\,\beta_t(s)}{p(y\mid x)},\qquad \operatorname{occ}_t^k=\sum_{s:\,y'_s=k}\gamma_t(s),\qquad \frac{\partial\mathcal{L}}{\partial\ell_t^k}=y_t^k-\operatorname{occ}_t^k. \tag{76}$$

Equation 76 is what makes CTC cheap to train: the gradient is the difference between what the network predicted and what the alignment lattice expects, so no explicit differentiation of the recursion is needed.

Both recursions run in the logarithmic domain, because a product of up to 160 probabilities underflows 64-bit arithmetic. Additions therefore become

$$\operatorname{LSE}(a,b)=m+\ln\!\left(e^{a-m}+e^{b-m}\right),\qquad m=\max(a,b),\qquad \operatorname{LSE}(-\infty,-\infty)=-\infty, \tag{77}$$

with a forbidden transition carried as $-\infty$; the last case of Equation 77 must be special-cased, since the general expression would evaluate $\infty-\infty$. Figure 29 gives the recursion and the

gradient as pseudocode.

```
input:  log-probabilities lp[t,k] for T steps and 75 symbols
        label y of length U
build:  y' <- blank, y[1], blank, y[2], ... , y[U], blank      (S = 2U+1)

logAlpha[1,.] <- -infinity everywhere except the first two positions
for t = 2 .. T:
    for every position s of y' simultaneously:
        stay      <- logAlpha[t-1, s]
        advance   <- logAlpha[t-1, s-1]
        skip      <- logAlpha[t-1, s-2]
        if y'[s] is blank or y'[s] = y'[s-2]:  skip <- -infinity
        logAlpha[t,s] <- LSE(stay, advance, skip) + lp[t, y'[s]]

logBeta  <- the same recursion run backwards from t = T
logP     <- LSE( logAlpha[T,S], logAlpha[T,S-1] )

gamma    <- exp( logAlpha + logBeta - logP )
occ[t,k] <- sum of gamma[t,s] over the positions s with y'[s] = k
gradient <- probabilities - occ
```

Figure 29. The CTC forward recursion and the loss gradient as pseudocode. The three shifted reads are the three transitions of Equation 75, which the implementation evaluates for all S positions at once rather than in a loop over s; `logP` is $\ln p(y \mid x)$ and the last two lines are the occupancy and gradient of Equation 76. The backward recursion differs only in direction and in excluding the emission at the current step.

Three conventions of the implementation matter for the losses quoted in Section 3.5. Each line contributes its whole negative log-likelihood and the loss routine divides only by the batch size, so a quoted loss is a mean *per line* and not per character; a long line therefore counts for more than a short one, and the figure cannot be compared with a per-character loss from elsewhere. Lines with no valid path are given zero loss, and every line uses the full $T = 160$ steps including the padding.

7.0.0 Pseudocode of the routines that carry the arithmetic

Four routines account for essentially all of the training time and all of the places where an error would be silent, so each is given here as pseudocode: the recurrent step, the convolution and its gradient, the training loop, and the greedy decoder.

A recurrent layer is evaluated one step at a time, and the four gates are read as four blocks of rows out of a single matrix product rather than computed separately, which halves the number of products per step.

The convolution is evaluated as a matrix product on rearranged patches, which is the im2col form of Equation 26. Its backward pass is the place where a defect is most easily hidden, for the reason given in the next sub-subsection.

The training loop shows where an image problem becomes a sequence problem, which is the single structural decision of the architecture.

At inference the decoder implements the collapse map $\mathcal{B}$ directly. The 74 symbols are the 26 lower-case and 26 upper-case letters, the ten digits and twelve symbols (space and `. , ; : ' " ! ? ( ) -`), with index 0 reserved for the CTC blank, which is why the output layer has 75 classes. Characters outside the set are removed from every label, lines that become empty

```
for t = 1 .. T:
    gates <- W_input . x[t] + W_recurrent . h[t-1] + biases     (4n rows)
    i, f  <- sigmoid( rows 1..n ),   sigmoid( rows n+1..2n )
    g     <- tanh( rows 2n+1..3n )
    o     <- sigmoid( rows 3n+1..4n )
    c[t]  <- f * c[t-1] + i * g
    h[t]  <- o * tanh( c[t] )
    cache[t] <- ( x[t], h[t-1], c[t-1], i, f, g, o, tanh(c[t]) )
```

Figure 30. One direction of one LSTM layer as pseudocode, where $n = 256$ is the hidden width and $*$ is element-wise multiplication. A step costs one matrix product for the input and one for the recurrent state. The tuple written to `cache` is exactly what the backward pass needs, so no activation is recomputed when the gradient is propagated back through time; the bidirectional layer runs the same procedure on the reversed sequence and concatenates the two results.

```
forward:
    cols <- every 3x3 patch of the zero-padded input, as a column
    out  <- reshape( W . cols ) + bias                      (one product)

backward, given dOut:
    dW    <- dOut . cols^T , summed over the batch
    dBias <- dOut , summed over batch, height and width
    dCols <- W^T . dOut
    dInput <- zeros
    for i = 1..3, for j = 1..3:                  # col2im MUST accumulate
        dInput[ :, :, i..i+H, j..j+W ] += dCols[ :, :, i, j, :, : ]
```

Figure 31. Forward and backward pass of the two-dimensional convolution as pseudocode. The forward line is Equation 26; `dW`, `dBias` and `dCols` are its three gradients, and the double loop is the col2im scatter-add. The batch dimension is carried by one broadcast matrix product rather than a loop over samples, which was measured to be about 35 % faster for a realistic training batch.

are dropped, and lines over 120 characters, which indicate two merged lines, are dropped.

7.0.0 The backward passes and their verification

Section 3.5 states that every backward pass was written for this project and verified numerically. This sub-subsection gives those gradients, the two places where the implementation rather than the algebra is the risk, and the full result of the check.

With dA denoting $\partial \mathcal{L} / \partial A$, the gradients are the convolution's (Figure 31), batch normalisation's [29], the rectifier's [30], dropout's [48] and the max-pool routing's, Equation 78, the adaptive pools', Equation 79, the linear layer's, Equation 80, the bidirectional LSTM's [31] and the two losses', Equation 76 for CTC [9] and Equation 38 for cross-entropy.

$$\mathrm{d}x = \mathrm{d}Y \odot \mathbb{1}[x > 0] \ \text{(rectifier)}, \qquad \mathrm{d}x = \mathrm{d}Y \odot M, \quad M_k = \frac{\mathbb{1}[\xi_k \geq p]}{1-p} \ \text{(dropout)}, \tag{78}$$

$$\mathrm{d}F[n,c,i,t] = \mathbb{1}\left[i = r^{\star}(n,c,t) \right] \mathrm{d}z_t[n,c], \qquad \mathrm{d}m[n,c,t] = \frac{1}{T} \mathrm{d}f[n,c], \tag{79}$$

$$\mathrm{d}W = \mathrm{d}Y^{\top} X, \qquad \mathrm{d}b = \sum_{m=1}^{M} \mathrm{d}Y[m, \cdot], \qquad \mathrm{d}X = \mathrm{d}Y\, W. \tag{80}$$

```
for each shuffled mini-batch of line images and target strings:
    logits, cache <- Forward( images )
    loss, dLogits <- CtcLoss( logits, targets )
    gradients     <- Backward( dLogits, cache )
    Optimiser.Step( gradients )

inside Forward:
    features <- convolutional backbone            (B, 128, 16, 160)
    pooled   <- maximum over the height axis      (B, 128,  1, 160)
    sequence <- move the width axis to the front  (160, B, 128)
                # <- the width of the feature map becomes the CTC time axis
    sequence <- two stacked bidirectional LSTM layers
    logits   <- linear projection to 75 classes, then log-softmax
```

Figure 32. One training step and the forward pass it calls, as pseudocode. Every routine named here is one written for this project; nothing in the loop comes from a machine-learning framework. The commented line is the step that turns an image problem into a sequence problem: after it, one time step is a four-pixel-wide column of the original line.

```
best <- argmax over the 75 symbols at each of the 160 time steps
previous <- none ;  text <- empty
for each token in best:
    if token is not the blank and token differs from previous:
        append the symbol of token to text
    previous <- token
```

Figure 33. The greedy decoder as pseudocode, which is the map $\mathcal{B}$ of Equation 32: a symbol is kept only when it is neither the blank nor a repeat of the previous step. There is no lexicon, language model or beam search.

In Equation 79 $r^\star$ is the winning row cached by the height maximum, so a feature that is never the column maximum receives nothing, and batch normalisation's input gradient couples all $N = BHW$ elements of a channel. Each of these is an exact derivative of a forward equation of Section 3.5, which makes the check that follows a check of the algebra rather than a measurement of an error.

Two of them need care in the implementation rather than in the algebra, and both would fail silently. The col2im step must accumulate, because for a 3×3 kernel one input pixel appears in up to nine columns, so it is nine slice additions into a zero array; a reshape alone would keep one contribution in nine and is the likeliest place for a convolution backward pass to be quietly wrong. In the bidirectional layer the backward direction runs on the reversed sequence, so its recursion must be driven by the reversed incoming gradient and its returned input gradient reversed back; omitting either yields a gradient wrong only in time ordering, which a falling loss curve would hide.

No automatic differentiation tool checks these backward passes, so each was verified against a

central-difference estimate,

$$\frac{\partial \mathcal{L}}{\partial \theta_i} \approx \frac{\mathcal{L}(\theta + \varepsilon e_i) - \mathcal{L}(\theta - \varepsilon e_i)}{2\varepsilon}, \qquad \varepsilon = 10^{-5}, \tag{81}$$

$$\frac{\max_i |g_i^{\mathrm{a}} - g_i^{\mathrm{n}}|}{\max\bigl(1,\ \max_i |g_i^{\mathrm{a}}| + \max_i |g_i^{\mathrm{n}}|\bigr)} < 10^{-6}, \tag{82}$$

a configuration being accepted when the relative difference of Equation 82 stays below 10^{-6} for the analytic gradient g^{a} against the numerical one g^{n} of Equation 81. A configuration is one pair of layer variant and tensor, and the scalar loss is $\mathcal{L} = \sum_j o_j R_j$ for fixed random R, so that $\partial \mathcal{L} / \partial o = R$ exactly. Of the 78 configurations, 77 passed, with relative differences between 4.9×10^{-12} and 1.4×10^{-10}, four to six orders of magnitude inside the tolerance. All arithmetic is in double precision and up to 40 random elements of each tensor are probed, 2044 in total; on the stricter per-element measure the worst case over all 2044 is 2.0×10^{-7}, on a recurrent LSTM weight whose gradient is itself near zero, where the central difference has no significant digits left. Max-pooling and the rectifier were given inputs from a permuted linear spacing, so that no element lies within ε of the kink; dropout was checked with the mask fixed; batch normalisation was checked in its training form, so the folded inference map is not relied upon; and the check runs against the batched form the trainer uses.

The one configuration that failed is a genuine defect, reported here rather than removed. The CTC loss does not survive an empty label sequence: for $U = 0$ the blank-interleaved target has $S = 1$ entry, but two of the shifted arrays carrying the recursion are built with two entries regardless of S, so the routine writes a two-element array into a one-element row and raises a shape error instead of returning the loss. Those arrays implement the $s - 1$ and $s - 2$ terms of Equation 75, so the fault is confined to the lattice indexing and touches neither the transition rule nor the gradient of Equation 76; a single-character target and a repeated-character target, which exercises the forbidden skip, both pass. Its practical effect is small, because a line with no transcription never reaches the trainer, but the function is not safe against one and the fault is listed in Section 5. The check also confirmed that with no possible alignment, $2U + 1 > T$, the loss is infinite with undefined gradients, so such samples must be removed before training.

7.0.0 Data sets and their leakage assessment

Every accuracy of Section 3.5 belongs to one of four data sets, each with a different split rule and a different reason why its figure may be optimistic. Table 29 states the rule and the leakage it leaves, so that no figure in the report is read as more general than its data set allows.

The labels of the database pages were generated by aligning the printed prompt words with the detected ink word runs by dynamic programming. An earlier proportional labelling assigned wrong text to lines, and correcting the evaluation labels alone raised the measured ten-author hold-out accuracy from 42.7 % to 89.4 %, which establishes that label quality, not the network, had limited the earlier measurements; that is the reason the recogniser was later trained on a public line-level set with clean labels [3].

Training images only are augmented after the resize with at most one of four geometric transformations, filled with white: a horizontal shear with $k \sim U(-0.6, 0.6)$, a rotation in $U(-2.5^\circ, 2.5^\circ)$, an elastic distortion with displacement fields smoothed at $\sigma \in \{3, 4\}$ and scaled by $\alpha \in \{15, 20\}$, and a perspective distortion moving each corner by up to $d W / 2$ and $d H / 2$ with $d \sim U(0.03, 0.08)$. Each is enabled independently with probability 0.5 and one of the enabled ones is then chosen uniformly, so that $P(\text{no augmentation}) = 1/16 = 6.25\,\%$. No colour, contrast, noise or blur aug-

Data set	Split rule	Caveat on the validation figure
Lines segmented by this project's own block from scanned database pages, labels generated by alignment (first stage only)	The last labelled page of every writer is held out.	Same writers in training and hold-out, so the figure is writer-dependent, and the labels are project-generated and noisy.
Public line-level IAM transcription set: 6 482 training, 976 validation, 2 915 test lines (6 481, 976, 2 915 after label cleaning)	Official training, validation and test partitions.	The validation set drives learning-rate decay and checkpoint choice, so its accuracy is mildly optimistic; the test set is used for no decision. The partitions are believed, but not verified, to be writer-independent, and the warm start may have seen some of the same forms.
13 photographed pages of the two personal writers (8 and 5 pages), 355 transcription lines of which one is a diagram marker. **[MUST STILL BE COMPLETED BY ME: confirm which personal set the deployed recogniser was fine-tuned on — this row's two writers, 13 pages and 355 lines, or the five writers, 30 pages and 669 lines that Section 4 reports on — and correct this row and the one below to match; the saved checkpoints do not say.]**	Per page, 15 % of the lines (at least one) drawn at random with a fixed seed; about 52 validation lines in total.	Validation lines come from the same pages, pen, paper, lighting and vocabulary as the training lines, so the figure measures same writer, same notebook, new lines, and is optimistic. It also takes part in checkpoint selection.
Ten-author hold-out (eight database writers, two personal)	Evaluation set only.	The held-out database lines may have been seen in the line-level training data and their labels are project-generated, so this is not a writer-independent measure.

Table 29. Data sets of the text recogniser and their leakage assessment. No figure in the report is measured on a split that is both writer-independent and labelled by hand; the public test partition is the closest, and it is the only set used for no decision.

mentation and no synthetic pre-training data are used.

7.0.0 Edit distance as the accuracy measure

The character accuracies of Section 3.5 are one minus a micro-averaged character error rate, and the error rate is built on the Levenshtein edit distance computed by the dynamic programme

$$D[i][j] = \min\big(D[i-1][j]+1,\; D[i][j-1]+1,\; D[i-1][j-1]+\mathbb{1}[a_i \neq b_j]\big), \tag{83}$$

with $d_{\mathrm{lev}}(a,b) = D[\,|a|\,][\,|b|\,]$, $D[i][0] = i$ and $D[0][j] = j$. The three branches of Equation 83 are a deletion, an insertion and a match or substitution, so the distance is the smallest number of single-character edits between the two strings. The error rate and the word measure are then

$$\mathrm{CER} = \frac{\sum_n d_{\mathrm{lev}}(\hat{y}_n, y_n)}{\sum_n |y_n|}, \qquad \mathrm{WordAcc} = \frac{\mathrm{LCS}(\text{reference words}, \text{predicted words})}{\#\text{reference words}}, \tag{84}$$

so the character error rate of Equation 84 is the total edit distance over the total reference length, and the measure is case-sensitive. The word measure of Equation 84 is a longest-common-subsequence recall and not $1 - \mathrm{WER}$, as Appendix 6.4 sets out at length. Section 4 averages instead the floored per-line accuracy $\max(0, 1 - d_{\mathrm{lev}}/|y|)$ after lower-casing, so the subsystem

figures of Section 3.5 and the qualification figures of Section 4 are not computed identically and should not be differenced.

Supporting detail of the writer classifiers

Section 3.6 describes the two writer classifiers, the reason a second one was needed and the accuracies measured for each. This appendix carries the parameter budget of the head against the shared backbone, the optimiser the two trainers use, and pseudocode of the deployed forward pass and of the stroke normalisation on which the shape classifier depends.

7.0.0 Parameter budget

Both classifiers reuse the recogniser's convolutional backbone and replace its recurrent stack and output head with two pools and two fully connected layers, so almost all of the capacity is shared rather than new.

Block	Output shape	Trainable parameters
Convolutional backbone, 12 layers as Table 28	$128 \times 16 \times 160$	952 800
Height max-pool, width mean-pool (Eq. 35)	128	0
FC 128→96, ReLU, dropout 0.2 (Eq. 36)	96	12 384
FC 96→13	13	1 261
Total (head 13 645)		**966 445**

Table 30. What the writer classifiers add to the recogniser's backbone, FC denoting a fully connected layer. Every backbone shape and parameter count is that of Table 28, the recurrent stack and the CTC head being replaced by the two pools and the two layers above; the shapes are identical but none of the 84 backbone tensors hold the same values. Batch normalisation adds 2 176 non-trainable running statistics, so the network holds 968 621 numbers. A forward pass needs about 3.79×10^9 multiply–accumulate operations and the parameters occupy about 3.9 MB as 32-bit floating-point numbers.

The ratio in Table 30 is what makes the training fault of Section 3.6 consequential: fitting the head alone fits 13 645 of 966 445 parameters, that is 1.4 % of the model, on features that a backbone trained for a different task produced.

7.0.0 The optimiser

Both trainers use AdamW, the decoupled-decay form [33] of Adam [35], with cosine annealing of the learning rate. The moment estimates and their bias corrections are

$$m_s = \beta_1 m_{s-1} + (1-\beta_1) g_s, \qquad \hat{m} = \frac{m_s}{1-\beta_1^{\,s}},$$
$$v_s = \beta_2 v_{s-1} + (1-\beta_2) g_s^2, \qquad \hat{v} = \frac{v_s}{1-\beta_2^{\,s}}, \tag{85}$$

and the parameter update with its schedule is

$$\theta \leftarrow \theta - \eta \Big(\frac{\hat{m}}{\sqrt{\hat{v}} + \varepsilon} + \lambda_{\mathrm{d}} \theta \Big), \qquad \eta_t = \frac{\eta_0}{2} \Big(1 + \cos \frac{\pi t}{T_{\max}} \Big), \tag{86}$$

with g_s the gradient at step s, m_s and v_s the running means of the gradient and of its square, $\hat{m}$ and $\hat{v}$ their bias-corrected forms, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, decay $\lambda_{\mathrm{d}} = 10^{-4}$ and the rate falling from η_0 to zero over $T_{\max}$ epochs. The bias correction of Equation 85 matters most in the first few dozen steps, where m_s and v_s are still dominated by their zero initial values and an uncorrected update would be far too small; the decoupled decay of Equation 86 is what distinguishes AdamW from Adam with an L2 penalty, since adding the penalty to g_s instead would let the adaptive

denominator scale it differently for every parameter. The ink classifier ran at $\eta_0 = 10^{-3}$ for 20 epochs on batches of 16 images with the gradient-norm clip at 5. The superseded frozen-head run used $\eta_0 = 3 \times 10^{-3}$, 500 epochs, batches of 64 feature vectors, no clipping and only the head parameters on 3 680 stored vectors, and is the configuration that reached 83.36 %.

7.0.0 Pseudocode of the forward pass and the stroke normalisation

The deployed forward pass is the same code the trainer runs for validation, which is why no separate inference implementation had to be verified.

```
Classify( batch of line images ):
    x <- convolutional stage 1, then 2x2 max-pool
    x <- convolutional stage 2, then 2x2 max-pool
    x <- convolutional stage 3                        (B, 128, 16, 160)
    x <- maximum over the height axis                 (B, 128, 160)
    f <- mean over the width axis                     (B, 128)
    h <- ReLU( linear( f ) )                          (B, 96)
    # the dropout layer between the two linear layers is inactive here
    return linear( h )                                (B, 13) raw scores
```

Figure 34. Deployed forward pass of both writer classifiers as pseudocode. The two pools are Equation 35 and the two linear layers Equation 36; the recurrent layers of the recogniser are absent, because writer identity does not depend on the letter sequence. Taking a maximum over height and a mean over width makes the feature a position-independent bag of local stroke patterns. The weights come from a plain array archive written by the trainer, with no third-party model format parsed.

Stroke normalisation is the transform that makes the shape classifier possible, since it removes the thickness cue by construction rather than by augmentation.

```
StrokeNormalize( grey crop, penWidthXh = 0.14 ):
    ink  <- Otsu binarisation, ruled lines removed, hairlines closed,
            components under 6 pixels dropped
    band <- first to last row whose ink count reaches 0.45 of the maximum
    xh   <- height of that band
    if xh < 4:  xh <- max( 8, crop height / 3 )        # fallback
    w    <- round( penWidthXh . xh ),  clipped to [2, 9]
    k    <- w made odd
    return Dilate( Skeletonize( ink ), k )
    # a sample with no ink left is dropped rather than classified
```

Figure 35. Stroke normalisation as pseudocode. The band is the core band of Equation 40, the dense band of the lower-case bodies; `Skeletonize` is the Zhang–Suen thinning [34] and `Dilate` the re-inking at the constant width. Because the width is a fixed fraction of the measured x-height, the result is independent of scan resolution: an x-height of 20 pixels gives a three-pixel pen whatever the camera.

Supporting detail of the style builder and the synthesiser

Sections 3.7 and 3.8 describe how a writer's hand is reduced to a library of pen paths and how a new line is then constructed from it. This appendix carries the geometric criteria that turn a thinned image into ordered pen paths, which Section 3.7 states in one sentence each, together with the schema of a stored profile, the constants of the synthesiser with the reason for every value, and the measured cost of the search. The tracing criteria are given here as explicit

conditions because each is a threshold on a measurable geometric quantity, and a reader checking the extraction needs the quantity and not the description.

7.0.0 From a thinned image to ordered pen paths

Thinning leaves a one-pixel-wide set of ink pixels, which is a picture of the pen path and not yet an ordered path. Four operations recover the ordering, and each rests on the 8-neighbourhood degree of a skeleton pixel p,

$$\deg(p) = \#\big\{q \in \mathcal{N}_8(p) \;:\; q \text{ is a skeleton pixel}\big\}, \tag{87}$$

which classifies every pixel by Equation 87: an *end* has deg = 1, an *interior* pixel deg = 2, and a *junction* deg ≥ 3.

Tracing. A chain is grown from each unvisited end or junction and extended through interior pixels only, stopping at the next end or junction, so that the chain set covers every skeleton pixel exactly once and each chain is an ordered pixel sequence. Stopping at junctions rather than choosing a branch there is what makes the decomposition unique: a junction is a point at which the true pen order is not recoverable from the image, so the decision is deferred to the re-joining rule below rather than taken arbitrarily.

Spur pruning. Thinning a stroke of finite width leaves short false branches at the ends of real strokes. A chain is discarded as a spur when

$$\operatorname{len}(C) < 2.5 \text{ pixels} \quad \wedge \quad \exists\, p \in \{p_1, p_{|C|}\} \;:\; \deg(p) = 1, \tag{88}$$

that is when it is shorter than 2.5 pixels *and* has at least one free end. The conjunction in Equation 88 matters: a short chain between two junctions is the crossbar of a letter such as A or the waist of an e and must be kept, whereas a short chain hanging off a junction carries no pen motion.

Junction re-joining. A self-crossing stroke, such as the loop of an e or the ascender of an l, breaks into several chains at the junction where it crosses itself, although the pen never left the paper. Two chain ends i and j are re-joined into one chain when they are close and nearly collinear,

$$\|p_i - p_j\| \le 1.5 \text{ pixels} \quad \wedge \quad \arccos\!\big(-\mathbf{d}_i \cdot \mathbf{d}_j\big) < 75^\circ, \tag{89}$$

where $\mathbf{d}_i$ and $\mathbf{d}_j$ are the unit outward directions of the two chains at those ends. The negative sign in Equation 89 is the continuation test rather than a parallelism test: two chains continue each other when one runs *into* the junction along the direction the other runs *out* of it, so their outward directions are nearly opposed and $-\mathbf{d}_i \cdot \mathbf{d}_j$ is near +1. A threshold of 75° admits the curvature a pen shows through a crossing while rejecting a genuine corner, at which the two strokes meet at nearer a right angle.

Curve simplification. A traced chain holds one point per pixel, far more than a pen path needs, and its points lie on a lattice. Each chain is therefore simplified by the Douglas–Peucker recursion [19]: for a chain from p_1 to p_n, the vertex of greatest perpendicular distance from the straight line joining its ends,

$$d_{\max} = \max_{1<i<n} \frac{\big|(p_n - p_1) \times (p_i - p_1)\big|}{\|p_n - p_1\|}, \tag{90}$$

is found; if $d_{\max} \le \varepsilon$ the whole chain is replaced by its two end points, and otherwise it is split at that vertex and both halves simplified in the same way. With $\varepsilon = 0.6$ pixels, which is 0.05 to 0.12 mm at 4 mm per x-height, the deviation of the simplified path from the traced one is bounded by ε everywhere by construction. Equation 90 measures distance to the infinite line through the two ends, which fails on a closed chain: there the two ends coincide, the denominator

vanishes and the whole loop would collapse to two points. A chain whose ends lie within $\max\bigl(1.5, 0.25\max_i \|p_i - p_0\|\bigr)$ pixels of each other is therefore split at its farthest vertex first and simplified in two open halves, which is what keeps the loops of e, l and o.

Resampling and smoothing. Thinning leaves a staircase on a diagonal stroke, whose median turn angle is 8.9° with a 90th percentile of 59.5°; drawn directly, that staircase appears as tremor. Kept variants are therefore resampled at 0.055 x-heights of arc length and smoothed by two passes of the three-point average

$$q_i \leftarrow \tfrac{1}{4} q_{i-1} + \tfrac{1}{2} q_i + \tfrac{1}{4} q_{i+1}, \tag{91}$$

with the end points held fixed. Equation 91 is a binomial kernel, so it attenuates the highest representable spatial frequency — the alternating lattice staircase — completely while leaving a smooth curve almost unchanged, and fixing the ends preserves the entry and exit points that the synthesiser's connectors attach to.

```
ForcedAlign( log-probabilities, transcript ):
    build y' <- blank-interleaved transcript            (S = 2U+1)
    delta[1,.] <- -infinity except the first two positions
    for t = 2 .. T:
        for every position s simultaneously:
            stay    <- delta[t-1, s]
            advance <- delta[t-1, s-1]
            skip    <- delta[t-1, s-2]
            if y'[s] is blank or y'[s] = y'[s-2]:  skip <- -infinity
            best    <- the largest of the three          # MAX, not a sum
            backpointer[t,s] <- which one won
            delta[t,s] <- best + logProb[t, y'[s]]
    back-trace from the better of the last two positions
    character i owns the frames whose winning position is 2i

CutCostPerColumn( column ink profile, baseline, x-height ):
    smoothed <- profile averaged over max(3, round(0.12 . xh)) columns, odd
    thin     <- smoothed / 60th percentile of the non-empty columns
    topAbove <- (baseline - highest ink row) / xh,  clipped to [0, 3]
    cost     <- thin + 1.15 . topAbove
    cost     <- 0 wherever the column holds no ink      # a true gap is free
```

Figure 36. The two routines of the builder that locate each character, as pseudocode. The first is Equation 42: it differs from the CTC loss of Equation 75 in taking the maximum where the loss takes a sum, which is what turns a probability into a single best alignment, and it keeps back-pointers so that the path itself can be recovered. The second is the cut cost of Equation 44 evaluated for every column, which is minimised near the boundary to place the cut at a ligature, that is at a column that is both thin and low.

7.0.0 The five quality gates

About 2 122 to 4 031 raw instances are extracted per writer and roughly 10 % survive, the gates being applied per character in this order. *Geometry* imposes size limits, at most six strokes, a class prior on height, the absence of a swallowed neighbour, and a pen length above the adaptive floor $\max(0.30, 0.5\,\tilde{p}_c)$ measured on that character's own median pen length $\tilde{p}_c$; the floor is adaptive because one fixed value either kept neighbour bleed or discarded genuinely short letters. *Outliers* requires the width and height to lie within fixed ratios of the 40th-percentile width and the median height. *The cross-writer prior* of Equation 47 keeps the 70 % nearest the pooled shape. *Consensus* keeps the nearest 75 % to the L1-medoid of the kept descriptors, which removes glyphs contaminated by a neighbour. *Typicality* keeps the best 12 by closeness to the reference

width and height. The cascade is ordered cheapest-first and most-local-first: the geometric tests reject on properties of one glyph, the prior on agreement with all thirteen writers, and the last two on agreement with that writer's other instances of the same letter, so a glyph is only compared with its peers once it is known to be a plausible letter at all.

7.0.0 Schema of a stored style profile

A profile is one text file per writer, and Table 31 lists the keys that carry design meaning, with the values of one writer as an example.

Key	Meaning	Example (153)
`slantDeg`, `xHeightPx`, `strokeWidthXh`	Median slant (Equation 45), median pixel x-height and ink area divided by skeleton length in x-heights.	30.0, 33.0, 0.189
`ascender`, `descender`	Typical reach of tall and descending lower-case glyphs, in x-heights (defaults 1.7 and −0.6).	1.59, −1.10
`wordSpaceXh`, `connectedness`	Word gap measured on the real ink, in x-heights; fraction of adjacent in-word letter pairs that the pen joins.	1.59, 0.961
`letterAdvance`	Median advance of each character, in x-heights (45 keys).	`T`: 1.438, `e`: 1.147
`slantMeasRef`, `ascMeasRef`, `descMeasRef`	Slant and reach of the real full lines, measured with the estimators later used on synthesised renders; input of the style calibration (Equation 51).	30.0, 1.933, −1.468
`glyphs`, `idealGlyphs`, `idealMedoid`	Variant library: per character up to 12 kept variants and a prepended prototype, each holding its `strokes` as $[x, y]$ points in x-heights (baseline $y = 0$, upright) with the width, advance, lead, reach, entry and exit coordinates, connection flags and priorD the synthesiser needs; then the per-stroke median prototype and the medoid, which is the anchor.	37 characters, 283 variants; 23 and 29 prototypes
`legibilityLambda`	Legibility parameter λ of Equations 48 and 49; written by the tuner after the build, *not* by the builder. *Absent from all thirteen stored profiles*, so the deployed code falls back to $\lambda = 0$.	not present

Table 31. Keys of a style-profile file (JavaScript Object Notation) that carry design meaning; bookkeeping keys and the grey-level reference statistics are omitted. The example column was re-read from the stored file `StyleProfiles10/153.json`. The whole library is geometry in x-height units, which is why one profile serves any pen size.

7.0.0 Constants of the synthesiser

Every constant of the glyph choice, the placement, the joins and the randomness is listed in Table 32 with the design reason for its value, so that Section 3.8 can describe the construction without interrupting it to justify a number.

7.0.0 Cost of the candidate search

A candidate costs one synthesis, one rendering, one recogniser pass and one classifier pass, so the search time is

$$T \approx N\left(T_{\text{syn}} + T_{\text{ren}} + T_{\text{text}} + T_{\text{wid}}\right) + (\text{up to three repair candidates}), \tag{92}$$

each network pass being about 4×10^9 multiply–accumulate operations (Equation 74, Table 30). Table 33 resolves Equation 92 into its terms on the development laptop, and the result decides where any optimisation must go: synthesis costs 11 to 16 ms per candidate against 0.5 to 4 s for the two judges, so the search is entirely judge-bound, and within the judges the dominant cost is thinning the full-resolution render rather than either network.

Were the candidate scores independent with distribution function F, then $\Pr\left[\max_k S_k \le \tau\right] = F(\tau)^N$ and the gain from raising N would grow roughly logarithmically, which is the usual order-statistics argument for a best-of-N scheme. The seeds share one profile, so independence is only approximate and no per-N curve was kept; the measured values at $N = 6$ and $N = 14$ in Table 13 are consistent with a diminishing return but do not establish its form.

Constant	Value	Reason for the value
Eligible variants per letter	5 best by priorD	80.5 against 75.5 % writer identification over all twelve
Anchor taken from the medoid	priorD ≤ 0.30	above it, not clean enough; skeleton used instead
Cursive anchor threshold	$c \ge 0.5$, off above 1	0.3 lowered the evaluation score
Anchor re-proportioning	$y \mapsto 1 + (y-1)\max(0.35, \frac{A-1}{0.75})$, $y \mapsto y\max(0.35, \frac{\lvert D\rvert}{0.55})$	skeleton takes the writer's reach; floor 0.35 keeps it from flattening
Pen advance	the glyph's own advance	never the character median
Anchor shear cap	$\pm\tan 30°$, $\pm\tan 22°$ print	40° collides even for a clean letter
Collision guard	$0.05\,x_{\mathrm{mm}}$	shift right rather than let ink touch
Word gap	$g\,x_{\mathrm{mm}}(0.85 + 0.3u)$, $u \sim U(0,1)$	so words do not fall on a grid
Line width; pitch	185 mm in the server; L (Eq. 48)	wrap inside the work area; writer's line spacing
Connector sag	$\min(0.11x_{\mathrm{mm}}, 0.16\lvert\Delta x\rvert)$, plus noise	span-proportional: a fixed dip loops a short join
Polyline resampling	0.35 mm of arc length	even plotter motion
Jitter rotation; scale; translation	$2.2°J$; $0.05J$; $0.022J$ and $0.0132J$ x-heights	one amplitude sets how unsteady the hand looks
Jitter amplitude J	0.15 default, 0.5 search, 0.15 anchors	the search needs spread; anchors exist to stay legible
Baseline drift	walk of $0.05J \cdot 0.35$ per glyph, clipped ±0.25 x-heights	wanders slowly, clipped to stay on the line
Judges' render settings	0.085 x-height pen, blur 0.30 of it, 18 px/mm, pad $0.18\,x_{\mathrm{mm}}$	identical ink for every candidate (Section 3.8.3)

Table 32. Constants of the synthesiser, with the design reason for each. Lengths in x_{mm} scale with the pen size; A, D, c and g are the profile's ascender, descender, connectedness and word gap. At the $J = 0.5$ the search uses, the jitter comes to a rotation of $1.1°$, a scale change of 2.5 % and a translation spread below 0.05 mm, which perturbs a letter without changing its identity.

Quantity (56-character sentence)	Writer 150	Writer 153	Writer 551
Render size (pixels)	3585×225	5508×207	7876×277
Synthesis of one candidate (ms)	11	13	16
Rendering (ms)	20	27	56
Text recogniser (s)	0.50	0.49	0.74
Writer classifier incl. stroke normalisation (s)	1.03	1.65	3.74
Complete search, $N = 14$, with repair attempts (s)	23	37	137

Table 33. Indicative cost of the search on the development laptop (single profiling run, unknown background load, jitter 0.5); times on the single-board computer were not recorded. The spread across the three writers follows the render width, which follows the writer's own letter size and spacing, so the cost of a search is set by the hand being imitated and not only by N.

Supporting detail of the hardware design

Section 3.3 gives the complete circuit with every component value on it, the resolution the transmission yields and the two design faults found during bring-up. This appendix carries the derivations behind those values: the pin assignment, the three interface-circuit calculations that size the bulk capacitor, the pull-downs and the pen-motor switching stage, the stepper-and-belt dynamics that the torque requirement rests on, the interpolation and calibration arithmetic, and pseudocode of the motion program. Every number quoted in Section 3.3 that was calculated rather than measured is derived here.

7.0.0 Pin assignment

Twelve lines on one chip carry the whole interface, and all twelve are requested in a single call so that a second instance of the program cannot claim them during a job.

Function	Offset / header pin	Direction, initial level
X motor DIR, STEP	64/7, 68/11	output, high / low
Y motor DIR, STEP	81/12, 69/13	output, high / low
X_MIN, X_MAX, Y_MIN, Y_MAX	72/15, 65/16, 66/18, 67/22	input
Pen switch, pen motor	70/33, 71/35	input; output, low

Table 34. Pin assignment on `/dev/gpiochip0`; all twelve are requested in one call, so a second instance cannot claim them during a job. Header numbers are unchecked against the board pinout.

7.0.0 Stepper and belt dynamics behind the torque requirement

Section 3.3 states that the torque margin of this design cannot be stated, because neither the carriage mass nor the friction force was measured. This sub-subsection sets out the calculation that those two measurements would complete, so that what is missing is a pair of numbers rather than a method.

A belt axis converts motor torque to carriage force through the pulley pitch radius alone, there being no gear reduction, so with r from Equation 1 the force available at the carriage is $F = T/r$ and the torque the motor must supply is

$$T = r\left(m_{\text{eff}}\,a + F_f\right) + J_{\text{rot}}\,\frac{a}{r}, \tag{93}$$

which is Equation 2 with the rotational inertia of the pulleys, the cross-shaft and the rotor added. The second term of Equation 93 is not negligible for this machine: the lower axis turns a cross-shaft with a pulley at each end, so its rotating inertia is carried at the same acceleration as the carriage, and referring it to the carriage gives an equivalent mass J_{rot}/r^2. With $r = 6.37$ mm the division by r^2 is a multiplication by about 2.5×10^4 m^{-2}, so even a small rotor inertia contributes an equivalent mass of the same order as the carriage itself; a design that ignored it would understate the torque required. The effective mass of the lower axis is therefore

$$m_{\text{eff}} = m_{\text{carriage}} + m_{\text{upper axis}} + m_{\text{pen}} + \frac{J_{\text{rot}}}{r^2}, \tag{94}$$

the upper axis riding on the lower one, which is the asymmetry noted in Section 3.3: the two axes do not have the same load, so a torque margin established on the upper axis does not transfer to the lower one.

The torque a stepper can actually deliver falls with step rate, because the winding inductance limits how fast current is established. For a winding of inductance L and resistance R driven

from a rail V, the current reaches its regulated value with time constant L/R but is forced by the chopper at a rate V/L, so the fraction of rated current reached within one step interval τ of Equation 4 falls as τ shortens, and with it the available torque:

$$\frac{T(\tau)}{T_{\text{hold}}} \approx 1 - \exp\!\left(-\frac{V\,\tau}{L I_{\text{coil}}}\right), \tag{95}$$

which is why the 24 V rail was chosen over a 12 V one for the same motor, as Section 3.3 states: at fixed τ, doubling V in Equation 95 moves the knee of the torque–speed curve to roughly twice the step rate. Equation 95 is an approximation that ignores back-electromotive force and the chopper's own regulation, and it is given here to show the shape of the dependence rather than to predict a value; the manufacturer's pull-out torque curve is the quantity a design should check against, and it was not obtained. A pass would require the torque of Equation 93, evaluated with the measured mass of Equation 94 and the measured friction force at the 400 mm/s^2 of Equation 54, to lie below the pull-out torque at both of the step rates the interpreter uses, 1200 and 3200 steps per second, with margin. Because the coil current is deliberately derated to about 1 A per phase for thermal reasons (Section 3.3), the holding torque available is itself below the datasheet figure in roughly the same proportion, and that derating must be applied before the margin is taken. This is also the reason the design carries no acceleration ramp: a ramp is sized from the torque margin, and with no margin established there is no basis on which to choose one.

7.0.0 The three interface-circuit derivations

Three component values in Figure 6 follow from calculation rather than from a datasheet recommendation, and each is derived here.

Bulk capacitor. The chopper draws current in bursts that the supply leads cannot follow, so a capacitor sits at the drivers; the rail sag over one 1.68 A burst of 10 µs is

$$\Delta V = \frac{I\,\Delta t}{C} = \frac{1.68 \cdot 10 \times 10^{-6}}{100 \times 10^{-6}} = 0.17\ \text{V}, \tag{96}$$

which is small against 24 V and clear of the driver's undervoltage threshold [25]. Three gaps remain in this part of the design: one capacitor is shared where one belongs at each driver, no ceramic logic-supply capacitor is documented, and the supply current was never budgeted. **[MUST STILL BE COMPLETED BY ME: budget four phases at the derated coil current plus pen motor, board and camera against the supply rating, and confirm whether each driver has its own bulk and logic capacitor.]**

Coil current and its derating. The datasheet relation [25] between the trimmer reference, the carrier's two sense resistors and the regulated current gives

$$I_{\text{coil}} = \frac{V_{\text{ref}}}{8\,R_S} \quad\Longrightarrow\quad V_{\text{ref}} = 8\,R_S\,I_{\text{coil}} = 8 \cdot 0.1 \cdot 1.68 = 1.34\ \text{V} \tag{97}$$

for the 1.68 A rating [23] and the 0.1 Ω resistors usually fitted. That rating is deliberately not used. Each driver dissipates about $2I_{\text{coil}}^2 R_{DS(\text{on})}$, which has only the small carrier board to leave through, so an unheatsinked carrier is held near 1 A per phase and V_{ref} of Equation 97 is derated from 1.34 V to about 0.8 V, trading holding torque for a driver that will not shut down thermally in the middle of a page. **[MUST STILL BE COMPLETED BY ME: confirm the sense resistor fitted (0.05, 0.068 and 0.1 Ω are all sold) and the measured reference voltage, give the coil current Equation 97 then returns as a fraction of 1.68 A, and say whether the drivers have heatsinks.]**

Switch pull-downs. With one switch closed, and with all four end-stops closed, the pull-down

current is

$$I_{\text{pull}} = \frac{V_{DD}}{R} = \frac{3.3}{10000} = 0.33\ \text{mA}, \qquad 4I_{\text{pull}} = 1.3\ \text{mA}, \tag{98}$$

negligible on this board. The 10 kΩ of Equation 98 is bounded in both directions: a smaller value wastes current, and a larger one lets pin leakage lift the open-switch voltage, so $R \le V_{IL}/I_{\text{leak}}$ [49]. Its RC time constant is microseconds against the 6 ms re-read of Section 3.3, so the debounce belongs in the program and not in the network. **[MUST STILL BE COMPLETED BY ME: take V_{IL} and the input leakage from the ODROID N2+ documentation [49] and state the upper bound they put on the pull-down.] [MUST STILL BE COMPLETED BY ME: give the board revision and date of that document; the bibliography entry has neither.] [MUST STILL BE COMPLETED BY ME: confirm each switch is wired to 3.3 V, not 5 V, and give the header pin that rail comes from; Figure 6 infers it from program comments.]**

Pen-motor switching stage. A header pin cannot supply the pen motor's current, so the motor is switched by a bipolar transistor in the common-emitter configuration. With the pin at 3.3 V and $V_{BE(\text{sat})}$ taken as 0.7 V [50], the base current through the 1 kΩ resistor is

$$I_B = \frac{V_{\text{pin}} - V_{BE(\text{sat})}}{R_B} = \frac{3.3 - 0.7}{1000} = 2.6\ \text{mA}, \tag{99}$$

and with $h_{FE(\min)} = 250$ for the −40 gain group [50] and an overdrive factor k_{od} of three to five for hard saturation, the switchable collector current is

$$I_{C(\max)} = \frac{h_{FE(\min)}\, I_B}{k_{od}} = \frac{250 \cdot 2.6\ \text{mA}}{k_{od}} = \frac{650\ \text{mA}}{k_{od}}, \tag{100}$$

which is 217 mA at $k_{od} = 3$ and 130 mA at $k_{od} = 5$. Equation 100 is comfortable for a motor drawing up to about 200 mA, and the saturated dissipation $P = V_{CE(\text{sat})} I_C \approx 0.7 \cdot 0.2 = 0.14$ W is well inside the unheatsinked TO-92 package [50]. The overdrive factor is the design decision here: dividing by k_{od} rather than using the full $h_{FE} I_B$ keeps the transistor in hard saturation at the minimum gain of the group, so the collector–emitter drop stays at $V_{CE(\text{sat})}$ and the dissipation stays at the figure above even for the worst-case device. **[MUST STILL BE COMPLETED BY ME: give the manufacturer (onsemi and Nexperia both supply this part), part number and datasheet revision of the transistor fitted, and check the gain group, $V_{BE(\text{sat})}$, $V_{CE(\text{sat})}$ and TO-92 dissipation against it.] [MUST STILL BE COMPLETED BY ME: measure the pen motor's free-running and stall current against Equation 100; above about 200 mA the stage is not hard saturated and R_B must be reduced.]**

7.0.0 Interpolation and calibration arithmetic

Stepping the two axes independently would draw an L-shaped path, so the program interpolates with Bresenham's integer line algorithm [26]: an error term e is carried along the major axis of $a_x = |\Delta x|$ and $a_y = |\Delta y|$ steps, the major axis is pulsed every iteration and the minor axis whenever e would go negative. Keeping e in $[0, a_x)$ gives the minor-axis count after k major steps as

$$m_k = \left\lceil \frac{k\,a_y - \lfloor a_x/2 \rfloor}{a_x} \right\rceil, \tag{101}$$

which is within one step of the ideal $k a_y / a_x$, about 6 µm, using integer arithmetic only. Equation 101 is the reason no floating-point division appears in the inner loop: the deviation from the true line is bounded by construction rather than by rounding behaviour.

Calibration replaces the assumed resolution of Equation 3 with each axis's measured value, and it is written so that it never needs to know which direction leads to which switch. It drives one way until *either* switch of that axis triggers, reverses, and counts the steps N to the *other*. With

the hand-measured travel L, the resolution, the initial step counter and the position in millimetres are

$$\kappa_{\text{axis}} = \frac{N}{L}, \qquad n_0 = N \text{ if the second switch was MAX else } 0, \qquad x_{\text{mm}} = \frac{n}{\kappa} - 5, \tag{102}$$

so the zero of Equation 102 lies 5 mm inside the minimum switch, the usable area is 194×252 mm, and each axis retracts 5 mm off its last switch. An already triggered switch, an unexpected switch of the other axis, or 200 000 steps without a trigger aborts calibration, after which jobs are refused. The two traverses need $(204 + 262)/7.8 \approx 60$ s, derived and not measured. The result is deliberately not stored between sessions, since a bump between sessions would silently invalidate it.

An error δL in the hand-measured travel scales every commanded position by $x\,\delta L/L$, so to keep that scale error below half of the 0.5 mm allowed by R5 over the 194 mm of usable X travel,

$$\delta L \le \frac{0.25 \text{ mm}}{194 \text{ mm}} L = \frac{0.25}{194} \cdot 204 \text{ mm} = 0.26 \text{ mm} \quad (0.13\ \%), \tag{103}$$

and switch-trigger repeatability adds directly to the zero error on top of that. Equation 103 is the tightest tolerance anywhere in the hardware design, and it falls on a measurement taken with a hand-held rule. **[MUST STILL BE COMPLETED BY ME: state the method and uncertainty of the L_X and L_Y measurement, which Equation 103 requires better than 0.26 mm.]**

7.0.0 Pseudocode of the motion program

Three routines of the motion program carry the behaviour that Section 3.3 describes: the debounced end-stop read inside the stepping loop, the direction-agnostic calibration, and the pen toggle.

Supporting detail of the G-code generator and the software architecture

Sections 3.9 and 3.11 describe how a trajectory becomes a machine program and how the blocks form one application. This appendix carries the generator's configuration with the role of every field, the apportionment of the two timing requirements across the stages that spend them, and pseudocode of the emitting loop and of the start-up warm-up. The apportionment is given here because it is a design-stage calculation rather than a result: it states what each stage was *allowed* to cost, against which the measurements of Section 4 can be read.

7.0.0 Configuration of the G-code generator

Every constant of the generator is listed in Table 35 with its role. The motor, driver and transmission values are assumptions recorded in the program comments, not measurements; the calibrated steps per millimetre of Section 3.3.4 replace the derived ones in the interpreter, so a scale error in the generator's assumption does not reach the machine.

7.0.0 Apportionment of the two timing budgets

Requirements R3 and R4 bound the *total* time of a job, and Equations 56 and 57 decompose each total into the stages that spend it. A budget is only useful once it is apportioned, and Table 36 states the allowance this design assigned to each stage, together with the laptop figure measured for it. The apportionment was made at design time from the structure of the chain, not fitted to the measurements: the recognition and classification terms were given the whole per-character allowance of R4 because they are the only terms that grow with x, leaving the fixed 2 s for capture, conditioning and segmentation; and in R3 the physical writing was given three quarters of the per-character allowance, since it is the one term that cannot be made faster by a faster computer.

```
TriggeredEndstop( ignore ):            # three reads, 6 ms apart
    for each end-stop not in ignore:
        if it does not read closed: continue
        re-read it twice more, 6 ms apart
        if it read closed every time: return that end-stop
    return none

StepSegment( dx, dy, feed ):           # x taken as the major axis
    delay <- max( (distance / feed) / majorSteps,  2 microseconds )
    e <- majorSteps / 2
    repeat majorSteps times:
        if TriggeredEndstop() is not none:  stop at once
        pulse the major-axis STEP pin ;  e <- e - minorSteps
        if e < 0:  pulse the minor-axis STEP pin ;  e <- e + majorSteps
        sleep( delay )

Calibrate( axis ):                     # no direction is ever named
    drive one way until EITHER switch of this axis triggers -> first
    reverse, counting steps, until the OTHER one triggers    -> N
    stepsPerMm <- N / measuredTravel
    position   <- N  if the second switch was MAX  else 0
    retract 5 mm off the switch that is still closed

SetPen( wantUp ):
    if any end-stop is triggered: refuse
    if the pen switch already reads wantUp: return   # motor must NOT run
    energise the pen motor
    until the deadline: if the pen switch reads wantUp: stop; break
    if it reached the up state: wait 40 ms more to seat
    de-energise the pen motor
```

Figure 37. The four routines of the motion program as pseudocode. In `StepSegment`, `delay` is τ of Equation 4 and `e` is the error term of Equation 101; the end-stop read before every major-axis pulse is what sets the latency of Equation 6. In `Calibrate`, `N` and the two assignments after it are the quantities of Equation 102. `SetPen` must check the switch first because the cam turns one way only, so an unnecessary pulse would toggle the pen to the wrong state rather than leave it alone.

Field	Value	Meaning
Steps, microstepping, travel per revolution; steps per millimetre	200, 16, 40.0 mm; 80.000	Assumed 1.8° stepper, driver setting and 2 mm-pitch belt on a 20-tooth pulley, giving $200 \cdot 16/40$.
Work area; origin and park	0 to 200 mm in X and Y; (10, 180) mm	Limits of the clamp; top-left corner of the text block.
Draw and travel feed	900, 2400 mm/min	Pen down (15 mm/s), pen up (40 mm/s, emitted once).
Acceleration; minimum segment	400 mm/s^2; 0.12 mm	Used only by the time model; shorter moves are collapsed.
Pen pulse and settling; pen codes and power-on state	260 and 90 ms; `M5`, `M3`, up	Assumed time for a 90° rotation and the dwell after a toggle; pen up and pen down, the up state at power-on being assumed.

Table 35. Configuration of the G-code generator. Only the work area and the two feeds are design choices of this project; the rest are assumptions about parts, which is why the interpreter re-measures the resolution rather than trusting the figure in the file header.

Stage	Design-stage allowance	Measured on the laptop, and the verdict
R4: capture, conditioning and segmentation	the whole fixed 2 s, being independent of x	24.8 to 69.1 s per page, 12 to 35 times the allowance: the term that fails R4, and it fails by an order of magnitude rather than marginally
R4: recognition and writer classification	the whole 25 ms per character, being the only terms growing with x	8.86 ms per character for recognition and 4.6 to 10.3 ms for classification, so about half the allowance together: met with margin
R3: synthesis	1 s per character, a quarter of the allowance	0.4 to 0.7 s per character at $N = 1$, rising to 1.8 to 3.4 s at $N = 30$: met only at small N, which is why N is the parameter that decides whether R3 can be met at all
R3: G-code generation	negligible by design, the generator doing no search	0.006 to 0.014 s per sentence: met by three orders of magnitude
R3: physical writing	3 s per character, three quarters of the allowance, being the irreducible term	2.7 to 4.0 s per character from the generator's model, no stopwatch reading: cannot be declared met, and the upper end of the model already exceeds the allowance

Table 36. Apportionment of the budgets of Equations 56 and 57 across the stages, with the laptop figures of Section 4 against each. No stage was timed on the single-board computer, so every verdict here is provisional on hardware that is faster than the target. The two stages that fail are the two that were never redesigned after first being written: page segmentation and the physical writing.

Read against Table 36, the failure of R4 is localised rather than general: four of the five stages are inside their allowance, two of them by a wide margin, and the whole shortfall sits in page segmentation. That is also the one stage for which a remedy was demonstrated but not deployed, the two routines reimplemented in C99 having reduced the retired segmenter by 25 to 32 % end to end (Section 3.4); a saving of that size would not by itself close a factor of twelve, so the conclusion the apportionment supports is that segmentation needs a different algorithm or a different language, not a faster implementation of the same one.

7.0.0 Pseudocode of the emitting loop and the start-up warm-up

The emitting loop is short, and its one subtlety is the pen state: because the lift is a one-way toggle motor, a pulse emitted where none is needed inverts the pen for the remainder of the job, so the pen command is emitted only on a change of state.

```
emit the comment header, G21 (millimetres) and G90 (absolute)
emit the travel feed once

for each stroke of at least two points:
    SetPen( up )                 # emits M5 and a dwell ONLY on a change
    rapid move to the first point of the stroke
    SetPen( down )               # emits M3 and a dwell ONLY on a change
    emit the draw feed
    for every remaining point of the stroke:
        emit a linear move to that point, three decimals

SetPen( up ) ;  move to the park position ;  emit M2
```

Figure 38. The emitting loop of the G-code generator as pseudocode. `SetPen` returns nothing at all when the pen is already in the wanted state, which is what keeps the one-way toggle motor of Section 3.3.5 in step with the program's model of it; the dwell it emits on a change is the $260 + 90$ ms of Table 35. Fault 2 of Section 3.9.3 is visible here: the draw feed is emitted before every stroke but the travel feed only once, so every travel after the first runs at the draw feed.

The start-up warm-up is the one path in the application that can reach a machine-learning library, as Section 3.11 records.

```
Prewarm( ):                  # a background thread, not the inference path
    for each writer, in order:
        try:
            build or fetch the cached glyph grid for that writer
            build or fetch two cached sample crops for that writer
        except any error:
            warn and continue              # a failure must not stop start-up
        mark that writer's cache as warm

start Prewarm in a background thread at start-up
any route needing a cached file answers "still preparing" until it is marked
```

Figure 39. The start-up warm-up as pseudocode. The two cache-building calls are the ones that, on a miss, reach into the retired source tree and import a machine-learning library; because three writers have no cached files as delivered, that miss happens on every start, which is the defect Section 3.11 records. Running it on a background thread is what keeps the delay off the first request, and marking each writer separately is what lets the routes that do not need a cache answer immediately.